

O'REILLY®

Паттерны разработки на Python

TDD, DDD и событийно-ориентированная
архитектура



Гарри Персиваль
Боб Грегори

Architecture Patterns with Python

*Enabling Test-Driven Development,
Domain-Driven Design, and
Event-Driven Microservices*

Harry Percival and Bob Gregory

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Паттерны разработки на Python

TDD, DDD и событийно-ориентированная
архитектура

Гарри Персиваль
Боб Грегори



Санкт-Петербург • Москва • Минск

2022

ББК 32.988.02-018
УДК 004.738.5
П27

Персиваль Гарри, Грегори Боб

П27 Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура. — СПб.: Питер, 2022. — 336 с.: ил. — (Серия «Для профессионалов»).

ISBN 978-5-4461-1468-9

Популярность Python продолжает расти, а значит, проекты, созданные на этом языке программирования, становятся все масштабнее и сложнее. Многие разработчики проявляют интерес к высокоуровневым паттернам проектирования, таким как чистая и событийно-управляемая архитектура и паттерны предметно-ориентированного проектирования (DDD). Но их адаптация под Python не всегда очевидна. Гарри Персиваль и Боб Грегори познакомят вас с проверенными паттернами, чтобы каждый питонист мог управлять сложностью приложений и получать максимальную отдачу от тестов. Теория подкреплена примерами на чистом Python, лишенном синтаксической избыточности Java и C#.

В этой книге:

- «Инверсия зависимостей» и ее связи с портами и адаптерами (гексагональная/чистая архитектура).
- Различия между паттернами «Сущность», «Объект-значение» и «Агрегат» в рамках DDD.
- Паттерны «Репозиторий» и «UoW», обеспечивающие постоянство хранения данных.
- Паттерны «Событие», «Команда» и «Шина сообщений».
- Разделение ответственности на команды и запросы (CQRS).
- Событийно-управляемая архитектура и реактивные расширения.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.738.5

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

ISBN 978-1492052203 англ. Authorized Russian translation of the English edition of *Architecture Patterns with Python* ISBN 9781492052203 © 2020 Harry Percival and Bob Gregory. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-5-4461-1468-9 © Перевод на русский язык ООО Издательство «Питер», 2022
© Издание на русском языке, оформление
ООО Издательство «Питер», 2022
© Серия «Для профессионалов», 2022

Краткое содержание

Предисловие	12
-------------------	----

Введение	21
----------------	----

Часть I. Создание архитектуры для поддержки моделирования предметной области

Глава 1. Моделирование предметной области	32
---	----

Глава 2. Паттерн «Репозиторий»	53
--------------------------------------	----

Глава 3. О связанности и абстракциях	74
--	----

Глава 4. Первый вариант использования: API фреймворка Flask и сервисный слой	90
--	----

Глава 5. TDD на повышенной и пониженной передачах	109
---	-----

Глава 6. Паттерн UoW	120
----------------------------	-----

Глава 7. Агрегаты и границы согласованности	136
---	-----

Часть II. Событийно-управляемая архитектура

Глава 8. События и шина сообщений	162
---	-----

Глава 9. Катимся в город на шине сообщений	180
--	-----

Глава 10. Команды и обработчик команд	200
---	-----

Глава 11. Событийно-управляемая архитектура: использование событий для интеграции микросервисов	211
---	-----

Глава 12. Разделение обязанностей команд и запросов	226
---	-----

Глава 13. Внедрение зависимостей (и начальная загрузка)	246
---	-----

Эпилог	268
--------------	-----

Приложение А. Сводная диаграмма и таблица	290
---	-----

Приложение Б. Шаблонная структура проекта	292
---	-----

Приложение В. Замена инфраструктуры: делаем все с помощью CSV	302
---	-----

Приложение Г. Паттерны «Репозиторий» и UoW с Django	308
---	-----

Приложение Д. Валидация	318
-------------------------------	-----

Об авторах	329
------------------	-----

Об обложке	330
------------------	-----

Оглавление

Предисловие	12
Управлять сложностью, решая бизнес-задачи	12
Почему Python?	13
TDD, DDD и событийно-управляемая архитектура	14
Для кого эта книга	15
Краткий обзор книги	16
Дополнительные материалы	17
Примеры кода и работа с ним	17
Условные обозначения	19
Благодарности	19
От издательства	20

Введение	21
Почему в проекте что-то идет не так?	21
Инкапсуляции и абстракции	22
Разделение на слои	24
Принцип инверсии зависимостей	25
Место для всей бизнес-логики: модель предметной области	27

ЧАСТЬ I. СОЗДАНИЕ АРХИТЕКТУРЫ ДЛЯ ПОДДЕРЖКИ МОДЕЛИРОВАНИЯ ПРЕДМЕТНОЙ ОБЛАСТИ

Глава 1. Моделирование предметной области	32
Что такое модель предметной области	32
Изучение языка предметной области	36
Юнит-тестирование моделей предметных областей	38
Не все должно быть объектом: функция службы предметной области	48

Глава 2. Паттерн «Репозиторий»	53
Организация постоянного хранения модели предметной области.....	54
Немного псевдокода: что нам потребуется?	55
Применение принципа инверсии зависимостей для доступа к данным	55
Напоминание: наша модель.....	57
Введение паттерна «Репозиторий».....	63
Теперь поддельный репозиторий для тестов создается просто!.....	69
Что такое порт и что такое адаптер в Python.....	70
Выводы	71
 Глава 3. О связанности и абстракциях.....	74
Абстрагирование состояния способствует тестопригодности	76
Выбор правильной(-ых) абстракции(-й).....	79
Реализация выбранных абстракций	81
Выводы	89
 Глава 4. Первый вариант использования: API фреймворка Flask и сервисный слой.....	90
Связываем приложение с реальным миром.....	92
Первый сквозной тест.....	93
Простая реализация	94
Состояния ошибок, требующие проверки базы данных	95
Введение сервисного слоя и использование поддельного репозитория для юнит-теста.....	97
Почему все называется службой?	102
Складываем все в папки, чтобы понять, где что находится	103
Выводы	105
 Глава 5. TDD на повышенной и пониженной передачах.....	109
Как выглядит пирамида тестирования	110
Должны ли тесты слоя предметной области перейти в сервисный слой? ...	110
Какие тесты писать	112
Повышенная и пониженная передачи.....	113
Устранение связей между тестами сервисного слоя и предметной областью.....	113

Дальнейшее улучшение с помощью сквозных тестов.....	117
Выводы	118
Глава 6. Паттерн UoW	120
Паттерн UoW работает с репозиторием	121
Тестирование UoW интеграционными тестами.....	123
UoW и его контекстный менеджер.....	124
Использование паттерна UoW в сервисном слое.....	127
Явные тесты для форм поведения по фиксации/откату	128
Явные и неявные фиксации	129
Примеры: использование паттерна UoW для группировки многочисленных операций в атомарную единицу	131
Приведение в порядок интеграционных тестов.....	132
Выводы	133
Глава 7. Агрегаты и границы согласованности	136
Почему бы просто не записать все в электронную таблицу?.....	137
Инварианты, ограничения и согласованность	138
Что такое агрегат	139
Выбор агрегата.....	141
Один агрегат = один репозиторий.....	145
А что насчет производительности?	146
Оптимистичная конкурентность с номерами версий.....	148
Тестирование правил целостности данных	152
Выводы	155
Итоги части I.....	157
 ЧАСТЬ II. СОБЫТИЙНО-УПРАВЛЯЕМАЯ АРХИТЕКТУРА	
Глава 8. События и шина сообщений	162
Как избежать беспорядка	164
Принцип единственной обязанности	166
Катимся на шине сообщений!	167

Вариант 1: сервисный слой берет события из модели и помещает их в шину сообщений.....	171
Вариант 2: сервисный слой инициирует собственные события.....	172
Вариант 3: UoW публикует события в шине сообщений.....	173
Выводы	177
Глава 9. Катимся в город на шине сообщений	180
Новое требование приводит к новой архитектуре.....	182
Рефакторинг функций служб для обработчиков сообщений.....	184
Реализация нового требования.....	191
Тест-драйв нового обработчика.....	192
Необязательно: юнит-тест обработчиков событий в изоляции с помощью поддельной шины сообщений.....	196
Выводы	198
Глава 10. Команды и обработчик команд	200
Команды и события	200
Различия в обработке исключений	202
События, команды и обработка ошибок.....	204
Синхронное восстановление после ошибок.....	208
Выводы	210
Глава 11. Событийно-управляемая архитектура: использование событий для интеграции микросервисов	211
Распределенный комок грязи, или Мыслить существительными	212
Обработка ошибок в распределенных системах.....	216
Альтернатива: временное устранение связанности при помощи асинхронного обмена сообщениями	218
Использование канала «издатель/подписчик» хранилища Redis для интеграции	219
Тестирование с помощью сквозного теста	219
Внутренние события против внешних.....	224
Выводы	224

Глава 12. Разделение обязанностей команд и запросов.....	226
Модели предметной области для записи.....	226
Большинство пользователей не собираются покупать вашу мебель.....	228
PRG и разделение команд и запросов	230
Хватайте свой обед, ребята	232
Тестирование представлений CQRS	233
«Очевидная» альтернатива 1: использование существующего репозитория.....	234
Модель предметной области не оптимизирована для операций чтения	235
«Очевидная» альтернатива № 2: использование ORM.....	236
SELECT N+1 и другие соображения по поводу производительности.....	237
Время прыгать через акулу	238
Изменить реализацию модели чтения очень просто	242
Выводы	244
 Глава 13. Внедрение зависимостей (и начальная загрузка).....	246
Неявные зависимости против явных	249
Разве явные зависимости не кажутся странными и Java-подобными?	250
Подготовка обработчиков: внедрение зависимостей вручную с помощью замыканий и частичных применений	252
Альтернатива с использованием классов	254
Сценарий начальной загрузки	255
Шина сообщений получает обработчики во время выполнения	258
Использование начальной загрузки в точках входа	259
Внедрение зависимостей в тестах.....	260
«Правильное» создание адаптера: рабочий пример	262
Выводы	266
 Эпилог	268
И что теперь?	268
Как мне добраться туда?	268
Разделение запутанных обязанностей	269
Определение агрегатов и ограниченных контекстов.....	273

Подход на основе событий для перехода к микросервисам через паттерн «Душитель»	277
Как убедить стейкхолдеров попробовать что-то новое	281
Вопросы наших научных редакторов, которые мы не включили в основной текст	284
Выстрел в ногу	287
Книги для обязательного прочтения.....	289
Выводы	289
Приложение А. Сводная диаграмма и таблица.....	290
Приложение Б. Шаблонная структура проекта	292
Приложение В. Замена инфраструктуры: делаем все с помощью CSV	302
Приложение Г. Паттерны «Репозиторий» и UoW с Django.....	308
Приложение Д. Валидация.....	318
Об авторах	329
Об обложке	330

Предисловие

Вам интересно, кто мы такие и почему написали эту книгу?

В конце своей последней книги «Python. Разработка на основе тестирования» (O'Reilly) Гарри задался множеством вопросов об архитектуре, например: «Как лучше структурировать приложение, чтобы его было легко тестировать. Точнее, как сделать так, чтобы ваша стержневая бизнес-логика была охвачена юнит-тестами и чтобы можно было минимизировать число необходимых интеграционных и сквозных тестов?» Он вскользь упомянул о «гексагональной архитектуре», «портах и адаптерах» и архитектуре «функциональное ядро — императивная оболочка», но по справедливости он должен был признать, что не особо разбирался в этих вещах и уж тем более не применял их на практике.

Но, к счастью, он встретил Боба, у которого нашлись ответы на эти вопросы.

Бобу пришлось стать архитектором ПО, потому что в его команде таких специалистов не было. Как и следовало ожидать, поначалу выходило не очень. Но Боб встретил Яна Купера, научившего его по-новому писать код и в целом смотреть на программирование.

Управлять сложностью, решая бизнес-задачи

Мы оба работаем в MADE.com — европейской компании, занимающейся онлайн-продажами мебели. В своей работе мы применяем описанные в этой книге приемы для создания распределенных систем, моделирующих реальные бизнес-задачи. Рассматриваемый нами пример предметной области — первая система, которую Боб создал для MADE. Эта книга — попытка записать все те штуки, которые должны освоить новички в наших командах.

Компания MADE.com управляет глобальной цепочкой поставок, состоящей из перевозчиков и производителей. В целях экономии издержек мы стараемся оптимизировать доставку товаров на склады, чтобы избежать простоя.

В идеале нужный вам диван должен прибыть в порт в тот самый день, когда вы решите его купить. Мы же должны отправить его прямо к вам домой, минуя этап хранения на складе.

Чтобы правильно выбрать время, необходимо учитывать то, что с момента отправки и до момента прибытия товара морем уходит три месяца. По пути товары ломаются или портятся от воды, штормы становятся причиной неожиданных задержек, транспортные компании неправильно обращаются с грузом, документы пропадают, клиенты меняют решения и корректируют заказы и т. д.

Мы решаем такие проблемы, создавая интеллектуальное ПО, которое представляет виды реальных операций. Благодаря этому мы можем максимально автоматизировать бизнес-процессы.

Почему Python?

Если вы читаете эту книгу, то нет смысла убеждать вас в том, что Python просто великолепен. Настоящий вопрос вот в чем: зачем сообществу Python нужна эта книга? Ответ кроется в популярности и зрелости этого языка: хотя популярность Python растет такими темпами, что он приближается к верхним строкам рейтингов, сам язык только начинает брать на себя задачи, над которыми C# и Java работали в течение многих лет. Стартапы становятся реальным бизнесом; веб-приложения и сценарии автоматизации становятся (по большому секрету) *корпоративным ПО*.

В мире Python мы часто цитируем Дзен Python: «Должен существовать один — и желательно только один — очевидный способ сделать это»¹. К сожалению, по мере роста проекта наиболее очевидный способ выполнения задач не всегда помогает управлять сложностью и меняющимися требованиями.

В этой книге нет ни одного нового паттерна или приема, но для мира Python они все же в новинку. Наша книга не заменит классику вроде «Предметно-ориентированного проектирования» Эрика Эванса (Eric Evans, Domain-Driven Design) или «Шаблонов корпоративных приложений» Мартина Фаулера (Martin Fowler, Patterns of Enterprise Application Architecture), на которые мы часто ссылаемся и которые рекомендуем прочитать.

¹ `python -c "import this"`.

Но все классические примеры, как правило, написаны на Java или C++/#, и если вы питонист и долго (или вообще никогда) не использовали ни один из этих языков, то разобраться в этих примерах будет непросто. Вот почему в последнем издании еще одной классической книги, «Рефакторинг. Улучшение проекта существующего кода» Фаулера (Refactoring, Addison-Wesley Professional), примеры были на JavaScript.

TDD, DDD и событийно-управляемая архитектура

Ниже в порядке известности приведены три методологии, применяющиеся для управления сложностью.

1. *Разработка через тестирование* (test-driven development, TDD) помогает писать правильный код, делать рефакторинг или добавлять новые возможности, не опасаясь регрессии. Но иногда бывает трудно использовать тесты по максимуму: как сделать так, чтобы они выполнялись как можно быстрее? Так, чтобы мы могли увеличить покрытие и получить больше информации от быстрых юнит-тестов, свободных от зависимостей, при этом уменьшив число более медленных нестабильных сквозных тестов?
2. *Предметно-ориентирование проектирование* (domain-driven design, DDD) предлагает нам сосредоточить усилия на создании модели хорошей предметной области. Но как освободить модели от инфраструктурной функциональности, чтобы они не становились все менее пригодными для внесения изменений?
3. Слабосвязанные (микро)сервисы, интегрированные посредством сообщений (иногда их называют *реактивными микросервисами*), являются хорошо зарекомендовавшим себя методом при решении задач, связанных с управлением сложностью в многочисленных приложениях или предметных областях бизнеса. Но не всегда очевидно, как вписать их в инструменты мира Python — Flask, Django, Celery и т. д.



Не пугайтесь, если не работаете с микросервисами (или не заинтересованы в них). Подавляющее большинство паттернов, которые мы обсуждаем, включая большую часть материалов по событийно-управляемой архитектуре, применимы в монолитной архитектуре.

Цель этой книги — представить несколько классических паттернов разработки и показать, как они поддерживают разработку через тестирование (TDD), предметно-ориентированное проектирование (DDD) и событийно-управляемые службы. Мы надеемся, что она послужит ориентиром для реализации этих паттернов на Python и подтолкнет программистов к дальнейшему изучению этой области.

Для кого эта книга

Вот как мы представляем вас, дорогой читатель:

- Вы имели какое-то отношение к разработке нескольких достаточно комплексных приложений на Python.
- Разобраться с этой комплексностью было непросто.
- Вы можете ничего не знать о предметно-ориентированном проектировании или любом из классических паттернов разработки.

Мы структурируем изучение паттернов разработки вокруг примера приложения, выстраивая его глава за главой. В своей работе мы используем разработку через тестирование (TDD), поэтому предпочитаем сначала показывать листинги тестов, а затем реализацию. Если вы не привыкли работать по принципу «сначала тест», то поначалу это может показаться странным, но мы надеемся, что прежде, чем смотреть на то, как код устроен изнутри, вы скоро привыкнете смотреть на то, как он «используется» (то есть снаружи).

Мы применяем несколько питоновских фреймворков и технологий, включая веб-фреймворк Flask, инструментарий по работе с SQL и объектно-реляционное отображение (ORM) SQLAlchemy, библиотеку pytest, а также Docker и Redis. Хорошо, если вы уже с ними знакомы, но вообще это не обязательно. Одна из главных целей этой книги состоит в том, чтобы выстроить архитектуру, для которой конкретные варианты выбора технологии становятся второстепенными деталями реализации.

Краткий обзор книги

Книга делится две части; ниже приводится обзор тем и глав.

Часть I. Создание архитектуры для поддержки моделирования предметной области

Моделирование предметной области и предметно-ориентированное проектирование (главы 1 и 7)

Понятно, что сложные бизнес-задачи должны отражаться в коде в виде модели предметной области. Но почему всегда кажется, что это так трудно сделать, не запутавшись в инфраструктурных возможностях, веб-фреймворках или чем-то еще? В первой главе мы даем широкий обзор *моделирования предметной области* и DDD, а также показываем, как начать работу с моделью, которая не имеет внешних зависимостей и быстрых юнит-тестов. Позже мы вернемся к паттернам предметно-ориентированного проектирования, чтобы обсудить вопрос выбора правильного агрегата и того, как этот выбор связан с целостностью данных.

Паттерны «Репозиторий», «Сервисный слой» и UoW (Unit of Work) (главы 2, 4 и 5)

В этих трех главах мы приводим три тесно связанных и взаимоподкрепляющих паттерна, которые позволяют сохранять модель свободной от посторонних зависимостей. Мы создаем слой абстракции вокруг системы постоянного хранения данных, воздвигаем сервисный слой, чтобы определить точки входа в систему и уловить основные варианты использования. Мы покажем, как этот слой позволяет легко создавать прозрачные точки входа в систему, будь то API веб-фреймворка Flask или CLI.

Некоторые соображения насчет тестирования и абстракций (главы 3 и 6)

После введения первой абстракции (паттерна «Репозиторий») мы обсудим вопрос выбора абстракций и их роли в принятии решения о степени связанности ПО. После введения паттерна «Сервисный слой» немного поговорим о создании *тестовой пирамиды* и написании юнит-тестов на самом высоком уровне абстракции.

Часть II. Событийно-управляемая архитектура

Событийно-управляемая архитектура (главы 8–11)

Здесь вводим еще три взаимно усиливающих паттерна: «События предметной области», «Шина сообщений» и «Обработчик». Паттерн «События предметной области» — это способ передать идею о том, что некоторые взаимодействия с системой являются триггерами для других. Мы используем «Шину сообщений», чтобы дать действиям возможность запускать события и вызывать надлежащие «Обработчики». Затем перейдем к обсуждению вариантов использования событий в качестве паттерна для интеграции между службами в архитектуре, построенной на основе микросервисов. Наконец, покажем различие между *командами* и *событиями*. И наше приложение по сути будет являться системой обработки сообщений.

Разделение ответственности между командами и запросами (глава 12)

Приведем пример *разделения ответственности между командами и запросами* с событиями и без событий.

Внедрение зависимостей (глава 13)

Приведем в порядок явные и неявные зависимости и реализуем простой фреймворк внедрения зависимостей.

Дополнительные материалы

Как мне добраться туда? (эпилог)

На простом примере реализовать паттерны проектирования с нуля всегда легко, но многих из вас, вероятно, интересует, как применить эти принципы к уже имеющемуся ПО. В эпилоге мы дадим несколько указаний и ссылок для дальнейшего изучения темы.

Примеры кода и работа с ним

Сейчас вы читаете книгу, но, думаем, согласитесь, что лучший способ узнать о коде — это писать его. Всему, что нам известно, мы научились благодаря командной работе и обучению на практике. В этой книге мы хотели бы максимально воссоздать для вас этот опыт.

Структурно книга строится вокруг примера одного проекта (хотя иногда мы рассматриваем и другие). Мы будем от главы к главе достраивать этот проект, как если бы вы были с нами в команде. На каждом этапе мы будем объяснять наши действия.

Но для того, чтобы по-настоящему разобраться с паттернами, нужно поработать с кодом самостоятельно и почувствовать, как он работает. Вы найдете весь код на GitHub — каждая глава имеет собственную ветвь. Там же можно найти список¹ ветвей.

Ниже приводятся три способа работы с кодом в процессе чтения книги.

- Сделайте собственный репозиторий и попробуйте создать приложение, как это делаем мы, следуя примерам и заглядывая в наш репозиторий за подсказками. Но предупреждаем: по сравнению с предыдущей книгой Гарри, где можно было писать код только на основе прочитанного, здесь нужно много разбираться самостоятельно. Возможно, придется брать за основу рабочие версии на GitHub.
- Попробуйте применить каждый паттерн к собственному, желательно небольшому проекту и посмотреть, работает ли он в вашем случае. Здесь работает принцип «серьезный риск / серьезная награда» (и «серьезные усилия» к тому же). Возможно, ввиду специфики вашего проекта придется изрядно потрудиться, чтобы привести все в рабочее состояние, но, с другой стороны, так вы узнаете гораздо больше.
- В каждой главе мы даем «Упражнение для читателя» и ссылку на GitHub, откуда можно скачать «скелет» кода для главы с несколькими недостающими частями, которые нужно дописать самостоятельно.

Если вы собираетесь применять некоторые из этих паттернов в своих проектах, то отработка простых примеров поможет набить руку.



При чтении каждой главы как минимум перенесите код из репозитория в отдельную ветку командой `git checkout`. Возможность посмотреть на код в контексте реально работающего приложения поможет ответить на массу вопросов. Инструкции, как это сделать, вы найдете в начале каждой главы.

¹ См. <https://github.com/cosmicpython/code/branches/all>

Условные обозначения

В книге используются следующие условные обозначения:

Курсив

Так выделяются новые термины и важные слова.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для ссылки на элементы программ, такие как переменные или имена функций, базы данных, типы данных, переменные среды, инструкции и ключевые слова.



Так обозначаются подсказки или советы.



Так обозначаются замечания общего характера.



Так обозначаются предупреждения или предостережения.

Благодарности

Наши научные редакторы Дэвид Седдон, Эд Юнг и Хайнек Шлавак! Признаемся, что абсолютно не заслуживаем вас. Вы все невероятно преданы своему делу, добросовестны и строги. Каждый из вас чрезвычайно умен, и ваши разные точки зрения были полезны и дополняли друг друга. Спасибо вам от всего сердца.

Выражаем огромную благодарность читателям ранней редакции книги за их комментарии и предложения, в том числе: Иэну Куперу, Абдулле Ариффу, Джонатану Мейеру, Гилу Гонсалвешу, Мэтью Чоплину, Бену Джадсону,

Джеймсу Грегори, Лукашу Леховичу, Клинтону Рою, Виторино Араужо, Сьюзан Гудбоди, Джошу Харвуду, Дэниелю Батлеру, Лю Хайбину, Джимми Дэвису, Игнасио Вергара Каузелу, Гайя Канестрани, Ренне Роша, Педро Аби, Ашии Завадук, Йостейну Лейре, Брэндону Родсу и многим другим. Приносим извинения, если пропустили кого-то в этом списке.

Супермегаспасибо нашему редактору Корбину Коллинзу за его добродушную критику и за то, что он был неустойчивым защитником читателя. Также выражаем наивысшую благодарность команде издательства Кэтрин Тозер, Шэрон Уилки, Эллен Траутман-Зайг и Ребекке Демарест за вашу преданность делу, профессионализм и внимание к деталям. Эта книга неизмеримо улучшилась благодаря вам.

Любые оставшиеся в книге ошибки на нашей совести.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Введение

Почему в проекте что-то идет не так?

О чем вы думаете, когда слышите слово «хаос»? Возможно, о шумной фондовой бирже или о своей кухне по утрам. При слове «*порядок*» представьте пустую комнату — такую, в которой безмятежно и спокойно. Однако для ученых хаос характеризуется однородностью (одинаковостью), а порядок — сложностью (различием).

Например, ухоженный сад — это очень упорядоченная система. Садоводы определяют границы дорожками и заборами, разбивают клумбы или грядки. Со временем сад разрастается, становясь все гуще и разнообразнее; но без ухода он будет дичать. Сорняки и трава начнут вытеснять другие растения до тех пор, пока в итоге все не вернется в исходное состояние — дикое и неуправляемое.

Программные системы также тяготеют к хаосу. Начиная создавать новую систему, мы преисполнены идеей сделать код чистым и упорядоченным, но со временем обнаруживаем, что он вбирает в себя всякий мусор и крайности. В конечном итоге такой код превращается в болото управленческих классов и модулей. Оказывается, что разумно организованная многослойная архитектура развалилась, как намокшее печенье. Функции хаотических программных систем схожи: обработчики API, которые отправляют электронную почту и выполняют логирование, обладая знаниями о предметной области; классы «бизнес-логики», которые ничего не вычисляют, но зато выполняют операции ввода-вывода, и все это связано со всем остальным так запутанно, что вносить изменения в любую часть системы становится опасно. Такая ситуация настолько распространена, что программисты придумали термин для обозначения хаоса: антипаттерн «большой комок грязи» (рис. В.1).



Большой комок грязи — это естественное состояние софта, точно так же, как сорняки и заросли — естественное состояние вашего сада. Предотвращение коллапса потребует усилий.

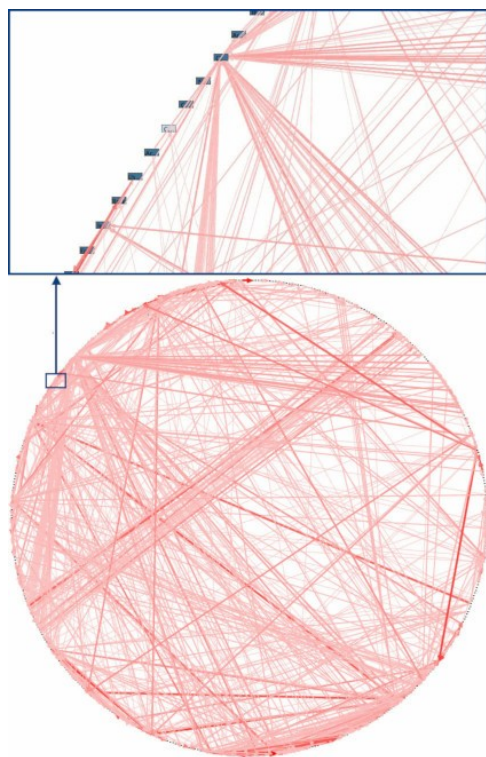


Рис. В.1. Диаграмма зависимостей в реальной жизни
(источник: «Корпоративная зависимость: большой комок грязи»¹ Алекса Пападимулиса)

К счастью, приемы, позволяющие избежать образования комка грязи, не так сложны.

Инкапсуляции и абстракции

Инкапсуляция и абстракция — это то, к чему интуитивно стремятся все программисты, даже если явно не говорят об этом. Давайте рассмотрим эти концепции подробнее, поскольку они проходят красной нитью через всю книгу.

Термин *инкапсуляция* охватывает две тесно связанные идеи: упрощение поведения и сокрытие данных. В нашем случае мы будем иметь в виду первое. Мы инкапсулируем поведение, определяя задачу, которую необ-

¹ См. Enterprise Dependency: Big Ball of Yarn, <https://oreil.ly/dbGTW>

ходимо выполнить в коде, и передавая эту задачу хорошо определенному объекту или функции. Мы называем этот объект или функцию *абстракцией*.

Взгляните на следующие два фрагмента кода:

Выполнить поиск с помощью urllib

```
import json
from urllib.request import urlopen
from urllib.parse import urlencode

params = dict(q='Sausages', format='json')
handle = urlopen('http://api.duckduckgo.com' + '?' + urlencode(params))
raw_text = handle.read().decode('utf8')
parsed = json.loads(raw_text)

results = parsed['RelatedTopics']
for r in results:
    if 'Text' in r:
        print(r['FirstURL'] + ' - ' + r['Text'])
```

Выполнить поиск с помощью requests

```
import requests

params = dict(q='Sausages', format='json')
parsed = requests.get('http://api.duckduckgo.com/', params=params).json()

results = parsed['RelatedTopics']
for r in results:
    if 'Text' in r:
        print(r['FirstURL'] + ' - ' + r['Text'])
```

Оба листинга делают одно и то же: отправляют значения, закодированные в форме, на URL-адрес, чтобы воспользоваться API поисковой машины. Но второй код воспринимается легче, потому что работает на более высоком уровне абстракции.

Мы можем пойти еще дальше, определив и назвав задачу, которую должен выполнять код. Чтобы сделать ее более явной, мы используем еще более высокоуровневую абстракцию.

Выполнить поиск с помощью модуля duckduckgo

```
import duckduckgo

for r in duckduckgo.query('Sausages').results:
    print(r.url + ' - ' + r.text)
```

Инкапсулирование поведения с помощью абстракций — это мощный инструмент для того, чтобы делать код выразительнее, проще в техническом сопровождении и удобнее в тестировании.



В литературе, посвященной объектно ориентированному (ОО) миру, одна из классических характеристик этого подхода называется *проектированием на основе обязанностей* (RDD-проектирование — Responsibility-Driven-Design)¹; в нем используются слова *роли* и *обязанности*, а не *задачи*. Главное — думать о коде в терминах поведения, а не данных или алгоритмов².

АБСТРАКЦИИ И АБСТРАКТНЫЕ БАЗОВЫЕ КЛАССЫ

В традиционном объектно ориентированном языке вроде Java или C# вы можете использовать абстрактный базовый класс (abstract base class, ABC) или интерфейс для определения абстракции. В Python вы можете (мы тоже иногда так делаем) использовать абстрактные базовые классы или опереться на утиную (неявную) типизацию.

Абстракция может просто означать «публичный API вещи, которую вы используете» — например, имя функции плюс несколько аргументов.

Большинство паттернов в этой книге предусматривают абстракцию, и поэтому вы увидите массу примеров в каждой главе. Кроме того, в главе 3 подробно рассматривается несколько общих эвристик для выбора абстракций.

Разделение на слои

Инкапсуляция и абстракция помогают скрывать детали и защищать согласованность данных, но мы также должны обращать внимание и на взаимодействие между объектами и функциями. Когда одна функция (модуль или объект) использует другую, мы говорим, что одна функция *зависит* от другой. Эти зависимости образуют своего рода сеть, или граф.

В большом комке грязи зависимости выходят из-под контроля (как вы видели на рис. В.1). Внося изменения в один узел графа, мы рискуем повлиять

¹ См. <http://www.wirfs-brock.com/Design.html>

² Если вы сталкивались с CRC-картами (class-responsibility-collaborator), то замечали, что все они ведут к одному и тому же: размышляя об *обязанностях*, вы тем самым помогаете себе принимать правильные решения о том, как разделять вещи.

на многие другие части системы. Многослойные архитектуры — это один из способов решения указанной проблемы. В многослойной архитектуре мы разделяем код на отдельные категории или роли и вводим правила, указывающие, какие категории кода могут вызывать друг друга.

Одним из наиболее распространенных примеров является *трехслойная архитектура*, показанная на рис. В.2.

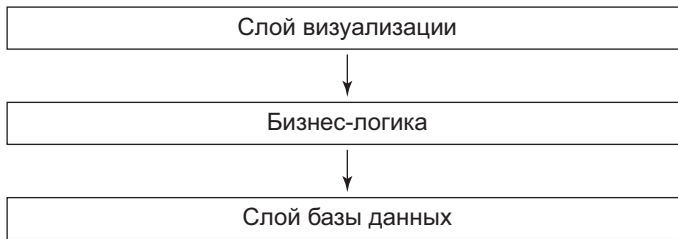


Рис. В.2. Многослойная архитектура

Многослойная архитектура является, пожалуй, наиболее распространенным паттерном для создания корпоративного ПО. В этой модели у нас есть компоненты пользовательского интерфейса, которые могут быть веб-страницей, API или командной строкой; эти компоненты взаимодействуют со слоем бизнес-логики, который содержит бизнес-правила и рабочие потоки; и, наконец, у нас есть слой, который отвечает за хранение и извлечение данных.

В оставшейся части этой книги мы будем систематически выворачивать эту модель наизнанку, следуя одному простому принципу.

Принцип инверсии зависимостей

Возможно, вы уже знакомы с *принципом инверсии зависимостей* (dependency inversion principle, DIP), потому что это буква D из аббревиатуры SOLID¹.

¹ SOLID — это аббревиатура пяти принципов объектно ориентированного дизайна Роберта К. Мартина: одна-единственная обязанность, открытость для расширения, но закрытость для модификации, принцип подстановки Барбары Лисков, принцип разделения интерфейса и инверсия зависимостей. См. «S.O.L.I.D: первые 5 принципов объектно ориентированного дизайна» Сэмюэля Олорунтобы (S.O.L.I.D: The First 5 Principles of Object-Oriented Design, Samuel Oloruntoba, <https://oreil.ly/UFM7U>).

К сожалению, мы не можем проиллюстрировать DIP, используя три крошечных листинга, как это было сделано для инкапсуляции. Однако вся часть I — это, по сути, пошагово проработанный пример реализации указанного принципа во всем приложении, так что в итоге вы получите полные и конкретные примеры.

А пока дадим формальное определение принципа инверсии зависимостей.

1. Высокоуровневые модули не должны зависеть от низкоуровневых. И то и другое должно зависеть от абстракций.
2. Абстракции не должны зависеть от деталей. Вместо этого детали должны зависеть от абстракций.

Но что это значит? Давайте разберемся.

Высокоуровневые модули — это код, который в действительности интересуется вашей организацией. Возможно, вы работаете в фармацевтической компании и ваши высокоуровневые модули работают с пациентами и испытаниями. Если вы работаете в банке, то ваши высокоуровневые модули управляют сделками и биржами. Высокоуровневые модули программной системы — это функции, классы и пакеты, которые работают с объективно существующими концепциями.

В отличие от них, *низкоуровневые модули* — это код, который не интересуется вашей организацией. Вряд ли ваш отдел кадров будет в восторге от файловых систем или сетевых сокетов. Вам нечасто придется обсуждать SMTP, HTTP или протокол AMQP с вашим финансовым отделом. Для стейкхолдеров-нетехнарей предприятия эти низкоуровневые концепции неинтересны и не относятся к делу. Их интересует только одно: правильно или нет работают высокоуровневые концепции. Если расчет заработной платы выполняется вовремя, то ваш бизнес вряд ли будет заботиться о том, что за этим стоит — планировщик стоп или функция, работающая на Kubernetes.

«Зависит от» не обязательно означает «импортирует» или «вызывает», а скорее выражает более общую идею о том, что один модуль *знает* о другом модуле или *нуждается* в нем.

И наконец, *абстракции*, которые мы уже упоминали: это упрощенные интерфейсы, которые инкапсулируют поведение, подобно тому как модуль duckduckgo инкапсулировал API поисковой машины.

Все проблемы в информатике можно решить, добавив еще один уровень абстракции.

Дэвид Уилер

Итак, первая часть принципа инверсии зависимостей говорит о том, что бизнес-код не должен зависеть от технических деталей; вместо этого и тот и другие должны использовать абстракции.

Почему? В широком смысле потому, что мы хотим иметь возможность изменять их независимо друг от друга. Вносить изменения в высокоуровневые модули в соответствии с потребностями бизнеса должно быть легко. Вносить изменения в низкоуровневые модули (детали) часто на практике сложнее: представьте, что вы перерабатываете код с целью изменить имя функции в противовес определению миграции, тестированию и развертыванию базы данных с целью изменить имя столбца. Мы не хотим, чтобы внесение изменений в бизнес-логику замедлялось, потому что они тесно связаны с деталями низкоуровневой инфраструктуры. Но точно так же важно иметь возможность при надобности вносить изменения в инфраструктуру низкого уровня (например, подумайте о шардировании базы данных) без необходимости изменять бизнес-слой. Добавление между ними абстракции (знаменитого дополнительного слоя косвенности) позволяет им изменяться (более) независимо друг от друга.

Вторая часть является еще более загадочной. Утверждение «абстракции не должны зависеть от деталей» кажется понятным, но вот «детали должны зависеть от абстракций» трудно себе представить. Каким образом можно иметь абстракцию, не зависящую от деталей, которые она абстрагирует? К тому времени, как мы перейдем к главе 4, у нас будет конкретный пример, который должен все это немного прояснить.

Место для всей бизнес-логики: модель предметной области

Но прежде, чем мы сможем вывернуть трехслойную архитектуру наизнанку, нужно еще поговорить о среднем слое: о высокоуровневых модулях, или бизнес-логике. Одна из наиболее распространенных причин, по которым проектирование заходит не туда, заключается в том, что бизнес-логика

распространяется по всем слоям приложения, что затрудняет ее идентификацию, понимание и изменение.

В главе 1 показано, как создать бизнес-слой с паттерном «Модель предметной области». Остальные паттерны в части I показывают то, как благодаря правильному выбору абстракций и непрерывному применению принципа инверсии зависимостей можно поддерживать легко изменяемую и свободную от низкоуровневых зависимостей модель предметной области.

ЧАСТЬ I

Создание архитектуры для поддержки моделирования предметной области

Большинство разработчиков никогда не видело модель предметной области, только модель данных.

Сирил Мартрайр, DDD EU 2017

Большинство разработчиков, с которыми мы говорили об архитектуре, думают, что все могло бы быть лучше. Они часто хотят спасти систему, которая каким-то образом запуталась, и пытаются придать некую структуру комку грязи. Они знают, что их бизнес-логика не должна распространяться повсюду, но понятия не имеют, как это исправить.

Мы обнаружили, что многие разработчики при создании новой системы сразу же начинают создавать схему базы данных, а идея создать объектную модель приходит, как правило, с запозданием. Вот тут-то все и начинает разваливаться. Вместо этого на первом месте должно быть *поведение, которое управляет требованиями к хранилищу*. В конце концов, клиентов совсем не интересуют модели данных. Их интересует то, что система делает; в противном случае они просто использовали бы электронную таблицу.

В первой части книги мы рассмотрим вопрос построения насыщенной объектной модели с помощью разработки через тестирование (в главе 1), а затем покажем, как отвязать эту модель от технических обязанностей. Мы также расскажем, как написать изолированный от используемой системы постоянного хранения данных код (*persistence-ignorant code*) и как создавать стабильные API вокруг предметной области, тем самым обеспечивая возможность эффективно перерабатывать код. Для этого мы представим четыре ключевых паттерна:

- паттерн «Репозиторий» — абстракцию в области обеспечения постоянного хранения данных;
- паттерн «Сервисный слой» для четкого определения того, где начинаются и заканчиваются варианты использования;
- паттерн UoW для обеспечения атомарных операций;
- паттерн «Агрегат» для обеспечения целостности данных.

Если вы хотите получить представление о том, куда мы движемся, то взгляните на рис. I.1. Не волнуйтесь, если все это пока непонятно! Мы будем поочередно представлять каждый блок на приведенном ниже рисунке на протяжении всей первой части книги.

Мы также уделим немного времени разговору о связанности и абстракциях и дадим простой пример, который показывает, как и почему мы выбираем абстракции.

Три приложения в конце книги помогут лучше изучить часть I.

- Приложение Б — это описание инфраструктуры для кода примеров: как создавать и выполнять образы Docker, в которых мы управляем информацией о конфигурации, и как выполнять разные типы тестов.
- Приложение В — это своего рода «не попробуешь — не узнаешь», показывающее, как легко изъять всю инфраструктуру — API веб-фреймворка Flask, ORM и СУБД Postgres, — поменяв ее на совершенно другую модель ввода-вывода, включающую CLI и CSV.
- Наконец, приложение Г пригодится, если вам интересно знать о том, как эти паттерны могут выглядеть при использовании Django вместо Flask и объектно-реляционного отображения SQLAlchemy.

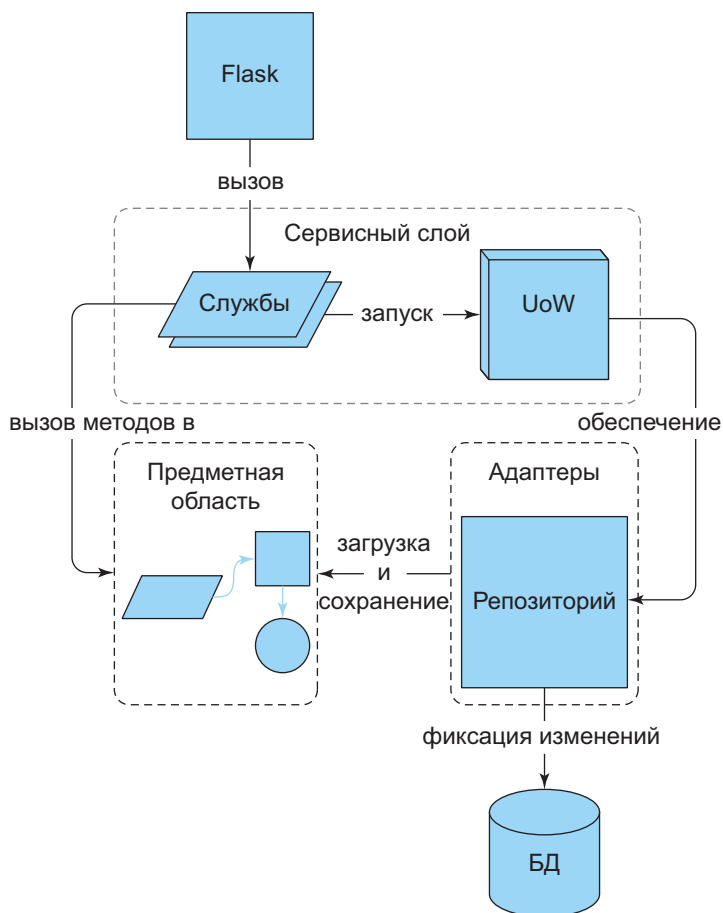


Рис. I.1. В конце первой части диаграмма компонентов приложения будет иметь такой вид

Моделирование предметной области

В этой главе рассматриваются возможности моделирования бизнес-процессов с помощью кода таким образом, чтобы он был полностью совместим с разработкой через тестирование (TDD). Мы обсудим причину, по которой моделирование предметной области так важно, и рассмотрим несколько ключевых паттернов для моделирования предметной области: «Сущность», «Объект-значение» и «Служба предметной области».

Рис. 1.1 представляет собой визуализацию паттерна «Модель предметной области». В этой главе мы добавим к ней несколько деталей, а в дальнейшем и вовсе окружим ее разными компонентами. Но вы всегда сможете разглядеть эту базовую схему.

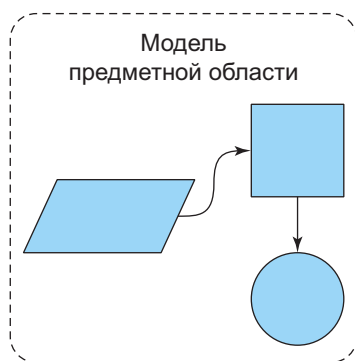


Рис. 1.1. Визуализация модели предметной области

Что такое модель предметной области

Во введении мы использовали термин «слой бизнес-логики» для описания центрального слоя трехслойной архитектуры. В оставшейся части книги

мы будем использовать термин «модель предметной области». Этот термин пришел из предметно-ориентированного проектирования, или DDD, и лучше всего объясняет смысл того, что мы хотим вам объяснить (более подробную информацию о DDD см. ниже, во врезке).

Предметная область (domain) — это своеобразный способ выразить задачу, которую вы пытаетесь решить. Например, авторы работают в мебельном интернет-магазине. В зависимости от того, о какой системе идет речь, предметная область может быть закупочной и снабженческой деятельностью, или дизайном продукта, или логистикой и доставкой. Большинство программистов тратят кучу времени, пытаясь улучшить или автоматизировать бизнес-процессы. Предметная область — это набор действий, которые эти процессы поддерживают.

Модель (model) — это карта процесса или явления, которая отражает полезное свойство. Люди исключительно хороши в создании ментальных моделей. Например, когда кто-то бросает вам мяч, вы бессознательно можете предсказать направление его движения, потому что у вас есть модель движения объектов в пространстве. Ваша модель ни в коем случае не является совершенной. У людей есть ужасные интуитивные представления о том, как объекты ведут себя на околосветовых скоростях или в вакууме, потому что модель вовсе не была предназначена для того, чтобы охватывать эти случаи. Это не означает, что модель неверна, просто некоторые предсказания выходят за пределы ее предметной области.

Модель предметной области — это мысленный образ бизнеса. У всех деловых людей есть такие образы — так люди думают о сложных процессах. Можно сразу понять, когда они оперируют этими образами, по тому, как используется деловая лексика. Жаргон естественным образом возникает среди людей, которые работают над сложными системами.

Представьте себе, что вы, наш несчастный читатель, внезапно перенеслись на много световых лет от Земли на борту космического корабля со своими друзьями и семьей и теперь должны разобраться, как добраться домой.

В первые несколько дней вы, возможно, будете просто нажимать кнопки наобум, но вскоре поймете, как работает каждая кнопка, и сможете давать друг другу инструкции. Сначала вы, возможно, будете говорить: «Нажми красную кнопку рядом с мигающей штуkenцией, а затем перебрось этот большой рычаг рядом с радарной фиговиной».

ЭТО КНИГА НЕ О ПРЕДМЕТНО-ОРИЕНТИРОВАННОМ ПРОЕКТИРОВАНИИ

Предметно-ориентированное проектирование популяризировало концепцию моделирования предметных областей¹ и было чрезвычайно успешным движением в сторону преобразования способов разработки ПО, фокусируя внимание разработчиков на ключевой сфере бизнеса. Многие паттерны проектирования, которые мы рассмотрим в этой книге, включая «Сущность», «Агрегат», «Объект-значение» (см. главу 7) и «Репозиторий» (в следующей главе), происходят из традиционного подхода к проектированию на основе предметной области.

В двух словах, DDD говорит о том, что самое важное в ПО — это то, что оно обеспечивает полезную модель задачи. Если мы правильно понимаем эту модель, то наше ПО приносит пользу и дает новые возможности.

Если мы понимаем модель неправильно, то она становится препятствием, которое приходится обходить. В этой книге мы можем показать основы проектирования модели предметной области и создания вокруг нее архитектуры, которая оставляет модель максимально свободной от внешних ограничений, благодаря чему ее легко развивать и изменять.

Однако в понятии «предметно-ориентированное проектирование» и в процессах, инструментах и технических приемах разработки модели предметной области есть гораздо больше, чем мы рассказываем в этой книге. Мы затронем тему лишь поверхностно. Для более глубокого изучения обратитесь к этим источникам.

- Оригинальная «синяя книга», «Предметно-ориентированное проектирование» Эрика Эванса (Domain-Driven Design, Eric Evans, Addison-Wesley Professional).
- «Красная книга» — «Реализация методов предметно-ориентированного проектирования» Вона Вернона (Implementing Domain-Driven Design, Vaughn Vernon, Addison-Wesley Professional).

Через пару недель вы станете лучше изъясняться, так как подобрали слова для описания функций корабля: «увеличить уровень кислорода в третьем грузовом отсеке» или «включить малые двигатели». Через несколько месяцев вы уже освоите лексику для описания сложных процессов: «начать посадку» или «подготовить варп-двигатель». Этот процесс будет про-

¹ Предметные области впервые появились не в предметно-ориентированном проектировании (DDD). Эрик Эванс ссылается на книгу 2002 года «Объектное проектирование» Ребекки Вирфсброк и Алана Маккина (Object Design, Rebecca WirfsBrock, Alan McKean, Addison-Wesley Professional). В ней был представлен дизайн на основе обязанностей, в котором DDD является частным случаем, связанным с предметными областями. Но даже эта книга была уже слишком запоздалой, и энтузиасты ОО предложат вам заглянуть в еще более далекое прошлое, обратившись к Ивару Якобсону и Грейди Бучу; этот термин существует с середины 1980-х годов.

исходить совершенно естественно, без каких-либо усилий по созданию общего глоссария.

Так и в мире бизнеса. Используемая в деловых кругах терминология представляет собой дистиллированное понимание модели предметной области, где сложные идеи и процессы сводятся к одному слову или фразе.

Когда мы слышим от стейкхолдеров незнакомые слова или термины, мы должны постараться извлечь смысл из сказанного и реализовать эти идеи в софте.

В этой книге мы будем использовать реальную модель предметной области, в частности из нашей настоящей работы. Компания MADE.com является успешным продавцом мебели. Мы поставляем мебель от производителей со всего мира и продаем ее по всей Европе.

Когда вы покупаете диван или журнальный столик, мы должны выяснить, как лучше всего доставить товар из Польши, Китая или Вьетнама в вашу гостиную.

На высоком уровне отдельные системы отвечают за приобретение товарных запасов, продажу и доставку клиентам имеющегося в наличии товара. Система в середине должна координировать процесс, распределяя товар по клиентским заказам (см. рис. 1.2).

Представим, что предприятие решает внедрить новый способ размещения заказов. До сих пор бизнес представлял товарные запасы и сроки выполнения заказов на основе того, что физически доступно на складе. Если и когда товар заканчивается, продукт помечается как «товара нет в наличии» до тех пор, пока не поступит следующая партия от производителя.

И вот в чем новшество: если у нас есть система, которая может отслеживать все морские поставки и время прибытия судов, то мы можем рассматривать товары на судах как реальный товарный запас, просто с чуть более длительным временем выполнения заказа. В наличии будет больше товаров, мы будем продавать больше, и бизнес может сэкономить деньги, храня меньше товаров на внутреннем складе.

Но размещение заказов больше не сводится лишь к уменьшению количества товаров в складской системе. Нам нужен более сложный механизм размещения. Самое время обратиться к моделированию предметной области.

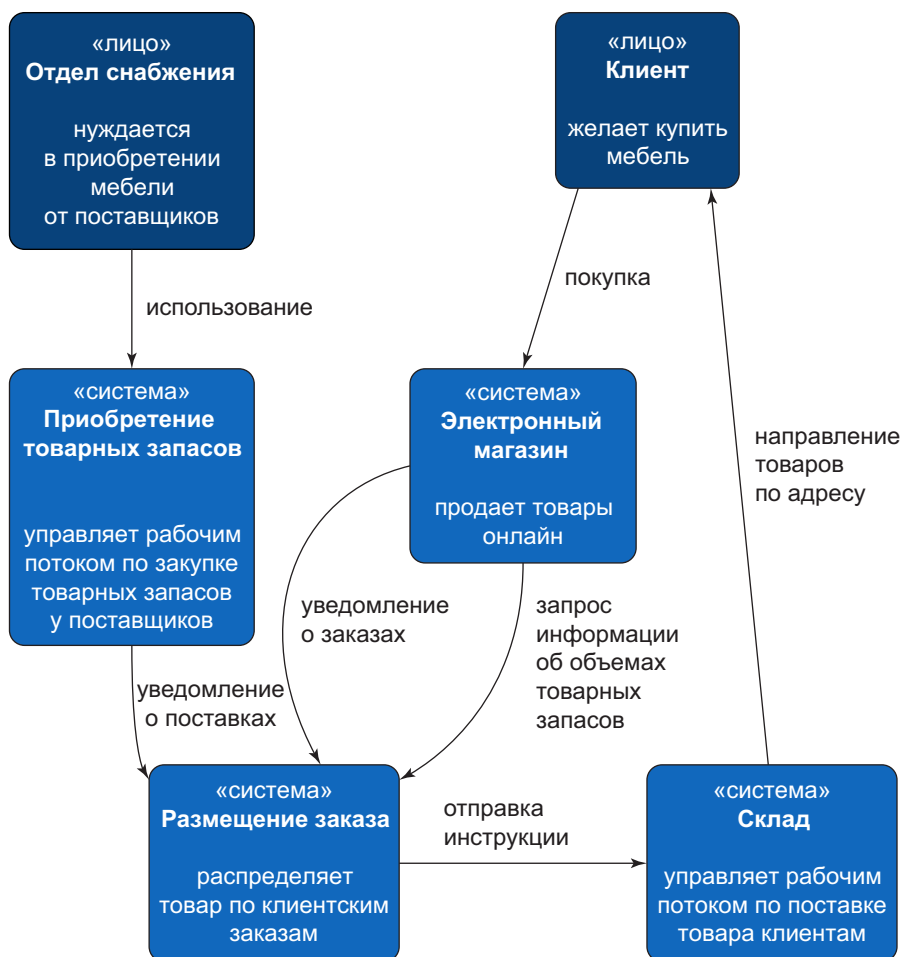


Рис. 1.2. Контекстная диаграмма для службы размещения заказов

Изучение языка предметной области

На то, чтобы разобраться в модели предметной области, уходит много времени, терпения и стикеров для заметок. Мы предварительно беседуем с бизнес-экспертами и согласовываем глоссарий и несколько правил для первой минимальной версии модели предметной области. По возможности мы просим привести конкретные примеры, иллюстрирующие каждое правило.

Мы обязательно выражаем эти правила на деловом жаргоне (*едином языке* в терминологии DDD). Выбираем запоминающиеся идентификаторы для объектов, чтобы было проще говорить о примерах.

Во врезке «Несколько слов о размещении заказов» показаны примеры, которые мы могли бы записать во время разговора с экспертами в нашей предметной области.

НЕСКОЛЬКО СЛОВ О РАЗМЕЩЕНИИ ЗАКАЗОВ

Продукт идентифицируется по артикулу, или *единице складского учета* (stock-keeping unit, SKU). *Клиенты* (customers) делают *заказы* (orders). Заказ идентифицируется *ссылкой на заказ* (order reference) и состоит из нескольких товарных позиций, или *строк* (order lines), где каждая товарная позиция имеет *артикул* и *количество* (quantity). Например:

- 10 шт. артикула СТУЛ-КРАСНЫЙ;
- 1 шт. артикула ЛАМПА-БЕЗВКУСНАЯ.

Отдел закупок заказывает малые *партии* (batches) товара. *Партия* товара имеет уникальный идентификатор, именуемый *ссылкой*, *артикулом* и *количеством*.

Нужно разместить *товарные позиции заказа* (allocate order lines) в *партиях* товара. После того как мы это сделали, мы отправляем товар из этой конкретной партии на клиентский адрес доставки. Когда мы размещаем x штук в партии товара, то его *располагаемое количество* (available quantity) уменьшается на x . Например:

- у нас есть партия из 20 шт. артикула СТОЛ-МАЛЫЙ, и мы размещаем в ней товарную позицию на 2 шт. артикула СТОЛ-МАЛЫЙ;
- в партии товара должно остаться 18 шт. артикула СТОЛ-МАЛЫЙ.

Мы не можем размещать товарные позиции заказа, если располагаемое количество меньше количества в товарной позиции. Например:

- у нас есть партия из 1 шт. артикула ПОДУШКА-СИНЯЯ и товарная позиция на 2 шт. артикула ПОДУШКА-СИНЯЯ;
- мы не можем разместить эту позицию в партии товара.

Мы не можем размещать одну и ту же товарную позицию заказа дважды. Например:

- у нас есть партия из 10 шт. артикула ВАЗА-СИНЯЯ, и мы размещаем товарную позицию на 2 шт. артикула ВАЗА-СИНЯЯ;
- если снова разместить эту позицию в той же партии товара, то указанная партия все равно должна иметь количество 8 шт.

Партии товара имеют *предполагаемый срок прибытия* (estimated arrival time, ETA), если они в настоящее время в пути, либо они могут быть *на складе* (warehouse stock). Складские партии имеют приоритет в размещении. Приоритет партий в пути зависит от их предполагаемого срока прибытия — чем раньше срок, тем более партия приоритетна к размещению.

Юнит-тестирование моделей предметных областей

В этой книге мы не собираемся показывать вам, как происходит разработка через тестирование, но хотим продемонстрировать, как мы выстроили бы модель из этого делового разговора.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Почему бы не попробовать решить эту задачу самостоятельно? Напишите несколько юнит-тестов, чтобы понять, сможете ли вы передать суть этих бизнес-правил в красивом, чистом коде.

Вы найдете несколько юнит-тестов-заглушек на GitHub¹, но можно просто начать с нуля или совместить/переписать их как вам нравится.

Вот как может выглядеть один из первых тестов:

Первый тест на размещение (test_batches.py)

```
def test_allocating_to_a_batch_reduces_the_available_quantity():
    batch = Batch("batch-001", "SMALL-TABLE", qty=20, eta=date.today())
    line = OrderLine('order-ref', "SMALL-TABLE", 2)

    batch.allocate(line)

    assert batch.available_quantity == 18
```

Название юнит-теста описывает поведение, которое мы хотим видеть в системе, а имена классов и переменных, которые мы используем, взяты из делового жаргона. Мы могли бы показать этот код нетехническим коллегам, и они согласились бы, что он правильно описывает поведение системы.

А вот и модель предметной области, отвечающая требованиям:

Первая примерка модели предметной области для партий товара (model.py)

```
@dataclass(frozen=True) ❶ ❷
class OrderLine:
    orderid: str
    sku: str
    qty: int
```

¹ См. https://github.com/cosmicpython/code/tree/chapter_01_domain_model_exercise

```
class Batch:
    def __init__(
        self, ref: str, sku: str, qty: int, eta: Optional[date] ❷
    ):
        self.reference = ref
        self.sku = sku
        self.eta = eta
        self.available_quantity = qty

    def allocate(self, line: OrderLine):
        self.available_quantity -= line.qty ❸
```

❶ `OrderLine` — это немутулируемый класс данных без какого-либо поведения¹.

❷ В большинстве листингов мы не показываем инструкции импорта, чтобы примеры выглядели чисто. Мы надеемся, вы догадаетесь, что это делается с помощью инструкции `from dataclasses import dataclass`; схожим образом это происходит и в случае `typing.Optional` и `datetime.date`. Если вы хотите что-то перепроверить, то можете свериться с полным рабочим кодом в ветке каждой главы (например, в `chapter_01_domain_model`).

❸ В мире Python аннотации типов по-прежнему остаются предметом споров. Относительно моделей предметных областей они иногда помогают прояснить или документально подтвердить ожидаемые аргументы, и разработчики с IDE нередко за них благодарны. Возможно, вы решите, что цена удобочитаемости слишком высока.

Здесь реализация тривиальна: класс `Batch` просто обертывает целочисленное количество `available_quantity` и мы уменьшаем это значение при размещении. Довольно много кода для простого вычитания одного числа из другого, но думаем, что подобное моделирование предметной области точно окупится².

Давайте напишем несколько новых неудачных тестов.

¹ В предыдущих версиях языка Python мы могли бы использовать именованный кортеж, `namedtuple`. Вы также можете попробовать отличный пакет `attrs` (<https://pypi.org/project/attrs>) Хайнека Шлавака.

² Или, может быть, вы думаете, что там недостаточно кода? Как насчет какой-нибудь проверки соответствия артикула (SKU) в товарной позиции `OrderLine` артикулу в партии товара `Batch.sku`? Мы высказали несколько мыслей по поводу проверки в приложении Д в конце книги.

Тестирование логики на предмет того, что можно разместить (test_batches.py)

```
def make_batch_and_line(sku, batch_qty, line_qty):
    return (
        Batch("batch-001", sku, batch_qty, eta=date.today()),
        OrderLine("order-123", sku, line_qty)
    )

def test_can_allocate_if_available_greater_than_required():
    large_batch, small_line = make_batch_and_line("ELEGANT-LAMP", 20, 2)
    assert large_batch.can_allocate(small_line)

def test_cannot_allocate_if_available_smaller_than_required():
    small_batch, large_line = make_batch_and_line("ELEGANT-LAMP", 2, 20)
    assert small_batch.can_allocate(large_line) is False

def test_can_allocate_if_available_equal_to_required():
    batch, line = make_batch_and_line("ELEGANT-LAMP", 2, 2)
    assert batch.can_allocate(line)

def test_cannot_allocate_if_skus_do_not_match():
    batch = Batch("batch-001", "UNCOMFORTABLE-CHAIR", 100, eta=None)
    different_sku_line = OrderLine("order-123", "EXPENSIVE-TOASTER", 10)
    assert batch.can_allocate(different_sku_line) is False
```

Тут нет ничего неожиданного. Мы сделали рефакторинг набора тестов, чтобы не дублировать строки кода для создания партии товара и товарной позиции заказа для одного и того же артикула, и написали четыре простых теста для нового метода `can_allocate`. Опять же обратите внимание, что используемые нами имена отражают язык экспертов в анализируемой сфере деятельности (предметной области), а согласованные с ними примеры записаны непосредственно в код.

Мы также можем реализовать это прямолинейно, написав метод `can_allocate` класса `Batch`.

Новый метод в модели (model.py)

```
def can_allocate(self, line: OrderLine) -> bool:
    return self.sku == line.sku and self.available_quantity >= line.qty
```

Пока что мы можем управлять реализацией, просто увеличивая и уменьшая количество `Batch.available_quantity`, но, как только мы перейдем к тестам `deallocate()`, то нам понадобится более изящное решение.

Для этого теста нужна более умная модель (test_batches.py)

```
def test_can_only_deallocate_allocated_lines():
    batch, unallocated_line = make_batch_and_line("DECORATIVE-TRINKET",
                                                    20, 2)
    batch.deallocate(unallocated_line)
    assert batch.available_quantity == 20
```

В этом тесте мы убеждаемся, что отмена размещения товарной позиции заказа в партии не имеет никакого эффекта в случае, если эта позиция не была ранее размещена в этой партии. Для этого класс `Batch` должен понимать, какие товарные позиции заказа были размещены, а какие — нет. Давайте посмотрим на реализацию.

Модель предметной области теперь отслеживает размещения (model.py)

```
class Batch:
    def __init__(
        self, ref: str, sku: str, qty: int, eta: Optional[date]
    ):
        self.reference = ref
        self.sku = sku
        self.eta = eta
        self._purchased_quantity = qty
        self._allocations = set() # тип 'множество': Set[OrderLine]

    def allocate(self, line: OrderLine):
        if self.can_allocate(line):
            self._allocations.add(line)

    def deallocate(self, line: OrderLine):
        if line in self._allocations:
            self._allocations.remove(line)

    @property
    def allocated_quantity(self) -> int:
        return sum(line.qty for line in self._allocations)

    @property
    def available_quantity(self) -> int:
        return self._purchased_quantity - self.allocated_quantity

    def can_allocate(self, line: OrderLine) -> bool:
        return self.sku == line.sku and self.available_quantity >=
            line.qty
```

На рис. 1.3 показана модель на языке UML.

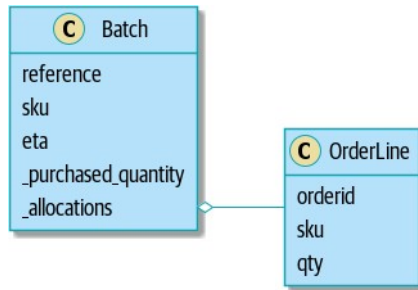


Рис. 1.3. Модель на UML

Мы кое-чего добились! Класс `Batch` теперь отслеживает множество объектов `OrderLine`, набор размещенных товарных позиций заказа. В случае если во время размещения у нас хватает товара, мы просто добавляем в структуру данных `set`. Количество `available_quantity` теперь является вычисляемым свойством: закупленное количество минус размещенное.

Разумеется, мы могли бы сделать гораздо больше. Немного сбивает с толку, что и `allocate()`, и `deallocate()` могут отказать без уведомления, но зато у нас есть основа.

Кстати, использование структуры данных `set` для размещений `_allocations` упрощает нам работу с последним тестом, поскольку элементы во множестве являются уникальными.

Последний тест партии товара! (`test_batches.py`)

```
def test_allocation_is_idempotent():
    batch, line = make_batch_and_line("ANGULAR-DESK", 20, 2)
    batch.allocate(line)
    batch.allocate(line)
    assert batch.available_quantity == 18
```

Справедливости ради сейчас модель предметной области слишком тривиальна, чтобы беспокоиться о DDD (или даже об объектном ориентировании!). В реальной жизни может возникнуть сколько угодно бизнес-правил и граничных случаев: клиенты могут запросить доставку в конкретные будущие даты, так что мы, возможно, не захотим их размещать в самой ранней партии. Некоторые артикулы не продаются партиями, а заказыва-

ются по требованию непосредственно у поставщиков, поэтому у них другая логика. В зависимости от местоположения клиента, мы можем размещать товар только в подмножестве складов и партий товара, которые находятся в его регионе, — за исключением некоторых артикулов, которые мы рады доставить с другого склада, если товара нет в наличии в домашнем регионе. И так далее. Реальный бизнес умеет городить сложности быстрее, чем мы можем показать здесь.

Но, взяв эту простую модель предметной области в качестве заглушки — замены чего-то более сложного, мы собираемся расширить ее в остальной части книги и подключить ее к реальному миру API-интерфейсов, баз данных и электронных таблиц. Мы увидим, как строгое следование нашим принципам инкапсуляции и тщательно продуманному разделению на слои поможет избежать комка грязи.

ДОПОЛНИТЕЛЬНЫЕ ТИПЫ ДЛЯ АННОТАЦИЙ ТИПОВ

Если вы и правда хотите оттянуться по полной с аннотациями типов, то можете зайти так далеко, что будете обертывать примитивные типы, используя `typing.NewType`.

Ты просто слишком далеко зашел, Боб

```
from dataclasses import dataclass
from typing import NewType

Quantity = NewType("Quantity", int)
Sku = NewType("Sku", str)
Reference = NewType("Reference", str)
...
```

```
class Batch:
    def __init__(self, ref: Reference, sku: Sku, qty: Quantity):
        self.sku = sku
        self.reference = ref
        self._purchased_quantity = qty
```

К примеру, это позволило бы проверке типов запретить передачу артикула `Sku`, когда ожидается ссылка `Reference`.

Независимо от того, считаете ли вы такой подход замечательным или ужасным, вопрос остается спорным¹.

¹ Это ужасно. Умоляю, не делайте этого. — *Гарри*.

Dataclasses (классы данных) отлично подходят для объектов-значений

В предыдущих листингах мы широко использовали товарную позицию заказа `line` буквально, но что такое товарная позиция? Выражаясь деловым языком, заказ состоит из нескольких товарных позиций, у каждой из которых есть артикул и количество. Скажем, простой файл YAML, содержащий информацию о заказе, мог бы выглядеть так:

Информация о заказе на YAML

```
Order_reference: 12345
```

```
Lines:
```

- sku: RED-CHAIR
 qty: 25
- sku: BLU-CHAIR
 qty: 25
- sku: GRN-CHAIR
 qty: 25

Обратите внимание, что, в отличие от заказа, у которого есть уникальная ссылка для идентификации, товарная позиция указана без такой ссылки. (Даже если добавить в класс `OrderLine` ссылку, она не будет однозначно идентифицировать саму товарную позицию.)

Всякий раз, когда у нас есть концепция, которая имеет данные, но не идентичность, мы часто выбираем для ее представления паттерн «Объект-значение». *Объект-значение* — это любой объект предметной области, который уникально идентифицируется содержащимися в нем данными; обычно мы делаем их немутуруемыми.

Товарная позиция заказа `OrderLine` — это объект-значение

```
@dataclass(frozen=True)
```

```
class OrderLine:
```

```
    orderid: OrderReference  
    sku: ProductReference  
    qty: Quantity
```

Одна из приятных вещей, которые дают нам `dataclasses` (или `namedtuples`), — это *эквивалентность значений*, причудливый способ сказать, что «две позиции заказа с одинаковыми `orderid`, `sku` и `qty` идентичны».

Еще примеры объектов-значений

```
from dataclasses import dataclass
from typing import NamedTuple
from collections import namedtuple

@dataclass(frozen=True)
class Name:
    first_name: str
    surname: str

class Money(NamedTuple):
    currency: str
    value: int

Line = namedtuple('Line', ['sku', 'qty'])

def test_equality():
    assert Money('gbp', 10) == Money('gbp', 10)
    assert Name('Harry', 'Percival') != Name('Bob', 'Gregory')
    assert Line('RED-CHAIR', 5) == Line('RED-CHAIR', 5)
```

Эти объекты-значения совпадают с нашими интуитивными представлениями о работе их значений. Совсем не важно, о *какой* банкноте в 10 фунтов мы говорим, потому что все они имеют одинаковый номинал. Схожим образом два полных имени эквивалентны, если совпадают имя и фамилия, и две товарных позиции эквивалентны, если они имеют один и тот же клиентский заказ, код продукта и количество. Вместе с тем объекту-значению по-прежнему можно задавать сложное поведение. На самом деле широко принято поддерживать операции со значениями, например математические операторы.

Вычисления с объектами-значениями

```
fiver = Money('gbp', 5)
tenner = Money('gbp', 10)

def can_add_money_values_for_the_same_currency():
    assert fiver + fiver == tenner

def can_subtract_money_values():
    assert tenner - fiver == fiver

def adding_different_currencies_fails():
    with pytest.raises(ValueError):
        Money('usd', 10) + Money('gbp', 10)
```

```
def can_multiply_money_by_a_number():
    assert fiver * 5 == Money('gbp', 25)

def multiplying_two_money_values_is_an_error():
    with pytest.raises(TypeError):
        tenner * fiver
```

Объекты-значения и сущности

Товарная позиция имеет уникальные идентификатор, артикул и количество; если мы изменим одно из этих значений, то получим новую товарную позицию. Все дело в определении объекта-значения: это любой объект, который идентифицируется только его данными и не имеет долговременной идентичности. Но как насчет партии? Она идентифицируется по ссылке.

Мы используем термин «сущность» для описания объекта предметной области, который имеет долговременную идентичность. На предыдущей странице мы представили класс `Name` как объект-значение. Если мы возьмем имя Гарри Персиваль и изменим одну букву, то получим новый объект класса `Name`, Барри Персиваль.

Очевидно, что Гарри Персиваль не эквивалентен Барри Персивалю.

Имя как таковое не может меняться...

```
def test_name_equality():
    assert Name("Harry", "Percival") != Name("Barry", "Percival")
```

Но как насчет Гарри как *личности*? Люди и правда меняют свои имена, семейное положение и даже пол, но мы продолжаем признавать в них одного и того же человека. И все потому, что люди, в отличие от имен, имеют постоянную *идентичность*.

Но человек может!

```
class Person:
    def __init__(self, name: Name):
        self.name = name

def test_barry_is_harry():
    harry = Person(Name("Harry", "Percival"))
    barry = harry

    barry.name = Name("Barry", "Percival")

    assert harry is barry and barry is harry
```

Сущности, в отличие от значений, обладают *эквивалентностью идентичности* (identity equality), или ее тождественностью. Мы можем изменить их значения, и они по-прежнему остаются узнаваемо теми же самими. Партии в нашем примере являются сущностями. Мы можем разместить товарные позиции заказа в партии или изменить дату, когда она должна прибыть, и это будет все та же самая сущность.

В коде обычно мы делаем это явным образом, реализуя для сущностей операторы эквивалентности.

Реализация операторов эквивалентности (model.py)

```
class Batch:
    ...

    def __eq__(self, other):
        if not isinstance(other, Batch):
            return False
        return other.reference == self.reference

    def __hash__(self):
        return hash(self.reference)
```

В Python магический метод `__eq__` определяет поведение класса для оператора `==`¹.

Рассуждая о сущностях и объектах-значениях, также стоит подумать о том, как будет работать метод `__hash__`. Этот магический метод используется в Python для управления поведением объектов, когда вы добавляете их в коллекции или используете в качестве ключей словаря dict; дополнительную информацию можно найти в документации по Python².

Для объектов-значений хеш должен основываться на всех атрибутах-значениях, и мы должны обеспечивать немутуируемость объектов. Это легко сделать, указав в классе данных `@frozen=True`.

Для сущностей самый легкий вариант — просто сказать, что хеш равен `None`, имея в виду, что объект не является хешируемым и не может, например, использоваться во встроенной в язык структуре данных `set` (коллекции).

¹ Метод `__eq__` произносится как «дандер-eq». По крайней мере некоторыми.

² См. <https://oreil.ly/YUzg5>

Если по какой-то причине вы решите, что и правда хотите использовать с сущностями операции над множествами и словарями, то есть над структурами данных `set` или `dict`, то хеш должен быть основан на атрибуте (-ах) вроде `.reference`, который определяет уникальную идентичность сущности. Также нужно попытаться каким-то образом сделать этот атрибут «только для чтения».



Это тонкий лед; не следует изменять `__hash__` без изменения `__eq__`. Если вы не уверены в том, что делаете, то рекомендуем изучить эту тему. Можно начать со статьи «Хеши и эквивалентность в Python»¹ нашего научного редактора Шлавака Хайнека.

Не все должно быть объектом: функция службы предметной области

Мы создали модель, которая представляет партии товара, но на самом деле нам нужно размещать товарные позиции заказа в конкретном наборе партий, образующих все товарные запасы.

Не все на свете сводится к вещам и предметам.

Эрик Эванс,

Предметно-ориентированное проектирование

Эванс обсуждает понятие операций службы предметной области, которые не входят в сущности или в объекты-значения². То, что размещает товарную позицию заказа при наличии набора партий, очень смахивает на функцию. Мы можем воспользоваться тем фактом, что Python является многопарадигмальным языком, и действительно написать такую функцию. Давайте посмотрим, как можно было бы протестировать ее.

¹ Python Hashes and Equality. См. по ссылке <https://oreil.ly/vxkgX>

² Службы предметной области — совсем не то же самое, что службы из служебного слоя, хотя зачастую они тесно связаны. Служба предметной области представляет бизнес-концепцию или процесс, тогда как служба из служебного слоя представляет способ использования вашего приложения. Часто служебный слой вызывает службу предметной области.

Тестирование службы предметной области (test_allocate.py)

```
def test_prefers_current_stock_batches_to_shipments():
    in_stock_batch = Batch("in-stock-batch", "RETRO-CLOCK", 100,
                           eta=None)
    shipment_batch = Batch("shipment-batch", "RETRO-CLOCK", 100,
                           eta=tomorrow)
    line = OrderLine("oref", "RETRO-CLOCK", 10)

    allocate(line, [in_stock_batch, shipment_batch])

    assert in_stock_batch.available_quantity == 90
    assert shipment_batch.available_quantity == 100

def test_prefers_earlier_batches():
    earliest = Batch("speedy-batch", "MINIMALIST-SPOON", 100,
                    eta=today)
    medium = Batch("normal-batch", "MINIMALIST-SPOON", 100,
                  eta=tomorrow)
    latest = Batch("slow-batch", "MINIMALIST-SPOON", 100, eta=later)
    line = OrderLine("order1", "MINIMALIST-SPOON", 10)

    allocate(line, [medium, earliest, latest])

    assert earliest.available_quantity == 90
    assert medium.available_quantity == 100
    assert latest.available_quantity == 100

def test_returns_allocated_batch_ref():
    in_stock_batch = Batch("in-stock-batch-ref", "HIGHBROW-POSTER",
                           100, eta=None)
    shipment_batch = Batch("shipment-batch-ref", "HIGHBROW-POSTER",
                           100, eta=tomorrow)
    line = OrderLine("oref", "HIGHBROW-POSTER", 10)
    allocation = allocate(line, [in_stock_batch, shipment_batch])
    assert allocation == in_stock_batch.reference
```

И служба могла бы выглядеть следующим образом:

Автономная функция для службы предметной области (model.py)

```
def allocate(line: OrderLine, batches: List[Batch]) -> str:
    batch = next(
        b for b in sorted(batches) if b.can_allocate(line)
    )
    batch.allocate(line)
    return batch.reference
```

Магические методы Python позволяют использовать модели вместе с идиомами Python

Вы можете по-разному относиться к использованию `next()` в предыдущем листинге, но наверняка согласитесь с тем, что возможность использовать `sorted()` в списке партий товара — это хорошая идиома Python.

Чтобы все сработало как надо, в модели предметной области мы реализуем `__gt__`.

Магические методы могут выражать семантику предметной области (`model.py`)

```
class Batch:
    ...

    def __gt__(self, other):
        if self.eta is None:
            return False
        if other.eta is None:
            return True
        return self.eta > other.eta
```

Замечательно.

Исключения тоже могут выражать понятия предметной области

Осталось разобраться с последней концепцией: исключения тоже могут использоваться для выражения понятий предметной области. В наших беседах с экспертами мы узнали, что заказ нельзя разместить, если товара *нет в наличии*, и мы можем охватить это, инициировав соответствующее *исключение из предметной области*.

Тестирование исключения «товар отсутствует на складе» (`test_allocate.py`)

```
def test_raises_out_of_stock_exception_if_cannot_allocate():
    batch = Batch('batch1', 'SMALL-FORK', 10, eta=today)
    allocate(OrderLine('order1', 'SMALL-FORK', 10), [batch])

    with pytest.raises(OutOfStock, match='SMALL-FORK'):
        allocate(OrderLine('order2', 'SMALL-FORK', 1), [batch])
```

Не будем слишком утомлять вас реализацией, а отметим главное — мы уделяем особое внимание тому, чтобы использовать тот же деловой язык для имен исключений, что и для сущностей, объектов-значений и служб.

ОБЗОР МОДЕЛИРОВАНИЯ ПРЕДМЕТНОЙ ОБЛАСТИ

Моделирование предметной области

Это та часть кода, которая наиболее близка к бизнесу, наиболее подвержена изменениям и представляет наибольшую ценность для бизнеса. Делайте ее легкой для восприятия и внесения изменений.

Отличие сущностей от объектов-значений

Объект-значение определяется его атрибутами. Обычно он лучше всего реализуется как немутуируемый тип. Если вы измените атрибут объекта-значения, то он будет представлять другой объект. Напротив, сущность имеет атрибуты, которые могут меняться со временем, и это все равно будет та же самая сущность. Важно определить то, что уникально идентифицирует сущность (обычно это какое-то имя или поле ссылки).

Не все должно быть объектом

Python — это многопарадигмальный язык, поэтому пусть «глаголы» в вашем коде будут функциями. Для каждого `FooManager`, `BarBuilder` или `BazFactory` есть более выразительная и читабельная `manage_foo()`, `build_bar()` или `get_baz()`.

Самое время применить свои лучшие принципы ОО дизайна

Пересмотрите пять принципов объектно ориентированного программирования (SOLID) и прочие хорошие эвристики, такие как «наличие противоположного is-a», «предпочтение композиции перед наследованием» и т. д.

Стоит также подумать о границах согласованности и агрегатах

Но это тема главы 7.

Вызов исключения домена (`model.py`)

```
class OutOfStock(Exception):
    pass

    def allocate(line: OrderLine, batches: List[Batch]) -> str:
        try:
            batch = next(
                ...
            )
        except StopIteration:
            raise OutOfStock(f'Артикула {line.sku} нет в наличии')
```

На рис. 1.4 показано, что у нас в итоге получилось.

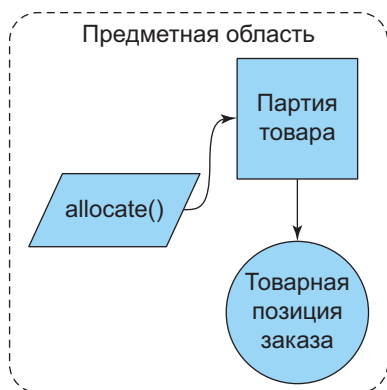


Рис. 1.4. Модель предметной области в конце главы

Пожалуй, пока хватит. У нас есть служба предметной области, которую мы можем задействовать для первого варианта использования. Но сначала понадобится база данных...

Паттерн «Репозиторий»

Пришло время выполнить обещание использовать принцип инверсии зависимостей как способ устранения связанности ключевой логики от инфраструктурных обязанностей.

Мы представим паттерн «Репозиторий», упрощающую абстракцию хранения данных, которая позволяет устранить связанность слоя модели и слоя данных. Приведем конкретный пример того, как эта упрощающая абстракция делает систему более легкой в тестировании, скрывая сложности базы данных.

На рис. 2.1 дан предварительный обзор того, что мы собираемся создать: объект `Repository`, который находится между моделью предметной области и базой данных.

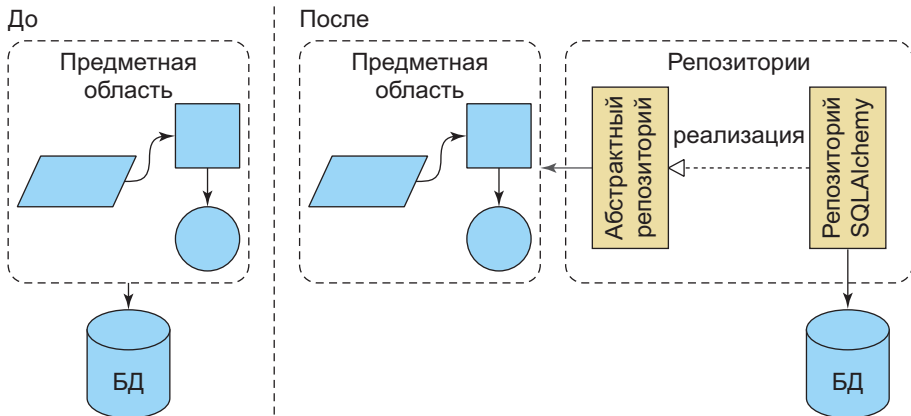


Рис. 2.1. Вид модели до и после применения паттерна «Репозиторий»



Код для этой главы находится в ветке `chapter_02_repository` на GitHub¹.

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_02_repository
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_01_domain_model
```

Организация постоянного хранения модели предметной области

В главе 1 мы создали простую модель предметной области, которая может размещать заказы в партиях товара. Легко писать тесты для этого кода, потому что не требуется никакой настройки зависимостей или инфраструктуры. Если бы нужно было управлять базой данных или обращаться к API и создавать тестовые данные, то писать и поддерживать тесты было бы труднее.

К сожалению, в какой-то момент придется отдать нашу крохотную совершенную модель в руки пользователей и бороться с реальным миром электронных таблиц, веб-браузеров и условий гонки. В последующих главах мы рассмотрим способы соединения идеализированной модели предметной области с внешним состоянием.

Мы собираемся работать по принципам Agile, поэтому приоритет состоит в том, чтобы как можно быстрее заполучить минимально жизнеспособный продукт. В нашем случае это будет веб-API. В реальном проекте вы, возможно, сразу же погрузитесь в работу с несколькими сквозными тестами и начнете подключать веб-фреймворк, тестируя элементы кода снаружи.

Но мы знаем — несмотря ни на что, нам понадобится какая-то форма постоянного хранения, а вы работаете с учебником, а не над проектом, поэтому мы можем позволить себе потратить чуть больше времени на разработку «снизу вверх» и начать думать о хранении и базах данных.

¹ См. <https://oreil.ly/6STDu>

Немного псевдокода: что нам потребуется?

Когда мы создаем первую конечную точку API, то знаем, что у нас будет некий код, который выглядит примерно так:

Как будет выглядеть первая конечная точка API

```
@flask.route.gubbins
def allocate_endpoint():
    # извлечь товарную позицию заказа из запроса
    line = OrderLine(request.params, ...)
    # загрузить все партии товара из БД
    batches = ...
    # вызвать профильную службу
    allocate(line, batches)
    # затем каким-то образом сохранить размещение обратно в БД
    return 201
```



Мы использовали веб-фреймворк Flask, потому что он легок в применении, но для того, чтобы понять эту книгу, вам не обязательно надо быть пользователем Flask. На самом деле мы вам покажем, как можно сделать выбор фреймворка не таким уж важным.

Нам понадобится способ, который позволяет извлекать информацию о партиях товара из базы данных и создавать из нее экземпляры объектов модели предметной области. А еще надо найти способ, который позволяет сохранять их обратно в базу данных.

Что-что? А-а, gubbins в инструкции @flask.route.gubbins — это британский вариант слова stuff («штуки», «фигня»). Не обращайтесь на это внимания. Это псевдокод.

Применение принципа инверсии зависимостей для доступа к данным

Как уже упоминалось во введении, подход на основе многослойной архитектуры широко распространен при структурировании системы, которая имеет пользовательский интерфейс, некоторую логику и базу данных (рис. 2.2).

Структура «модель — вид — шаблон» (Model-View-Template) фреймворка Django тесно связана, как и «модель — вид — контролер» (Model-View-Controller, MVC). В любом случае, цель состоит в том, чтобы держать слои разделенными (что хорошо) и чтобы каждый слой зависел только от расположенного ниже.

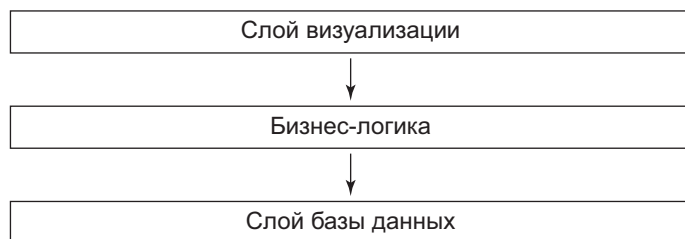


Рис. 2.2. Многослойная архитектура

Но мы хотим, чтобы модель предметной области не имела *никаких зависимостей*¹. Мы не хотим, чтобы инфраструктурные обязанности растекались по модели предметной области и замедляли юнит-тесты или внесение изменений.

Вместо этого, как обсуждалось во введении, мы будем считать, что модель находится «внутри» и зависимости втекают в нее; это то, что иногда называют *луковой архитектурой* (рис. 2.3).

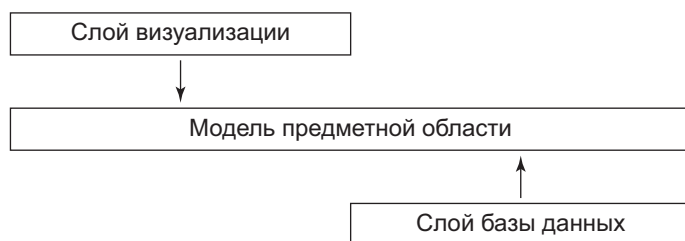


Рис. 2.3. Луковая архитектура

¹ Здесь я предполагаю, что нам не нужно «никаких зависимостей с поддержкой внутреннего состояния». Вполне нормально зависеть от вспомогательной библиотеки, а вот зависеть от ORM или веб-фреймворков — нет.

ЭТО ПОРТЫ И АДАПТЕРЫ?

Если вы читали о паттернах проектирования, то, возможно, задаете себе такие вопросы:

Это порты и адаптеры? Или это гексагональная архитектура? Разве это то же самое, что и луковая архитектура? А как быть с чистой архитектурой? Что такое порт и что такое адаптер? Народ, почему у вас так много слов для одного и того же?

Хотя некоторые любят придирается к различиям, все это в значительной степени означает одно и то же и сводится к принципу инверсии зависимостей: высокоуровневые модули (предметная область) не должны зависеть от низкоуровневых (инфраструктура)¹.

Далее поговорим о некоторых тонкостях, связанных с «зависимостью от абстракций», и о том, существует ли питоновский эквивалент интерфейсов. См. также раздел «Что такое порт и что такое адаптер в Python» на с. 70.

Напоминание: наша модель

Давайте вспомним, как выглядит наша модель предметной области (рис. 2.4): размещение — это концепция связывания позиции заказа (OrderLine) с партией товара (Batch). Мы храним размещения в виде коллекции на объекте Batch.

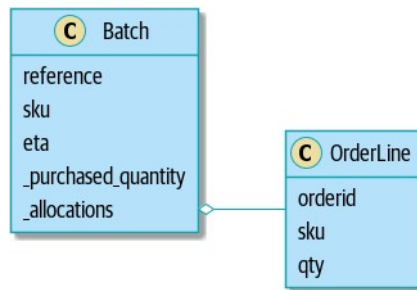


Рис. 2.4. Наша модель

Давайте посмотрим, как мы могли бы передать все это в реляционную базу данных.

¹ У Марка Симанна есть отличный пост на эту тему: <https://oreil.ly/LpFS9>.

«Нормальный» способ ORM: модель зависит от отображения

В наши дни команды разработчиков практически не выполняют свои SQL-запросы вручную. Для этого наверняка используется какой-либо фреймворк для генерирования SQL на основе объектов модели.

Эти фреймворки называются *объектно-реляционными отображениями* (object-relational mapper, ORM), поскольку они существуют для преодоления концептуального разрыва между миром объектов и моделирования предметной области и миром баз данных и реляционной алгебры.

Самая важная вещь, которую дает нам объектно-реляционное отображение, — это *неосведомленность об используемой системе постоянного хранения*: ее суть в том, что наша капризная модель предметной области не должна ничего знать о способах загрузки или хранения данных. Это помогает держать модель независимой от конкретных технологий баз данных¹.

Но если вы будете следовать типичному tutorialу по ORM SQLAlchemy, то в итоге получите что-то вроде этого:

«Декларативный» синтаксис SQLAlchemy, модель зависит от ORM (orm.py)

```
from sqlalchemy import Column, ForeignKey, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import relationship
```

```
Base = declarative_base()
```

```
class Order(Base):
    id = Column(Integer, primary_key=True)

    class OrderLine(Base):
        id = Column(Integer, primary_key=True)
        sku = Column(String(250))
        qty = Integer(String(250))
        order_id = Column(Integer, ForeignKey('order.id'))
        order = relationship(Order)
```

```
class Allocation(Base):
    ...
```

¹ В этом смысле использование ORM уже является примером принципа инверсии зависимостей. Вместо того чтобы зависеть от жестко запрограммированного SQL, мы зависим от абстракции, ORM. Но в данной книге нам и этого мало!

Вам вовсе не нужно быть экспертом в SQLAlchemy, чтобы увидеть, что первоначальная модель теперь полна зависимостей от ORM и, кроме того, начинает выглядеть адски скверно. Можем ли мы и вправду сказать, что эта модель не осведомлена о базе данных? Каким образом она может быть отделена от обязанностей по хранению данных, когда свойства модели напрямую связаны со столбцами БД?

ORM ФРЕЙМВОРКА DJANGO, ПО СУЩЕСТВУ, ТАКОЕ ЖЕ, НО ИМЕЕТ БОЛЕЕ ОГРАНИЧИТЕЛЬНЫЙ ХАРАКТЕР

Если вы привыкли к Django, то предыдущий «декларативный» фрагмент кода SQLAlchemy переводится примерно вот так:

Пример объектно-реляционного отображения в Django

```
class Order(models.Model):
    pass

class OrderLine(models.Model):
    sku = models.CharField(max_length=255)
    qty = models.IntegerField()
    order = models.ForeignKey(Order)

class Allocation(models.Model):
    ...
```

Суть в том же — классы модели наследуют непосредственно от классов ORM, поэтому модель зависит от него. Мы хотим, чтобы все было наоборот.

Django не предоставляет эквивалента классическому попарному ORM SQLAlchemy, но примеры применения инверсии зависимостей и паттерна «Репозиторий» к Django можно найти в приложении Г книги.

Инвертирование зависимости: ORM зависит от модели

К счастью, это не единственный способ использовать SQLAlchemy. Альтернативой является отдельное определение вашей схемы и определение явного попарного отображения, задачей которого будет конвертирование между схемой и моделью предметной области. В SQLAlchemy это называется классическим попарным отображением¹.

¹ См. <https://oreil.ly/ZucTG>

Явное ORM с участием объектов Table инструмента SQLAlchemy (orm.py)

```
from sqlalchemy.orm import mapper, relationship

import model ❶

metadata = MetaData()

order_lines = Table( ❷
    'order_lines', metadata,
    Column('id', Integer, primary_key=True, autoincrement=True),
    Column('sku', String(255)),
    Column('qty', Integer, nullable=False),
    Column('orderid', String(255)),
)

...

def start_mappers():
    lines_mapper = mapper(model.OrderLine, order_lines) ❸
```

❶ Объектно-реляционное отображение импортирует (или «зависит от», или «осведомлено о») модель предметной области, а не наоборот.

❷ Мы определяем таблицы и столбцы базы данных с помощью абстракций SQLAlchemy¹.

❸ Когда мы вызываем функцию `mapper`, SQLAlchemy творит свои загадочные манипуляции, чтобы привязать классы модели предметной области к различным таблицам, которые мы определили.

В итоге будет происходить следующее: если мы вызовем `start_mappers`, то сможем легко загружать и сохранять экземпляры модели предметной области из базы данных и в базу данных. Но если функция так и не будет вызвана, то классы модели предметной области остаются в блаженном неведении о базе данных.

Это дает нам все преимущества SQLAlchemy, включая возможность использовать `alembic` для миграции и делать запросы прозрачно с помощью классов предметной области, в чем мы позже убедимся.

¹ Даже в проектах, где мы не используем ORM, мы часто применяем SQLAlchemy наряду с легковесным инструментом миграции Alembic для декларативного создания схем в Python и управления миграциями, соединениями и сеансами.

Когда вы впервые пытаетесь создать конфигурацию ORM, порой неплохо писать для нее тесты, как в следующем примере:

Тестирование ORM напрямую (одноязычные тесты) (test_orm.py)

```
def test_orderline_mapper_can_load_lines(session): ❶
    session.execute(
        'INSERT INTO order_lines (orderid, sku, qty) VALUES '
        '("order1", "RED-CHAIR", 12),'
        '("order1", "RED-TABLE", 13),'
        '("order2", "BLUE-LIPSTICK", 14)'
    )
    expected = [
        model.OrderLine("order1", "RED-CHAIR", 12),
        model.OrderLine("order1", "RED-TABLE", 13),
        model.OrderLine("order2", "BLUE-LIPSTICK", 14),
    ]
    assert session.query(model.OrderLine).all() == expected

def test_orderline_mapper_can_save_lines(session):
    new_line = model.OrderLine("order1", "DECORATIVE-WIDGET", 12)
    session.add(new_line)
    session.commit()

    rows = list(session.execute('SELECT orderid, sku, qty FROM
                                "order_lines"'))
    assert rows == [("order1", "DECORATIVE-WIDGET", 12)]
```

❶ Если вы не знакомы с библиотекой `pytest`, то нам стоит объяснить использование аргумента `session` в этом тесте. Вообще-то для понимания этой книги вам не нужно разбираться в деталях `pytest` или ее фикстурах. Мы ограничимся лишь кратким объяснением. Вы можете определять общие зависимости для своих тестов как фикстуры (`fixture`) и `pytest` введет их в тесты, где они нужны, посмотрев на их функциональные аргументы. В данном случае это сеанс базы данных `SQLAlchemy`.

Скорее всего, эти тесты вам не пригодятся — как вы вскоре увидите, инвертировав зависимость от ORM и модели предметной области, останется сделать лишь крохотный дополнительный шаг для реализации еще одной абстракции — паттерна «Репозиторий», для которого будет легче писать тесты и который позже обеспечит простой шаблон интерфейса для тестирования.

Но мы уже достигли инверсии традиционной зависимости — модель предметной области остается «чистой» и свободной от инфраструктурных обязанностей. Мы можем отказаться от SQLAlchemy и использовать другое ORM или совершенно другую систему постоянного хранения, и модель предметной области вообще не нуждается в изменении.

В зависимости от того, что вы делаете в своей модели предметной области, и в особенности если вы отклоняетесь от объектно ориентированной парадигмы, вам будет труднее добиться нужного поведения ORM. Потребуется модифицировать свою модель предметной области¹. Как это часто бывает с архитектурными решениями, придется пойти на компромисс. Как гласит Дзен Python, «практичность важнее безупречности»!

Но на данный момент конечная точка API может выглядеть примерно так, и мы могли бы без проблем заставить ее работать:

Использование SQLAlchemy непосредственно в конечной точке API

```
@flask.route.gubbins
def allocate_endpoint():
    session = start_session()

    # извлечь товарную позицию заказа из запроса
    line = OrderLine(
        request.json['orderid'],
        request.json['sku'],
        request.json['qty'],
    )

    # загрузить все партии товара из БД
    batches = session.query(Batch).all()

    # вызвать службу предметной области
    allocate(line, batches)

    # сохранить размещение в базе данных
    session.commit()

    return 201
```

¹ Привет удивительно полезным специалистам по сопровождению SQLAlchemy и Майку Байеру в частности.

Введение паттерна «Репозиторий»

Паттерн «Репозиторий» — это абстракция поверх системы постоянного хранения. Он скрывает скучные детали доступа к данным, делая вид, что все данные находятся прямо в памяти.

Если бы в ноутбуках была бесконечная память, то в неуклюжих базах данных не было бы нужды. Можно было бы просто использовать объекты когда угодно. Как бы это выглядело?

Вы должны откуда-то получить свои данные

```
import all_my_data

def create_a_batch():
    batch = Batch(...)
    all_my_data.batches.add(batch)

def modify_a_batch(batch_id, new_quantity):
    batch = all_my_data.batches.get(batch_id)
    batch.change_initial_quantity(new_quantity)
```

Даже если объекты будут находиться прямо в памяти, мы должны их куда-то поместить, чтобы найти снова. Данные в памяти позволят добавлять новые объекты как список или множество. Поскольку объекты находятся в памяти, нам никогда не придется вызывать метод `.save()`; мы просто извлекаем объект, который нас интересует, и модифицируем его в памяти.

Хранилище в абстрактном виде

Простейший репозиторий имеет всего два метода: `add()`, чтобы поместить новый элемент в репозиторий, и `get()`, чтобы вернуть ранее добавленный элемент¹. Мы твердо придерживаемся использования этих методов для доступа к данным в модели и в слое служб предметной области. Это условие не позволяет нам прицепить модель предметной области к базе данных.

¹ Вы можете спросить: «А как насчет операций выведения списка (`list`), удаления (`delete`) или обновления (`update`)?» Однако в идеальном мире мы модифицируем модельные объекты по одному за раз, и удаление обычно обрабатывается как мягкое удаление — то есть `batch.cancel()`. Наконец, как вы увидите в главе 6, обновлением занимается паттерн UoW.

Вот как будет выглядеть абстрактный базовый класс (ABC) для репозитория:

Самый простой из возможных репозиториях (repository.py)

```
class AbstractRepository(abc.ABC):  
  
    @abc.abstractmethod ❶  
    def add(self, batch: model.Batch):  
        raise NotImplementedError ❷  
  
    @abc.abstractmethod  
    def get(self, reference) -> model.Batch:  
        raise NotImplementedError
```

❶ Совет: `@abc.abstractmethod` — это одна из немногих вещей, которая действительно заставляет абстрактные базовые классы «работать» в Python. Язык не позволит вам создавать экземпляр класса, который не реализует все абстрактные методы `abstractmethod`, заданные в его родительском классе¹.

❷ Можно использовать `raise NotImplementedError`, чтобы понять ошибку отсутствующей реализации, но это не обязательно и недостаточно. На самом деле ваши абстрактные методы могут иметь реальное поведение, которое подклассы могут вызывать, если вам это и вправду нужно.

АБСТРАКТНЫЕ БАЗОВЫЕ КЛАССЫ, УТИНАЯ ТИПИЗАЦИЯ И ПРОТОКОЛЫ

Мы используем абстрактные базовые классы в этой книге по дидактическим причинам: мы надеемся, что они помогут объяснить, что такое интерфейс абстракции репозитория.

В реальной жизни мы порой удаляем абстрактные базовые классы из производственного кода, потому что Python слишком легко их игнорирует и они в конечном счете остаются без поддержки, а в худшем случае вводят в заблуждение. На практике в целях заедирования абстракции мы часто просто опираемся на утиную типизацию Python. Для питониста репозиторий — это любой объект, имеющий методы `add(thing)` и `get(id)`.

В качестве альтернативы можно рассмотреть протоколы PEP 544². Благодаря им можно делать типизацию без возможности наследования, что особенно понравится поклонникам идеи «композиция лучше наследования».

¹ Для того чтобы по-настоящему извлечь пользу из абстрактных базовых классов (какими бы они ни были), используйте помощников, таких как `pylint` и `mypy`.

² См. <https://oreil.ly/q9EPC>

В чем компромисс?

Знаете, говорят, что экономисты знают цену всему и ценность ничего. Скажем так, программисты знают выгоду от всего и ценность компромиссов.

Рич Хикки

Всякий раз, когда в этой книге мы вводим паттерн, мы всегда спрашиваем: «А что мы за это получаем и чего это стоит?»

Как минимум мы вводим дополнительный слой абстракции. Хотя мы и надеемся, что это уменьшит сложность в целом, этот слой все же добавляет сложность локально и имеет свою цену с точки зрения количества движущихся частей, из которых состоит вся система¹, а также объема текущего технического сопровождения.

И все же паттерн «Репозиторий», вероятно, является одним из самых простых, если вы уже идете по пути DDD и инверсии зависимостей. Что касается кода, то на самом деле мы просто изымаем абстракцию SQLAlchemy (`session.query(Batch)`), меняя ее на другую разработанную нами абстракцию (`batches_repo.get`).

Нам придется писать несколько строк кода в классе репозитория всякий раз, когда мы добавляем новый объект модели предметной области, который хотим получить, но взамен получаем простую абстракцию над слоем хранения, который мы контролируем. Паттерн «Репозиторий» позволит легко вносить фундаментальные изменения в способ хранения (см. приложение В), и как мы увидим, его легко подделывать для юнит-тестов.

Кроме того, паттерн «Репозиторий» настолько распространен в мире предметно-ориентированного проектирования, что ваши коллеги, пришедшие в Python из мира Java и C#, скорее всего, его узнают. Он приведен на рис. 2.5.

Как всегда, начинаем с теста. Указанный тест, вероятно, был бы классифицирован как интеграционный, так как мы выполняем проверку на правильность интегрирования нашего кода (репозитория) с базой данных;

¹ В англоязычной литературе по объектно ориентированному программированию для обозначения частей, составляющих систему (работающие вместе серверы, конфигурации, фреймворки и т. д.), используется термин *moving parts*, по аналогии с деталями автомобильного двигателя. — *Примеч. ред.*

следовательно, тесты склонны смешивать сырой SQL с вызовами и подтверждениями истинности (инструкциями `assert`) в собственном коде.

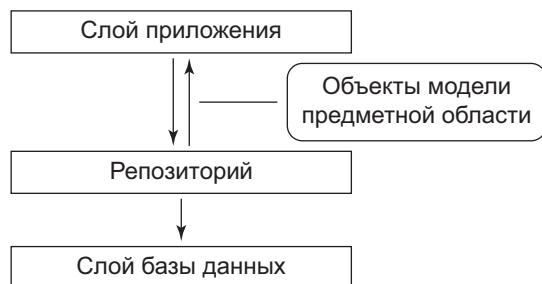


Рис. 2.5. Паттерн «Репозиторий»



В отличие от предыдущих тестов ORM, эти тесты лучше надолго оставить в вашей кодовой базе, особенно если из-за каких-либо частей вашей модели предметной области объектно-реляционное сопоставление будет нетривиальным.

Тест репозитория на сохранение объекта (`test_repository.py`)

```
def test_repository_can_save_a_batch(session):
    batch = model.Batch("batch1", "RUSTY-SOAPDISH", 100, eta=None)

    repo = repository.SqlAlchemyRepository(session)
    repo.add(batch) ❶
    session.commit() ❷

    rows = list(session.execute(
        'SELECT reference, sku, _purchased_quantity, eta FROM "batches"' ❸
    ))
    assert rows == [("batch1", "RUSTY-SOAPDISH", 100, None)]
```

❶ `repo.add()` — это тестируемый здесь метод.

❷ Мы держим `.commit()` за пределами репозитория и делаем операцию фиксации обязанностью вызывающей части кода. В этом есть свои плюсы и минусы; некоторые из наших доводов станут яснее в главе 6.

❸ Мы используем сырой SQL для проверки сохранения нужных данных.

Следующий тест предусматривает извлечение партий товара и размещений, поэтому он сложнее:

Тест репозитория на извлечение сложного объекта (test_repository.py)

```
def insert_order_line(session):
    session.execute( ❶
        'INSERT INTO order_lines (orderid, sku, qty)'
        ' VALUES ("order1", "GENERIC-SOFA", 12)'
    )
    [[orderid_id]] = session.execute(
        'SELECT id FROM order_lines WHERE orderid=:orderid AND'
        ' sku=:sku',
        dict(orderid="order1", sku="GENERIC-SOFA")
    )
    return orderline_id

def insert_batch(session, batch_id): ❷
    ...

def test_repository_can_retrieve_a_batch_with_allocations(session):
    orderline_id = insert_order_line(session)
    batch1_id = insert_batch(session, "batch1")
    insert_batch(session, "batch2")
    insert_allocation(session, orderline_id, batch1_id) ❸

    repo = repository.SqlAlchemyRepository(session)
    retrieved = repo.get("batch1")

    expected = model.Batch("batch1", "GENERIC-SOFA", 100, eta=None)
    assert retrieved == expected # Batch.__eq__ сравнивает только
                                # ссылку ❹
    assert retrieved.sku == expected.sku ❺
    assert retrieved._purchased_quantity == expected._purchased_quantity
    assert retrieved._allocations == { ❻
        model.OrderLine("order1", "GENERIC-SOFA", 12),
    }
}
```

❶ Здесь тестируется сторона чтения, поэтому сырой SQL готовит данные, которые затем будут прочитаны методом `repo.get()`.

❷ Избавим вас от подробностей методов `insert_batch` и `insert_allocation`; их суть в том, чтобы создать несколько партий товара и для интересующей нас партии выделить одну существующую строку заказа.

❸ И это то, что мы здесь проверяем. Первая инструкция, `assert ==`, проверяет, совпадают ли типы и является ли ссылка одинаковой (потому что, как вы помните, `Batch` — это сущность и у нас есть для нее собственный `eq`).

❹ Поэтому мы также явным образом проверяем его главные атрибуты, в том числе `._allocations`, который представляет собой питоновское множество объектов-значений `OrderLine`.

Независимо от того, старательно ли вы пишете тесты для каждой модели или нет, это остается на ваше усмотрение. После того как вы протестировали один класс на создание/модификацию/сохранение, вы с легким сердцем можете продолжить в том же духе и протестировать другие минимальным тестом «туда-обратно»¹ или даже вообще ничего не делать, если все они следуют аналогичной схеме. В нашем случае конфигурация ORM, которая настраивает множество `._allocations`, выглядит сложновато, поэтому заслуживает специального теста. В итоге получаем что-то вроде этого:

Типичный репозиторий (`repository.py`)

```
class SQLAlchemyRepository(AbstractRepository):
```

```
    def __init__(self, session):
        self.session = session

    def add(self, batch):
        self.session.add(batch)

    def get(self, reference):
        return self.session.query(model.Batch).filter_
            by(reference=reference).one()

    def list(self):
        return self.session.query(model.Batch).all()
```

И теперь конечная точка Flask, возможно, будет выглядеть примерно так:

Использование репозитория непосредственно в конечной точке API

```
@flask.route.gubbins
def allocate_endpoint():
    batches = SQLAlchemyRepository.list()
    lines = [
```

¹ Тест «туда-обратно» (round-trip test) — это тест, который взаимодействует только через «входную дверь» (публичный интерфейс) тестируемой системы. — *Примеч. пер.*

```
        OrderLine(1['orderid'], 1['sku'], 1['qty'])
        for l in request.params...
    ]
    allocate(lines, batches)
    session.commit()
    return 201
```

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

На днях мы столкнулись с другом на конференции по DDD. Он сказал, что не использовал ORM уже десять лет. Паттерн «Репозиторий» и ORM действуют как абстракции перед сырым SQL, поэтому применять один на фоне другого на самом деле совсем не обязательно. Почему бы не попробовать реализовать репозиторий без использования объектно-реляционного отображения? Код ищите на GitHub¹.

Мы оставили тесты репозитория, но выбор SQL зависит от вас. Может быть, это будет труднее, чем вы думаете, а может быть, и легче. Но самое приятное, что остальной части вашего приложения просто все равно.

Теперь поддельный репозиторий для тестов создается просто!

Вот одна из самых больших выгод от паттерна «Репозиторий».

Простой поддельный репозиторий с использованием множества set (repository.py)

```
class FakeRepository(AbstractRepository):

    def __init__(self, batches):
        self._batches = set(batches)

    def add(self, batch):
        self._batches.add(batch)

    def get(self, reference):
        return next(b for b in self._batches if b.reference == reference)

    def list(self):
        return list(self._batches)
```

Поскольку он представляет собой простую оболочку вокруг структуры данных set, все методы являются однострочными.

¹ См. https://github.com/cosmicpython/code/tree/chapter_02_repository_exercise

Использовать поддельный репозиторий в тестах и вправду несложно, и у нас есть простая абстракция, которую просто использовать и о которой легко рассуждать.

Пример использования поддельного репозитория (`test_api.py`)

```
fake_repo = FakeRepository([batch1, batch2, batch3])
```

Вы увидите эту подделку в действии в следующей главе.



Изготовление подделок для ваших абстракций — это отличный способ оценить дизайн: если делать подделки трудно, то абстракция, вероятно, является слишком сложной.

Что такое порт и что такое адаптер в Python

Не будем слишком подробно останавливаться на терминологии, потому что главное, на чем стоит заострить внимание, — это инверсия зависимостей, а вот специфика используемого вами технического приема не имеет большого значения. Кроме того, мы знаем, что разные люди используют разные определения.

Порты и адаптеры пришли из объектно ориентированного мира, и мы придерживаемся определения, которое состоит в том, что *порт* — это *интерфейс* между приложением и тем, что мы хотим абстрагировать, а *адаптер* — это *реализация*, стоящая за этим интерфейсом или абстракцией.

Так вот, Python не имеет интерфейсов как таковых, поэтому, хотя адаптер обычно легко идентифицировать, определить порт бывает сложнее. Если вы используете абстрактный базовый класс, то это порт. Если нет, то порт — это просто утиный тип, которому подчиняются ваши адаптеры и который ваше стержневое приложение ожидает — имена используемых функций и методов, а также имена и типы аргументов.

Если говорить конкретно, то в данной главе `AbstractRepository` — это порт, а `SqlAlchemyRepository` и `FakeRepository` — адаптеры.

Выводы

Держа в голове цитату Рича Хикки, в каждой главе мы резюмируем издержки и выгоды каждого вводимого нами паттерна. Поймите нас правильно: мы не говорим, что каждое отдельное приложение должно быть построено именно таким образом; сложность приложения и предметной области лишь иногда делает стоящим вложение времени и усилий в добавление этих новых слоев косвенности.

В табл. 2.1 приведены некоторые плюсы и минусы паттерна «Репозиторий» и модели, неосведомленной о системе постоянного хранения данных.

Таблица 2.1. Паттерн «Репозиторий» и неосведомленность о системе постоянного хранения: компромиссы

Плюсы	Минусы
• Есть простой интерфейс между системой постоянного хранения и моделью предметной области. • Легко изготовить поддельную версию репозитория для юнит-тестирования или изъять разные решения по хранению данных благодаря полному отделению модели от особенностей инфраструктуры. • Написание модели предметной области перед планированием обеспечения постоянства данных помогает сосредоточиться на текущей бизнес-задаче. Если мы когда-нибудь захотим радикально поменять подход, то сможем сделать это в модели, не беспокоясь о внешних ключах или миграциях вплоть до самого последнего момента. • Схема базы данных является очень простой, потому что у нас есть полный контроль над тем, как мы сопоставляем объекты с таблицами	• Объектно-реляционное отображение (ORM) частично устраняет связанность. Вполне возможно, что поменять внешние ключи будет непросто, но при этом будет довольно легко менять MySQL на Postgres и наоборот, если вам когда-нибудь это понадобится. • Поддержка объектно-реляционных сопоставлений вручную требует дополнительной работы и дополнительного кода. • Любой дополнительный слой косвенности всегда увеличивает издержки на техническое сопровождение и добавляет неизвестный фактор WTF для программистов на Python, которые никогда раньше не сталкивались с паттерном «Репозиторий»

На рис. 2.6 показан базисный тезис: да, для простых случаев работа над отцепленной моделью предметной области дается сложнее, чем простой паттерн «Объектно-реляционное отображение/активная запись»¹.

¹ Указанный график вдохновлен статьей Роба Вэнса «Глобальная сложность, локальная простота» (Global Complexity, Local Simplicity, см. <https://oreil.ly/fQXkP>) в его блоге.

Но чем сложнее предметная область, тем больше окупятся усилия по освобождению от инфраструктурных вопросов, ведь тогда становится проще вносить изменения.



Если ваше приложение представляет собой простую оболочку CRUD (create-read-update-delete) вокруг базы данных, то модель предметной области или репозиторий вам не нужны.

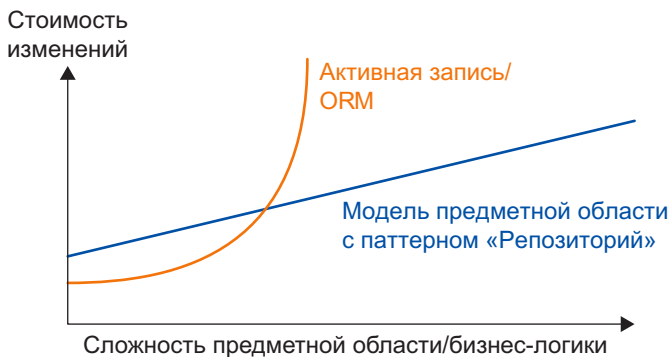


Рис. 2.6. График компромиссов модели предметной области

Приведенный пример кода недостаточно сложен, чтобы в полной мере показать, как выглядит правая сторона графика, но намеки есть. Представьте себе, например, что однажды мы решим вносить изменения размещения так, чтобы они располагались в объекте `OrderLine`, а не в объекте `Batch`: если бы мы использовали, скажем, Django, то пришлось бы определять и продумывать миграцию базы данных, прежде чем мы могли бы выполнять какие-либо тесты. Как бы то ни было, поскольку модель представлена обыкновенными объектами Python, мы можем изменить `set()`, сделав его новым атрибутом, и побеспокоиться о базе данных лишь в самом конце.

Вам интересно, как мы создаем экземпляры этих репозиториев, поддельных или настоящих? Как на самом деле будет выглядеть приложение Flask? Вы узнаете об этом в следующей захватывающей части, где рассказывается о паттерне «Слой служб».

Но сначала небольшое отступление.

О ПАТТЕРНЕ «РЕПОЗИТОРИЙ»

Применяйте инверсию зависимостей к ORM

Модель предметной области должна быть свободной от вопросов инфраструктуры, поэтому ORM должно импортировать вашу модель, а не наоборот.

Паттерн «Репозиторий» — это простая абстракция вокруг системы постоянного хранения данных

Репозиторий дает вам иллюзию коллекции объектов, находящихся в памяти. Это позволяет легко создавать поддельный репозиторий `FakeRepository` для тестирования и изымать фундаментальные детали вашей инфраструктуры, не нарушая работу вашего стержневого приложения. См. пример в приложении В в конце книги.

0 связанности и абстракциях

Давайте-ка, дорогой читатель, сделаем небольшое отступление от абстракций. О них уже было сказано немало. Например, паттерн «Репозиторий» — это абстракция поверх системы постоянного хранения данных. Но что делает абстракцию хорошей? Чего мы хотим от абстракций? И как они связаны с тестированием?



Код для этой главы находится в ветке `chapter_03_abstractions` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
git checkout chapter_03_abstractions
```

Ключевая тема этой книги заключается в том, что мы можем использовать простые абстракции, чтобы скрывать беспорядочные детали. Когда мы пишем код для развлечения или же на ката по программированию², мы можем позволить себе свободно играть с идеями, дорабатывать их и потом делать активный рефакторинг. Но в крупномасштабной системе мы становимся заложниками решений, принятых в других частях системы.

Когда мы не можем изменить компонент А из-за боязни нарушить компонент Б, мы говорим, что компоненты стали *связанными* (coupled). В локальном плане связанность — это хорошая вещь, являющаяся признаком того, что в коде все работает взаимосвязано, каждый компонент поддерживает другие, все они подходят друг к другу, как шестеренки часов. На жаргоне мы

¹ См. <https://oreil.ly/k6MmV>

² Ката по программированию — небольшой конкурс по программированию, часто используемый для того, чтобы попрактиковаться в разработке через тестирование (TDD). См. статью Питера Провоста «Kata — это единственный способ выучить TDD» (Kata — The Only Way to Learn TDD, <https://oreil.ly/vhiju>).

говорим, что код работает, когда существует высокое *сцепление* (cohesion) между связанными элементами.

В глобальном плане связанность — это неприятность: она увеличивает риск и «стоимость» изменения исходного кода, иногда до такой степени, что мы чувствуем себя вообще неспособными внести какие-либо изменения. Вот в чем проблема с антипаттерном «большой комок грязи»: если мы по мере роста приложения не сможем предотвратить связанность между элементами, которые не имеют сцепления, то эта связанность будет увеличиваться сверхлинейно до тех пор, пока мы больше не сможем эффективно вносить изменения в систему.

Мы можем уменьшить степень связанности внутри системы (рис. 3.1), абстрагируясь от деталей (рис. 3.2).

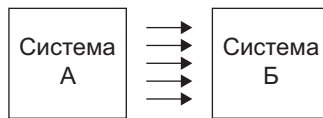


Рис. 3.1. Очень большая связанность

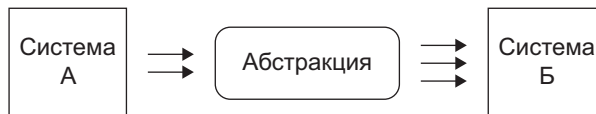


Рис. 3.2. Меньше связанности

На обоих рисунках есть пара подсистем, причем одна из них зависит от другой. На рис. 3.1 показана высокая степень связанности между ними; число стрелок указывает на разные виды зависимостей между ними. Если нам нужно изменить систему Б, то с огромной вероятностью изменение затронет и систему А.

А вот на рис. 3.2 мы уменьшили степень связанности, вставив новую, более простую абстракцию. Поскольку система А проще, она имеет меньше видов зависимостей от абстракции. Абстракция служит для того, чтобы защитить нас от вносимых изменений, скрывая сложные детали работы системы Б, — мы можем изменить стрелки справа, не меняя стрелки слева.

Абстрагирование состояния способствует тестопригодности

Рассмотрим пример. Представим, что мы хотим написать код для синхронизации двух файловых каталогов, которые мы будем называть *источником* (source) и *местом назначения* (destination):

- если файл существует в источнике, но отсутствует в месте назначения, скопируйте его;
- если файл существует в источнике, но имеет другое имя, чем в месте назначения, переименуйте файл назначения в соответствующий;
- если файл существует в месте назначения, но не существует в источнике, удалите его.

Первое и третье требования довольно просты: всего-то сравнить два списка путей. Но вот со вторым дело обстоит посложнее. Для того чтобы обнаружить переименования, мы должны проверить содержимое файлов. Для этого мы можем использовать функцию хеширования, такую как MD5 или SHA-1. Код для генерирования хеша SHA-1 из файла относительно прост:

Хеширование файла (sync.py)

```
BLOCKSIZE = 65536
```

```
def hash_file(path):  
    hasher = hashlib.sha1()  
    with path.open("rb") as file:  
        buf = file.read(BLOCKSIZE)  
        while buf:  
            hasher.update(buf)  
            buf = file.read(BLOCKSIZE)  
    return hasher.hexdigest()
```

Теперь нам нужно написать часть, которая принимает решения о том, что делать, — бизнес-логику, если угодно.

Когда нам приходится решать задачу начиная с первых принципов, мы обычно пытаемся написать простую реализацию, а затем сделать рефакторинг кода в сторону более качественного дизайна. Мы будем использовать этот подход на протяжении всей книги, потому что именно так мы пишем код в реальных условиях: начинаем с решения крохотной

части задачи, а затем итеративно делаем решение насыщеннее и конструктивно изощреннее.

Первый подход выглядит примерно так:

Базовый алгоритм синхронизации (sync.py)

```
import hashlib
import os
import shutil
from pathlib import Path

def sync(source, dest):
    # Обойти исходную папку и создать словарь имен и их хешей

    source_hashes = {}
    for folder, _, files in os.walk(source):
        for fn in files:
            source_hashes[hash_file(Path(folder) / fn)] = fn

    seen = set() # отслеживать файлы, найденные в целевой папке

    # Обойти целевую папку и получить имена файлов и хеши
    for folder, _, files in os.walk(dest):
        for fn in files:
            dest_path = Path(folder) / fn
            dest_hash = hash_file(dest_path)
            seen.add(dest_hash)

            # если в целевой папке есть файл, которого нет
            # в источнике, то удалить его
            if dest_hash not in source_hashes:
                dest_path.remove()

            # если в целевой папке есть файл, который имеет другой
            # путь в источнике, то переместить его в правильный путь
            elif dest_hash in source_hashes and fn != source_hashes[dest_hash]:
                shutil.move(dest_path, Path(folder) / source_hashes[dest_hash])

    # каждый файл, который появляется в источнике, но не в месте
    # назначения, скопировать в целевую папку
    for src_hash, fn in source_hashes.items():
        if src_hash not in seen:
            shutil.copy(Path(source) / fn, Path(dest) / fn)
```

Фантастика! У нас появилось немного кода, и он *выглядит* нормально, но прежде, чем мы выполним его, возможно, следует его протестировать. И как же такое тестировать?

Немного сквозных тестов (test_sync.py)

```
def test_when_a_file_exists_in_the_source_but_not_the_destination():
    try:
        source = tempfile.mkdtemp()
        dest = tempfile.mkdtemp()

        content = "Я — очень полезный файл"
        (Path(source) / 'my-file').write_text(content)

        sync(source, dest)

        expected_path = Path(dest) / 'my-file'
        assert expected_path.exists()
        assert expected_path.read_text() == content

    finally:
        shutil.rmtree(source)
        shutil.rmtree(dest)

def test_when_a_file_has_been_renamed_in_the_source():
    try:
        source = tempfile.mkdtemp()
        dest = tempfile.mkdtemp()

        content = "Я — файл, который переименовали"
        source_path = Path(source) / 'source-filename'
        old_dest_path = Path(dest) / 'dest-filename'
        expected_dest_path = Path(dest) / 'source-filename'
        source_path.write_text(content)
        old_dest_path.write_text(content)

        sync(source, dest)

        assert old_dest_path.exists() is False
        assert expected_dest_path.read_text() == content

    finally:
        shutil.rmtree(source)
        shutil.rmtree(dest)
```

Вот это да! Да тут уйма настроек для двух простых случаев! Проблема в том, что предметная логика — «выяснить разницу между двумя каталога-

ми» — тесно связана с кодом ввода-вывода данных. Мы не можем запустить разношерстный алгоритм без вызова модулей `pathlib`, `shutil` и `hashlib`.

И беда в том, что даже с учетом текущих требований мы не написали достаточно тестов: у нынешней реализации есть целый ряд проблем (к примеру, метод `shutil.move()` неправильный). Чтобы охватить все нюансы, придется написать еще больше тестов, но если они все будут такими же громоздкими, как в примере выше, то это вскоре станет сильно раздражать.

Кроме того, такой код сложно расширять. Представьте себе, что вы пытаетесь реализовать флаг `-dry-run`, который заставляет код просто вывести на экран то, что он собирается делать, не делая этого на самом деле. А что, если мы захотим синхронизироваться с удаленным сервером или облачным хранилищем?

Высокоуровневый код связан с низкоуровневыми деталями, что усложняет жизнь. По мере усложнения рассматриваемых сценариев тесты будут становиться все более громоздкими. Мы определенно можем их переработать (например, часть очистки может перейти в фикстуры `pytest`), но пока мы выполняем операции с файловой системой, они будут оставаться медленными и трудными для чтения и записи.

Выбор правильной(-ых) абстракции(-й)

Как можно переписать код, чтобы сделать его более тестопригодным?

Прежде всего, подумаем о том, что нужно коду от файловой системы. Внимательно просмотрев код, можно увидеть, что происходят три четко различимые вещи. Мы можем рассматривать их как три отдельные *обязанности* (responsibilities) кода:

1. Мы опрашиваем файловую систему с помощью `os.walk` и определяем хеши для серии путей. Этот код похож в случае и исходного каталога, и каталога места назначения.
2. Мы определяем, является ли файл новым, переименованным или лишним.
3. Мы копируем, перемещаем или удаляем файлы в соответствии с исходным каталогом.

Напомним, что мы хотим найти *упрощающие абстракции* для каждой из этих обязанностей. Это позволит скрыть беспорядочные детали, чтобы можно было сконцентрироваться на интересующей нас логике¹.



В этой главе мы сделаем рефакторинг корявого кода и превратим его в более тестопригодную структуру, выявляя отдельные части работы, которую нужно сделать, и предоставляя каждую часть четко определенному исполнителю, как в примере с duckduckgo.

Для шагов 1 и 2 мы уже интуитивно начали использовать абстракцию — словарь хешей для путей. Возможно, вы уже подумали: почему бы не создать словарь для папки места назначения, а также для исходной, а затем просто сравнить два словаря? Этот способ кажется неплохим, чтобы абстрагировать текущее состояние файловой системы:

```
source_files = {'hash1': 'path1', 'hash2': 'path2'}  
dest_files = {'hash1': 'path1', 'hash2': 'pathX'}
```

А как насчет перехода от шага 2 к шагу 3? Как абстрагироваться от фактического взаимодействия, связанного с перемещением/копированием/удалением в файловой системе?

Применим здесь трюк, который позже масштабируем. Мы собираемся отделить то, *что* хотим сделать, от того, *как* это сделать. Мы собираемся сделать так, чтобы программа выводила список команд, которые выглядят следующим образом:

```
("COPY", "sourcepath", "destpath"),  
("MOVE", "old", "new"),
```

Теперь мы могли бы написать тесты, которые используют два словаря файловой системы в качестве ввода, и в выводе мы можем получить списки кортежей со строковыми значениями, представляющими действия.

Вместо того чтобы говорить: «Если у нас есть эта реальная файловая система, то, когда мы выполняем свою функцию, мы проверяем произошедшие действия», мы говорим: «Если у нас есть эта абстракция файловой системы, то какая абстракция действий файловой системы произойдет?»

¹ Если вы привыкли мыслить в терминах интерфейсов, то это как раз то, что мы пытаемся здесь определить.

Упрощенные операции ввода и вывода в тестах (test_sync.py)

```
def test_when_a_file_exists_in_the_source_but_not_the_destination():
    src_hashes = {'hash1': 'fn1'}
    dst_hashes = {}
    expected_actions = [('COPY', '/src/fn1', '/dst/fn1')]
    ...

def test_when_a_file_has_been_renamed_in_the_source():
    src_hashes = {'hash1': 'fn1'}
    dst_hashes = {'hash1': 'fn2'}
    expected_actions == [('MOVE', '/dst/fn2', '/dst/fn1')]
    ...
```

Реализация выбранных абстракций

Все это очень хорошо, но как на самом деле писать эти новые тесты и как изменить реализацию так, чтобы все заработало?

Наша цель состоит в том, чтобы изолировать хитроумную часть системы и иметь возможность тщательно протестировать ее без необходимости организовывать реальную файловую систему. Мы создадим «ядро» кода, которое не зависит от внешнего состояния, а затем посмотрим, как оно реагирует, когда мы даем ему входные данные извне (такой подход был охарактеризован Гэри Бернхардтом как «функциональное ядро — императивная оболочка» (Functional Core, Imperative Shell, FCIS)¹.

Давайте начнем с разделения кода, чтобы отделить части с внутренним состоянием от логики. Высокоуровневая функция не будет содержать почти никакой логики вообще; это просто императивная серия шагов: собрать входные данные, вызвать логику, применить результаты.

Разделяем код на три части (sync.py)

```
def sync(source, dest):
    # шаг 1 с императивным ядром: собрать входные данные
    source_hashes = read_paths_and_hashes(source) ❶
    dest_hashes = read_paths_and_hashes(dest) ❶

    # шаг 2: вызвать функциональное ядро
    actions = determine_actions(source_hashes, dest_hashes, source,
                                dest) ❷
```

¹ См. <https://oreil.ly/wnad4>

```
# шаг 3 с императивным ядром: применить операции ввода-вывода данных
for action, *paths in actions:
    if action == 'copy':
        shutil.copyfile(*paths)
    if action == 'move':
        shutil.move(*paths)
    if action == 'delete':
        os.remove(paths[0])
```

❶ Это первая функция, которую мы перерабатываем, `read_paths_and_hashes()`. Она изолирует часть приложения, связанную с операциями ввода-вывода.

❷ Здесь мы выкраиваем функциональное ядро, бизнес-логику.

Код для построения словаря путей и хешей теперь пишется тривиально.

Функция, которая просто выполняет операции ввода-вывода (`sync.py`)

```
def read_paths_and_hashes(root):
    hashes = {}
    for folder, _, files in os.walk(root):
        for fn in files:
            hashes[hash_file(Path(folder) / fn)] = fn
    return hashes
```

Функция `determine_actions()` будет ядром бизнес-логики, которая говорит: «Что теперь следует скопировать/переместить/удалить, получив эти два множества хешей и имен файлов?» Она берет простые структуры данных и возвращает простые структуры данных.

Функция, которая просто выполняет бизнес-логику (`sync.py`)

```
def determine_actions(src_hashes, dst_hashes, src_folder, dst_folder):
    for sha, filename in src_hashes.items():
        if sha not in dst_hashes:
            sourcepath = Path(src_folder) / filename
            destpath = Path(dst_folder) / filename
            yield 'copy', sourcepath, destpath

        elif dst_hashes[sha] != filename:
            olddestpath = Path(dst_folder) / dst_hashes[sha]
            newdestpath = Path(dst_folder) / filename
            yield 'move', olddestpath, newdestpath

    for sha, filename in dst_hashes.items():
        if sha not in src_hashes:
            yield 'delete', dst_folder / filename
```

Теперь тесты действуют непосредственно на функцию `determine_actions()`.

Более симпатичные тесты (`test_sync.py`)

```
def test_when_a_file_exists_in_the_source_but_not_the_destination():
    src_hashes = {'hash1': 'fn1'}
    dst_hashes = {}
    actions = determine_actions(src_hashes, dst_hashes, Path('/src'),
                                Path('/dst'))
    assert list(actions) == [('copy', Path('/src/fn1'), Path('/dst/fn1'))]

...

def test_when_a_file_has_been_renamed_in_the_source():
    src_hashes = {'hash1': 'fn1'}
    dst_hashes = {'hash1': 'fn2'}
    actions = determine_actions(src_hashes, dst_hashes, Path('/src'),
                                Path('/dst'))
    assert list(actions) == [('move', Path('/dst/fn2'), Path('/dst/fn1'))]
```

Поскольку мы распутали логику программы — код для идентификации изменений, освободив ее от низкоуровневых деталей ввода-вывода, то можем легко протестировать ядро кода.

С помощью этого подхода мы перешли от тестирования главной функции точки входа, `sync()`, к тестированию более низкоуровневой функции, `determine_actions()`. Возможно, вы решите, что это нормально, потому что `sync()` очень простая. Или же решите попридержаться несколько интеграционных/приемочных тестов для того, чтобы протестировать эту `sync()`. Но есть и другой вариант, который заключается в изменении функции `sync()` так, чтобы ее можно было тестировать модульно и от начала до конца; такой подход Боб называет тестированием от *края до края* (*edge-to-edge testing*).

Edge-to-edge-тестирование с помощью подделок и внедрения зависимостей

Когда мы начинаем писать новую систему, то часто фокусируем наше внимание сначала на ключевой логике, управляя ею с помощью прямых юнит-тестов. Но в какой-то момент мы хотим протестировать более крупные куски системы вместе.

Мы *могли бы* вернуться к сквозным тестам, но их по-прежнему трудно писать и сопровождать. Вместо этого мы часто пишем тесты, которые ак-

тивизируют все систему целиком, но подделывают операции ввода-вывода как бы *от края до края*.

Явные зависимости (sync.py)

```
def sync(reader, filesystem, source_root, dest_root): ❶

    source_hashes = reader(source_root) ❷
    dest_hashes = reader(dest_root)

    for sha, filename in src_hashes.items():
        if sha not in dest_hashes:
            sourcepath = source_root / filename
            destpath = dest_root / filename
            filesystem.copy(destpath, sourcepath) ❸

        elif dest_hashes[sha] != filename:
            olddestpath = dest_root / dest_hashes[sha]
            newdestpath = dest_root / filename
            filesystem.move(olddestpath, newdestpath)

    for sha, filename in dst_hashes.items():
        if sha not in source_hashes:
            filesystem.delete(dest_root/filename)
```

❶ Верхнеуровневая функция теперь выставляет наружу две новые зависимости: читателя `reader` и файловую систему `filesystem`.

❷ Вызываем `reader`, чтобы создать словарь с файлами.

❸ Вызываем `filesystem`, чтобы применить обнаруженные изменения.



Хотя мы используем внедрение зависимостей, определять абстрактный базовый класс или какой-либо явный интерфейс не нужно. В этой книге мы часто показываем абстрактные базовые классы: так мы надеемся объяснить, что такое абстракция. Но использовать их необязательно. Динамическая природа языка Python означает, что всегда можно полагаться на утиную типизацию.

Тесты с внедрением зависимостей

```
class FakeFileSystem(list): ❶

    def copy(self, src, dest): ❷
        self.append(('COPY', src, dest))
```

```
def move(self, src, dest):
    self.append(('MOVE', src, dest))

def delete(self, dest):
    self.append(('DELETE', src, dest))

def test_when_a_file_exists_in_the_source_but_not_the_destination():
    source = {"sha1": "my-file" }
    dest = {}
    filesystem = FakeFileSystem()

    reader = {"/source": source, "/dest": dest}
    synchronise_dirs(reader.pop, filesystem, "/source", "/dest")

    assert filesystem == [("COPY", "/source/my-file", "/dest/my-file")]

def test_when_a_file_has_been_renamed_in_the_source():
    source = {"sha1": "renamed-file" }
    dest = {"sha1": "original-file" }
    filesystem = FakeFileSystem()

    reader = {"/source": source, "/dest": dest}
    synchronise_dirs(reader.pop, filesystem, "/source", "/dest")

    assert filesystem == [("MOVE", "/dest/original-file", "/dest/renamed-file")]
```

❶ Боб *обожает* использовать списки для построения простых тестовых двойников, даже если это бесит его коллег. Это означает, что мы можем писать тесты вроде проверки того, что `foo` нет в базе данных, `assert foo not in database`.

❷ Каждый метод в поддельной файловой системе, `FakeFileSystem`, просто добавляет что-то в список, чтобы мы могли проверить его позже. Это пример объекта-шпиона.

Преимущество этого подхода в том, что тесты работают с той же самой функцией, которая используется производственным кодом. Недостатком является то, что мы должны сделать компоненты с внутренним состоянием явными и передавать их туда-сюда. Дэвид Хайнемайер Ханссон, создатель `Ruby on Rails`, как известно, описал это как «спровоцированное тестом повреждение дизайна» (`test-induced design damage`).

В любом случае теперь можно работать над исправлением всех ошибок в реализации; перечислять тесты для всех крайних случаев теперь намного проще.

Почему бы просто не пропатчить его?

В этот момент вы можете призадуматься: почему бы нам просто не использовать `mock.patch` и не избавить себя от лишней работы?

В этой книге и в продакшене мы избегаем использования имитационных, или подставных, объектов `mock`. Мы не собираемся ввязываться в священную войну, но инстинкт подсказывает, что фреймворки имитационных объектов, в особенности с обезьяньими заплатками, — это признак кода «с душком»¹.

Вместо этого мы предпочитаем четко определять обязанности в кодовой базе и разделять эти обязанности на небольшие сфокусированные объекты, которые легко заменить тестовым двойником.



Вы увидите пример в главе 8, где мы используем `mock.patch()` в модуле отправки электронной почты, но впоследствии заменим его явной инъекцией зависимостей в главе 13.

Наше предпочтение основывается на трех тесно связанных причинах:

1. Наложение заплаток на используемые нами зависимости позволяет проводить юнит-тестирование кода, но это никак не улучшает дизайн. Использование `mock.patch` не позволит вашему коду работать с флагом `--dry-run`, а также не поможет вам работать с FTP-сервером. Для этого вам нужно будет ввести абстракции.
2. Тесты с использованием имитаций *тяготеют* к большей связанности с деталями реализации кодовой базы. Это обусловлено тем, что имитационные тесты проверяют взаимодействие между элементами кода: правильные ли аргументы мы использовали при вызове `shutil.rmtree()`? По нашему опыту, такая связанность между кодом и тестом, *как правило*, делает тесты ненадежными.
3. Чрезмерное использование имитаций приводит к сложным тестам, которые не способны объяснять код.

¹ Обезьянья заплатка (monkey patch), или утиный патч, — в программировании возможность подмены методов и значений атрибутов классов во время выполнения программы. Признак кода «с душком». — *Примеч. пер.*



Проектирование с учетом тестопригодности на самом деле означает проектирование с учетом расширяемости. Мы получаем немного больше сложности в обмен на более чистый дизайн, который допускает новые варианты использования.

ИМИТАЦИИ ПРОТИВ ПОДДЕЛОК; КЛАССИЧЕСКАЯ ШКОЛА TDD ПРОТИВ ЛОНДОНСКОЙ

Вот краткое и несколько упрощенное определение разницы между имитациями и подделками:

- Имитации (mocks), или подставные объекты, используются для проверки того, *каким образом* что-то используется; у них есть такие методы, как `assert_called_once_with()` (проверяет, действительно ли метод был вызван только один раз). Они связаны с лондонской школой TDD.
- Подделки (fakes) — это рабочие реализации того, что они заменяют. Подделки предназначены для использования только в тестах. Они не будут работать «в реальной жизни»; репозиторий, расположенный прямо в памяти, — хороший тому пример. Но вы можете использовать их, чтобы делать утверждения о конечном состоянии системы, а не о поведении в процессе, поэтому они связаны с классическим стилем TDD.

Здесь мы немного мешаем в кучу имитации со шпионами и подделки с заглушками, и вы можете прочитать длинный правильный ответ в классическом эссе Мартина Фаулера на эту тему под названием «Имитации — это не заглушки»¹.

Вероятно, не помогает и то, что объекты `MagicMock`, предоставляемые методом `unittest.mock`, — это, строго говоря, не имитации, а шпионы, если уж на то пошло. Но они также часто используются в качестве заглушек или муляжей. Ну вот мы, как и обещали, закончили придирааться к терминологии тестовых двойников.

А как насчет технологии TDD в стиле лондонской школы против классического стиля? Вы можете узнать о них больше из вышеупомянутой статьи Мартина Фаулера, а также на веб-сайте по инженерии ПО², но в этой книге мы уверенно занимаем позицию «классиков». Нам нравится создавать тесты на основе состояния как при настройке, так и в утверждениях, и работать на максимально возможном уровне абстракции, а не проверять поведение промежуточных участников³.

Подробнее об этом — в разделе «Какие тесты писать» на с. 112.

¹ См. Mocks Aren't Stubs, <https://oreil.ly/yYjBN>

² См. Software Engineering Stack Exchange, https://oreil.ly/H2im_

³ Но это вовсе не означает, что приверженцы лондонской школы ошибаются. Некоторые чертовски толковые люди работают именно так. Просто это не то, к чему мы привыкли.

Мы рассматриваем TDD в первую очередь как практику проектирования и только во вторую — как практику тестирования. Тесты помогают зафиксировать наш выбор дизайна и дать объяснение системе, когда мы возвращаемся к коду после долгого отсутствия.

Если тесты слишком перегружаются установочным кодом из-за большого числа имитаций, мы не видим интересующую нас информацию.

В своем выступлении «Разработка на основе тестов» Стив Фримен привел отличный пример тестов с чрезмерным числом имитаций¹. Вам также стоит посмотреть выступление «Подводные камни применения имитаций и патчей»² на конференции RuCon нашего уважаемого научного редактора Эда Юнга, который также рассматривает использование имитации и ее альтернативы. И раз уж мы рекомендуем выступления, то не пропустите Брэндона Родса с «Поднятием ваших операций ввода-вывода на новый уровень»³, где на простом примере действительно неплохо охватываются обсуждаемые нами трудности.



В этой главе мы потратили много времени на замену сквозных тестов (E2E-тестов) юнит-тестами. Это вовсе не означает, что вы никогда не должны использовать сквозные тесты! В этой книге мы показываем методы, которые помогут создать достойную тестовую пирамиду с максимально возможным числом юнит-тестов и минимальным числом E2E-тестов, необходимых вам для того, чтобы чувствовать себя уверенно. Более подробную информацию см. во врезке «Эмпирические правила для разных типов тестов» на с. 119.

ТАК ЧТО ЖЕ МЫ ИСПОЛЬЗУЕМ В ЭТОЙ КНИГЕ? ФУНКЦИОНАЛЬНУЮ ИЛИ ЖЕ ОБЪЕКТНО ОРИЕНТИРОВАННУЮ КОМПОЗИЦИЮ?

Обе. Модель предметной области полностью свободна от зависимостей и побочных эффектов и потому является функциональным ядром. Сервисный слой, который мы создаем вокруг него (в главе 4), позволяет управлять системой от начала до конца, и мы используем внедрение зависимостей, чтобы предоставлять этим службам компоненты с внутренним состоянием. Поэтому по-прежнему можно провести для них юнит-тесты.

Более подробную информацию о том, как сделать внедрение зависимостей более явно выраженным и централизованным, см. в главе 13.

¹ См. Test-Driven Development, <https://oreil.ly/jAmtr>

² См. Mocking and Patching Pitfalls, <https://oreil.ly/s3e05>

³ См. Hoisting Your I/O, <https://oreil.ly/oiXJM>

Выводы

В книге мы будем встречать эту идею снова и снова: можно упростить тестирование и обслуживание наших систем, упростив интерфейс между бизнес-логикой и беспорядочным вводом-выводом. Найти правильную абстракцию трудно, но вот несколько эвристик и вопросов, которые нужно себе задать:

- Могу ли я выбрать знакомую мне структуру данных Python для представления состояния запутанной системы, а затем попытаться представить себе единственную функцию, которая может возвращать это состояние?
- Где я могу провести грань между моими системами и вырезать шов^{1,2}, чтобы вставить эту абстракцию?
- Какой есть разумный способ разделения элементов кода на компоненты с разными обязанностями? Какие неявные концепции можно сделать явными?
- Каковы зависимости и какова ключевая бизнес-логика?

Практика — путь к меньшему несовершенству! А теперь вернемся к обычному программированию...

¹ См. <https://oreil.ly/zNUGG>

² Шов (seam) — это место, где можно изменить поведение своей программы без редактирования. — *Примеч. пер.*

Первый вариант использования: API фреймворка Flask и сервисный слой

Вернемся к нашему проекту размещения заказов. На рис. 4.1 показана схема, к которой мы пришли в конце главы 2. Напомним, что посвящена она была паттерну «Репозиторий».

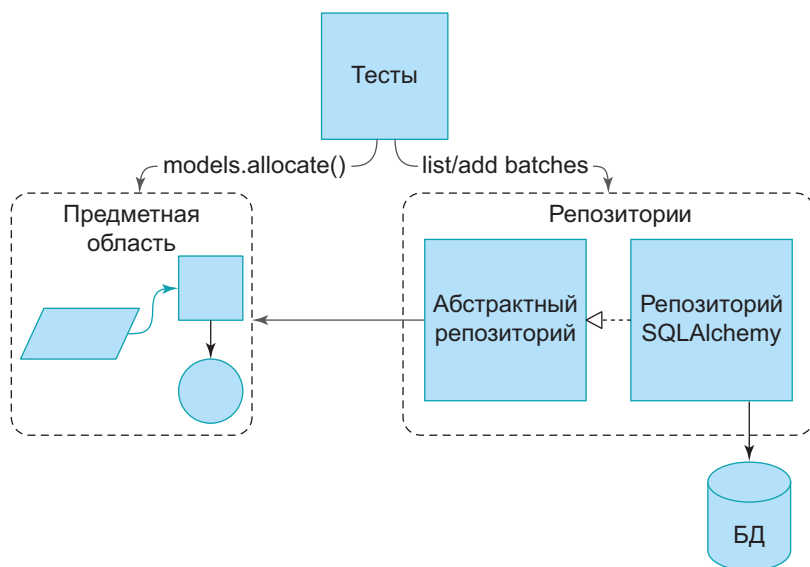


Рис. 4.1. Прежний вид: мы управляем приложением, обращаясь к репозиториям и модели предметной области

В этой главе мы обсудим различия между логикой оркестровки, бизнес-логикой и интерфейсным кодом, а также представим паттерн «Сервисный

слой» для оркестровки рабочих потоков и определения вариантов использования системы.

Мы также обсудим тестирование: объединив абстракцию «Сервисный слой» с абстракцией «Репозиторий» над базой данных, мы сможем писать быстрые тесты не только для модели предметной области, но и для всего процесса работы нашего примера.

На рис. 4.2 показано то, к чему мы стремимся: мы собираемся добавить API фреймворка Flask, который будет взаимодействовать с сервисным слоем, а он станет служить точкой входа в модель предметной области. Поскольку сервисный слой зависит от абстрактного репозитория, `AbstractRepository`, мы можем сделать юнит-тест поддельного репозитория, `FakeRepository`, но при этом выполнять производственный код с помощью репозитория `SQLAlchemyRepository`.

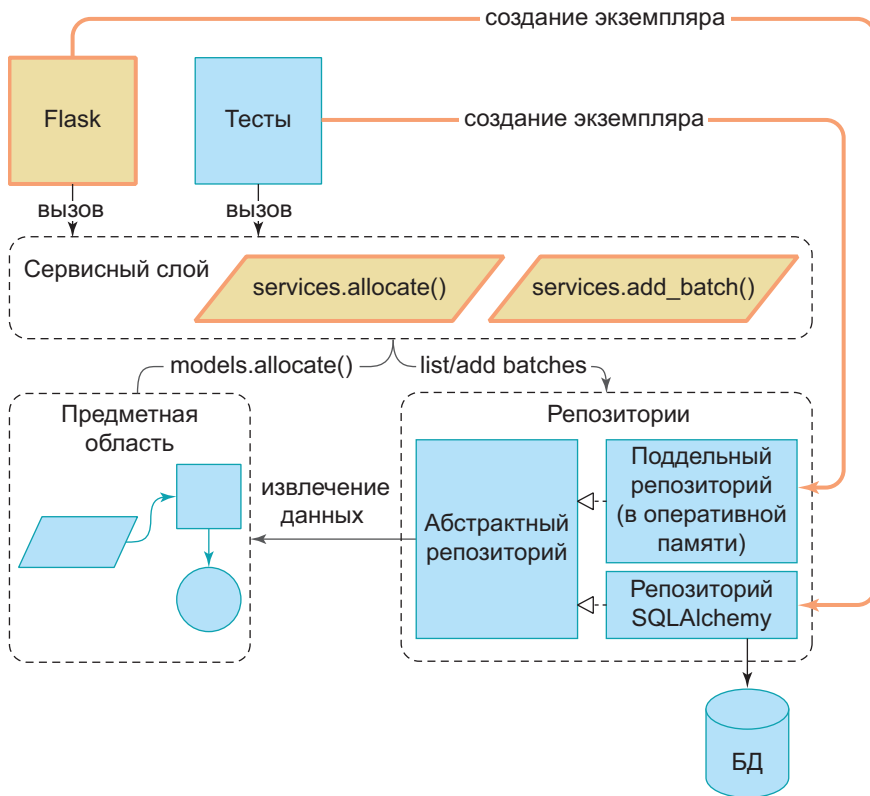


Рис. 4.2. Слой служб станет главным путем в наше приложение

На рисунках новые компоненты выделяются жирным шрифтом/линиями.



Код для этой главы находится в ветке `chapter_04_service_layer` на GitHub:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_04_service_layer
# или, если пишете код по ходу чтения, возьмите за основу
# материал из главы 2:
git checkout chapter_02_repository
```

Связываем приложение с реальным миром

Как и любая хорошая agile-команда, мы стараемся выпустить минимально жизнеспособный продукт и начать собирать фидбек пользователей. У нас есть ядро предметной области и служба предметной области, которая нужна для размещения заказов, плюс интерфейс репозитория для постоянного хранения данных.

Давайте как можно быстрее соединим все части, а затем сделаем рефакторинг и добьемся более чистой архитектуры. План таков:

1. Использовать Flask, чтобы поместить конечную точку API перед службой предметной области `allocate`. Подсоединить сеанс базы данных и репозиторий. Протестировать его с помощью E2E-теста и какого-нибудь чернового SQL для подготовки тестовых данных.
2. Выполнить рефакторинг сервисного слоя, который может выполнять роль абстракции для захвата варианта использования и будет находиться между Flask и нашей моделью предметной области. Создать несколько тестов сервисного слоя и показать, как они могут использовать поддельный репозиторий, `FakeRepository`.
3. Поэкспериментировать с разными типами параметров для функций сервисного слоя; показать, что использование примитивных типов данных позволяет отделить клиентов сервисного слоя (тесты и API фреймворка Flask) от слоя модели.

Первый сквозной тест

Никому не интересно долго спорить о различиях в терминологии, когда речь заходит о сравнении сквозных (E2E), функциональных, приемочных, интеграционных и юнит-тестов. Различные проекты нуждаются в разных комбинациях тестов, и мы были свидетелями того, как совершенно успешные проекты просто делили все тесты на «быстрые» и «медленные».

Пока что мы хотим написать один или два теста, в которых будет использоваться «реальная» конечная точка API (используя HTTP) и связываться с реальной базой данных. Давайте назовем их *сквозными тестами* (E2E), потому что это название говорит само за себя.

Ниже показана первая попытка сделать это.

Первый тест API (test_api.py)

```
@pytest.mark.usefixtures('restart_api')
def test_api_returns_allocation(add_stock):
    sku, othersku = random_sku(), random_sku('other') ❶
    earlybatch = random_batchref(1)
    laterbatch = random_batchref(2)
    otherbatch = random_batchref(3)
    add_stock([ ❷
        (laterbatch, sku, 100, '2011-01-02'),
        (earlybatch, sku, 100, '2011-01-01'),
        (otherbatch, othersku, 100, None),
    ])
    data = {'orderid': random_orderid(), 'sku': sku, 'qty': 3}
    url = config.get_api_url() ❸
    r = requests.post(f'{url}/allocate', json=data)
    assert r.status_code == 201
    assert r.json()['batchref'] == earlybatch
```

❶ random_sku(), random_batchref() и т. д. — это малые вспомогательные функции, которые с помощью модуля uuid генерируют рандомизированные символы. Поскольку мы сейчас работаем с реальной базой данных, это один из способов предотвратить влияние различных тестов и запусков друг на друга.

❷ add_stock — это вспомогательный инструмент, который просто скрывает детали ручной вставки строк в базу данных с помощью SQL. Позже в этой главе мы покажем более аккуратный способ.

❸ config.py — это модуль, в котором хранится информация о конфигурации.

Все решают эти задачи по-разному, но понадобится какой-то способ «разгона» Flask, возможно, в контейнере, и обмена информацией с базой данных Postgres. Если вы хотите посмотреть, как это сделали мы, то ознакомьтесь с приложением Б.

Простая реализация

Реализовав приложение самым очевидным образом, можно получить что-то вроде этого:

Первая версия приложения на основе Flask (flask_app.py)

```
from flask import Flask, jsonify, request
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

import config
import model
import orm
import repository

orm.start_mappers()
get_session = sessionmaker(bind=create_engine(config.get_postgres_uri()))
app = Flask(__name__)

@app.route("/allocate", methods=['POST'])
def allocate_endpoint():
    session = get_session()
    batches = repository.SqlAlchemyRepository(session).list()
    line = model.OrderLine(
        request.json['orderid'],
        request.json['sku'],
        request.json['qty'],
    )

    batchref = model.allocate(line, batches)

    return jsonify({'batchref': batchref}), 201
```

Пока все идет хорошо. Возможно, вы подумали: «Ну что, Боб и Гарри, не нужна нам ваша лишняя “архитектурно-космическая” чепуха».

Но минуточку — здесь нет команды фиксации транзакции базы данных. На самом деле мы не сохраняем размещение заказа в базе данных. Теперь

нужен второй тест — тот, который будет проверять состояние базы данных после (не очень похожей на черный ящик) операции либо, возможно, тот, который проверяет, что мы не можем разместить вторую товарную позицию заказа, если после размещения первой партия закончилась.

Распределение тестов сохраняется (test_api.py)

```
@pytest.mark.usefixtures('restart_api')
def test_allocations_are_persisted(add_stock):
    sku = random_sku()
    batch1, batch2 = random_batchref(1), random_batchref(2)
    order1, order2 = random_orderid(1), random_orderid(2)
    add_stock([
        (batch1, sku, 10, '2011-01-01'),
        (batch2, sku, 10, '2011-01-02'),
    ])
    line1 = {'orderid': order1, 'sku': sku, 'qty': 10}
    line2 = {'orderid': order2, 'sku': sku, 'qty': 10}
    url = config.get_api_url()

    # первый заказ исчерпывает все товары в партии 1
    r = requests.post(f'{url}/allocate', json=line1)
    assert r.status_code == 201
    assert r.json()['batchref'] == batch1

    # второй заказ должен пойти в партию 2
    r = requests.post(f'{url}/allocate', json=line2)
    assert r.status_code == 201
    assert r.json()['batchref'] == batch2
```

Не столь красиво, но это вынудит нас добавить фиксацию.

Состояния ошибок, требующие проверки базы данных

Но если мы будем продолжать в том же духе, то все примет еще более скверный оборот.

Предположим, что мы хотим добавить часть с обработкой ошибок. Что делать, если модель предметной области вызывает ошибку для артикула, которого нет в наличии? Или как насчет артикула, которого даже не существует? Это даже не то, о чем модель не знает, да и не должна знать.

Это скорее проверка исправности, которую мы должны реализовать на уровне базы данных, прежде чем вызывать службу предметной области.

Теперь мы рассмотрим еще два сквозных теста:

Еще больше тестов в слое сквозных тестов (test_api.py)

```
@pytest.mark.usefixtures('restart_api')
def test_400_message_for_out_of_stock(add_stock): ❶
    sku, small_batch, large_order = random_sku(), random_batchref(),
    random_orderid()
    add_stock([
        (small_batch, sku, 10, '2011-01-01'),
    ])
    data = {'orderid': large_order, 'sku': sku, 'qty': 20}
    url = config.get_api_url()
    r = requests.post(f'{url}/allocate', json=data)
    assert r.status_code == 400
    assert r.json()['message'] == f'Артикула {sku} нет в наличии'

@pytest.mark.usefixtures('restart_api')
def test_400_message_for_invalid_sku(): ❷
    unknown_sku, orderid = random_sku(), random_orderid()
    data = {'orderid': orderid, 'sku': unknown_sku, 'qty': 20}
    url = config.get_api_url()
    r = requests.post(f'{url}/allocate', json=data)
    assert r.status_code == 400
    assert r.json()['message'] == f'Недопустимый артикул {unknown_sku}'
```

❶ В первом тесте мы пытаемся разместить больше товаров, чем есть в наличии.

❷ Во втором случае артикула просто не существует (потому что мы ни разу не вызывали `add_stock`), и для приложения он недействителен.

Конечно, мы могли бы реализовать это и в приложении Flask.

Приложение Flask начинает портиться (flask_app.py)

```
def is_valid_sku(sku, batches):
    return sku in {b.sku for b in batches}

@app.route("/allocate", methods=['POST'])
def allocate_endpoint():
    session = get_session()
    batches = repository.SqlAlchemyRepository(session).list()
    line = model.OrderLine(
```



```
        request.json['orderid'],
        request.json['sku'],
        request.json['qty'],
    )

    if not is_valid_sku(line.sku, batches):
        return jsonify({'message': f'Недопустимый артикул {line.sku}'}),
        400

    try:
        batchref = model.allocate(line, batches)
    except model.OutOfStock as e:
        return jsonify({'message': str(e)}), 400

    session.commit()
    return jsonify({'batchref': batchref}), 201
```

Все начинает выглядеть немного громоздким. И число сквозных тестов постепенно выходит из-под контроля. Вскоре мы получим перевернутую тестовую пирамиду (или «модель в виде рожка мороженого», как называет ее Боб).

Введение сервисного слоя и использование поддельного репозитория для юнит-теста

Если посмотреть на действия приложения Flask, то мы увидим довольно многое из того, что можно назвать *оркестровкой* — извлечение материала из репозитория, проверка входных данных по состоянию базы данных, обработка ошибок и фиксация в успешном результате сценария процесса (счастливом пути). Большинство из этих вещей не имеют ничего общего с наличием конечной точки веб-API (они понадобятся, например, при создании CLI; см. приложение В), и на самом деле это не то, что стоит тестировать сквозными тестами.

Часто имеет смысл разделить сервисный слой, иногда именуемый *слоем оркестровки* (orchestration layer), или *слоем варианта использования* (use-case layer).

Помните поддельный репозиторий, FakeRepository, который мы подготовили в главе 3?

Поддельный репозиторий, коллекция партий товара в памяти (test_services.py)

```
class FakeRepository(repository.AbstractRepository):
```

```
    def __init__(self, batches):
        self._batches = set(batches)

    def add(self, batch):
        self._batches.add(batch)

    def get(self, reference):
        return next(b for b in self._batches if b.reference == reference)

    def list(self):
        return list(self._batches)
```

Вот где он будет полезен — он позволяет тестировать слой служб с помощью хороших, быстрых юнит-тестов.

Юнит-тестирование с подделками в слое служб (test_services.py)

```
def test_returns_allocation():
    line = model.OrderLine("o1", "COMPLICATED-LAMP", 10)
    batch = model.Batch("b1", "COMPLICATED-LAMP", 100, eta=None)
    repo = FakeRepository([batch]) ❶

    result = services.allocate(line, repo, FakeSession()) ❷ ❸
    assert result == "b1"

def test_error_for_invalid_sku():
    line = model.OrderLine("o1", "NONEXISTENTSKU", 10)
    batch = model.Batch("b1", "AREALSKU", 100, eta=None)
    repo = FakeRepository([batch]) ❶

    with pytest.raises(services.InvalidSku, match="Недопустимый артикул NONEXISTENTSKU"):
        services.allocate(line, repo, FakeSession()) ❷ ❸
```

❶ Поддельный репозиторий `FakeRepository` содержит объекты `Batch`, которые будут использоваться в тесте.

❷ Модуль служб (`services.py`) определит функцию `allocate()` сервисного слоя. Он будет находиться между функцией `allocate_endpoint()` в слое API и функцией `allocate()` службы предметной области из модели¹.

¹ Термины «службы сервисного слоя» и «службы предметной области» и вправду имеют обескураживающе схожие названия. Мы рассмотрим эту тему позже в разделе «Почему все называется службой?» на с. 102.

❸ Также нужен `FakeSession`, чтобы подделывать сеанс базы данных, как показано в следующем фрагменте кода:

Поддельный сеанс базы данных (`test_services.py`)

```
class FakeSession():
    committed = False

    def commit(self):
        self.committed = True
```

Указанный поддельный сеанс — лишь временное решение. В главе 6 мы от него избавимся и вскоре сделаем все еще лучше. А пока поддельный `.commit()` позволяет перенести третий тест из сквозного слоя (E2E-слоя).

Второй тест в сервисном слое (`test_services.py`)

```
def test_commits():
    line = model.OrderLine('o1', 'OMINOUS-MIRROR', 10)
    batch = model.Batch('b1', 'OMINOUS-MIRROR', 100, eta=None)
    repo = FakeRepository([batch])
    session = FakeSession()

    services.allocate(line, repo, session)
    assert session.committed is True
```

Типичная функция службы

Напишем функцию службы, которая выглядит примерно так:

Базовая служба размещения заказов (`services.py`)

```
class InvalidSku(Exception):
    pass

def is_valid_sku(sku, batches):
    return sku in {b.sku for b in batches}

def allocate(line: OrderLine, repo: AbstractRepository, session) -> str:
    batches = repo.list() ❶
    if not is_valid_sku(line.sku, batches): ❷
        raise InvalidSku(f'Недопустимый артикул {line.sku}')
    batchref = model.allocate(line, batches) ❸
    session.commit() ❹
    return batchref
```

Типичные функции сервисного слоя имеют похожие этапы:

- ❶ Извлекаем несколько объектов из репозитория.
- ❷ Делаем несколько проверок или утверждений относительно запроса текущего состояния мира.
- ❸ Вызываем службу предметной области.
- ❹ Если все хорошо, то сохраняем/обновляем любое состояние, которое изменили.

Этот последний этап пока что не очень хорош, поскольку слой служб тесно связан со слоем базы данных. Мы исправим это в главе 6 с помощью паттерна UoW.

ЗАВИСИМОСТЬ ОТ АБСТРАКЦИЙ

Обратите внимание еще на одну вещь, которая касается функции слоя служб:

```
def allocate(line: OrderLine, repo: AbstractRepository, session) -> str:
```

Она зависит от репозитория. Мы решили сделать зависимость явной и использовали подсказку типа, чтобы сказать, что зависим от абстрактного репозитория, `AbstractRepository`. Это означает, что она будет работать и тогда, когда тесты передают ей поддельный репозиторий, `FakeRepository`, и когда приложение Flask передает ей репозиторий `SqlAlchemyRepository`.

Из раздела «Принцип инверсии зависимостей» вы, возможно, помните, что это то, что мы имеем в виду, когда говорим, что должны «зависеть от абстракций». *Высокоуровневый модуль*, сервисный слой, зависит от абстракции репозитория. И *детали* реализации для выбранной нами конкретной системы постоянного хранения также зависят от этой же абстракции. См. рис. 4.3 и 4.4.

См. также приложение В, где описан рабочий пример замены деталей выбора системы постоянного хранения при том, что абстракции остаются нетронутыми.

Но самое необходимое для уровня обслуживания есть, и приложение Flask теперь выглядит намного чище:

Приложение Flask передает полномочия в сервисный слой (flask_app.py)

```
@app.route("/allocate", methods=['POST'])
def allocate_endpoint():
    session = get_session() ❶
    repo = repository.SqlAlchemyRepository(session) ❶
    line = model.OrderLine(
```

```

        request.json['orderid'], ❷
        request.json['sku'], ❷
        request.json['qty'], ❷
    )
    try:
        batchref = services.allocate(line, repo, session) ❷
    except (model.OutOfStock, services.InvalidSku) as e:
        return jsonify({'message': str(e)}), 400 ❸

    return jsonify({'batchref': batchref}), 201 ❸

```

❶ Создаем экземпляр сеанса базы данных и несколько объектов репозитория.

❷ Извлекаем из веб-запроса команды пользователя и передаем их в службу предметной области.

❸ Возвращаем несколько ответов в формате JSON с соответствующими кодами статуса.

Обязанности приложения Flask связаны со всякими веб-штуками: сеансом по запросу, разбором информации из параметров POST, кодами статуса ответа и JSON. Вся логика оркестровки находится в слое варианта использования/служб, а логика предметной области остается в пределах предметной области.

Наконец, мы можем уверенно урезать сквозные тесты, оставив всего два: один для счастливого пути и другой — для несчастливого.

Сквозные тесты проверяют только счастливые и несчастливые пути (test_api.py)

```

@pytest.mark.usefixtures('restart_api')
def test_happy_path_returns_201_and_allocated_batch(add_stock):
    sku, othersku = random_sku(), random_sku('other')
    earlybatch = random_batchref(1)
    laterbatch = random_batchref(2)
    otherbatch = random_batchref(3)
    add_stock([
        (laterbatch, sku, 100, '2011-01-02'),
        (earlybatch, sku, 100, '2011-01-01'),
        (otherbatch, othersku, 100, None),
    ])
    data = {'orderid': random_orderid(), 'sku': sku, 'qty': 3}
    url = config.get_api_url()
    r = requests.post(f'{url}/allocate', json=data)

```

```
assert r.status_code == 201
assert r.json()['batchref'] == earlybatch

@pytest.mark.usefixtures('restart_api')
def test_unhappy_path_returns_400_and_error_message():
    unknown_sku, orderid = random_sku(), random_orderid()
    data = {'orderid': orderid, 'sku': unknown_sku, 'qty': 20}
    url = config.get_api_url()
    r = requests.post(f'{url}/allocate', json=data)
    assert r.status_code == 400
    assert r.json()['message'] == f'Недопустимый артикул {unknown_sku}'
```

Мы успешно разделили тесты на две большие категории: тесты, касающиеся всего, что связано с вебом, которые мы реализуем в сквозном порядке, то есть от начала до конца; и тесты, касающиеся оркестровки, которые мы можем выполнять в отношении сервисного слоя прямо в памяти.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Теперь, когда у нас есть служба для размещения заказов, почему бы не создать службу для отмены размещений, `deallocate`? Мы добавили сквозной тест и несколько тестов-заглушек слоя служб, чтобы вы могли начать работу на GitHub¹.

Если этого недостаточно, то продолжайте работу со сквозными тестами и `flask_app.py`, и сделайте рефакторинг адаптеров Flask, чтобы получить более RESTful-ориентированный архитектурный стиль. Обратите внимание, что для этого не требуется никаких изменений в сервисном слое или слое предметной области!

Если вы решите создать конечную точку только для чтения информации о размещении заказа, то просто сделайте «самое простое, что сможет работать», а именно `repo.get()`, прямо в обработчике Flask.

В главе 12 мы еще поговорим о том, что такое чтение и запись.

Почему все называется службой?

Некоторые из вас, вероятно, сейчас ломают голову, пытаясь понять, в чем именно заключается разница между службой предметной области и сервисным слоем (слоем служб).

¹ См. https://github.com/cosmicpython/code/tree/chapter_04_service_layer_exercise

Нам очень жаль — эти имена придумали не мы, иначе у нас были бы гораздо более прикольные и дружелюбные названия для таких вещей.

В этой главе мы используем две вещи, именуемые *службами*. Первая — это *служба приложения* (слой служб). Ее работа заключается в обработке запросов извне и оркестрировании операции. Мы имеем в виду, что сервисный слой управляет приложением по следующему сценарию:

- получает немного данных из БД;
- обновляет модель предметной области;
- сохраняет любые изменения в постоянном хранилище.

Эта довольно-таки скучная работа должна выполняться для каждой операции в вашей системе. Хранение ее отдельно от бизнес-логики помогает поддерживать порядок.

Второй тип службы — это *служба предметной области*. Это название используется для той части логики, которая принадлежит модели предметной области, но сама по себе не находится внутри объекта с внутренним состоянием или объекта-значения. Например, если вы создаете приложение для корзины товаров, то, возможно, решите создать правила налогообложения как службу предметной области. Расчет налога — это отдельная от обновления корзины работа, и она является важной частью модели, но, по-видимому, совершенно неправильно иметь ради этого сущность постоянного хранения. Вместо этого указанную работу может выполнять класс `TaxCalculator` без внутреннего состояния либо функция `calculate_tax`.

Складываем все в папки, чтобы понять, где что находится

По мере того как приложение становится все больше, нужно будет продолжать приводить в порядок его структуру директорий. Схема проекта дает нам полезные подсказки о том, какого рода объекты находятся в каждом файле.

Вот один из способов, с помощью которого мы могли бы все организовать:

Несколько папок

```
.
├── config.py
├── domain ❶
│   ├── __init__.py
│   └── model.py
├── service_layer ❷
│   ├── __init__.py
│   └── services.py
├── adapters ❸
│   ├── __init__.py
│   ├── orm.py
│   └── repository.py
├── entrypoints ❹
│   ├── __init__.py
│   └── flask_app.py
├── tests
│   ├── __init__.py
│   ├── conftest.py
│   ├── unit
│   │   ├── test_allocate.py
│   │   ├── test_batches.py
│   │   └── test_services.py
│   ├── integration
│   │   ├── test_orm.py
│   │   └── test_repository.py
│   └── e2e
│       └── test_api.py
```

❶ Создадим папку для модели предметной области. Пока что это просто один файл, но в случае более сложных приложений для каждого класса будет отдельный файл; возможно, у вас будут вспомогательные родительские классы для классов `Entity`, `ValueObject` и `Aggregate` и вы добавите файл `exceptions.py` для исключений слоя предметной области и, как мы увидим в части II, файлы `commands.py` и `events.py`.

❷ Выделим сервисный слой. Сейчас это всего лишь один файл под названием `services.py` для функций сервисного слоя. Здесь можно добавить исключения сервисного слоя и, как вы увидите в главе 5, файл `unit_of_work.py`.

❸ *Адаптеры* — это дань терминологии портов и адаптеров. Они заполняются любыми другими абстракциями вокруг операций внешнего ввода-вывода

(например, `redis_client.py`). Строго говоря, их можно было бы назвать *вторичными адаптерами* или *ведомыми адаптерами*, а иногда и адаптерами, *обращенными вовнутрь*.

④ Точки входа — это места, из которых мы управляем приложением. В официальной терминологии портов и адаптеров они также являются адаптерами и называются *первичными*, *ведущими* или *внешними* адаптерами.

А что насчет портов? Как вы помните, это абстрактные интерфейсы, которые реализуются адаптерами. Как правило, мы храним их в том же файле, что и реализующие их адаптеры.

Выводы

Добавление слоя служб и вправду дало нам множество преимуществ:

- Конечные точки API фреймворка Flask становятся очень тонкими и простыми в написании: их единственная обязанность состоит в выполнении всего, что связано с вебом, в частности парсинг JSON и создание правильных HTTP-кодов для счастливых или несчастливых случаев.
- Мы определили четкий API для предметной области, набор вариантов использования или точек входа, которые могут использоваться любым адаптером без необходимости знать что-либо о классах модели предметной области — будь то API, CLI (см. приложение В) или тесты! Они тоже являются адаптером для предметной области.
- Мы можем писать тесты в режиме «верхней передачи» посредством использования сервисного слоя, имея возможность свободно выполнять переработку модели предметной области любым способом, который считаем нужным. До тех пор пока мы предоставляем одни и те же варианты использования, мы можем экспериментировать с новыми вариантами дизайна без необходимости переписывать массу тестов.
- И в итоге тестовая пирамида выглядит хорошо, подавляющая часть тестов — это быстрые юнит-тесты с минимумом сквозных и интеграционных тестов.

Принцип инверсии зависимостей в действии

На рис. 4.3 показаны зависимости сервисного слоя: модель предметной области и абстрактный репозиторий, `AbstractRepository` (порт в терминологии портов и адаптеров).

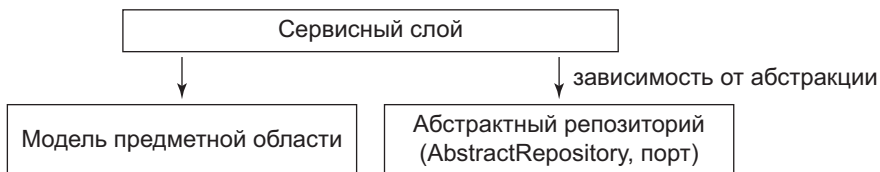


Рис. 4.3. Абстрактные зависимости сервисного слоя

На рис. 4.4 показано то, как реализуются абстрактные зависимости с помощью поддельного репозитория, `FakeRepository` (адаптера), когда мы выполняем тесты.

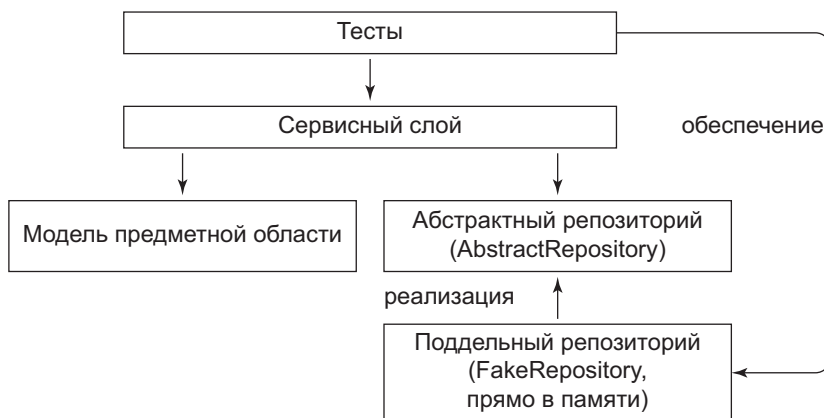


Рис. 4.4. Тесты обеспечивают реализацию абстрактной зависимости

И когда мы выполняем приложение по-настоящему, мы вносим «реальную» зависимость, показанную на рис. 4.5.



Рис. 4.5. Зависимости во время выполнения

Замечательно!

Давайте остановимся на табл. 4.1, в которой рассмотрим плюсы и минусы наличия сервисного слоя.

Таблица 4.1. Сервисный слой: компромиссы

Плюсы	Минусы
• Есть единое место для сбора всех сценариев использования нашего приложения. • Мы поместили хитроумную логику предметной области позади API, что позволяет нам свободно перерабатывать код. • Мы отделили «все то, что общается с HTTP» от «всего того, что общается с размещением заказов». • В сочетании с паттерном «Репозиторий» и поддельным репозиторием, FakeRepository, у нас появился хороший способ написания тестов на более высоком уровне, чем слой предметной области; можно тестировать больше рабочего потока без необходимости использовать интеграционные тесты (подробнее об этом — в главе 5)	• Если ваше приложение является исключительно веб-приложением, то контроллеры/функции представления могут быть единственным местом для сбора всех вариантов использования. • Это еще один слой абстракции. • Размещение слишком большого объема логики в слое служб может привести к антипаттерну «Анемичная предметная область». Лучше ввести этот слой после того, как вы обнаружите, что логика оркестровки начинает заползать в ваши контроллеры. • Вы можете получить преимуществ от наличия насыщенных моделей предметной области, просто выталкивая логику из ваших контроллеров и перемещая ее вниз, в слой модели, без необходимости добавлять дополнительный слой между ними (так называемые «толстые модели, тонкие контроллеры»)

Но есть еще кое-что, что нужно исправить:

- Сервисный слой по-прежнему тесно связан с предметной областью, поскольку его API выражается в терминах объектов `orderLine`. В главе 5 мы это исправим и поговорим о том, как сервисный слой обеспечивает более продуктивную TDD.
- Сервисный слой тесно связан с объектом `session`. В главе 6 мы представим еще один паттерн, который тесно взаимодействует с паттернами «Репозиторий» и «Слой служб», паттерн UoW, и все будет просто замечательно. Вот увидите!

TDD на повышенной и пониженной передачах

Мы ввели сервисный слой, чтобы захватить некоторые дополнительные обязанности по оркестровке, которые требуются от рабочего приложения. Сервисный слой помогает четко определять варианты использования и рабочий поток для каждого из них: что нужно получить из репозитория, какие предварительные проверки и оценку текущего состояния мы должны провести и что сохраняем в конце.

Но пока что многие наши юнит-тесты работают на более низком уровне, действуя непосредственно на модель. В этой главе мы обсудим компромиссы, связанные с перемещением этих тестов в сервисный слой, а также дадим несколько более общих рекомендаций по тестированию.

ГОВОРIT ГАРРИ: «Я ПРОЗРЕЛ, УВИДЕВ ТЕСТОВУЮ ПИРАМИДУ В ДЕЙСТВИИ»

Вот несколько слов от Гарри:

«Поначалу я относился ко всем паттернам Боба скептически, но вид настоящей тестовой пирамиды меня переубедил.

После того как вы внедрите моделирование предметной области и сервисный слой, вы действительно сможете перейти к тому, что количество юнит-тестов на порядок превышает количество интеграционных и сквозных. В прошлом я сталкивался с многочасовой сборкой сквозных тестов (читай «ждите до завтра»), так что не нахожу слов, чтобы выразить ту огромную разницу, когда можно выполнять все свои тесты за считанные минуты или секунды.

Прочтите несколько рекомендаций о том, как решить, какие тесты писать в зависимости от уровня. Мышление в стиле повышенной и пониженной передач действительно изменило мою жизнь как тестировщика».

Как выглядит пирамида тестирования

Давайте посмотрим, что этот переход к использованию сервисного слоя с его собственными тестами сделал с тестовой пирамидой:

Считаем типы тестов

```
$ grep -c test_ test_*.py
tests/unit/test_allocate.py:4
tests/unit/test_batches.py:8
tests/unit/test_services.py:3

tests/integration/test_orm.py:6
tests/integration/test_repository.py:2

tests/e2e/test_api.py:2
```

Неплохо! У нас пятнадцать юнит-тестов, восемь интеграционных тестов и всего два сквозных теста. Это уже здоровая тестовая пирамида.

Должны ли тесты слоя предметной области перейти в сервисный слой?

Посмотрим, что произойдет дальше. Поскольку мы можем тестировать программу для сервисного слоя, тесты для модели предметной области больше не нужны. Вместо этого можно переписать все тесты слоя предметной области из главы 1 для сервисного слоя:

Переписывание теста слоя предметной области в сервисном слое (tests/unit/test_services.py)

```
# тест слоя предметной области:
def test_prefers_current_stock_batches_to_shipments():
    in_stock_batch = Batch("in-stock-batch", "RETRO-CLOCK", 100,
                           eta=None)
    shipment_batch = Batch("shipment-batch", "RETRO-CLOCK", 100,
                           eta=tomorrow)
    line = OrderLine("oref", "RETRO-CLOCK", 10)

    allocate(line, [in_stock_batch, shipment_batch])

    assert in_stock_batch.available_quantity == 90
    assert shipment_batch.available_quantity == 100

# тест сервисного слоя:
def test_prefers_warehouse_batches_to_shipments():
```

```
in_stock_batch = Batch("in-stock-batch", "RETRO-CLOCK", 100,
                        eta=None)
shipment_batch = Batch("shipment-batch", "RETRO-CLOCK", 100,
                       eta=tomorrow)
repo = FakeRepository([in_stock_batch, shipment_batch])
session = FakeSession()

line = OrderLine('oref', "RETRO-CLOCK", 10)

services.allocate(line, repo, session)

assert in_stock_batch.available_quantity == 90
assert in_stock_batch.available_quantity == 100
```

Зачем это делать?

По идее тесты должны помогать нам без опаски менять систему, но мы часто видим, как разработчики пишут слишком много тестов для своей модели предметной области. Из-за этого появляются проблемы, когда они приступают к изменениям кодовой базы и обнаруживают, что нужно обновить десятки или даже сотни юнит-тестов.

Это имеет смысл, если забыть о предназначении автоматизированных тестов. Мы используем тесты, чтобы убедиться, что свойство системы не изменится во время работы. С помощью тестов мы проверяем, что API продолжает возвращать значение 200, что сеанс базы данных продолжает фиксировать транзакции и что заказы по-прежнему размещаются.

Если мы случайно изменим одну из этих форм поведения, то тесты сломаются. С другой стороны, если мы хотим изменить дизайн кода, то любые тесты, полагающиеся непосредственно на этот код, также перестанут работать.

По ходу чтения вы увидите, как сервисный слой формирует API системы, которой мы можем управлять многочисленными способами. Тестирование с помощью этого API уменьшает объем кода, в который нужно вносить изменения при рефакторинге модели предметной области. Если мы ограничимся тестированием только в сервисном слое, то у нас не будет тестов, которые непосредственно взаимодействуют с «приватными» методами или атрибутами модельных объектов, что дает нам больше свободы для их переработки.



Каждая строка кода, которую мы помещаем в тест, удерживает систему в определенной форме, словно капля клея. Чем больше будет низкоуровневых тестов, тем труднее будет что-то изменить.

Какие тесты писать

Вы можете спросить: «И что, теперь я должен переписывать все свои юнит-тесты? Что плохого в том, чтобы писать тесты для модели предметной области?» Чтобы ответить на эти вопросы, важно понять компромисс между связанностью и обратной связью при проектировании (рис. 5.1).

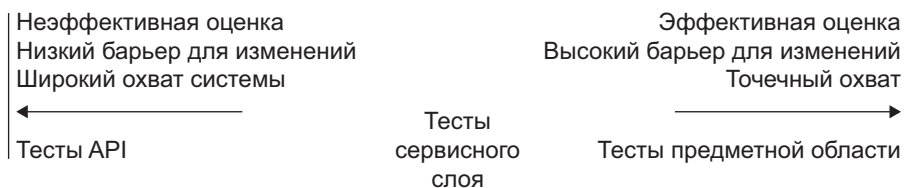


Рис. 5.1. Тестовый спектр

Экстремальное программирование (extreme programming, XP) призывает нас «прислушиваться к коду». Когда мы пишем тесты, то можем обнаружить, что код трудно использовать или же у него появился «душок». Это сигнал к тому, чтобы сделать рефакторинг и пересмотреть дизайн.

При этом мы можем заметить такие вещи только тогда, когда тесно работаем с целевым кодом. Тест для HTTP API ничего не говорит нам о мелких деталях дизайна объектов, потому что находится на гораздо более высоком уровне абстракции.

С другой стороны, даже если полностью переписать приложение и при этом не менять его URL или форматы запросов, то тесты HTTP все равно будут пройдены. Благодаря этому можно удостовериться, что крупномасштабные изменения, такие как изменение схемы базы данных, не нарушили код.

На другом конце спектра тесты, описанные в главе 1, помогли нам лучше понять необходимые объекты. С помощью этих тестов мы пришли к разумному дизайну, который использует язык предметной области. Когда тесты читаются на языке предметной области, мы уверены в том, что код соответствует нашему интуитивному пониманию задачи, которую нужно решить.

Поскольку тесты написаны на языке предметной области, они действуют как живая документация для модели. Новый участник команды прочтет

эти тесты и быстро поймет характер работы системы и взаимосвязь ее ключевых элементов.

Мы часто «делаем набросок» новых форм поведения, составляя тесты на этом уровне, чтобы увидеть, как будет выглядеть код. Но когда мы хотим улучшить дизайн кода, эти тесты нужно будет переписать или удалить, так как они тесно связаны с конкретной реализацией.

Повышенная и пониженная передачи

В большинстве случаев когда мы добавляем новую функцию или исправляем баг, сильно менять модель предметной области не приходится. В этих случаях мы предпочитаем писать тесты для служб по причине более низкой связанности и более широкого охвата.

Например, при добавлении функции `add_stock` или `cancel_order` мы можем работать быстрее и с меньшей связанностью, написав тесты для сервисного слоя.

Когда мы начинаем новый проект или сталкиваемся с особенно сложной задачей, то возвращаемся к написанию тестов для модели предметной области. Так можно лучше оценить свои планы и получить рабочий вариант документации.

Мы используем метафору переключения передач. Велосипед начинает двигаться на пониженной передаче, чтобы преодолеть инерцию. Затем можно ехать быстрее и эффективнее, переключаясь на повышенную передачу; но если мы вдруг сталкиваемся с крутым холмом или вынуждены замедлиться, то снова переключаемся на пониженную передачу, пока снова не наберем скорость.

Устранение связей между тестами сервисного слоя и предметной областью

В тестах сервисного слоя по-прежнему есть прямые зависимости от предметной области, потому что мы используем объекты предметной области для настройки тестовых данных и вызова функций сервисного слоя.

Для того чтобы иметь полностью не связанный с предметной областью сервисный слой, нужно переписать его API, чтобы работать с примитивами.

Пока что сервисный слой принимает объект предметной области `OrderLine`.

До: функция `allocate` принимает объект предметной области (`service_layer/services.py`)

```
def allocate(line: OrderLine, repo: AbstractRepository, session) -> str:
```

Как бы это выглядело, если бы все его параметры были примитивными типами?

После: функция `allocate` принимает строковые и целочисленные значения (`service_layer/services.py`)

```
def allocate(
    orderid: str, sku: str, qty: int, repo: AbstractRepository, session
) -> str:
```

Мы также переписываем тесты, чтобы в них были примитивы.

В тестах теперь используются примитивы при вызове функции (`tests/unit/test_services.py`)

```
def test_returns_allocation():
    batch = model.Batch("batch1", "COMPLICATED-LAMP", 100, eta=None)
    repo = FakeRepository([batch])

    result = services.allocate("o1", "COMPLICATED-LAMP", 10, repo,
                               FakeSession())
    assert result == "batch1"
```

Но тесты все еще зависят от предметной области, потому что мы по-прежнему вручную создаем экземпляры объектов `Batch`. И если однажды мы решим провести массовый рефакторинг работы модели партий товара `Batch`, то придется изменить кучу тестов.

Выход: сохраняйте все зависимости предметной области в фикстурах

Можно как минимум перенести все зависимости теста во вспомогательную функцию, или фикстуру. Ниже показано, как можно это сделать, добавив фабричную функцию, определенную в классе `FakeRepository`.

Фабричные функции для фикстур — это один из способов (tests/unit/test_services.py)

```
class FakeRepository(set):

    @staticmethod
    def for_batch(ref, sku, qty, eta=None):
        return FakeRepository([
            model.Batch(ref, sku, qty, eta),
        ])
    ...

def test_returns_allocation():
    repo = FakeRepository.for_batch("batch1", "COMPLICATED-LAMP",
                                     100, eta=None)
    result = services.allocate("o1", "COMPLICATED-LAMP", 10, repo,
                               FakeSession())
    assert result == "batch1"
```

По меньшей мере это переместило бы все зависимости тестов от предметной области в одно конкретное место.

Добавление отсутствующей службы

Но можно пойти еще дальше. Если бы у нас была служба для добавления товарных запасов, то мы могли бы использовать ее и выразить тесты сервисного слоя с помощью общепринятых вариантов использования, убрав все зависимости от предметной области.

Тест для новой службы add_batch (tests/unit/test_services.py)

```
def test_add_batch():
    repo, session = FakeRepository([]), FakeSession()
    services.add_batch("b1", "CRUNCHY-ARMCHAIR", 100, None, repo, session)
    assert repo.get("b1") is not None
    assert session.committed
```



Если вы обнаружите, что вам приходится делать что-то связанное со слоем предметной области непосредственно в тестах сервисного слоя, возможно, ваш сервисный слой недоработан.

И реализация занимает всего пару строк кода.

Новая служба для add_batch (service_layer/services.py)

```
def add_batch(
    ref: str, sku: str, qty: int, eta: Optional[date],
    repo: AbstractRepository, session,
):
    repo.add(model.Batch(ref, sku, qty, eta))
    session.commit()

def allocate(
    orderid: str, sku: str, qty: int, repo: AbstractRepository,
    session
) -> str:
    ...
```



Стоит ли добавлять новую службу только потому, что она поможет устранить зависимости из тестов? Скорее всего, нет. Но в данном случае нам почти наверняка в один прекрасный день понадобится служба `add_batch`.

Теперь можно переписать все тесты сервисного слоя исключительно с точки зрения самих служб, используя только примитивы без каких-либо зависимостей от модели.

В тестах служб теперь используются только службы (tests/unit/test_services.py)

```
def test_allocate_returns_allocation():
    repo, session = FakeRepository([]), FakeSession()
    services.add_batch("batch1", "COMPLICATED-LAMP", 100, None, repo,
    session)
    result = services.allocate("o1", "COMPLICATED-LAMP", 10, repo,
    session)
    assert result == "batch1"

def test_allocate_errors_for_invalid_sku():
    repo, session = FakeRepository([]), FakeSession()
    services.add_batch("b1", "AREALSKU", 100, None, repo, session)

    with pytest.raises(services.InvalidSku, match="Недопустимый
    артикул NONEXISTENTSKU"):
        services.allocate("o1", "NONEXISTENTSKU", 10, repo,
        FakeSession())
```

Теперь все действительно выглядит приятно. Тесты сервисного слоя зависят только от него самого, что дает нам полную свободу для рефакторинга модели по своему усмотрению.

Дальнейшее улучшение с помощью сквозных тестов

Когда мы добавили `add_batch`, это помогло устранить связанность тестов сервисного слоя с моделью. Точно так же если добавить конечную точку API для добавления партии товара, то фикстура `add_stock` больше не понадобится, а сквозные тесты освободятся от этих жестко закодированных SQL-запросов и прямой зависимости от базы данных.

Благодаря функции службы добавить конечную точку очень просто, требуется лишь немного работы с JSON и всего один вызов функции.

API для добавления партии товара (`entrypoints/flask_app.py`)

```
@app.route("/add_batch", methods=['POST'])
def add_batch():
    session = get_session()
    repo = repository.SqlAlchemyRepository(session)
    eta = request.json['eta']
    if eta is not None:
        eta = datetime.fromisoformat(eta).date()
    services.add_batch(
        request.json['ref'], request.json['sku'], request.json['qty'],
        eta,
        repo, session
    )
    return 'OK', 201
```



Вы наверняка думаете: «Отправка методом POST в `/add_batch`? Но это не совсем в стиле RESTful!» И будете совершенно правы. Мы несколько небрежны, но если вы хотите сделать все в стиле RESTful, то при отправке методом POST в `/batches` Flask вырубится! Поскольку Flask — это тонкий адаптер (простая абстракция), такое вполне вероятно. См. следующую врезку.

И жестко закодированные SQL-запросы из `conftest.py` заменяются несколькими вызовами API, а значит, тесты API не имеют никаких зависимостей, кроме как от API, что не может не радовать.

Тесты API теперь могут добавлять свои собственные партии (tests/e2e/test_api.py)

```
def post_to_add_batch(ref, sku, qty, eta):
    url = config.get_api_url()
    r = requests.post(
        f'{url}/add_batch',
        json={'ref': ref, 'sku': sku, 'qty': qty, 'eta': eta}
    )
    assert r.status_code == 201

@pytest.mark.usefixtures('postgres_db')
@pytest.mark.usefixtures('restart_api')
def test_happy_path_returns_201_and_allocated_batch():
    sku, othersku = random_sku(), random_sku('other')
    earlybatch = random_batchref(1)
    laterbatch = random_batchref(2)
    otherbatch = random_batchref(3)
    post_to_add_batch(laterbatch, sku, 100, '2011-01-02')
    post_to_add_batch(earlybatch, sku, 100, '2011-01-01')
    post_to_add_batch(otherbatch, othersku, 100, None)
    data = {'orderid': random_orderid(), 'sku': sku, 'qty': 3}
    url = config.get_api_url()
    r = requests.post(f'{url}/allocate', json=data)
    assert r.status_code == 201
    assert r.json()['batchref'] == earlybatch
```

Выводы

Создав сервисный слой, вы действительно можете перенести большую часть тестового охвата на юнит-тесты и разработать правильную пирамиду тестирования.

Вот что вам может помочь:

- Выражайте сервисный слой с помощью примитивов, а не объектов предметной области.
- В идеальном мире у вас будут все нужные службы для полного тестирования сервисного слоя, и вам не придется взламывать состояние через репозитории или базу данных. Это окупается и в сквозных тестах.

Вперед к следующей главе!

ЭМПИРИЧЕСКИЕ ПРАВИЛА ДЛЯ РАЗНЫХ ТИПОВ ТЕСТОВ

Стремитесь к одному сквозному тесту на одну функцию

К примеру, тест может быть написан для HTTP API. Цель состоит в том, чтобы продемонстрировать, что функциональное свойство работает и что все движущиеся части «склеены» правильно.

Пишите основную часть тестов для сервисного слоя

Все эти сквозные тесты предлагают хороший компромисс между охватом, временем выполнения и эффективностью. Каждый тест охватывает один кодовый путь функционального свойства и использует подделки для ввода-вывода. Это именно то место, где следует исчерпывающе охватить все крайние случаи и входы и выходы вашей бизнес-логики¹.

Поддерживайте малое ядро тестов, написанных для модели предметной области

Эти тесты имеют точечный охват и являются более хрупкими, но при этом дают самую эффективную оценку. Не бойтесь удалять эти тесты, если функциональность позже будет охвачена тестами в сервисном слое.

Обработка ошибок рассматривается как функциональное свойство

В идеале ваше приложение должно быть структурировано таким образом, чтобы все ошибки, которые всплывают во входных точках (например, в Flask), обрабатывались одинаково. Из этого следует, что по каждому функциональному свойству нужно тестировать только счастливый путь и оставлять один сквозной тест для всех несчастливых путей (и разумеется, много юнит-тестов для несчастливых путей).

¹ Некоторые справедливо отмечают, что когда речь идет о более сложных бизнес-схемах, высокоуровневое тестирование может привести к комбинаторному взрыву. В этих случаях можно попробовать перейти к юнит-тестам более низкого уровня для различных взаимодействующих объектов предметной области. Но см. также главу 8 и раздел «Не обязательно: юнит-тестирование обработчиков событий в изоляции с помощью поддельной шины сообщений» на с. 196.

Паттерн UoW

В этой главе мы рассмотрим последний кусочек пазла, который связывает паттерны «Репозиторий» и «Сервисный слой» воедино: паттерн UoW (Unit of Work — «единица работы»).

Если паттерн «Репозиторий» — это абстракция идеи постоянного хранения данных, то паттерн UoW — это абстракция идеи *атомарных операций*. Он позволит окончательно устранить связанность сервисного слоя со слоем данных.

На рис. 6.1 показано, что сейчас между нашими слоями инфраструктуры происходит интенсивный обмен сообщениями: API напрямую связывается со слоем базы данных для запуска сеанса, со слоем репозитория для инициализации `SQLAlchemyRepository` и с сервисным слоем, чтобы попросить его выполнить операцию размещения.



Код для этой главы находится в ветке `chapter_06_uow` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_06_uow
# или, если пишете код по ходу чтения, возьмите за основу
# материал из главы 4:
git checkout chapter_04_service_layer
```

На рис. 6.2 изображена схема нашей конечной идеи. API фреймворка Flask теперь делает только две вещи: инициализирует UoW и вызывает службу. Служба сотрудничает с паттерном UoW (нам нравится представлять его как часть сервисного слоя), но ни сама функция службы, ни Flask теперь не нуждаются в непосредственном обмене с базой данных.

И мы сделаем все это, применив прекрасный синтаксический элемент языка Python — контекстный менеджер.

¹ См. <https://oreil.ly/MoWdZ>

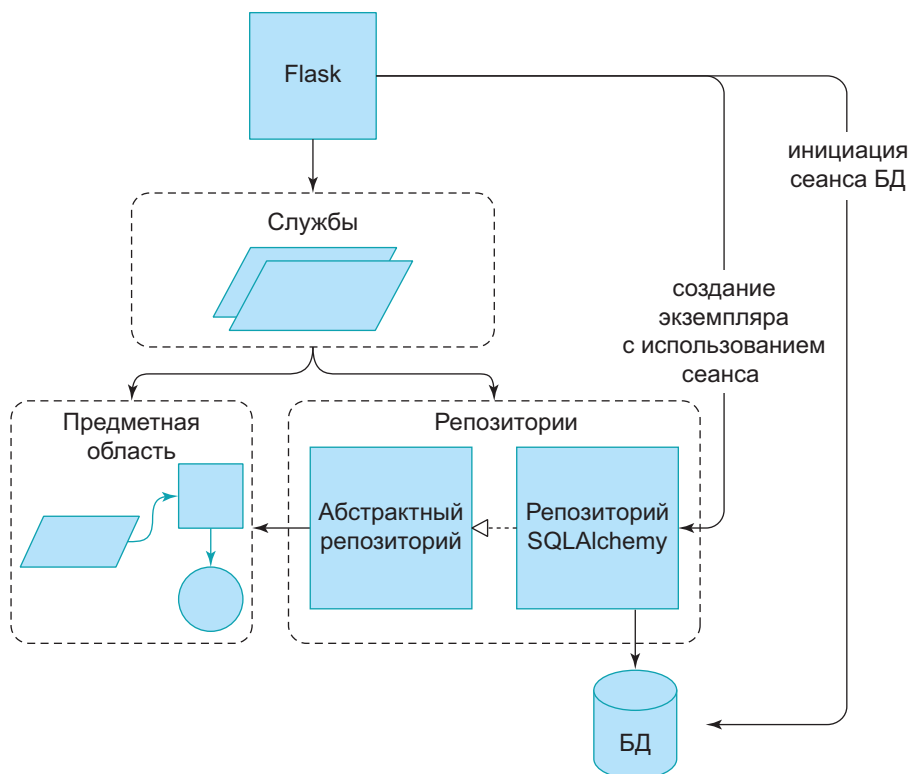


Рис. 6.1. Без паттерна UoW: API связывается непосредственно с тремя слоями

Паттерн UoW работает с репозиторием

Давайте посмотрим на UoW в действии. Вот как будет выглядеть сервисный слой, когда мы закончим:

Предварительный обзор UoW в действии (src/allocation/service_layer/services.py)

```
def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow: ❶
        batches = uow.batches.list() ❷
```

```
...  
batchref = model.allocate(line, batches)  
uow.commit() ❸
```

- ❶ Мы запустим UoW как контекстный менеджер.
- ❷ `uow.batches` — это репозиторий партий товара, так что паттерн UoW предоставляет нам доступ к системе постоянного хранения.
- ❸ В конце мы делаем коммит или откат, используя паттерн UoW.

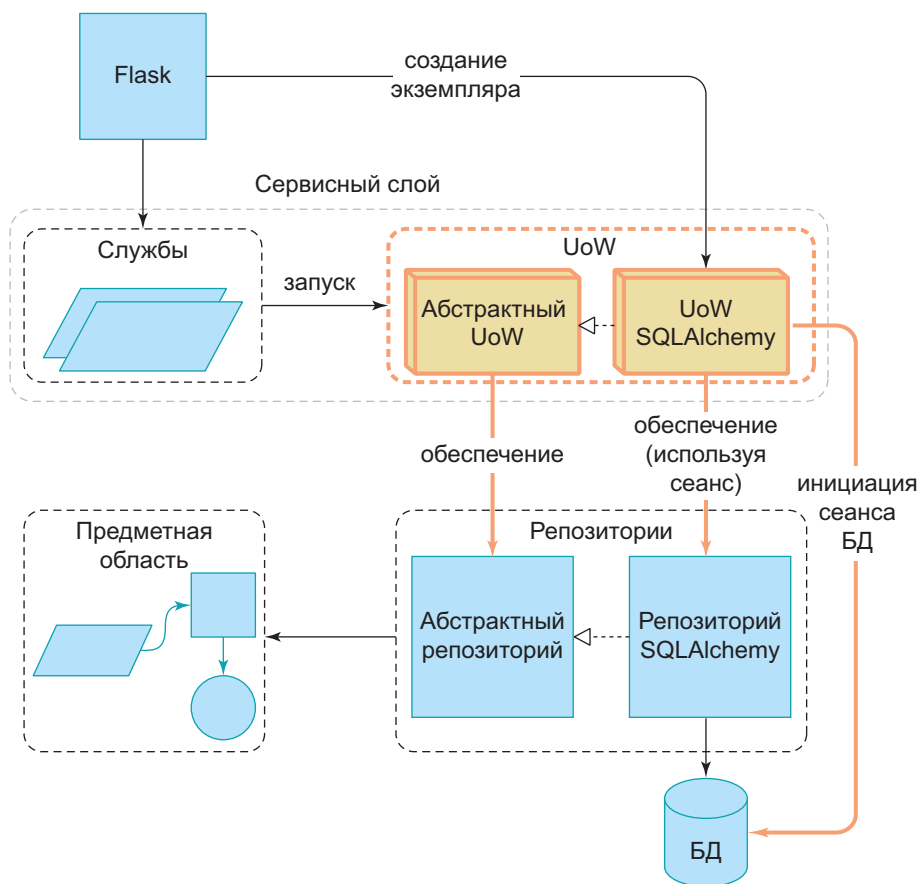


Рис. 6.2. С паттерном UoW: теперь UoW управляет состоянием базы данных

Паттерн UoW действует как единая точка входа в систему постоянного хранения данных и отслеживает загрузку объектов и их последнее состояние¹.

Это дает нам три полезные вещи:

- стабильный моментальный снимок базы данных для работы, так что объекты, которые мы используем, не меняются на полпути;
- способ сохранять все изменения сразу, благодаря чему мы не окажемся в состоянии несогласованности, если что-то пойдет не так;
- простой API для обязанностей по обеспечению постоянства данных и удобное место для получения репозитория.

Тестирование UoW интеграционными тестами

Вот интеграционные тесты для паттерна UoW.

Базовый тест «туда-обратно» для паттерна UoW (tests/integration/test_uow.py):

```
def test_uow_can_retrieve_a_batch_and_allocate_to_it(session_factory):
    session = session_factory()
    insert_batch(session, 'batch1', 'HIPSTER-WORKBENCH', 100, None)
    session.commit()

    uow = unit_of_work.SqlAlchemyUnitOfWork(session_factory) ❶
    with uow:
        batch = uow.batches.get(reference='batch1') ❷
        line = model.OrderLine('o1', 'HIPSTER-WORKBENCH', 10)
        batch.allocate(line)
        uow.commit() ❸

    batchref = get_allocated_batch_ref(session, 'o1', 'HIPSTER-WORKBENCH')
    assert batchref == 'batch1'
```

❶ Инициализируем паттерн UoW с помощью пользовательской фабрики сеансов и возвращаем объект `uow` для использования в блоке `with`.

¹ В английском языке объекты, которые работают вместе, порой называют коллабораторами (collaborators), т. е. сотрудничающими. UoW и «Репозиторий» являются прекрасным примером сотрудничества в плане объектного моделирования. В проектировании на основе обязанностей кластеры объектов, которые взаимодействуют в своих ролях, называются *объектным соседством* (object neighborhood), что, по нашему профессиональному мнению, совершенно восхитительно.

❷ Паттерн UoW дает доступ к репозиторию партий товара через `uow.batches`.

❸ Когда дело сделано, мы вызываем для него метод `commit()`.

Для любопытных: помощники `insert_batch` и `get_allocated_batch_ref` выглядят следующим образом:

Помощники для выполнения всего, что связано с SQL (`tests/integration/test_uow.py`)

```
def insert_batch(session, ref, sku, qty, eta):
    session.execute(
        'INSERT INTO batches (reference, sku, _purchased_quantity, eta)'
        ' VALUES (:ref, :sku, :qty, :eta)',
        dict(ref=ref, sku=sku, qty=qty, eta=eta)
    )

def get_allocated_batch_ref(session, orderid, sku):
    [[orderid]] = session.execute(
        'SELECT id FROM order_lines WHERE orderid=:orderid AND'
        ' sku=:sku',
        dict(orderid=orderid, sku=sku)
    )
    [[batchref]] = session.execute(
        'SELECT b.reference FROM allocations JOIN batches AS b ON'
        ' batch_id = b.id'
        ' WHERE orderline_id=:orderid',
        dict(orderline_id=orderid)
    )
    return batchref
```

UoW и его контекстный менеджер

В тестах мы неявно определили интерфейс для работы, которую должен выполнять паттерн UoW. Давайте укажем это явным образом с помощью абстрактного базового класса.

Абстрактный контекстный менеджер UoW (`src/allocation/service_layer/unit_of_work.py`)

```
class AbstractUnitOfWork(abc.ABC):
    batches: repository.AbstractRepository ❶

    def __exit__(self, *args): ❷
        self.rollback() ❸
```

```

@abc.abstractmethod
def commit(self): ❸
    raise NotImplementedError

@abc.abstractmethod
def rollback(self): ❹
    raise NotImplementedError

```

❶ UoW предоставляет атрибут `.batches`, который обеспечит доступ к репозиторию партий товара.

❷ Если вы никогда не видели контекстный менеджер, то вот два волшебных метода — `__enter__` и `__exit__`, которые выполняются соответственно при входе в блок `with` и при выходе из него. Это фазы наладки и демонтажа.

❸ Вызовем этот метод, чтобы явно зафиксировать работу, когда мы будем готовы.

❹ Если мы не выполняем фиксацию или выходим из контекстного менеджера, инициировав ошибку, то выполняем откат. (Откат ни к чему не приводит, если была вызвана фиксация `commit()`. Далее в книге этот вопрос обсуждается подробнее.)

Настоящий UoW использует сеансы SQLAlchemy

В конкретной реализации добавлено главное, и это сеанс работы с базой данных.

Настоящий UoW с SQLAlchemy (src/allocation/service_layer/unit_of_work.py)

```

DEFAULT_SESSION_FACTORY = sessionmaker(bind=create_engine( ❶
    config.get_postgres_uri(),
))

class SQLAlchemyUnitOfWork(AbstractUnitOfWork):
    def __init__(self, session_factory=DEFAULT_SESSION_FACTORY):
        self.session_factory = session_factory ❶

    def __enter__(self):
        self.session = self.session_factory() # тип: Session ❷
        self.batches = repository.SqlAlchemyRepository(self.session) ❷
        return super().__enter__()

    def __exit__(self, *args):
        super().__exit__(*args)
        self.session.close() ❸

```

```
def commit(self): ❹
    self.session.commit()

def rollback(self): ❹
    self.session.rollback()
```

❶ В указанном модуле определяется фабрика сеансов по умолчанию, которая будет подключаться к базе данных Postgres, но мы позволяем их переопределять в интеграционных тестах, чтобы вместо нее можно было использовать SQLite.

❷ Метод `__enter__` отвечает за запуск сеанса базы данных и создание экземпляра реального репозитория, который может пользоваться этим сеансом.

❸ Закрываем сеанс на выходе.

❹ Наконец, мы добавляем конкретные методы `commit()` и `rollback()`, в которых используется сеанс базы данных.

Поддельный UoW для тестирования

Вот как мы используем поддельный UoW в тестах сервисного слоя:

Поддельный UoW (tests/unit/test_services.py)

```
class FakeUnitOfWork(unit_of_work.AbstractUnitOfWork):
```

```
    def __init__(self):
        self.batches = FakeRepository([]) ❶
        self.committed = False ❷

    def commit(self):
        self.committed = True ❷

    def rollback(self):
        pass

    def test_add_batch():
        uow = FakeUnitOfWork() ❸
        services.add_batch("b1", "CRUNCHY-ARMCHAIR", 100, None, uow) ❸
        assert uow.batches.get("b1") is not None
        assert uow.committed
```

```
def test_allocate_returns_allocation():
    uow = FakeUnitOfWork() ❸
    services.add_batch("batch1", "COMPLICATED-LAMP", 100, None, uow) ❸
    result = services.allocate("o1", "COMPLICATED-LAMP", 10, uow) ❸
    assert result == "batch1"
...

```

❶ FakeUnitOfWork и FakeRepository тесно связаны, как и настоящие классы UnitOfWork и Repository. Это прекрасно, потому что мы признаем, что объекты являются коллабораторами.

❷ Обратите внимание на сходство с поддельной функцией commit() из FakeSession (от которой мы теперь можем избавиться). Но это существенное улучшение, потому что теперь вместо стороннего кода мы подделываем собственный. Некоторые говорят: «Не владеешь — не имитируй»¹.

❸ В тестах можно создать экземпляр паттерна UoW и передавать его в сервисный слой, вместо того чтобы передавать репозиторий и сеанс. Такой способ гораздо легче.

Использование паттерна UoW в сервисном слое

Вот как выглядит новый сервисный слой:

Сервисный слой с паттерном UoW (src/allocation/service_layer/services.py)

```
def add_batch(
    ref: str, sku: str, qty: int, eta: Optional[date],
    uow: unit_of_work.AbstractUnitOfWork ❶
):
    with uow:
        uow.batches.add(model.Batch(ref, sku, qty, eta))
        uow.commit()

def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork ❶
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow:
        batches = uow.batches.list()

```

¹ Don't mock what you don't own. См. <https://oreil.ly/OLVj3>

```
if not is_valid_sku(line.sku, batches):
    raise InvalidSku(f'Недопустимый артикул {line.sku}')
batchref = model.allocate(line, batches)
uow.commit()
return batchref
```

❶ Сервисный слой теперь имеет только одну зависимость, опять же от абстрактного паттерна UoW.

НЕ ВЛАДЕЕШЬ — НЕ ИМИТИРУЙ

Почему нам проще имитировать UoW, а не сеанс? Обе подделки достигают одного и того же — позволяют изымать слой обеспечения постоянства данных, благодаря чему можно выполнять тесты прямо в памяти, вместо того чтобы связываться с реальной базой данных. Разница заключается в итоговом дизайне.

Если бы мы заботились только о написании быстрых тестов, то могли бы создать имитации, которые заменяют SQLAlchemy, и использовать их во всей кодовой базе. Проблема в том, что `Session` — это сложный объект, который содержит большой объем функциональности, связанной с обеспечением постоянства. `Session` легко использовать для выполнения произвольных запросов к базе данных, но это быстро приводит к тому, что код доступа к данным хаотично разбрасывается по всей кодовой базе. Чтобы этого не происходило, мы хотим ограничить доступ к слою обеспечения постоянства данных, чтобы каждый компонент обладал именно тем, что ему нужно, и ничем больше.

Устанавливая связь с интерфейсом `Session`, вы соглашаетесь на связанность со всей сложностью SQLAlchemy. Вместо этого лучше выбрать более простую абстракцию и использовать ее для четкого разделения обязанностей. Паттерн UoW намного проще, чем сеанс, и мы чувствуем себя комфортно с сервисным слоем, который способен запускать и останавливать UoW.

«Не владеешь — не имитируй» — это эмпирическое правило, которое заставляет нас строить простые абстракции над беспорядочными подсистемами. Этот подход дает такое же преимущество в производительности, что и имитация сеанса SQLAlchemy, но при этом побуждает нас тщательно обдумывать проекты.

Явные тесты для форм поведения по фиксации/откату

Для того чтобы убедиться, что формы поведения по фиксации/откату работают как надо, мы написали несколько тестов.

Интеграционные тесты для поведения по откату (tests/integration/test_uow.py)

```
def test_rolls_back_uncommitted_work_by_default(session_factory):
    uow = unit_of_work.SqlAlchemyUnitOfWork(session_factory)
    with uow:
        insert_batch(uow.session, 'batch1', 'MEDIUM-PLINTH', 100, None)

    new_session = session_factory()
    rows = list(new_session.execute('SELECT * FROM "batches"'))
    assert rows == []

def test_rolls_back_on_error(session_factory):
    class MyException(Exception):
        pass

    uow = unit_of_work.SqlAlchemyUnitOfWork(session_factory)
    with pytest.raises(MyException):
        with uow:
            insert_batch(uow.session, 'batch1', 'LARGE-FORK', 100, None)
            raise MyException()

    new_session = session_factory()
    rows = list(new_session.execute('SELECT * FROM "batches"'))
    assert rows == []
```



В нашем примере этого нет, но возможно, стоит протестировать пару-тройку не столь понятных действий базы данных, таких как транзакции, относительно «реальной» базы данных, то есть того же самого движка. Пока что нам хватает SQLite, но в главе 7 мы переключим некоторые тесты на использование реальной базы данных. Здорово, что класс UoW позволяет нам с легкостью сделать это!

Явные и неявные фиксации

Кратко остановимся на разных способах реализации паттерна UoW.

Можно представить себе немного другую версию UoW, которая по умолчанию выполняет фиксацию и откатывает только в том случае, если обнаруживает исключение.

Паттерн UoW с неявной фиксацией... (src/allocation/unit_of_work.py)

```
class AbstractUnitOfWork(abc.ABC):
```

```
    def __enter__(self):  
        return self
```

```
    def __exit__(self, exn_type, exn_value, traceback):  
        if exn_type is None:  
            self.commit() ❶  
        else:  
            self.rollback() ❷
```

❶ Должны ли мы иметь неявную фиксацию на счастливом пути?

❷ И откатывать только при исключении?

Это позволило бы не писать лишнюю строчку и удалить явную фиксацию из клиентского кода.

...Мы могли бы не писать лишнюю строчку (src/allocation/service_layer/services.py)

```
def add_batch(ref: str, sku: str, qty: int, eta: Optional[date], uow):  
    with uow:  
        uow.batches.add(model.Batch(ref, sku, qty, eta))  
        # uow.commit()
```

Это остается на ваше усмотрение, но мы предпочитаем запрос явной фиксации, чтобы выбор времени сброса состояния оставался за нами.

Да, у нас здесь дополнительная строка кода, зато с ней программа по умолчанию становится безопасной. По умолчанию принято *ничего не менять*. В свою очередь, код становится более обоснованным, потому что есть только один путь, который приводит к изменениям в системе: полный успех и явная фиксация работы. Любой другой путь, любое исключение, любой ранний выход из области видимости паттерна UoW приводит к безопасному состоянию.

Точно так же мы предпочитаем делать откат по умолчанию, потому что такой принцип легче понять; код откатывается до последней фиксации, поэтому либо пользователь ее сделал, либо мы удаляем его изменения. Четко и ясно.

Примеры: использование паттерна UoW для группировки многочисленных операций в атомарную единицу

Вот несколько примеров, показывающих паттерн UoW в деле. Хорошо видно, как с помощью него можно легко понять, какие блоки кода работают.

Пример 1. Повторное размещение

Предположим, что мы хотим иметь возможность отменять размещение, а затем размещать заказы повторно.

Функция службы повторного размещения

```
def reallocate(line: OrderLine, uow: AbstractUnitOfWork) -> str:
    with uow:
        batch = uow.batches.get(sku=line.sku)
        if batch is None:
            raise InvalidSku(f'Недопустимый артикул {line.sku}')
        batch.deallocate(line) ❶
        allocate(line)
        uow.commit()
```

❶ Если функция `deallocate()` не срабатывает, то мы, очевидно, не хотим вызывать функцию `allocate()`.

❷ Если функция `allocate()` не срабатывает, то мы, вероятно, также не хотим фактически фиксировать `deallocate()`.

Пример 2. Изменение количества товаров в партии

Нам звонят из судоходной компании и сообщают, что один из контейнеров открылся и половина наших диванов утонула в Индийском океане. Ой!

Изменить количество

```
def change_batch_quantity(batchref: str, new_qty: int, uow:
    AbstractUnitOfWork):
    with uow:
        batch = uow.batches.get(reference=batchref)
        batch.change_purchased_quantity(new_qty)
```

```
while batch.available_quantity < 0:  
    line = batch.deallocate_one() ❶  
uow.commit()
```

❶ Здесь, возможно, понадобится отменять размещение какого-либо числа товарных позиций заказа. Если на каком-нибудь этапе программа не работает, то, вероятно, не захотим фиксировать никаких изменений.

Приведение в порядок интеграционных тестов

Теперь у нас есть три набора тестов, все они, по существу, указывают на базу данных: `test_orm.py`, `test_repository.py` и `test_uow.py`. Может, стоит от них избавиться?

```
├─ tests  
│   ├── conftest.py  
│   ├── e2e  
│   │   └─ test_api.py  
│   ├── integration  
│   │   ├── test_orm.py  
│   │   ├── test_repository.py  
│   │   └─ test_uow.py  
│   ├── pytest.ini  
│   └─ unit  
│       ├── test_allocate.py  
│       ├── test_batches.py  
│       └─ test_services.py
```

Всегда нужно без колебаний избавляться от тестов, если вы считаете, что в долгосрочной перспективе они не будут приносить пользу, «добавочную стоимость». Мы бы сказали, что `test_orm.py` был в первую очередь вспомогательным инструментом для изучения работы SQLAlchemy, так что надолго он нам не понадобится, особенно если главная работа, которую он делает, охвачена `test_repository.py`. Последний тест вы можете оставить, но мы определенно убедимся, что лучше просто сохранить все на максимально возможном уровне абстракции (как это делалось для юнит-тестов).

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

В этой главе лучше всего попробовать реализовать паттерн UoW с нуля. Код, как всегда, находится на GitHub¹. Вы могли бы либо строго придерживаться имеющейся модели, либо поэкспериментировать с отделением паттерна (чья обязанности заключаются в фиксации `commit()`, откате `rollback()` и предоставлении репозитория партий товара `.batches`) от контекстного менеджера, чья работа состоит в инициализировании, а затем выполнении фиксации либо отката на выходе. Если вы хотите сконцентрироваться на функциях, а не возиться со всеми этими классами, то можете воспользоваться `@contextmanager` из `contextlib`.

Мы удалили настоящий паттерн UoW, равно как и подделки, а также сократили абстрактный паттерн UoW. Пришлите нам ссылку на ваш репозиторий, если вы придумали что-то, чем особенно гордитесь!



Очередное доказательство идеи из главы 5: если у нас есть хорошая абстракция, мы можем писать тесты именно для нее, что позволяет свободно менять лежащие в основе детали.

Выводы

Надеюсь, мы убедили вас в полезности паттерна UoW и в том, что контекстный менеджер — это и вправду хороший питоновский способ визуальной группировки кода в блоки, которые мы хотим сделать атомарными.

Этот паттерн настолько полезен, что SQLAlchemy уже использует его внутри контура объекта `Session`. Объект `Session` в SQLAlchemy показывает то, как приложение загружает данные из БД.

Всякий раз, когда вы загружаете новую сущность из базы данных, сеанс начинает отслеживать изменения в сущности, и когда сеанс сбрасывается, все ваши изменения сохраняются. Зачем пытаться абстрагировать сеанс SQLAlchemy, если в нем уже реализован нужный нам паттерн? В табл. 6.1 приведены преимущества и недостатки паттерна UoW.

¹ См. https://github.com/cosmicpython/code/tree/chapter_06_uow_exercise

Таблица 6.1. Паттерн UoW: компромиссы

Плюсы	Минусы
• Есть хорошая абстракция над концепцией атомарных операций, и контекстный менеджер позволяет наглядно видеть атомарную группировку блоков кода. • Есть явный контроль над временем начала и окончания транзакции, и приложение отказывает таким образом, который безопасен по умолчанию. Нам никогда не придется беспокоиться о частичном завершении операции. • Это хорошее место для размещения всех репозиторийев, куда сможет обращаться клиентский код. • Как вы увидите в последующих главах, атомарность не только касается транзакций, но и помогает нам работать с событиями и шиной сообщений	• Ваш ORM, вероятно, уже имеет несколько совершенных, хороших абстракций вокруг атомарности. В SQLAlchemy есть даже контекстные менеджеры. Можно долго ходить кругами, просто передавая сеанс. • Мы сделали так, чтобы все выглядело легко, но следует хорошенько подумать о таких вещах, как откаты, многопоточная обработка и вложенные транзакции. Возможно, вы упростите свою жизнь, просто придерживаясь того, что дает вам Django или Flask-SQLAlchemy

РЕЗЮМЕ О ПАТТЕРНЕ UOW

Паттерн UoW является абстракцией вокруг целостности данных

Он помогает обеспечивать согласованность модели предметной области и повышает производительность, позволяя выполнять одну операцию сброса в конце транзакции.

Он тесно взаимодействует с паттернами «Репозиторий» и «Сервисный слой»

Паттерн UoW завершает абстракции над доступом к данным, представляя атомарные обновления. Каждый вариант использования сервисного слоя выполняется в одном элементарном UoW, который завершается успешно или безуспешно как единый блок.

Это прекрасная возможность для применения контекстного менеджера

Контекстные менеджеры в Python являются идиоматическим способом определения области видимости. Мы можем использовать контекстный менеджер для автоматического отката работы в конце запроса. Следовательно, система по умолчанию будет безопасной.

В SQLAlchemy этот паттерн уже реализован

Мы вводим еще более простую абстракцию над объектом класса `Session` объектно-реляционного отображения SQLAlchemy, чтобы «сузить» интерфейс между ним и исходным кодом. Это способствует тому, что код остается слабо связанным.

С одной стороны, API сеанса является функционально насыщенным и поддерживает операции, которые мы не хотим видеть в нашей предметной области. Класс `UnitOfWork` упрощает сеанс до его существенного ядра: сеанс может запускаться, фиксироваться или выбрасываться.

С другой стороны, мы используем класс `UnitOfWork` для доступа к объектам класса `Repository`. Для разработчика это бесподобный элемент практичности, который мы не могли бы сделать в `SQLAlchemy` с помощью его простого класса `Session`.

Наконец, мы снова руководствуемся принципом инверсии зависимостей: сервисный слой зависит от тонкой абстракции, и мы прикрепляем конкретную реализацию к внешнему краю системы. Это прекрасно согласуется с внутренними рекомендациями `SQLAlchemy`¹.

Поддерживайте жизненный цикл сеанса (и обычно транзакции) как отдельный и внешний. При наиболее полном подходе, рекомендуемом для важных приложений, детали управления сеансами, транзакциями и исключениями удерживают как можно дальше от деталей программы, выполняющей свою работу.

Документация SQLAlchemy «Основы сеанса»

¹ См. <https://oreil.ly/tS0E0>

Агрегаты и границы согласованности

В этой главе мы хотели бы вернуться к модели предметной области, чтобы поговорить об инвариантах и ограничениях и посмотреть, как объекты предметной области могут поддерживать свою внутреннюю согласованность, как концептуально, так и в системе постоянного хранения. Обсудим понятие *границы согласованности* (consistency boundary) и покажем, как сделать ее явной, чтобы создавать высокопроизводительное и при этом легкое в сопровождении ПО.

На рис. 7.1 показано, что мы получим в итоге: мы представим новый модельный объект `Product` вокруг партий товара, а также сделаем старую профильную службу `allocate()` его методом.

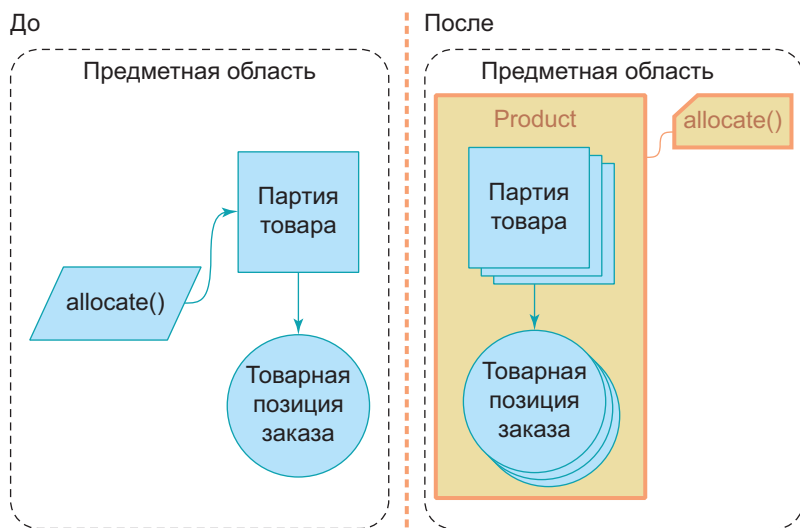


Рис. 7.1. Добавление агрегата `Product`

Зачем? Давайте выясним.



Код для этой главы находится в ветке `appendix_csvs` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout appendix_csvs
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_06_uow
```

Почему бы просто не записать все в электронную таблицу?

В любом случае, какой смысл в модели предметной области? Какую фундаментальную задачу мы пытаемся решить?

Разве нельзя записать все в электронную таблицу? Многие пользователи были бы от этого в восторге. Бизнесмены *любят* электронные таблицы — они просты, понятны и в то же время чрезвычайно мощны.

На самом деле огромным числом бизнес-процессов и вправду управляют с помощью ручной отправки таблиц по электронной почте. Такая архитектура, «CSV по SMTP», довольно проста, но ее не так-то легко масштабировать из-за трудностей в применении логики и поддержании согласованности.

У кого есть доступ на просмотр этого конкретного поля? А на его обновление? Что происходит, когда мы пытаемся заказать –350 стульев или 10 000 000 столов? Может ли сотрудник иметь отрицательную зарплату?

Таковы ограничения системы. Большая часть логики предметной области, которую мы создаем, нужна для обеспечения соблюдения этих ограничений, чтобы поддерживать инварианты системы. *Инварианты* — это то, что должно быть истинным всякий раз, когда мы заканчиваем операцию.

¹ См. <https://oreil.ly/vlnGg>

Инварианты, ограничения и согласованность

Эти два слова в некоторой степени взаимозаменяемы, но *ограничение* (constraint) — это правило, лимитирующее возможные состояния модели, в то время как *инвариант* (invariant) определяется немного точнее: это условие, которое всегда является истинным.

Если бы мы создавали систему бронирования номеров в гостинице, то установили бы ограничение, не допускающее двойного бронирования. Оно подтверждается инвариантом, что номер не может быть забронирован более одного раза на ту же ночь.

Иногда, возможно, придется временно нарушить правила, например перетасовать размещения из-за VIP-бронирования. Так мы можем оказаться в ситуации, когда номер будет забронирован дважды, но модель предметной области должна быть построена таким образом, чтобы в итоге все находилось в окончательном согласованном состоянии с соблюдением инвариантов. Если разместить всех гостей не получается, то следует инициировать ошибку и отказаться от завершения операции.

Давайте рассмотрим несколько конкретных примеров из бизнес-требований. Начнем с такого.

Товарная позиция заказа может размещаться только в одной партии товара за раз.

Бизнес

Это бизнес-требование накладывает инвариант. Он заключается в том, что товарная позиция заказа размещается в одной партии товара либо вообще не размещается. Других опций не предусмотрено. Нам нужно, чтобы код никогда случайно не вызывал `Batch.allocate()` на двух разных партиях для одной и той же товарной позиции заказа. Пока что нет ничего, что явно помешало бы нам это сделать.

Инварианты, конкурентность и блокировки

Давайте рассмотрим еще одно требование.

Мы не можем размещать заказ в партии, если доступное количество товара в ней меньше количества товарной позиции заказа.

Бизнес

Здесь ограничение состоит в том, что показывать в партии больше товарных запасов, чем действительно есть в наличии. Так два клиента не смогут заказать, к примеру, одну и ту же подушку. Каждый раз, когда мы обновляем состояние системы, код должен гарантировать, что мы не нарушим инвариант, то есть доступное количество товара должно быть больше или равно нулю.

В однопоточном однопользовательском приложении этот инвариант поддерживается относительно легко. Можно просто размещать товар по одной товарной позиции за раз и инициировать ошибку, если товара нет в наличии.

Все становится намного сложнее, когда мы вводим идею *параллелизма* (concurrency). Мы, возможно, будем размещать товарные запасы в нескольких товарных позициях заказа одновременно. Может, даже будем размещать товарные позиции заказа в одно и то же время с обработкой изменений в самих партиях товара.

Обычно мы решаем эту проблему, применяя *блокировки* (locks) к нашим таблицам базы данных. Это предотвращает одновременное выполнение двух операций в одной строке или одной таблице.

А что насчет масштабирования приложения? Понятно, что модель размещения товарных позиций заказа относительно всех доступных партий товара, вероятно, не будет масштабироваться. Если мы обрабатываем десятки тысяч заказов в час и сотни тысяч товарных позиций в заказах, то не сможем заблокировать всю таблицу партий — как минимум возникнут тупиковые ситуации или проблемы с производительностью.

Что такое агрегат

Итак, блокировать всю базу данных всякий раз, когда нужно разместить товарную позицию заказа, не выйдет. И что же делать? Мы хотим защитить инварианты системы, но при этом обеспечить максимальную степень параллелизма. Соблюдение инвариантов неизбежно означает предотвращение параллельных операций записи; если несколько пользователей одновременно разместят заказ на артикул ЛОЖКА-СМЕРТОНОСНАЯ в одно и то же время, то мы рискуем выбиться из реального количества.

С другой стороны, мы вполне можем размещать заказ на артикул ЛОЖКА-СМЕРТОНОСНАЯ в то же время, что и на артикул СТОЛ-ХЛИПКИЙ. Такая ситуация безопасна, потому что нет инварианта, который охватывает оба товара. Их согласованность друг с другом не нужна.

Паттерн «Агрегат» родом из предметно-ориентированного проектирования как раз и помогает разрешить эту напряженность. *Агрегат* — это просто объект предметной области, который содержит другие ее объекты и позволяет рассматривать всю коллекцию как единое целое.

Единственный способ изменять объекты внутри агрегата — загружать его целиком и вызывать методы самого агрегата.

Чем сильнее разрастается модель и чем больше в ней появляется сущностей и объектов-значений, которые ссылаются друг на друга в запутанном графе, тем сложнее становится понять, что на что влияет. В таких ситуациях лучше назначать некоторые сущности в качестве единственной точки входа, чтобы вносить изменения в связанные с ними объекты, особенно если в модели есть коллекции, как у нас (партии товара — это коллекции). В результате система упрощается и ее становится легче понять.

Например, на коммерческом веб-сайте корзина может стать хорошим агрегатом: это набор сущностей, которые могут рассматриваться как единое целое. Важно отметить, что мы хотим загружать всю корзину из хранилища данных в виде одного большого объекта. Два запроса не должны одновременно изменять корзину, иначе есть риск вызвать странные ошибки параллелизма. Вместо этого нужно, чтобы каждое изменение корзины происходило при одной транзакции базы данных.

Нет смысла модифицировать несколько корзин в транзакции, потому что не бывает такого, чтобы корзины нескольких клиентов изменялись

одновременно. Каждая корзина представляет собой единую *границу согласованности*, отвечающую за поддержание собственных инвариантов.

Агрегат — это кластер связанных объектов, которые мы рассматриваем как единицу с целью изменения данных.

Эрик Эванс, Синяя книга по DDD

Согласно Эвансу, агрегат имеет корневую сущность (корзину), которая инкапсулирует доступ к элементам. Каждый товар можно идентифицировать отдельно, но другие части системы всегда будут ссылаться на корзину только как на неделимое целое.



Подобно тому как мы иногда используем `_передние_символы_подчеркивания` для обозначения методов или функций как «приватных», агрегаты можно представить как «публичные» классы модели, а остальные сущности и объекты-значения — как «приватные».

Выбор агрегата

Как определиться с агрегатом для системы? Это решение отчасти произвольное, но оттого не менее важное. Агрегат будет той границей, которая гарантирует, что каждая операция будет заканчиваться как положено. Это помогает нам рассуждать о софте и предотвращать странные условия гонки. Нам следует провести границу вокруг малого числа объектов — чем меньше, тем лучше для производительности, — которые должны быть согласованы друг с другом. А еще этой границе понадобится хорошее название.

Партия товара, `Batch`, — это объект, которым мы управляем. А как назвать коллекцию партий товара? Каким образом следует разделить все партии в системе на дискретные островки согласованности?

Мы *могли бы* использовать поставку товара, `Shipment`, в качестве границы. Каждая поставка товара содержит несколько партий, которые одновременно отправляются на склад. Или в качестве границы, возможно, мы могли бы использовать склад, `Warehouse`: на каждом складе хранится

много партий, и подсчет всех товарных запасов одновременно вполне может иметь смысл.

Но ничего из этого нам не подходит. У нас должна быть возможность размещать заказы на артикулы ЛОЖКА-СМЕРТОНОСНАЯ и СТОЛ-ХЛИПКИЙ одновременно, даже если они находятся на одном складе или в одной поставке. Эти понятия имеют неправильную гранулярность.

Когда мы размещаем товарную позицию заказа, нас интересуют только те партии, в которых есть тот же артикул, что и у товарной позиции. Какое-то подобие глобального товарного артикула, `GlobalSkuStock`, могло бы сработать: что-то вроде коллекции всех партий для данного артикула.

Но такое название слишком громоздкое, поэтому после некоторой бурной деятельности с `SkuStock`, `Stock`, `ProductStock` и т. д. мы решили просто назвать его продуктом, `Product`, — в конце концов, это слово было самым первым, с которым мы столкнулись в нашем исследовании языка предметной области еще в главе 1.

Итак, план таков: когда нужно разместить товарную позицию заказа, вместо подхода с рис. 7.2, где мы ищем все на свете объекты `Batch` и передаем их в профильную службу `allocate()`...

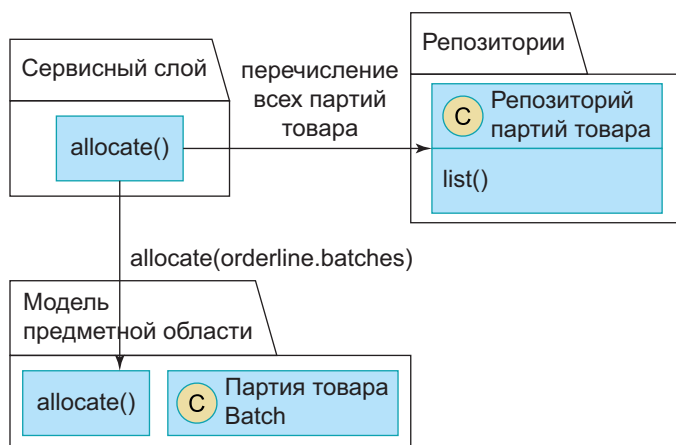


Рис. 7.2. Прежняя модель: размещение заказа относительно всех партий товара с помощью службы предметной области

...мы используем подход с рис. 7.3, где есть новый объект `Product` для конкретного артикула товарной позиции заказа, отвечающий за все партии товара *для этого артикула*, и можем вызывать метод `.allocate()` прямо в нем.

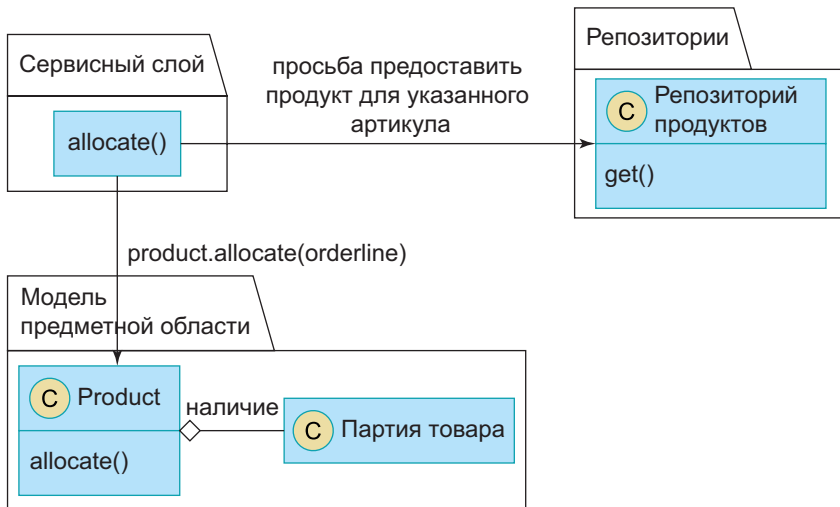


Рис. 7.3. Нынешняя модель: используем `Product`, чтобы разместить заказ относительно всех партий товара

Давайте посмотрим, как это выглядит в коде.

Выбранный нами агрегат, `Product` (`src/allocation/domain/model.py`)

`class Product:`

```
def __init__(self, sku: str, batches: List[Batch]):
    self.sku = sku ❶
    self.batches = batches ❷

def allocate(self, line: OrderLine) -> str: ❸
    try:
        batch = next(
            b for b in sorted(self.batches) if b.can_
            allocate(line)
        )
        batch.allocate(line)
        return batch.reference
    except StopIteration:
        raise OutOfStock(f'Артикула {line.sku} нет в наличии')
```

- ❶ Главным идентификатором продукта, `Product`, является артикул, `sku`.
- ❷ Класс `Product` содержит ссылку на коллекцию партий, `batches`, для этого артикула.
- ❸ Наконец, мы можем переместить службу предметной области `allocate()` в агрегат `Product` в качестве его метода.

АГРЕГАТЫ, ОГРАНИЧЕННЫЕ КОНТЕКСТЫ И МИКРОСЕРВИСЫ

Одним из наиболее ценных вкладов Эванса и сообщества DDD является понятие *ограниченных контекстов* (bounded context)¹.

По сути, это была реакция на попытки объединить весь бизнес в модель. Слово «клиент» означает разные вещи для специалистов в сфере продаж, сервиса, логистики, технической поддержки и т. д. Атрибуты, необходимые в одном контексте, не имеют смысла в другом; хуже того, вещи с одним и тем же названием могут иметь совершенно разные смыслы в зависимости от контекста. Вместо того чтобы пытаться построить единую модель (или класс, или базу данных), чтобы охватить все варианты использования, лучше иметь несколько моделей, очертить границы вокруг каждого контекста и обрабатывать перевод между разными контекстами явным образом.

Эта концепция очень хорошо переносится в мир микросервисов, каждый из которых может свободно иметь собственное понятие «клиент» и собственные правила для перевода этого понятия в другие микросервисы, с которыми они интегрируются.

В нашем примере служба размещения заказов имеет `Product(артикул, партии)`, тогда как электронная коммерция будет иметь `Product(артикул, описание, цена, url_изображения, габариты, и т.д...)`². Как правило, модели предметной области должны включать только те данные, которые необходимы для выполнения расчетов.

Независимо от того, построена ли ваша архитектура на основе микросервисов или нет, ключевым фактором при выборе агрегатов является также выбор ограниченного контекста, в котором они будут работать. Ограничивая контекст, вы можете уменьшить число агрегатов и управлять их размером.

Еще раз вынуждены сказать, что мы не можем уделить этому вопросу столько времени, сколько он заслуживает. Тем, кто заинтересовался этой темой, мы советуем начать со статьи Мартина Фаулера (ссылка дана в начале этой врезки). А еще в любой книге по DDD есть глава, а то и не одна, посвященная ограниченным контекстам.

¹ См. <https://martinfowler.com/bliki/BoundedContext.html>

² Соответственно `Product(sku, batches)` и `Product(sku, description, price, image_url, dimensions, etc...)`. — *Примеч. пер.*



Класс `Product`, возможно, выглядит не так, как вы ожидали. Ни цены, ни описания, ни размеров. Службе размещения заказов не важно ничего из вышеперечисленного. В этом и заключается вся мощь ограниченных контекстов; понятие продукта в разных приложениях может сильно различаться. Дополнительные сведения см. в следующей врезке.

Один агрегат = один репозиторий

Итак, мы определили некие сущности как агрегаты. Самое время вспомнить о правиле: эти сущности — единственные, доступ к которым публичный. Другими словами, единственные репозитории, к которым у нас есть доступ, должны быть теми, которые возвращают агрегаты.



Помните, что агрегаты должны быть единственным путем в вашу модель предметной области? Чтобы добиться этого, руководствуйтесь правилом: репозитории должны возвращать только агрегаты. Не нарушайте его!

В нашем случае мы переключимся с `BatchRepository` на `ProductRepository`.

Новый UoW и репозиторий (`unit_of_work.py` и `repository.py`)

```
class AbstractUnitOfWork(abc.ABC):
    products: repository.AbstractProductRepository

...

class AbstractProductRepository(abc.ABC):

    @abc.abstractmethod
    def add(self, product):
        ...

    @abc.abstractmethod
    def get(self, sku) -> model.Product:
        ...
```

Слой ORM стоит немного поправить для того, чтобы нужные партии товара автоматически загружались и связывались с объектами `Product`. Хорошая новость: благодаря паттерну «Репозиторий» нам пока не нужно об этом беспокоиться. Можно просто использовать поддельный репозиторий.

торий, `FakeRepository`, а затем передать новую модель в сервисный слой, чтобы проверить, как она выглядит с объектом `Product` в качестве основной точки входа.

Сервисный слой (`src/allocation/service_layer/services.py`)

```
def add_batch(
    ref: str, sku: str, qty: int, eta: Optional[date],
    uow: unit_of_work.AbstractUnitOfWork
):
    with uow:
        product = uow.products.get(sku=sku)
        if product is None:
            product = model.Product(sku, batches=[])
            uow.products.add(product)
        product.batches.append(model.Batch(ref, sku, qty, eta))
        uow.commit()

def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow:
        product = uow.products.get(sku=line.sku)
        if product is None:
            raise InvalidSku(f'Недопустимый артикул {line.sku}')
        batchref = product.allocate(line)
        uow.commit()
    return batchref
```

А что насчет производительности?

Мы уже несколько раз упоминали, что добавляем агрегаты в модели, потому что хотим получить высокопроизводительное ПО, но здесь мы загружаем *все* партии, когда требуется всего одна. Такое решение вполне может показаться неэффективным, но есть несколько причин, почему нас это устраивает.

Во-первых, мы целенаправленно моделируем данные так, чтобы делать один запрос к базе данных на чтение и один запрос на обновление для сохранения изменений. Системы с таким подходом работают намного лучше систем, которые выдают много специальных запросов. В послед-

них часто по мере развития ПО транзакции постепенно становятся все длиннее и сложнее.

Во-вторых, такие структуры данных совсем небольшие: они состоят из нескольких строковых и целочисленных значений в строке кода. Можно легко загружать десятки или даже сотни партий за несколько миллисекунд.

В-третьих, мы рассчитываем, что у нас одновременно будет лишь 20 партий товара или около того. Как только партия заканчивается, можно исключить ее из расчетов. Другими словами, объем получаемых нами данных со временем вряд ли выйдет из-под контроля.

А вот если бы мы *действительно* рассчитывали на тысячи активных партий товара, то у нас было бы несколько вариантов. Во-первых, мы могли бы использовать ленивую загрузку партий в `Product`. С точки зрения кода ничего не изменится, но в фоновом режиме `SQLAlchemy` будет листать данные за нас. В результате будет много запросов, каждый из которых будет доставать меньшее число строк. Поскольку нам нужно найти только одну партию с достаточным объемом для нашего заказа, то это, возможно, сработает неплохо.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Мы только что показали вам основные слои кода, так что вы вряд ли столкнетесь с трудностями. Но мы хотели бы, чтобы вы самостоятельно реализовали агрегат `Product` начиная с партии товара, `Batch`, как это сделали мы.

Конечно, можно схитрить и скопипастить код из предыдущих листингов, но даже в этом случае вам все равно придется решить несколько задач самостоятельно, например добавить модель в ORM и сделать так, чтобы все движущиеся части могли связываться друг с другом. Надеемся, что это упражнение вас чему-то все же научит.

Вы найдете код на GitHub¹. Мы вставили реализацию-подсказку в делегаты функции `allocate()` — она поможет вам как следует доработать код.

Мы отметили пару тестов с помощью `@pytest.skip()`. После прочтения остальной части этой главы вернитесь к этим тестам и попробуйте реализовать номера версий. Бонусные баллы получит тот, кто сможет заставить `SQLAlchemy` волшебным образом сделать их за вас!

¹ См. https://github.com/cosmicpython/code/tree/chapter_07_aggregate_exercise

Если бы все остальное отказало, то мы бы просто искали другой агрегат. Можно было бы разделить партии по регионам или по складам либо перестроить стратегию доступа к данным вокруг поставок. Паттерн «Агрегат» помогает справляться с некоторыми техническими ограничениями, связанными с согласованностью и производительностью. *Единственно* правильного агрегата нет, так что если вы заметили, что ограничения плохо влияют на производительность, можете смело выбрать другой вариант.

Оптимистичный параллелизм с номерами версий

У нас есть новый агрегат, так что мы решили концептуальную задачу выбора объекта, отвечающего за границы согласованности. Давайте теперь немного поговорим о том, как обеспечить целостность данных на уровне БД.



В этом разделе описано много деталей реализации, например некоторые из них относятся к Postgres. Но вообще-то мы показываем лишь один из способов справиться с проблемами параллелизма. Реальные требования в этой области сильно зависят от проекта. Не думайте, будто вы сможете просто скопировать код из книги и запустить его в производство.

Мы не хотим блокировать всю таблицу партий, *batches*, но как реализовать блокировку отдельных строк конкретного артикула?

Один из вариантов — добавить один атрибут в модели *Product*, который действует как маркер для полного изменения состояния, и использовать его в качестве единственного ресурса, за который могут соревноваться параллельные работники (*workers*). Если две транзакции одновременно считывают состояние мира для партий товара, *batches*, и хотят обновить таблицы размещений, *allocations*, то мы заставляем их также попытаться обновить номер версии, *version_number*, в таблице *products* таким образом, чтобы только одна из операций могла выиграть и мир оставался бы согласованным.

На рис. 7.4 показаны две параллельные транзакции, одновременно выполняющие операции чтения, поэтому они видят *Product*, например, с *version=3*. Обе вызывают *Product.allocate()*, чтобы изменить состояние. Но мы настроили правила целостности базы данных таким образом, чтобы

только одна из них могла фиксировать новый продукт с `version=4`, а другое обновление отклонялось.



Номера версий — это всего лишь один из способов реализации оптимистической блокировки. Вы можете добиться того же самого, задав в Postgres уровень изоляции транзакций равным `SERIALIZABLE`, но нередко это в итоге серьезно сказывается на производительности. Номера версий также делают неявные концепции явными.

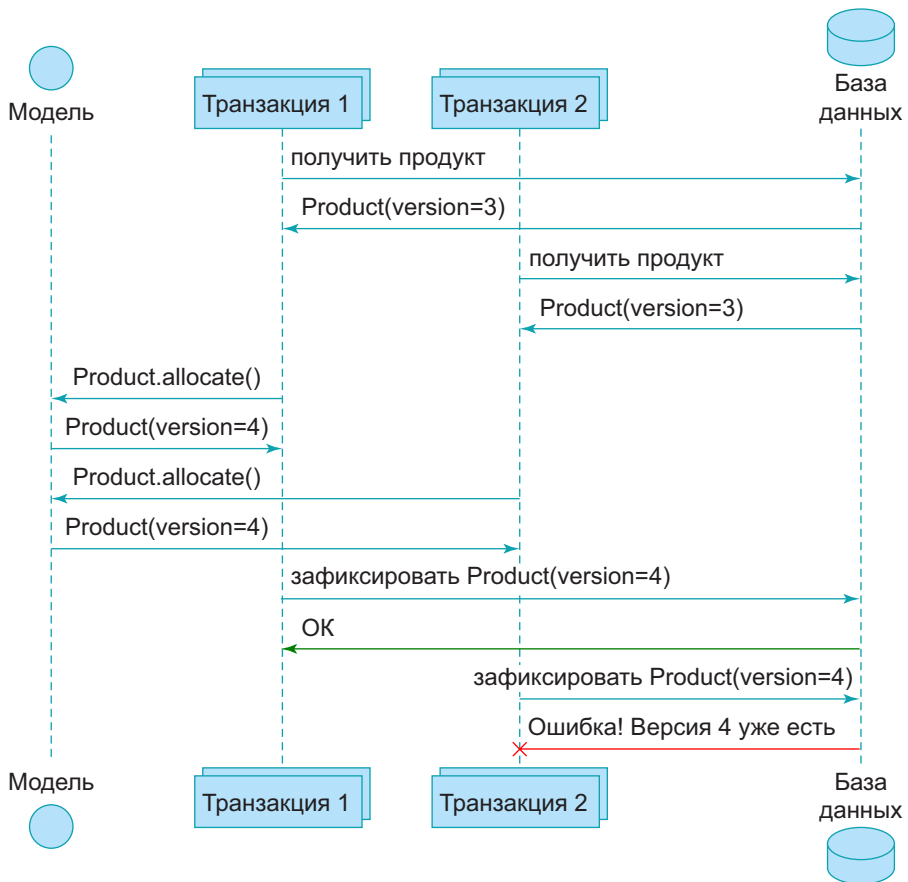


Рис. 7.4. Схема последовательности действий: две транзакции, пытающиеся параллельно выполнить операцию обновления продукта, Product

ОПТИМИСТИЧЕСКОЕ УПРАВЛЕНИЕ ПАРАЛЛЕЛИЗМОМ И ПОВТОРНЫЕ ПОПЫТКИ

Здесь мы реализовали то, что называется *оптимистическим* управлением параллелизмом, потому что по умолчанию допускаем: если два пользователя захотят внести изменения в базу данных, все будет хорошо. Мы считаем, что они вряд ли будут конфликтовать друг с другом, так что даем им продолжить и просто следим за тем, что сможем заметить проблемы, если они вдруг возникнут.

Пессимистическое управление параллелизмом основывается на предположении, что действия двух пользователей неизбежно приведут к конфликтам. Нам эти конфликты в любом случае не нужны, поэтому на всякий случай мы блокируем все. В нашем примере мы могли бы ради этого заблокировать всю таблицу `batches` или же использовать команду `SELECT FOR UPDATE` — мы делаем вид, что исключили их из соображений производительности, но в реальных условиях вы наверняка захотите оценить и измерить всё по-своему.

При пессимистической блокировке не нужно беспокоиться об обработке отказов, потому что база данных будет предотвращать их за вас (хотя вам стоит подумать о типовых ситуациях). При оптимистической блокировке нужно явно обрабатывать возможность отказов в случае конфликта (надеемся, они будут маловероятными). Типичный способ справиться с неудачной операцией — попробовать выполнить ее заново.

Представьте себе, что у нас есть два клиента, Гарри и Боб, и каждый делает заказ на артикул СТОЛ-БЛЕСТЯЩИЙ, `SHINY-TABLE`. Оба потока выполнения загружают продукт в версии 1 и находят соответствующий товарный запас. База данных предотвращает параллельное обновление, и заказ Боба завершается ошибкой. Когда мы повторяем операцию, заказ Боба загружает продукт в версии 2 и пытается снова его разместить. Если в запасах товар еще остался, то все в порядке, в противном случае Боб получит сообщение «нет в наличии», `OutOfStock`. Так происходит со всеми проблемами параллелизма.

Подробнее о повторных попытках читайте в разделах «Синхронное восстановление после ошибок» на с. 208 и «Выстрел в ногу» на с. 287.

Варианты реализации номеров версий

В сущности, для номеров версий имеются три варианта реализации:

1. `version_number` расположен в предметной области; мы добавляем его в конструктор класса `Product`, а метод `Product.allocate()` отвечает за его наращивание.
2. Это может делать сервисный слой! Предметная область *не обязана* отслеживать номер версии, поэтому вместо этого сервисный слой может предполагать, что текущий номер версии прикреплен репозиторием к `Product`, так что сервисный слой будет наращивать его перед выполнением `commit()`.

3. Поскольку эта обязанность, пожалуй, является инфраструктурной, паттерны UoW и «Репозиторий» могут наращивать номера версий самостоятельно. У репозитория есть доступ к номерам версий для любых продуктов, которые он извлекает, и когда UoW выполняет фиксацию, он может увеличивать номер версии для любых известных ему продуктов, допустив, что они изменились.

Вариант 3 неидеален, ведь сделать это можно, лишь допустив, что все продукты изменились, из-за чего мы будем вынуждены наращивать номера версий, когда нам это не нужно¹.

Во втором варианте обязанность по изменению состояния разделена между слоем служб и слоем предметной области, так что это тоже немного запутанно.

В конце концов, даже если предметная область не обязана отслеживать номера версий, это все же лучший вариант для их размещения.

Выбранный нами агрегат `Product` (`src/allocation/domain/model.py`)

`class Product:`

```
def __init__(self, sku: str, batches: List[Batch], version_
    number: int = 0): ❶
    self.sku = sku
    self.batches = batches
    self.version_number = version_number ❶

def allocate(self, line: OrderLine) -> str:
    try:
        batch = next(
            b for b in sorted(self.batches) if b.can_allocate(line)
        )
        batch.allocate(line)
        self.version_number += 1 ❶
        return batch.reference
    except StopIteration:
        raise OutOfStock(f'артикула {line.sku} нет в наличии')
```

❶ Вот он!

¹ Пожалуй, мы могли бы немного поколдовать с ORM/SQLAlchemy, чтобы найти объект, с которым что-то не так, но будет ли это работать во всех случаях, например для репозитория CSV-файлов, `CsvRepository`?



Если вся эта история с номерами версий ставит вас в тупик, просто помните, что номер неважен. Важно то, что строка в базе данных `Product` изменяется всякий раз, когда мы вносим изменения в агрегат `Product`. Номер версии — это простой и понятный человеку способ смоделировать что-то, что меняется при каждой операции записи, но всякий раз это точно так же мог бы быть и случайный `UUID`.

Тестирование правил целостности данных

Теперь убедимся, что программа ведет себя как задумано: если у нас есть две параллельные попытки размещения одного и того же продукта, `Product`, то сработает лишь одна из них, потому что они обе не могут обновить номер версии.

Во-первых, смоделируем «медленную» транзакцию с помощью функции, которая размещает заказ, а затем выполняет явный вызов `time.sleep()`¹.

Функция `time.sleep` может воспроизводить параллельное поведение (`tests/integration/test_uow.py`)

```
def try_to_allocate(orderid, sku, exceptions):
    line = model.OrderLine(orderid, sku, 10)
    try:
        with unit_of_work.SqlAlchemyUnitOfWork() as uow:
            product = uow.products.get(sku=sku)
            product.allocate(line)
            time.sleep(0.2)
            uow.commit()
    except Exception as e:
        print(traceback.format_exc())
        exceptions.append(e)
```

Затем тест вызывает это медленное размещение дважды, параллельно, используя потоки выполнения.

¹ В нашем случае функция `time.sleep()` работает хорошо, но этот способ воспроизведения ошибок параллелизма не самый надежный или эффективный. Чтобы наверняка гарантировать нужное поведение модели, подумайте о применении семафоров либо похожих примитивов синхронизации, совместно используемых между потоками выполнения.

Интеграционный тест на параллельное поведение (tests/integration/test_uow.py)

```

def test_concurrent_updates_to_version_are_not_allowed(postgres_
    session_factory):
    sku, batch = random_sku(), random_batchref()
    session = postgres_session_factory()
    insert_batch(session, batch, sku, 100, eta=None, product_version=1)
    session.commit()

    order1, order2 = random_orderid(1), random_orderid(2)
    exceptions = [] # тип: List[Exception]
    try_to_allocate_order1 = lambda: try_to_allocate(order1, sku,
                                                    exceptions)
    try_to_allocate_order2 = lambda: try_to_allocate(order2, sku,
                                                    exceptions)
    thread1 = threading.Thread(target=try_to_allocate_order1) ❶
    thread2 = threading.Thread(target=try_to_allocate_order2) ❶
    thread1.start()
    thread2.start()
    thread1.join()
    thread2.join()

    [[version]] = session.execute(
        "SELECT version_number FROM products WHERE sku=:sku",
        dict(sku=sku),
    )
    assert version == 2 ❷
    [exception] = exceptions
    assert 'не получилось сериализовать доступ из-за параллельного
    обновления' in str(exception) ❸

    orders = list(session.execute(
        "SELECT orderid FROM allocations"
        " JOIN batches ON allocations.batch_id = batches.id"
        " JOIN order_lines ON allocations.orderline_id = order_lines.id"
        " WHERE order_lines.sku=:sku",
        dict(sku=sku),
    ))
    assert len(orders) == 1 ❹
    with unit_of_work.SqlAlchemyUnitOfWork() as uow:
        uow.session.execute('select 1')

```

❶ Мы запускаем два потока выполнения, у которых точно будет нужное параллельное поведение: read1, read2, write1, write2.

❷ Мы убеждаемся, что номер версии был увеличен только один раз.

③ Мы также можем выполнить проверку какого-нибудь конкретного исключения, если захотим.

④ И мы перепроверяем, действительно ли прошло только одно размещение.

Обеспечение соблюдения правил параллелизма с помощью уровней изоляции транзакций базы данных

Чтобы тест прошел как есть, мы можем установить для сеанса уровень изоляции транзакций.

Установка уровня изоляции для сеанса (src/allocation/service_layer/unit_of_work.py)

```
DEFAULT_SESSION_FACTORY = sessionmaker(bind=create_engine(
    config.get_postgres_uri(),
    isolation_level="REPEATABLE READ",
))
```



Уровни изоляции транзакций — это сложная штука, так что стоит потратить время на того, чтобы разобраться в соответствующей документации Postgres¹.

Пример пессимистического управления параллелизмом: SELECT FOR UPDATE

К этому можно подойти несколькими способами, но мы покажем один. Команда `SELECT FOR UPDATE`² задает разное поведение: двум параллельным транзакциям будет запрещено одновременно выполнять чтение в одних и тех же строках таблицы.

Команда `SELECT FOR UPDATE` позволяет выбрать одну или несколько строк для использования в качестве блокировки (хотя эти строки не обязательно должны быть как раз теми, которые вы обновляете). Если две транзакции попытаются одновременно выполнить команду `SELECT FOR UPDATE` для строки, то одна одержит верх, а другая будет ждать до тех пор, пока

¹ Если вы не используете Postgres, то вам понадобится другая документация. Досадно, что разные базы данных имеют совершенно разные определения. Например, `SERIALIZABLE` в Oracle эквивалентно повторяемому чтению, `REPEATABLE READ`, в Postgres.

² См. <https://oreil.ly/i8wKL>

блокировка не будет снята. Ниже приводится пример пессимистического управления параллелизмом.

Вот как можно использовать DSL в SQLAlchemy для указания команды FOR UPDATE во время запроса.

with_for_update с SQLAlchemy (`src/allocation/adapters/repository.py`)

```
def get(self, sku):
    return self.session.query(model.Product) \
        .filter_by(sku=sku) \
        .with_for_update() \
        .first()
```

Это приведет к смене паттерна параллелизма с

```
read1, read2, write1, write2(fail)
```

на

```
read1, write1, read2, write2(succeed)
```

Некоторые называют это режимом отказа «чтение — изменение — запись». Хороший обзор этой темы можно найти в статье «Антипаттерны PostgreSQL: циклы чтения — изменения — записи»¹.

У нас действительно нет времени обсуждать все компромиссы между REPEATABLE READ и SELECT FOR UPDATE или оптимистической или пессимистической блокировкой в целом. Но если у вас есть тест вроде того, что мы показали, то вы можете задать желаемое поведение и посмотреть, как оно меняется. Также можно использовать тест в качестве основы для экспериментов с производительностью.

Выводы

Конкретные варианты управления конкурентностью сильно различаются в зависимости от условий работы бизнеса и выбора технологии хранения данных, но мы хотели вернуться в этой главе к концептуальной идее агрегата: мы моделируем объект явным образом как главную точку входа в некоторое подмножество модели и отвечаем за соблюдение инвариантов и бизнес-правил, применимых ко всем этим объектам.

¹ См. <https://oreil.ly/uXeZI>

Ключевым моментом здесь является выбор правильного агрегата, и это решение вы можете пересмотреть со временем. Вы можете дополнительно почитать другие книги на тему DDD, чтобы лучше изучить этот момент. Мы также рекомендуем следующие три статьи по эффективному проектированию агрегатов¹ Вона Вернона (автора «Красной книги»).

В табл. 7.1 сравниваются преимущества и недостатки паттерна «Агрегат».

Таблица 7.1. Агрегаты: компромиссы

Плюсы	Минусы
- Python может не иметь «официальных» публичных и частных методов, но в языке принято использовать подчеркивание, так как часто есть смысл указать, что именно предназначено для «внутреннего» использования, а что — для «внешнего кода». Агрегат — это просто еще один верхний уровень: он позволяет вам решить, какие из ваших классов модели предметной области публичные, а какие — нет. - Моделирование операций вокруг явных границ согласованности помогает избегать проблем с производительностью ORM. - Возлагая на агрегат исключительную обязанность по изменению состояния его вспомогательных моделей, рассуждать о системе и управлять инвариантами становится проще	- Еще одна новая концепция, которую должны учитывать новые разработчики. Объяснение сущностей в сравнении с объектами-значениями и так было непростым; теперь еще появился и третий тип объекта модели предметной области? - Строгое следование правилу изменения лишь одного агрегата за раз — большой сдвиг в сознании. - Бывает сложно работать с итоговой согласованностью (eventual consistency) между агрегатами

КРАТКО ОБ АГРЕГАТАХ И ГРАНИЦАХ СОГЛАСОВАННОСТИ

Агрегаты — это точки входа в модель предметной области

Ограничивая число способов что-то изменить, становится проще рассуждать о системе.

Агрегаты отвечают за границу согласованности

Задача агрегата — иметь возможность управлять бизнес-правилами в отношении инвариантов, поскольку они применяются к группе связанных объектов. Задачей именно агрегата является проверка согласованности объектов, входящих в его компетенцию, друг с другом и с правилами и отклонение изменений, которые нарушили бы эти правила.

Агрегаты и сложности параллелизма идут рука об руку

Когда мы планируем реализацию проверок согласованности, мы в итоге приходим к мысли о транзакциях и блокировках. Выбор правильного агрегата связан с производительностью, а также с концептуальной организацией вашей предметной области.

¹ См. https://dddcommunity.org/library/vernon_2011

Итоги части I

Помните диаграмму, которую мы показали в начале части I, чтобы понимать, куда мы движемся (рис. 7.5)?

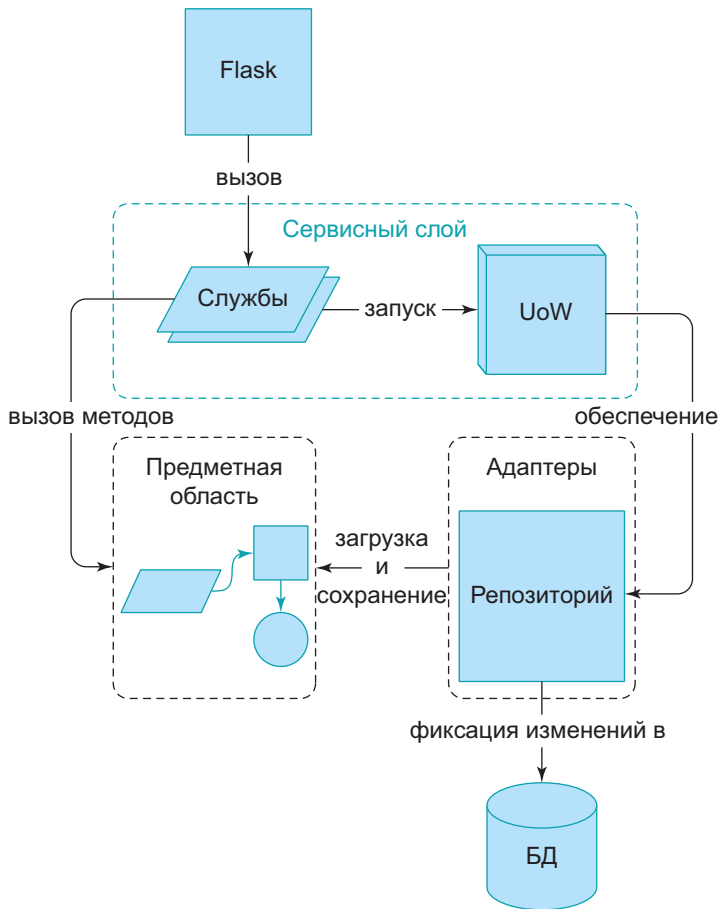


Рис. 7.5. Итоговая диаграмма компонентов приложения

Итак, вот где мы оказались. Чего мы достигли? Мы узнали, как создать модель предметной области, которая выполняется с помощью набора высокоуровневых юнит-тестов. Тесты представляют собой живую документацию: они описывают поведение системы — правила, о которых мы договорились со стейкхолдерами, — в удобочитаемом коде. Мы уверены,

что если бизнес-правила вдруг изменятся, юнит-тесты помогут нам продумать новую функциональность, а если новые разработчики присоединятся к проекту, эти тесты помогут им понять, как все работает.

Мы устранили связанность инфраструктурных частей системы, таких как база данных и обработчики API, чтобы подключить их к внешней стороне приложения. Это помогает содержать кодовую базу хорошо организованной и не скатываться в большой комок грязи.

Применяя принцип инверсии зависимостей и используя паттерны «Репозиторий» и UoW, мы сделали возможным применение TDD как на повышенной, так и на пониженной передаче и поддержание здоровой тестовой пирамиды. Можно тестировать систему от начала до конца, а потребность в интеграционных и сквозных тестах сведена к минимуму.

Наконец, мы поговорили об идее границ согласованности (consistency boundaries). Нам не нужно блокировать всю систему всякий раз при внесении изменений, поэтому следует определиться с тем, какие части согласуются друг с другом.

Для малой системы это все, что нужно, чтобы прочувствовать идеи DDD. Теперь у вас есть инструменты для создания моделей предметных областей, не осведомленных о лежащей в основании системе управления базами данных, которые представляют общий язык ваших профильных экспертов. Ура!



Рискуя слишком сильно углубиться в детали, мы изо всех сил старались подчеркнуть, что у каждого паттерна есть свои недостатки. Каждый слой косвенности имеет свою цену с точки зрения сложности и дублирования в коде и будет сбивать с толку программистов, которые никогда раньше не сталкивались с ними. Если ваше приложение по сути является простой оболочкой CRUD вокруг базы данных и вряд ли будет чем-то большим в обозримом будущем, то вам эти паттерны не нужны. Используйте Django и дальше и избавьте себя от многих хлопот.

В части II мы уменьшим масштаб и поговорим о более глобальной теме: если агрегаты — это граница и мы можем обновлять только по одному из них за раз, то как моделировать процессы, которые пересекают границы согласованности?

ЧАСТЬ II

Событийно-управляемая архитектура

Мне жаль, что когда-то давно я придумал термин «объекты» для этой темы, так как из-за него многие концентрируются на меньшей идее.

Главная же идея — это «обмен сообщениями» ... Ключ к созданию прекрасных масштабируемых систем — дизайн взаимодействия их модулей, а вовсе не выбор их внутренних свойств и поведения.

Алан Кей

Конечно, здорово, если вы можете написать *одну* модель предметной области для управления одним фрагментом бизнес-процесса, но что, если нужно *много* моделей? В реальном мире приложения — это часть целой организации. Они должны обмениваться информацией с другими частями системы. Посмотрите на рис. II.1.

Столкнувшись с этим требованием, многие команды решают использовать микросервисы, которые интегрируются через HTTP API. Но без должной осторожности такая система рискует превратиться в самый хаотичный беспорядок из всех — в большой распределенный комок грязи.

В части II мы покажем, как методы из части I можно применить в распределенных системах. Мы уменьшим масштаб и посмотрим, как можно составить систему из множества малых компонентов, которые взаимодействуют с помощью асинхронной передачи сообщений.

Мы увидим, как паттерны «Сервисный слой» и UoW позволяют перенастраивать приложение для работы в качестве асинхронного процессора сообщений и то, как событийно-управляемые системы помогают устранять связанность агрегатов с приложением.

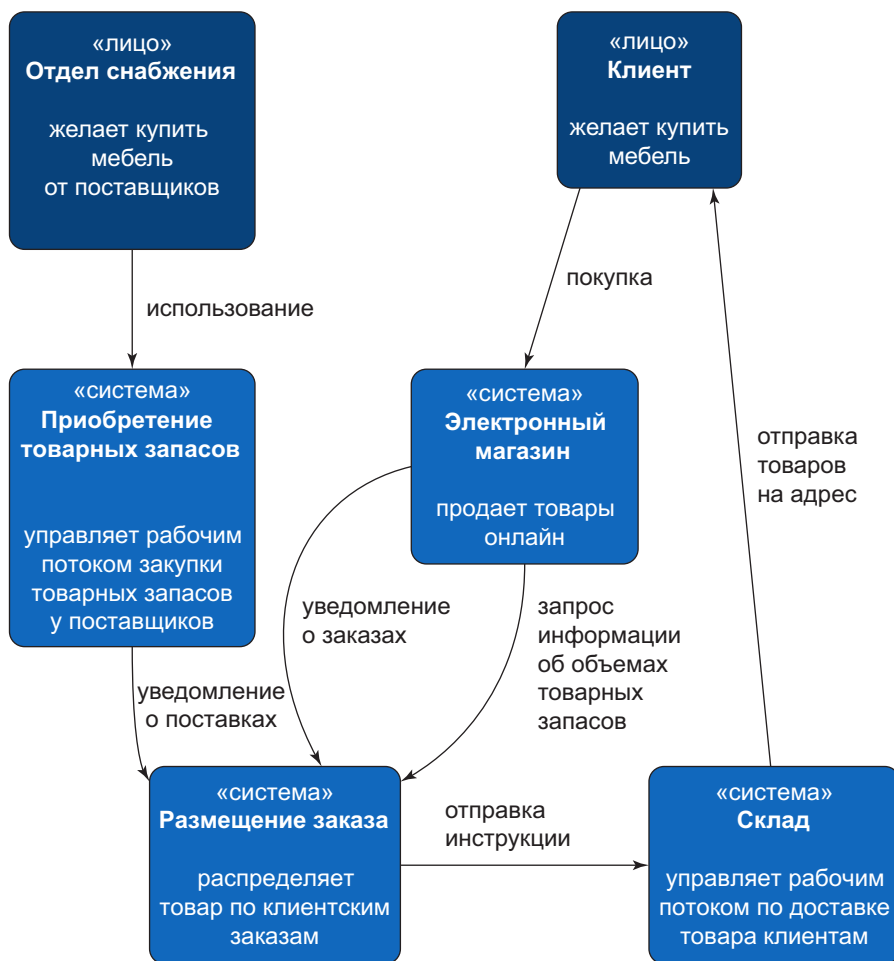


Рис. II.1. Но как именно все эти системы будут общаться друг с другом?

Рассмотрим следующие паттерны и приемы:

События предметной области

Запускают рабочие потоки, которые пересекают границы согласованности.

Шина сообщений

Обеспечивает унифицированный способ вызова вариантов использования из любой конечной точки.

Разделение ответственности команд и запросов (CQRS)

Разделение операций чтения и записи позволяет избежать компромиссов в событийно-управляемой архитектуре и повысить производительность и масштабируемость.

Кроме того, мы добавим фреймворк внедрения зависимостей. Он не имеет ничего общего с событийно-управляемой архитектурой как таковой, но зато подчищает великое множество хвостов.

События и шина сообщений

До сих пор у нас уходило много времени и сил на решение простой задачи, которую легко можно было бы решить с помощью Django. Вы спросите, *действительно ли* повышенная тестопригодность и выразительность стоят всех этих усилий.

Но на практике становится понятно, что беспорядок в кодовых базах вносят не очевидные функции, а всякие формальные мелочи. Это отчетность, разрешения и рабочие потоки, которые касаются огромного количества объектов.

Простой пример — типичное требование об уведомлении: когда мы не можем разместить заказ из-за отсутствия товара, мы должны предупредить об этом отдел снабжения. Они решат проблему, пополнив товарные запасы, и все будет хорошо.

Первым делом владелец продукта предлагает просто отправлять уведомление по имейлу.

Давайте посмотрим, как архитектура справляется с подключением нескольких внешних элементов, которые входят в состав столь многих систем.

Мы начнем с самого простого, самого быстрого решения и поговорим о том, почему именно из-за такого решения у нас получается большой комок грязи.

Затем мы покажем, как паттерн «События предметной области» помогает очистить варианты использования от побочных эффектов и как паттерн «Шина сообщений» активизирует поведение, основанное на этих событиях. Мы покажем несколько вариантов создания этих событий и способов их передачи в шину сообщений. Наконец, мы изменим паттерн UoW, чтобы элегантно соединить события, как показано на рис. 8.1.

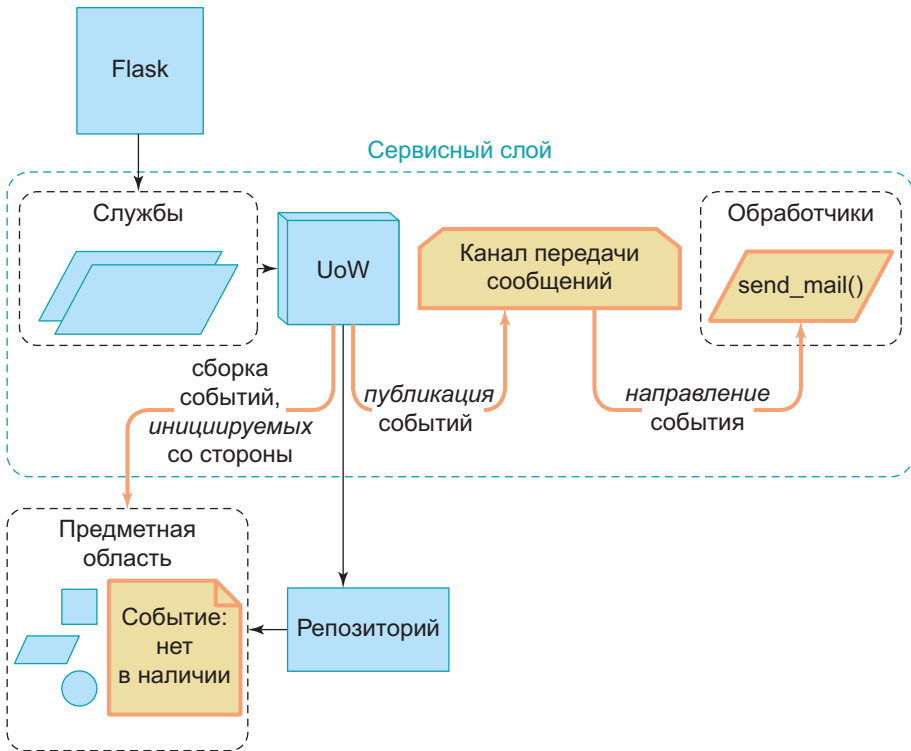


Рис. 8.1. События, проходящие через систему



Код для этой главы находится в ветке `chapter_08_events_and_message_bus` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_08_events_and_message_bus
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_07_aggregate
```

¹ См. <https://oreil.ly/M-JuL>

Как избежать беспорядка

Итак. Оповещения об отсутствии товара по имейлу. Когда появляются новые требования, которые *действительно* не имеют ничего общего с предметной областью как таковой, очень легко начать сбрасывать их в веб-контроллеры.

Для начала не будем вносить беспорядок в веб-контроллеры

Если провернуть такое всего один раз, *возможно*, все будет в порядке.

Просто воткни их в конечную точку — что не так-то? (src/allocation/entrypoints/flask_app.py)

```
@app.route("/allocate", methods=['POST'])
def allocate_endpoint():
    line = model.OrderLine(
        request.json['orderid'],
        request.json['sku'],
        request.json['qty'],
    )
    try:
        uow = unit_of_work.SqlAlchemyUnitOfWork()
        batchref = services.allocate(line, uow)
    except (model.OutOfStock, services.InvalidSku) as e:
        send_mail(
            'нет в наличии',
            'stock_admin@made.com',
            f'{line.orderid} - {line.sku}'
        )
        return jsonify({'message': str(e)}), 400

    return jsonify({'batchref': batchref}), 201
```

...Но несложно понять, как быстро такие патчи приведут к хаосу. Отправка имейлов не является задачей HTTP-слоя, да и юнит-тест этой функции не помешал бы.

И не будем вносить беспорядок в модель...

Нам нужны как можно более тонкие веб-контроллеры, так что не будем помещать в них этот код. Тогда можно подумать о том, чтобы поместить его прямо в источник в модели.

Код отправки сообщений в модели тоже не очень хорош (src/allocation/domain/model.py)

```
def allocate(self, line: OrderLine) -> str:
    try:
        batch = next(
            b for b in sorted(self.batches) if b.can_allocate(line)
        )
        #...
    except StopIteration:
        email.send_mail('stock@made.com', f'Артикула {line.sku} нет
            в наличии')
        raise OutOfStock(f'Артикула {line.sku} нет в наличии')
```

Но это еще хуже! Модель не должна зависеть от инфраструктурных обязанностей, таких как отправка имейлов `email.send_mail`.

Вся эта лишняя суета с отправкой электронной почты портит приятный чистый поток системы. Хотелось бы, чтобы модель предметной области была сфокусирована на правиле «нельзя размещать заказ на что-то в большем количестве, чем имеется на самом деле».

Работа модели предметной области состоит в том, чтобы знать, что у нас нет товара в наличии, но обязанность по отправке предупреждения лежит где-то в другом месте. Должна быть возможность включать или выключать эту функцию или вместо этого переключаться на SMS-уведомления так, чтобы при этом не изменять правила модели предметной области.

...и в сервисный слой!

Требование «попытаться разместить заказ на некоторый товар и отправить письмо, если не выйдет это сделать» — пример оркестровки рабочего потока: это набор шагов, которые система должна выполнять для достижения цели.

Мы написали сервисный слой для управления оркестровкой за нас, но даже здесь это функциональное свойство выглядит неуместным.

И в сервисном слое это тоже не в тему (src/allocation/service_layer/services.py)

```
def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
```

```
with uow:
    product = uow.products.get(sku=line.sku)
    if product is None:
        raise InvalidSku(f'Недопустимый артикул {line.sku}')
    try:
        batchref = product.allocate(line)
        uow.commit()
        return batchref
    except model.OutOfStock:
        email.send_mail('stock@made.com', f'Артикула {line.sku}
        нет в наличии')
        raise
```

Перехватить исключение и инициировать его повторно? Могло быть и хуже, но это определенно нас удручает. И почему же так трудно найти подходящее место для этого кода?

Принцип единственной обязанности

Все дело в том, что мы нарушаем *принцип единственной обязанности* (single responsibility principle, SRP)¹. Вариантом использования является размещение заказа на товар. Конечные точки, функции службы и методы предметной области называются `allocate` (разместить заказ), а не `allocate_and_send_mail_if_out_of_stock` (разместить заказ и отправить электронное сообщение, если товара нет в наличии).



Общее правило таково: если вы не можете описать работу функции, не используя такие слова, как «затем» или «и», то, скорее всего, нарушаете принцип единственной обязанности.

Одна из формулировок указанного принципа заключается в том, что каждый класс должен иметь только одну причину для изменения. Когда мы переключаемся с электронной почты на SMS, обновлять функцию `allocate()` не нужно, так как это явно отдельная обязанность.

Для того чтобы решить эту проблему, мы разделим оркестровку на отдельные шаги, чтобы разные обязанности не спутывались². Работа

¹ Этот принцип обозначен буквой S в аббревиатуре SOLID. См. <https://oreil.ly/AIdSD>

² Наш научный редактор Эд Юнг любит говорить, что переход от императивного управления потоком к событийному меняет то, что раньше было оркестровкой, на хореографию.

модели предметной области состоит в том, чтобы знать, что у нас нет товара в наличии, но обязанность по отправке предупреждения лежит где-то в другом месте. Мы должны иметь возможность включать или выключать эту функцию или вместо этого переключаться на SMS-уведомления без необходимости изменять правила модели предметной области.

Также в сервисном слое не стоит хранить детали реализации. Нам нужно применить принцип инверсии зависимостей к уведомлениям, чтобы сервисный слой зависел от абстракции точно так же, как мы избегаем зависимости от базы данных, используя паттерн UoW.

Катимся на шине сообщений!

Здесь мы собираемся ввести два паттерна. Это «События предметной области» и «Шина сообщений». Можно реализовать их несколькими способами, поэтому покажем пару из них, а потом остановимся на том, который нам больше нравится.

Модель регистрирует события

Вместо того чтобы беспокоиться об электронных письмах, модель будет отвечать за регистрацию *событий* — фактов произошедшего. Используем шину сообщений, через которую будем реагировать на события и вызывать новую операцию.

События — это простые классы данных

Событие — это вид *объекта-значения*. У событий нет поведения, потому что они представляют собой чистые структуры данных. Мы всегда называем события на языке предметной области и думаем о них как о части модели предметной области.

Можно хранить их внутри `model.py` либо в отдельном файле (пожалуй, самое время подумать о реструктуризации каталога `domain`, чтобы в итоге у нас были `domain/model.py` и `domain/events.py`).

Классы событий (src/allocation/domain/events.py)

```
from dataclasses import dataclass
```

```
class Event: ❶
    pass
```

```
@dataclass
class OutOfStock(Event): ❷
    sku: str
```

❶ Если событий много, есть смысл ввести родительский класс, который будет хранить общие атрибуты. Это также пригодится для подсказок типов в канале передачи сообщений, как вы увидите позже.

❷ dataclasses отлично подходят и для событий предметной области.

Модель иницирует события

Когда модель предметной области регистрирует произошедший факт, мы говорим, что она *инициирует событие*.

Вот как это будет выглядеть снаружи; если мы просим Product разместить заказ на товар, но у него не получается это сделать, то он должен *иницировать* событие.

Тест агрегата на предмет иницирования им события (tests/unit/test_product.py)

```
def test_records_out_of_stock_event_if_cannot_allocate():
    batch = Batch('batch1', 'SMALL-FORK', 10, eta=today)
    product = Product(sku="SMALL-FORK", batches=[batch])
    product.allocate(OrderLine('order1', 'SMALL-FORK', 10))

    allocation = product.allocate(OrderLine('order2', 'SMALL-FORK', 1))
    assert product.events[-1] == events.OutOfStock(sku="SMALL-FORK") ❶
    assert allocation is None
```

❶ Агрегат будет выявлять новый атрибут .events со списком случившихся фактов в форме объектов Event.

Вот как модель выглядит изнутри.

Модель иницирует событие предметной области (src/allocation/domain/model.py)

```
class Product:
```

```
    def __init__(self, sku: str, batches: List[Batch], version_
```



```
number: int = 0):
    self.sku = sku
    self.batches = batches
    self.version_number = version_number
    self.events = [] # тип: List[events.Event] ❶

def allocate(self, line: OrderLine) -> str:
    try:
        #...
    except StopIteration:
        self.events.append(events.OutOfStock(line.sku)) ❷
        # инициировать OutOfStock(f'Артикула {line.sku} нет
        в наличии') ❸
        return None
```

❶ Вот как используется новый атрибут `.events`.

❷ Вместо того чтобы напрямую вызывать какой-либо фрагмент кода с отправкой почты, мы регистрируем эти события в том месте, где они происходят, используя только язык предметной области.

❸ Мы также прекратим инициировать исключение для ситуации отсутствия товара в наличии. Событие выполнит ту работу, которую выполняло исключение.



На самом деле мы устраним прежний код «с душком», а именно исключения для потока управления¹. Как правило, если вы реализуете события предметной области, то не инициируйте исключения для описания той же концепции предметной области. Как вы увидите позже, когда мы будем обрабатывать события в паттерне UoW, это сбивает с толку, когда приходится рассуждать о событиях и исключениях вместе.

Шина сообщений попарно сопоставляет события с обработчиками

Шина сообщений, в сущности, говорит: «Когда я вижу это событие, я должна вызвать вот эту функцию-обработчик». Другими словами, это простая система «издатель — подписчик». Обработчики подписываются на получение событий, которые мы публикуем в канале. Это звучит сложнее, чем есть на самом деле, и обычно реализуется с помощью словаря.

¹ См. <https://oreil.ly/IQB51>

Простая шина сообщений (src/allocation/service_layer/messagebus.py)

```
def handle(event: events.Event):
    for handler in HANDLERS[type(event)]:
        handler(event)

def send_out_of_stock_notification(event: events.OutOfStock):
    email.send_mail(
        'stock@made.com',
        f'Артикула {event.sku} нет в наличии',
    )

HANDLERS = {
    events.OutOfStock: [send_out_of_stock_notification],
} # тип: Dict[Type[events.Event], List[Callable]]
```

ПОХОЖЕ НА CELERY?

Celery («сельдерей») — это популярный инструмент в мире Python для переноса автономных сегментов работы в асинхронную очередь задач¹. Краткий ответ на вопрос из заголовка: нет, не похоже; канал сообщений сильно отличается от Celery и имеет больше общего с приложением Node.js, событийным циклом пользовательского интерфейса или акторным фреймворком.

Если нужно перенести работу из главного потока выполнения, вы все равно можете применять наши событийные метафоры, но лучше использовать для этого внешние события. В табл. 11.1 этот вопрос обсуждается подробнее, но, по существу, если вы реализуете способ обеспечения постоянного хранения событий в централизованном хранилище, то можете подписать на них другие контейнеры или микросервисы. Тогда та же самая концепция использования событий для разделения обязанностей между UoW в рамках одного процесса/службы может быть распространена на многочисленные процессы, которые могут быть разными контейнерами в рамках одной и той же службы или совершенно разными микросервисами.

Если вы следуете нашему подходу, то вашим API для распределения задач будут классы-события — или же их представление в формате JSON. Это даст большую гибкость в распределении задач; они не обязательно должны быть службами Python. API Celery для распределения задач — это, по сути, «имя функции плюс аргументы», что является более ограничивающим инструментом и относится только к языку Python.

¹ См. <https://docs.celeryproject.org/en/stable/>



Обратите внимание на то, как реализована шина сообщений: она не обеспечивает согласованность, поскольку выполнится только по одному обработчику за раз. Цель состоит не в том, чтобы поддерживать параллельные потоки, а в том, чтобы разделить задачи концептуально и сделать каждый UoW как можно меньше. Это помогает лучше понимать кодовую базу, так как «рецепт» выполнения каждого варианта использования написан в одном месте. См. врезку выше.

Вариант 1: сервисный слой берет события из модели и помещает их в шину сообщений

Модель предметной области инициирует события, и шина сообщений будет вызывать нужные обработчики всякий раз, когда происходит событие. Теперь единственное, что осталось сделать, — это соединить модель и канал. Нужно что-то, что отлавливало бы события из модели и передавало их в шину сообщений, — этап *публикации* (publishing).

Самый простой способ сделать это — добавить немного кода в сервисный слой.

Сервисный слой с явно заданной шиной сообщений (src/allocation/service_layer/services.py)

```
from . import messagebus
...

def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow:
        product = uow.products.get(sku=line.sku)
        if product is None:
            raise InvalidSku(f'Недопустимый артикул {line.sku}')
        try: ❶
            batchref = product.allocate(line)
            uow.commit()
            return batchref
        finally: ❶
            messagebus.handle(product.events) ❷
```

❶ Мы оберегаем блок `try/finally` от неприглядной прежней реализации (мы еще не избавились от *всех* исключений, только `OutOfStock`).

❷ Но теперь, вместо того чтобы напрямую зависеть от инфраструктуры электронной почты, сервисный слой отвечает только за передачу событий из модели в шину сообщений.

Такая реализация выглядит гораздо лучше прежней. К тому же у нас есть несколько систем, работающих подобно этой, где сервисный слой собирает события из агрегатов явным образом и передает их в шину сообщений.

Вариант 2: сервисный слой инициирует собственные события

Еще один вариант — сделать так, чтобы сервисный слой непосредственно отвечал за создание и инициирование событий, а не за получение их из модели предметной области, которая их инициирует.

Сервисный слой вызывает `messagebus.handle` напрямую (`src/allocation/service_layer/services.py`)

```
def allocate(
    orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow:
        product = uow.products.get(sku=line.sku)
        if product is None:
            raise InvalidSku(f'Недопустимый артикул {line.sku}')
        batchref = product.allocate(line)
        uow.commit() ❶

        if batchref is None:
            messagebus.handle(events.OutOfStock(line.sku))
    return batchref
```

❶ Как и прежде, мы выполняем фиксацию (`.commit()`), даже если не удастся разместить заказ на товар, потому что в таком виде код проще и понятнее: мы всегда выполняем фиксацию, если что-то идет не так. Делать фиксацию, когда мы ничего не меняли, безопасно. К тому же это делает код более чистым.

Опять же в случае с нашими приложениями этот паттерн реализован именно так. То, что подойдет для вас, будет зависеть от конкретных компромиссов, с которыми вы столкнетесь. Мы же просто хотим показать вам то, что считаем наиболее элегантным решением, в котором паттерн UoW отвечает за сбор и инициирование событий.

Вариант 3: UoW публикует события в шине сообщений

Паттерн UoW уже включает блок `try/finally` и знает обо всех задействованных агрегатах, так как предоставляет доступ к репозиторию. Вполне подходящее место для обнаружения событий и передачи их в шину сообщений.

UoW вместе с шиной сообщений (`src/allocation/service_layer/unit_of_work.py`)

```
class AbstractUnitOfWork(abc.ABC):
    ...

    def commit(self):
        self._commit() ❶
        self.publish_events() ❷

    def publish_events(self): ❷
        for product in self.products.seen: ❸
            while product.events:
                event = product.events.pop(0)
                messagebus.handle(event)

    @abc.abstractmethod
    def _commit(self):
        raise NotImplementedError

    ...

class SQLAlchemyUnitOfWork(AbstractUnitOfWork):
    ...

    def _commit(self): ❶
        self.session.commit()
```

❶ Изменяем метод фиксации: будем требовать приватный метод `._commit()` из подклассов.

❷ После фиксации мы перебираем все объекты, которые репозиторий видел, и передаем их события в шину сообщений.

❸ Этот фрагмент опирается на репозиторий, который отслеживает агрегаты, загруженные с использованием нового атрибута `.seen`, как показано в следующем листинге.



А что произойдет, если один из обработчиков откажет? Обработку ошибок подробно обсудим в главе 10.

Репозиторий отслеживает проходящие через него агрегаты (src/allocation/adapters/repository.py)

```
class AbstractRepository(abc.ABC):

    def __init__(self):
        self.seen = set() # type: Set[model.Product] ❶

    def add(self, product: model.Product): ❷
        self._add(product)
        self.seen.add(product)

    def get(self, sku) -> model.Product: ❸
        product = self._get(sku)
        if product:
            self.seen.add(product)
        return product

    @abc.abstractmethod
    def _add(self, product: model.Product): ❷
        raise NotImplementedError

    @abc.abstractmethod ❸
    def _get(self, sku) -> model.Product:
        raise NotImplementedError

class SQLAlchemyRepository(AbstractRepository):

    def __init__(self, session):
        super().__init__()
        self.session = session

    def _add(self, product): ❷
        self.session.add(product)
```

```
def _get(self, sku): ❸
    return self.session.query
        (model.Product).filter_by(sku=sku).first()
```

❶ Чтобы паттерн UoW мог публиковать новые события, он должен иметь возможность узнавать у репозитория, какие объекты `Product` использовались во время этого сеанса. Для их хранения применяется множество под названием `.seen`. Это означает, что наши реализации должны вызывать `super().__init__()`.

❷ Родительский метод `add()` добавляет элементы в `.seen`, и теперь требует подклассы для реализации `._add()`.

❸ Схожим образом метод `.get()` делегирует в функцию `.get()`, которая должна быть реализована подклассами для того, чтобы захватывать встречавшиеся объекты.



Применение `._подчеркнутых()` методов и подклассов — далеко не единственный способ реализации паттернов. Попробуйте поэкспериментировать с вариантами, когда будете выполнять упражнение из этой главы.

После такой совместной работы репозитория и UoW по автоматическому отслеживанию объектов и обработке их событий сервисный слой может быть полностью освобожден от обязанностей по обработке событий.

Сервисный слой снова чист (`src/allocation/service_layer/services.py`)

```
def allocate(
   orderid: str, sku: str, qty: int,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(orderid, sku, qty)
    with uow:
        product = uow.products.get(sku=line.sku)
        if product is None:
            raise InvalidSku(f'Недопустимый артикул {line.sku}')
        batchref = product.allocate(line)
        uow.commit()
    return batchref
```

Еще нужно изменить подделки в сервисном слое и сделать так, чтобы в нужных местах он вызывал `super()`, а также реализовать подчеркнутые методы, но это уже мелочи.

Подделки сервисного слоя нуждаются в настройке (tests/unit/test_services.py)

```
class FakeRepository(repository.AbstractRepository):

    def __init__(self, products):
        super().__init__()
        self._products = set(products)

    def _add(self, product):
        self._products.add(product)

    def _get(self, sku):
        return next((p for p in self._products if p.sku == sku), None)

...

class FakeUnitOfWork(unit_of_work.AbstractUnitOfWork):
    ...

    def _commit(self):
        self.committed = True
```

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Вы считаете, что все эти методы `._add()` и `._commit()` являются «сверхгрубыми», выражаясь словами нашего любимого научного редактора Хайнека? Они «вызывают у вас желание настучать Гарри по голове плюшевой змеей»? Но слушайте, эти листинги нужны только для примеров, а не для идеального решения! Попробуйте придумать что-нибудь получше.

Один из способов в стиле «композиция важнее наследования» — реализация класса-оболочки.

Оболочка добавляет функциональность, а затем делегирует (src/adapters/repository.py)

```
class TrackingRepository:
    seen: Set[model.Product]

    def __init__(self, repo: AbstractRepository):
        self.seen = set() # type: Set[model.Product]
        self._repo = repo

    def add(self, product: model.Product): ❶
        self._repo.add(product) ❶
        self.seen.add(product)

    def get(self, sku) -> model.Product:
        product = self._repo.get(sku)
```



```
if product:
    self.seen.add(product)
return product
```

❶ Обернув репозиторий, мы можем вызывать методы `.add()` и `.get()`, избегая странных подчеркнутых методов.

Посмотрите, получится ли у вас применить похожий паттерн к классу `UoW`, чтобы избавиться и от этих Java-подобных методов `_commit()`. Код можно найти на GitHub¹.

Переключение всех абстрактных базовых классов на протокол типизирования `typing.Protocol` — это хороший способ заставить себя избегать использования наследования. Дайте знать, если вы придумаете что-то получше!

А вдруг техническое сопровождение всех этих подделок окажется довольно сложным делом? Нет никаких сомнений, что на это придется потратить кое-какие усилия, но, по нашему опыту, их будет не так уж и много. После того как проект доведен до рабочего состояния, интерфейс для абстракций репозитория и `UoW` и вправду не сильно изменится. И если вы используете абстрактные базовые классы, то они дадут вам знать, когда что-то выходит из синхронизации.

Выводы

События предметной области дают возможность обрабатывать рабочие потоки в системе. В разговорах с экспертами в предметной области мы часто слышим, как они выражают требования причинным или временным образом, например: «Когда мы пытаемся распределить товарные запасы, которых нет в наличии, мы должны отправить письмо команде закупок».

Волшебные слова «когда X, то Y» часто говорят нам о событии, которое можно воплотить в системе. Рассмотрение событий как объектов перво-степенной важности в модели делает код более тестопригодным и обзорным, а также помогает изолировать проблемы. В табл. 8.1 приведены плюсы и минусы.

¹ См. https://github.com/cosmicpython/code/tree/chapter_08_events_and_message_bus_exercise

Таблица 8.1. События предметной области: компромиссы

Плюсы	Минусы
• Шина сообщений дает хороший способ разделения обязанностей, когда следует предпринимать многочисленные действия в ответ на запрос. • Обработчики событий хорошо отцеплены от «стержневой» логики приложения, что позже позволяет легко вносить изменения в их реализацию. • События предметной области — отличный способ моделирования реального мира, и мы можем использовать их как часть делового языка при создании модели вместе со стейкхолдерами	• Шина сообщений — очередной элемент, над которым приходится ломать голову; реализация, в которой UoW инициирует события за нас, является изящной, но загадочной. Неочевидно, что при вызове <code>commit</code> мы также собираемся отправить имейл людям. • Более того, этот скрытый код обработки событий выполняется синхронно, а значит, функция сервисного слоя не завершится до тех пор, пока не будут завершены все обработчики для любых событий. Это может стать причиной неожиданных проблем с производительностью в конечных точках веб-приложения (можно добавить асинхронную обработку, но это сделает все еще более запутанным). • В более общем случае событийно-управляемые рабочие потоки могут сбивать с толку, потому что после того, как все разделено по очереди из нескольких обработчиков, в системе не окажется ни одного места, где можно понять, как будет выполняться запрос. • Вы также рискуете напороться на циклические зависимости между обработчиками событий и бесконечных циклов

КРАТКО О СОБЫТИЯХ ПРЕДМЕТНОЙ ОБЛАСТИ И ШИНЕ СООБЩЕНИЙ

События помогают придерживаться принципа единственной обязанности

Код запутывается, когда в одном месте смешивается несколько обязанностей. События помогают поддерживать порядок, отделяя первичные варианты использования от вторичных. Мы также используем события для обмена данными между агрегатами, благодаря чему не нужно выполнять длительные транзакции, которые блокируют нескольких таблиц.

Шина сообщений направляет сообщения обработчикам

Можно рассматривать шину сообщений как словарь, который попарно сопоставляет события с их потребителями. Она ничего не «знает» о сути событий; это просто элемент немой инфраструктуры для передачи сообщений по всей системе.

Вариант 1: сервисный слой инициирует события и передает их в шину сообщений

Самый простой способ начать использовать события в вашей системе — инициировать их из обработчиков, вызывая `bus.handle(некое_новое_событие)` после того, как вы зафиксируете свой UoW.

Вариант 2: модель предметной области инициирует события, сервисный слой передает их в шину сообщений

Логика в плане выбора момента инициации события действительно должна располагаться вместе с моделью, благодаря чему мы можем улучшить дизайн и тестопригодность системы, иницируя события из модели предметной обла-

сти. Обработчики легко собирают события с объектов модели после фиксации, `commit`, и передают их в канал.

Вариант 3: UoW собирает события из агрегатов и передает их в шину сообщений

Добавлять инструкцию `bus.handle(aggregate.events)` для каждого обработчика надоедает, поэтому можно подчистить код, обязав UoW инициировать события, которые были инициированы загруженными объектами. Этот вариант дизайна является самым сложным и, возможно, будет полагаться на загадочные манипуляции ORM, зато после настройки он чист и прост в использовании.

Но события полезны не только для отправки имейлов. В главе 7 мы потратили много времени, убеждая вас в том, что вы должны определять агрегаты, или границы, где мы гарантируем согласованность. Нас часто спрашивают: «Что делать, если нужно изменить несколько агрегатов в рамках запроса?» Теперь у нас есть инструменты, необходимые для ответа на этот вопрос.

Если есть что-то, что может быть транзакционно изолировано (например, заказ и продукт), то с помощью событий можно сделать их впоследствии согласованными. Когда заказ отменяется, следует найти продукты, на которые был размещен этот заказ, и удалить размещения.

В главе 9 мы рассмотрим эту идею подробнее, выстраивая более сложный рабочий поток с участием новой шины сообщений.

Катимся в город на шине сообщений

В этой главе мы начнем делать события более фундаментальными для внутренней структуры приложения. От текущего состояния на рис. 9.1, где события являются необязательным побочным эффектом, мы перейдем...

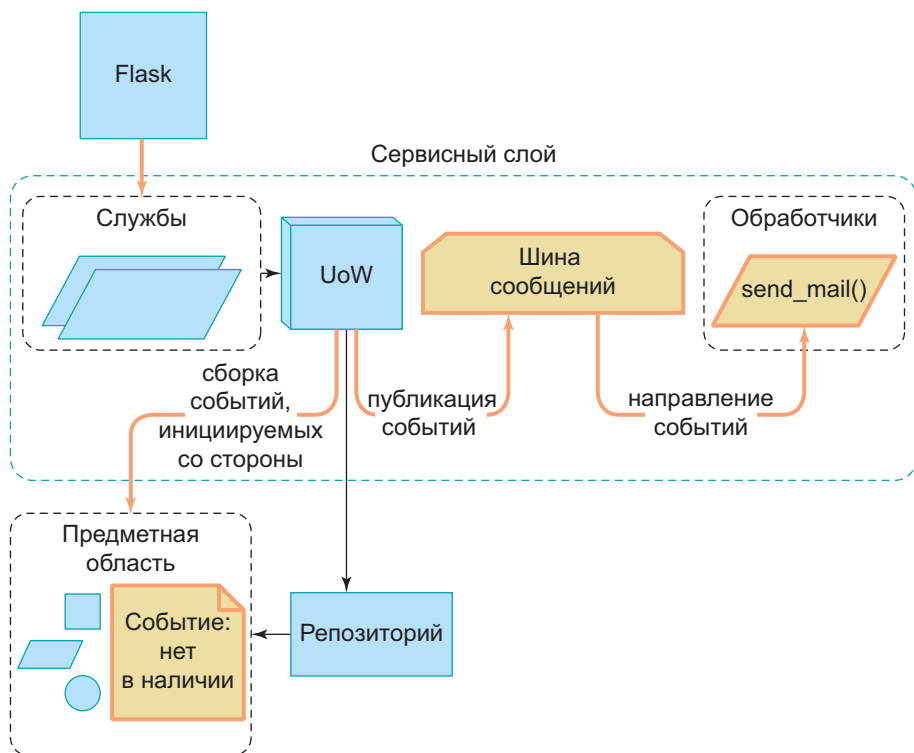


Рис. 9.1. Прежний вид: шина сообщений является необязательным дополнением

...к ситуации на рис. 9.2, где все идет через шину сообщений и приложение было фундаментально преобразовано в процессор сообщений.

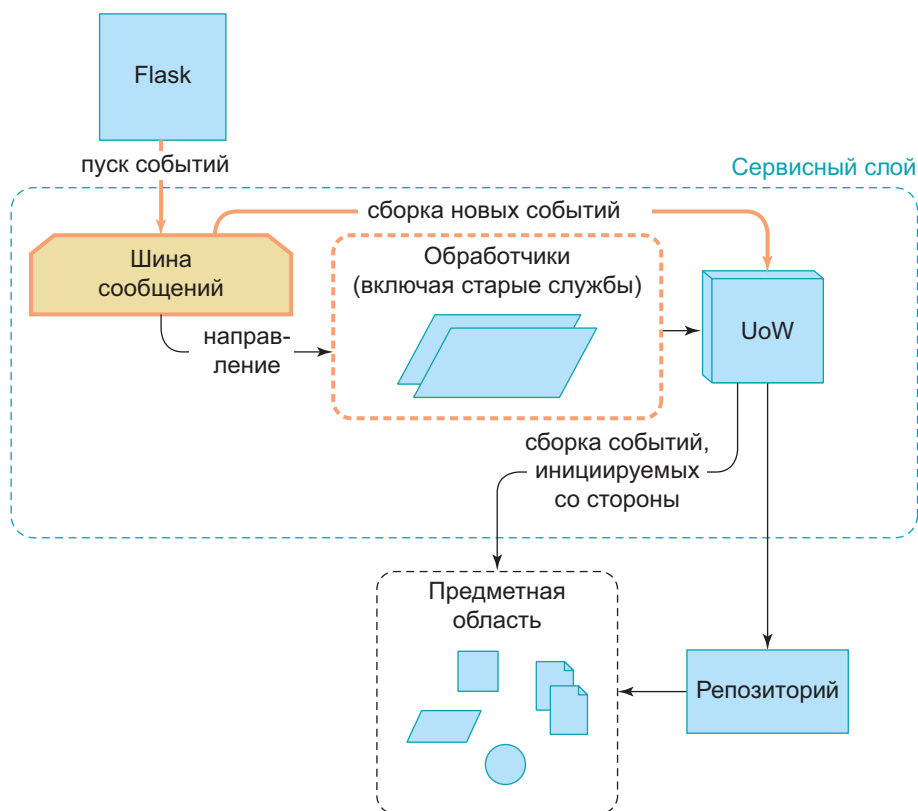


Рис. 9.2. Шина сообщений теперь является главной точкой входа в сервисный слой



Код для этой главы находится в ветке `chapter_09_all_messagebus` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_09_all_messagebus
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_08_events_and_message_bus
```

¹ См. <https://oreil.ly/oKNkn>

Новое требование приводит к новой архитектуре

Рич Хикки говорит о «ситуативном» (situated) ПО, имея в виду, что оно работает в течение длительных периодов, управляя реальным процессом. Примеры включают системы управления складами, логистические планировщики и системы расчета заработной платы.

Такое ПО сложно написать, потому что в реальном мире физических объектов и ненадежных людей внезапные вещи происходят постоянно. Например:

- во время инвентаризации мы обнаруживаем, что три артикула МАТ-РАС-ПРУЖИННЫЙ, SPRINGY-MATTRESS, промокли из-за протекающей на складе крыши;
- партия артикула ВИЛКА-НАДЕЖНАЯ, RELIABLE-FORK, не имеет необходимой документации и простаивает на таможне уже несколько недель. Три штуки артикула ВИЛКА-НАДЕЖНАЯ впоследствии не выдерживают испытания на безопасность и ломаются;
- глобальный дефицит блесков означает, что мы не можем произвести следующую партию артикула ШКАФ-СВЕРКАЮЩИЙ, SPARKLY-BOOKCASE.

В таких ситуациях мы узнаем о необходимости изменения размера партий товара, когда они уже находятся в системе. Возможно, кто-то ошибся номером в декларации или несколько диванов выпали из грузовика. После разговора со стейкхолдером¹ мы моделируем ситуацию, как показано на рис. 9.3.

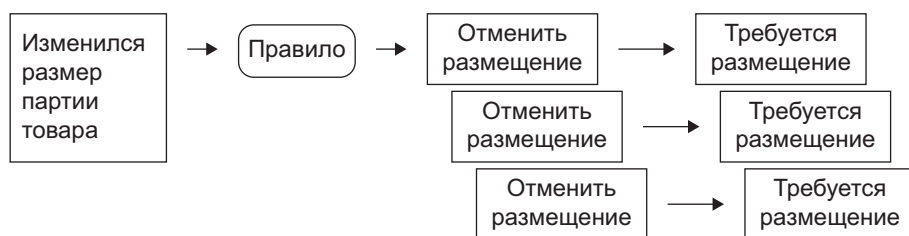


Рис. 9.3. Изменение размера партии товара означает отмену размещения и повторное размещение заказа

¹ Моделирование на основе событий настолько популярно, что для облегчения сбора требований на основе событий и разработки модели предметной области была разработана методика под названием «событийный штурм» (event storming).

Событие, которое мы назовем `BatchQuantityChanged`, должно привести к изменению размера партии, но не только. Мы также выполняем *бизнес-требование*: если в обновленной партии меньше товара, чем было заказано, то нужно *отменить* размещенные в ней заказы. Тогда каждый из этих заказов нужно разместить заново. Новое размещение можно захватывать как событие `AllocationRequired`.

Возможно, вы уже видите, что шина внутренних сообщений и события могли бы с этим помочь. Мы могли бы задать службу `change_batch_quantity`, которая знает, как корректировать размер партий товара, а также как отменять размещение заказа на лишние товарные позиции. Каждая такая отмена может инициировать событие `AllocationRequired`, которое будет перенаправляться в службу `allocate` в отдельных транзакциях. И снова обращаем внимание на то, что шина сообщений помогает обеспечивать соблюдение принципа единственной обязанности, что позволяет принимать взвешенные решения насчет транзакций и целостности данных.

Воображаемое изменение архитектуры: все будет обработчиком событий

Но прежде, чем приступить, подумайте о том, куда мы движемся. Через систему протекает два вида потоков:

- API-вызовы, которые обрабатываются функцией сервисного слоя.
- Внутренние события (которые могут быть инициированы как побочный эффект функции сервисного слоя) и их обработчики (которые, в свою очередь, вызывают функции сервисного слоя).

Разве не проще сделать все обработчиком событий? Если мы переосмыслим вызовы API как захват событий, то функции сервисного слоя также могут быть обработчиками событий, и тогда больше не нужно различать обработчики внутренних и внешних событий.

- `services.allocate()` может быть обработчиком события `AllocationRequired` и может порождать события `Allocated` на выходе.
- `services.add_batch()` может быть обработчиком события `BatchCreated`¹.

¹ Если вы немного читали о событийно-управляемой архитектуре, то, наверное, думаете, что некоторые из этих событий больше похожи на команды! Потерпите немного! Мы пытаемся вводить по одной концепции за раз. В следующей главе рассмотрим различие между командами и событиями.

Новое требование будет соответствовать той же схеме.

- Событие `BatchQuantityChanged` может активизировать обработчик `change_batch_quantity()`.
- Новые события `AllocationRequired`, которые он может инициировать, могут передаваться дальше в `services.allocate()`, поэтому нет никакой концептуальной разницы между совершенно новым размещением, поступающим из API, и повторным размещением, которое внутренне инициируется отменой размещения.

Звучит как-то чересчур, да? Давайте разбираться. Мы будем следовать рабочему процессу подготовительного рефакторинга¹, то есть «упрощать внесение изменений и потом вносить простые изменения».

1. Сделаем рефакторинг сервисного слоя в обработчики событий. Просто привыкните к идее, будто события описывают входы в систему. В частности, прежняя функция `services.allocate()` станет обработчиком для события `AllocationRequired`.
2. Создаем сквозной тест, который помещает события `BatchQuantityChanged` в систему и ищет выходящие события `Allocated`.
3. Реализация концептуально очень простая: это будет новый обработчик для событий `BatchQuantityChanged`, реализация которого будет порождать события `AllocationRequired`, которые, в свою очередь, будут обрабатываться точно таким же используемым в API обработчиком размещений.

По пути мы сделаем небольшую настройку шины сообщений и паттерна UoW, передав обязанность размещения новых событий в шине сообщений в саму шину сообщений.

Рефакторинг функций служб для обработчиков сообщений

Начнем с определения двух событий, которые захватывают текущие входы в API, — `AllocationRequired` и `BatchCreated`.

¹ См. <https://oreil.ly/W3RZM>

События BatchCreated и AllocationRequired (src/allocation/domain/events.py)

```
@dataclass
class BatchCreated(Event):
    ref: str
    sku: str
    qty: int
    eta: Optional[date] = None
```

...

```
@dataclass
class AllocationRequired(Event):
    orderid: str
    sku: str
    qty: int
```

Затем переименуем `services.py` в `handlers.py`, добавим прежний обработчик сообщений для `send_out_of_stock_notification` и, самое главное, поменяем все обработчики так, чтобы они имели одинаковые данные на входе, событие и `UoW`.

Обработчики и службы — это одно и то же (src/allocation/service_layer/handlers.py)

```
def add_batch(
    event: events.BatchCreated, uow: unit_of_work.AbstractUnitOfWork
):
    with uow:
        product = uow.products.get(sku=event.sku)
        ...

def allocate(
    event: events.AllocationRequired,
    uow: unit_of_work.AbstractUnitOfWork
) -> str:
    line = OrderLine(event.orderid, event.sku, event.qty)
    ...

def send_out_of_stock_notification(
    event: events.OutOfStock, uow: unit_of_work.AbstractUnitOfWork,
):
    email.send(
        'stock@made.com',
        f'Артикула {event.sku} нет в наличии',
    )
```

ОТ ОБЪЕКТОВ ПРЕДМЕТНОЙ ОБЛАСТИ ЧЕРЕЗ ОДЕРЖИМОСТЬ ПРИМИТИВАМИ К СОБЫТИЯМ В КАЧЕСТВЕ ИНТЕРФЕЙСА

Некоторые из вас, возможно, помнят раздел «Устранение связей в тестах сервисного слоя с предметной областью» на с. 113, в котором мы поменяли API сервисного слоя с точки зрения объектов предметной области на примитивы. Теперь же мы возвращаемся назад, но к другим объектам. Что это дает?

В объектно-ориентированных кругах говорят об одержимости примитивами как об антипаттерне: избегать примитивов в публичных API и вместо этого, как бы они выразились, обертывать их собственными классами-значениями. В мире Python многие люди относятся к этому весьма скептически. При бездумном применении такой подход лишь все усложняет. Конечно, мы таким не занимаемся.

Переход от объектов предметной области к примитивам помог устранить связи: клиентский код больше не связан непосредственно с предметной областью, поэтому сервисный слой может представлять API, который не изменится, даже если мы поменяем что-нибудь в модели, и наоборот.

Значит, это шаг назад? Ну, ключевые объекты модели предметной области по-прежнему могут меняться, зато внешний мир мы привязали к классам событий. Да, это тоже часть предметной области, но мы надеемся, что классы не придется менять очень уж часто, так что они выглядят подходящими кандидатами для связывания.

А какая нам выгода? Теперь при вызове варианта использования в приложении больше не нужно запоминать конкретную комбинацию примитивов. Держать в уме нужно лишь один класс события, который представляет собой вход в приложение. В концептуальном плане это удобно. Кроме того, как вы увидите в приложении Д в конце книги, указанные событийные классы могут быть хорошим местом для проверки входных данных.

Это изменение, возможно, будет нагляднее в таком виде¹:

Переход от служб к обработчикам (src/allocation/service_layer/handlers.py)

```
def add_batch(  
-     ref: str, sku: str, qty: int, eta: Optional[date],  
-     uow: unit_of_work.AbstractUnitOfWork  
+     event: events.BatchCreated, uow: unit_of_work.AbstractUnitOfWork  
) :  
    with uow:  
-         product = uow.products.get(sku=sku)  
+         product = uow.products.get(sku=event.sku)  
    ...
```

¹ Минус в начале строки означает удаление кода, плюс — добавление. — *Примеч. ред.*

```
def allocate(
-    orderid: str, sku: str, qty: int,
-     uow: unit_of_work.AbstractUnitOfWork
+     event: events.AllocationRequired, uow:
+     unit_of_work.AbstractUnitOfWork
) -> str:
-     line = OrderLine(orderid, sku, qty)
+     line = OrderLine(event.orderid, event.sku, event.qty)
    ...

+
+def send_out_of_stock_notification(
+     event: events.OutOfStock, uow: unit_of_work.AbstractUnitOfWork,
+):
+     email.send(
    ...
```

Попутно мы сделали API сервисного слоя более структурированным и последовательным. Раньше это была россыпь примитивов, теперь же в нем используются четко определенные объекты (см. врезку выше).

Шина сообщений теперь собирает события из UoW

Теперь обработчикам событий нужен UoW. Кроме того, поскольку шина сообщений занимает уже центральное место в приложении, имеет смысл возложить на нее обязанность по сбору и обработке новых событий явным образом. До сих пор существовала некоторая циклическая зависимость между UoW и шиной сообщений, так что это сделает ее односторонней.

Обработчик принимает UoW и управляет очередью (src/allocation/service_layer/messagebus.py)

```
def handle(event: events.Event, uow: unit_of_work.AbstractUnitOfWork): ❶
    queue = [event] ❷
    while queue:
        event = queue.pop(0) ❸
        for handler in HANDLERS[type(event)]: ❹
            handler(event, uow=uow) ❺
            queue.extend(uow.collect_new_events()) ❺
```

❶ Теперь UoW проходит через шину сообщений всякий раз, когда запускается.

❷ Когда мы начинаем обрабатывать первое событие, мы запускаем очередь.

❸ Мы извлекаем события из начала очереди и активизируем их обработчики (словарь `HANDLERS` не изменился; он по-прежнему попарно сопоставляет типы событий с функциями-обработчиками).

❹ Шина сообщений передает UoW дальше каждому обработчику.

❺ После завершения работы каждого обработчика мы собираем все новые сгенерированные события и добавляем их в очередь.

В модуле `unit_of_work.py` `publish_events()` становится менее активным методом `collect_new_events()`:

UoW больше не помещает события непосредственно в шину сообщений (`src/allocation/service_layer/unit_of_work.py`)

```
-from . import messagebus ❶
-

class AbstractUnitOfWork(abc.ABC):
@@ -23,13 +21,11 @@ class AbstractUnitOfWork(abc.ABC):

    def commit(self):
        self._commit()
-        self.publish_events() ❷

-    def publish_events(self):
+    def collect_new_events(self):
        for product in self.products.seen:
            while product.events:
-                event = product.events.pop(0)
-                messagebus.handle(event)
+                yield product.events.pop(0) ❸
```

❶ Модуль `unit_of_work` теперь больше не зависит от `messagebus`.

❷ Мы больше не публикуем события, `publish_events`, автоматически при фиксации. Вместо этого шина сообщений отслеживает очередь событий.

❸ И UoW больше не помещает события в шину сообщений, он просто делает их доступными.

Все тесты тоже написаны с помощью событий

Тесты теперь работают, создавая события и помещая их в шину сообщений вместо того, чтобы активизировать функции сервисного слоя напрямую.

В тестах обработчиков используются события (tests/unit/test_handlers.py)

```
class TestAddBatch:
```

```
    def test_for_new_product(self):
        uow = FakeUnitOfWork()
-         services.add_batch("b1", "CRUNCHY-ARMCHAIR", 100, None, uow)
+         messagebus.handle(
+             events.BatchCreated("b1", "CRUNCHY-ARMCHAIR", 100, None), uow
+         )
        assert uow.products.get("CRUNCHY-ARMCHAIR") is not None
        assert uow.committed

...

```

```
class TestAllocate:
```

```
    def test_returns_allocation(self):
        uow = FakeUnitOfWork()
-         services.add_batch("batch1", "COMPLICATED-LAMP", 100, None, uow)
-         result = services.allocate("o1", "COMPLICATED-LAMP", 10, uow)
+         messagebus.handle(
+             events.BatchCreated("batch1", "COMPLICATED-LAMP", 100,
+                 None), uow
+         )
+         result = messagebus.handle(
+             events.AllocationRequired("o1", "COMPLICATED-LAMP", 10), uow
+         )
        assert result == "batch1"

```

Уродливый костыль: шине сообщений приходится возвращать результаты

API и сервисный слой теперь хотят знать ссылку на размещенную партию товара, когда вызывают обработчик `allocate()`. Это означает, что нужно вставить костыль в шину сообщений, чтобы она смогла возвращать события.

Шина сообщений возвращает результаты (src/allocation/service_layer/messagebus.py)

```
def handle(event: events.Event, uow: unit_of_work.AbstractUnitOfWork):
+     results = []
    queue = [event]
    while queue:
        event = queue.pop(0)
        for handler in HANDLERS[type(event)]:
-             handler(event, uow=uow)
+             results.append(handler(event, uow=uow))
        queue.extend(uow.collect_new_events())
+     return results

```

Способ некрасивый, потому что в системе мы смешиваем обязанности по чтению и записи. Пофиксим все в главе 12.

Изменение API для работы с событиями

Замена в коде Flask на шину сообщений (src/allocation/entrypoints/flask_app.py)

```
@app.route("/allocate", methods=['POST'])
def allocate_endpoint():
    try:
        - batchref = services.allocate(
        -     request.json['orderid'], ❶
        -     request.json['sku'],
        -     request.json['qty'],
        -     unit_of_work.SqlAlchemyUnitOfWork(),
+     event = events.AllocationRequired( ❷
+         request.json['orderid'], request.json['sku'],
+         request.json['qty'],
        )
+     results = messagebus.handle(event,
+                                 unit_of_work.SqlAlchemyUnitOfWork()) ❸
+     batchref = results.pop(0)
    except InvalidSku as e:
```

❶ Вместо того чтобы вызывать сервисный слой со множеством примитивов, извлеченных из JSON-запроса...

❷ ...создаем экземпляр события.

❸ Затем передаем его в шину сообщений.

Ну вот, мы снова вернулись к абсолютно функциональному приложению, но теперь оно полностью управляется событиями.

- То, что раньше было функциями сервисного слоя, теперь является обработчиками событий.
- Это делает их такими же, как и функции, которые мы активизируем для обработки внутренних событий, инициированных моделью предметной области.
- Мы используем события в качестве специальной структуры для сбора данных на входе в систему, а также для передачи внутренних рабочих пакетов.

- Теперь самое подходящее описание для нашего приложения — процессор сообщений или процессор событий, если угодно. Поговорим об этом различии в следующей главе.

Реализация нового требования

С фазой рефакторинга разобрались. Теперь посмотрим, действительно ли мы «упростили внесение изменений». Реализуем новое требование, показанное на рис. 9.4: на входе будем получать новые события `BatchQuantityChanged` и передавать их обработчику, который будет порождать события `AllocationRequired`, а те, в свою очередь, будут возвращаться к существующему обработчику для повторного размещения.

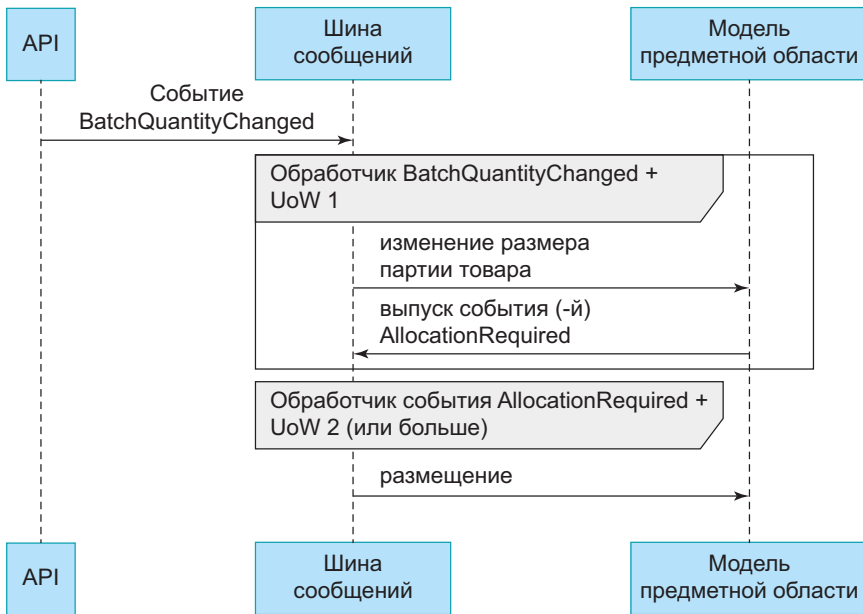


Рис. 9.4. Схема последовательности для потока повторного размещения заказа



Когда вы разделяете такие вещи на два UoW, у вас будут проходить две транзакции базы данных, и таким образом вы навлекаете на себя проблемы целостности данных: может возникнуть ситуация, в которой первая транзакция завершилась, а вторая — нет. Стоит подумать, насколько это приемлемо, нужно ли отслеживать такие вещи и что с ними делать. Подробности см. в разделе «Выстрел в ногу» на с. 287.

Новое событие

Событие, которое сообщает нам о том, что размер партии товара изменился, очень простое; ему нужна только ссылка на партию и ее новый размер.

Новое событие (src/allocation/domain/events.py)

```
@dataclass
class BatchQuantityChanged(Event):
    ref: str
    qty: int
```

Тест-драйв нового обработчика

Вспоминая урок из главы 4, мы можем «ехать на повышенной передаче» и писать юнит-тесты на максимально возможном уровне абстракции с помощью событий. Вот как они могут выглядеть:

Тесты обработчика для change_batch_quantity (tests/unit/test_handlers.py)

```
class TestChangeBatchQuantity:

    def test_changes_available_quantity(self):
        uow = FakeUnitOfWork()
        messagebus.handle(
            events.BatchCreated("batch1", "ADORABLE-SETTEE", 100,
                               None), uow
        )
        [batch] = uow.products.get(sku="ADORABLE-SETTEE").batches
        assert batch.available_quantity == 100 ❶

        messagebus.handle(events.BatchQuantityChanged("batch1", 50), uow)

        assert batch.available_quantity == 50 ❶

    def test_reallocates_if_necessary(self):
        uow = FakeUnitOfWork()
        event_history = [
            events.BatchCreated("batch1", "INDIFFERENT-TABLE", 50, None),
            events.BatchCreated("batch2", "INDIFFERENT-TABLE", 50,
                               date.today()),
            events.AllocationRequired("order1", "INDIFFERENT-TABLE", 20),
            events.AllocationRequired("order2", "INDIFFERENT-TABLE", 20),
        ]
        for e in event_history:
```



```

        messagebus.handle(e, uow)
    [batch1, batch2] = uow.products.get(
        sku="INDIFFERENT-TABLE").batches
    assert batch1.available_quantity == 10
    assert batch2.available_quantity == 50

messagebus.handle(events.BatchQuantityChanged("batch1", 25), uow)

# размещение заказа order1 или order2 будет отменено, и у нас
# будет 25 - 20
assert batch1.available_quantity == 5 ❷
# и 20 будет повторно размещено в следующей партии
assert batch2.available_quantity == 30 ❷

```

❶ Простой случай реализуется тривиально — просто изменяем размер партии.

❷ Если попытаться уменьшить размер партии, но при этом был размещен более крупный заказ, то придется отменить размещение по крайней мере одного заказа, чтобы затем повторно разместить его в новой партии товара.

Реализация

Новый обработчик очень прост.

Обработчик делегирует обязанности в слой модели (src/allocation/service_layer/handlers.py)

```

def change_batch_quantity(
    event: events.BatchQuantityChanged, uow:
        unit_of_work.AbstractUnitOfWork
):
    with uow:
        product = uow.products.get_by_batchref(batchref=event.ref)
        product.change_batch_quantity(ref=event.ref, qty=event.qty)
        uow.commit()

```

Мы понимаем, что понадобится новый тип запроса в репозиторий.

Новый тип запроса в репозиторий (src/allocation/adapters/repository.py)

```

class AbstractRepository(abc.ABC):
    ...

    def get(self, sku) -> model.Product:
        ...

```

```

def get_by_batchref(self, batchref) -> model.Product:
    product = self._get_by_batchref(batchref)
    if product:
        self.seen.add(product)
    return product

@abc.abstractmethod
def _add(self, product: model.Product):
    raise NotImplementedError

@abc.abstractmethod
def _get(self, sku) -> model.Product:
    raise NotImplementedError

@abc.abstractmethod
def _get_by_batchref(self, batchref) -> model.Product:
    raise NotImplementedError
...

class SqlAlchemyRepository(AbstractRepository):
    ...

    def _get(self, sku):
        return self.session.query(model.Product).filter_
            by(sku=sku).first()

    def _get_by_batchref(self, batchref):
        return self.session.query(model.Product).join
            (model.Batch).filter(
                orm.batches.c.reference == batchref,
            ).first()

```

И в поддельный репозиторий, FakeRepository, тоже.

Обновляем поддельный репозиторий (tests/unit/test_handlers.py)

```

class FakeRepository(repository.AbstractRepository):
    ...

    def _get(self, sku):
        return next((p for p in self._products if p.sku == sku), None)

    def _get_by_batchref(self, batchref):
        return next((
            p for p in self._products for b in p.batches
            if b.reference == batchref
        ), None)

```



Мы добавляем запрос в репозиторий, чтобы упростить реализацию этого варианта использования. До тех пор пока запрос возвращает один-единственный агрегат, никакие правила не нарушаются. Если же вы пишете сложные запросы к своим репозиториям, то подумайте об изменении дизайна. Такие методы, как «получить наиболее популярный продукт», `get_most_popular_products`, или «найти продукты по идентификатору заказа», `find_products_by_order_id`, помогут вам найти правильное решение. В главе 11 и эпилоге будет несколько советов по управлению сложными запросами.

Новый метод в модели предметной области

Мы добавляем в модель новый метод, который меняет размер партии и размещение в одной строке кода и публикует новое событие. Изменим и прежнюю функцию размещения с учетом публикации события.

Модель улучшается с учетом нового требования (`src/allocation/domain/model.py`)

```
class Product:
    ...

    def change_batch_quantity(self, ref: str, qty: int):
        batch = next(b for b in self.batches if b.reference == ref)
        batch._purchased_quantity = qty
        while batch.available_quantity < 0:
            line = batch.deallocate_one()
            self.events.append(
                events.AllocationRequired(line.orderid, line.sku,
                                          line.qty)
            )
        ...

class Batch:
    ...
    def deallocate_one(self) -> OrderLine:
        return self._allocations.pop()
```

Подключаем новый обработчик.

Шина сообщений растёт (`src/allocation/service_layer/messagebus.py`)

```
HANDLERS = {
    events.BatchCreated: [handlers.add_batch],
    events.BatchQuantityChanged: [handlers.change_batch_quantity],
    events.AllocationRequired: [handlers.allocate],
    events.OutOfStock: [handlers.send_out_of_stock_notification],
} # тип: Dict[Type[events.Event], List[Callable]]
```

И вот новое требование теперь полностью реализовано.

Необязательно: юнит-тест обработчиков событий в изоляции с помощью поддельной шины сообщений

Главный тест для рабочего потока повторного размещения выполняется «от края до края» (см. пример в разделе «Тест-драйв нового обработчика» на с. 192). В нем используется реальная шина сообщений и тестируется весь поток, где обработчик событий `BatchQuantityChanged` запускает отмену размещения и порождает новые события `AllocationRequired`, которые, в свою очередь, обрабатываются их собственными обработчиками. Один тест работает с цепочкой из нескольких событий и обработчиков.

В зависимости от сложности цепочки событий вы можете решить протестировать некоторые обработчики изолированно. Это можно сделать с помощью «поддельной» шины сообщений.

В нашем случае мы фактически вмешиваемся во внутреннее устройство, изменяя метод `publish_events()`, определенный на `FakeUnitOfWork`, и устраняем связь с реальной шиной сообщений, вместо этого делая так, чтобы она регистрировала то, что видит.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Отличный способ заставить себя по-настоящему понять любой код — выполнить его рефакторинг. При обсуждении тестирования обработчиков в изоляции мы использовали то, что называется поддельным UoW с поддельной шиной сообщений, `FakeUnitOfWorkWithFakeMessageBus`, что неоправданно сложно и нарушает принцип единственной обязанности.

Если мы реорганизуем шину сообщений в класс¹, то создавать `FakeMessageBus` будет проще.

Абстрактная шина сообщений и ее реальная и поддельная версии

```
class AbstractMessageBus:
    HANDLERS: Dict[Type[events.Event], List[Callable]]

    def handle(self, event: events.Event):
        for handler in self.HANDLERS[type(event)]:
            handler(event)
```

¹ В «простой» реализации в этой главе, по существу, сам модуль `messagebus.py` используется для реализации паттерна «Одиночка».

```
class MessageBus(AbstractMessageBus):
    HANDLERS = {
        events.OutOfStock: [send_out_of_stock_notification],
    }

class FakeMessageBus(messagebus.AbstractMessageBus):
    def __init__(self):
        self.events_published = [] # type: List[events.Event]
        self.handlers = {
            events.OutOfStock: [lambda e: self.events_
                                published.append(e)]
        }
```

Поэтому перейдите к коду на GitHub¹, посмотрите, сможете ли вы получить рабочую версию на основе класса, а затем напишите версию изолированного теста `test_reallocates_if_necessary_isolated()` из более ранних.

Если вам нужно больше вдохновения для поиска правильного решения, гляньте главу 13: там шина сообщений используется как класс.

Поддельная шина сообщений реализована в паттерне UoW (tests/unit/test_handlers.py)

```
class FakeUnitOfWorkWithFakeMessageBus(FakeUnitOfWork):
```

```
    def __init__(self):
        super().__init__()
        self.events_published = [] # тип: List[events.Event]

    def publish_events(self):
        for product in self.products.seen:
            while product.events:
                self.events_published.append(product.events.pop(0))
```

Теперь, когда мы активизируем `messagebus.handle()`, используя `FakeUnitOfWorkWithFakeMessageBus`, он запускает только обработчик для этого события. Таким образом, мы можем написать более изолированный юнит-тест: вместо того чтобы проверять все побочные эффекты, мы просто убеждаемся, что `BatchQuantityChanged` приводит к `AllocationRequired`, если размер партии становится ниже уже размещенного суммарного числа заказанных товаров.

¹ См. https://github.com/cosmicpython/code/tree/chapter_09_all_messagebus

Тестирование перераспределения изолированно (tests/unit/test_handlers.py)

```
def test_reallocates_if_necessary_isolated():
    uow = FakeUnitOfWorkWithFakeMessageBus()

    # тестовые условия, как и раньше
    event_history = [
        events.BatchCreated("batch1", "INDIFFERENT-TABLE", 50, None),
        events.BatchCreated("batch2", "INDIFFERENT-TABLE", 50,
            date.today()),
        events.AllocationRequired("order1", "INDIFFERENT-TABLE", 20),
        events.AllocationRequired("order2", "INDIFFERENT-TABLE", 20),
    ]
    for e in event_history:
        messagebus.handle(e, uow)
    [batch1, batch2] = uow.products.get(sku="INDIFFERENT-TABLE").batches
    assert batch1.available_quantity == 10
    assert batch2.available_quantity == 50

    messagebus.handle(events.BatchQuantityChanged("batch1", 25), uow)

    # подтвердить истинность на новых порожденных событиях,
    # а не на последующих побочных эффектах
    [reallocation_event] = uow.events_published
    assert isinstance(reallocation_event, events.AllocationRequired)
    assert reallocation_event.orderid in {'order1', 'order2'}
    assert reallocation_event.sku == 'INDIFFERENT-TABLE'
```

Необходимость прибегать к этому зависит от сложности цепочки событий. Начните с edge-to-edge-тестирования и прибегайте к нему только в случае необходимости.

Выводы

Подведем итоги.

Чего мы достигли

События — это простые классы данных, которые определяют структуры для данных на входе в систему и сообщений внутри системы. Они довольно мощные с точки зрения DDD, поскольку события часто очень хорошо переводятся на деловой язык (погуглите «событийный шторм», если еще этого не сделали).

Обработчики — это то, как мы реагируем на события. Они могут обращаться к модели либо к внешним службам. Если потребуется, то мы можем определить несколько обработчиков для одного события. Обработчики также могут инициировать другие события. Это позволяет поддерживать высокую гранулярность в отношении действий обработчика и по-настоящему придерживаться принципа единственной обязанности.

Зачем это нужно

Мы используем паттерны в попытке добиться того, чтобы сложность приложения росла медленнее, чем его размер. Когда мы ставим все на шину сообщений, то с точки зрения архитектурной сложности идем на компромисс (табл. 9.1), но получаем паттерн, который обрабатывает практически случайные сложные требования без необходимости каких-либо концептуальных или архитектурных изменений в процессах.

Мы добавили довольно сложный вариант использования (изменить размер партии, отменить размещение, начать новую транзакцию, разместить повторно, опубликовать внешнее уведомление), но в архитектурном плане сложнее не стало. Мы добавили новые события, обработчики и внешний адаптер (для электронной почты). Все они строятся на основе *существующих вещей* в нашей архитектуре. Мы можем легко понять эти вещи, обосновать их использование и объяснить их кому-либо со стороны. Каждая из частей выполняет одну задачу, они четко связаны друг с другом, и никаких неожиданных побочных эффектов нет.

Таблица 9.1. Все приложение — шина сообщений: компромиссы

Плюсы	Минусы
• Обработчики и службы — это одно и то же, так что это проще. • Есть изящная структура данных на входе в систему	• Шина сообщений по-прежнему является несколько непредсказуемым способом работы с точки зрения веб-процессов. Заранее не известно, когда все закончится. • Неизбежно дублирование между объектами модели и событиями, что усложняет обслуживание. Если добавить поле к одному из объектов, то придется добавлять поле и как минимум к одному из событий, и наоборот

Вам интересно, откуда берутся эти события `BatchQuantityChanged`? Ответ дадим через пару глав. Но сначала поговорим о событиях и командах.

Команды и обработчик команд

В предыдущей главе мы говорили об использовании событий как способа представления данных на входе в систему. Мы превратили наше приложение в машину для обработки сообщений.

Ради этого мы преобразовали все функции варианта использования в обработчики событий. Когда API получает POST для создания новой партии товара, он создает новое событие `BatchCreated` и обрабатывает его так, как если бы это было внутреннее событие. Это может показаться нелогичным. В конце концов, партия товара еще не создана, именно поэтому мы и вызывали API. Мы собираемся исправить этот концептуальный недостаток с помощью ввода команд и покажем, как они могут обрабатываться одной и той же шиной сообщений по другим правилам.



Код для этой главы находится в ветке `chapter_10_commands` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_10_commands
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_09_all_messagebus
```

Команды и события

Подобно событиям, *команды* представляют собой тип сообщений — инструкции, посылаемые одной частью системы другой. Мы обычно представляем команды немymi² структурами данных и можем обрабатывать их почти так же, как события.

¹ См. https://oreil.ly/U_VGa

² «Немыми» (*dumb*) нередко называются объекты типа `struct` или аналогичные им примитивные структуры данных. — *Примеч. пер.*

Но различия между командами и событиями очень важны.

Команды посылаются одним актором другому конкретному актору в надежде, что в результате произойдет то или иное событие. Когда мы отправляем форму в обработчик API, то посылаем команду. Командам даются имена в форме глаголов повелительного наклонения вроде «allocate stock» («разместить товарный запас») или «delay shipment» («задержать поставку»).

Команды улавливают *намерение* (intent). Они выражают то, чего мы хотим от системы. Когда команды не выполняются, отправитель в результате должен получить информацию об ошибке.

Акторы транслируют *события* всем заинтересованным слушателям (listeners). Когда мы публикуем событие `BatchQuantityChanged`, то не знаем, кто его подхватит. Событиям даются имена в форме глаголов прошедшего времени или причастных оборотов вроде «order allocated to stock» («заказ размещен в товарном запасе») или «shipment delayed» («отгрузка задержана»).

Мы часто используем события, чтобы сообщить об успешных командах.

События улавливают *факты* о том, что происходило в прошлом. Поскольку мы не знаем, кто будет обрабатывать событие, отправителей не должно волновать, удалось получателям выполнить команду или нет. В табл. 10.1 сравниваются события и команды.

Таблица 10.1. События в сравнении с командами

	Событие	Команда
Форма имени	Прошедшее время	Повелительное наклонение
Обработка ошибок	Независимый отказ	Заметный отказ
Отправка	Всем слушателям	Одному получателю

Какие команды сейчас есть в нашей системе?

Вытаскиваем несколько команд (src/allocation/domain/commands.py)

```
class Command:
    pass

@dataclass
class Allocate(Command): ❶
```

```

    orderid: str
    sku: str
    qty: int

@dataclass
class CreateBatch(Command): ❷
    ref: str
    sku: str
    qty: int
    eta: Optional[date] = None

@dataclass
class ChangeBatchQuantity(Command): ❸
    ref: str
    qty: int

```

- ❶ `commands.Allocate` заменит `events.AllocationRequired`.
- ❷ `commands.CreateBatch` заменит `events.BatchCreated`.
- ❸ `commands.ChangeBatchQuantity` заменит `events.BatchQuantityChanged`.

Различия в обработке исключений

Замена имен и глаголов — это прекрасно, но жонглирование словами не поменяет поведение системы. Мы хотим обращаться с событиями почти так же, как с командами, но не одинаково. Давайте посмотрим, как изменится шина сообщений.

Направляем события и команды по-разному (`src/allocation/service_layer/messagebus.py`)

```

Message = Union[commands.Command, events.Event]

def handle(message: Message, uow: unit_of_work.AbstractUnitOfWork): ❶
    results = []
    queue = [message]
    while queue:
        message = queue.pop(0)
        if isinstance(message, events.Event):
            handle_event(message, queue, uow) ❷
        elif isinstance(message, commands.Command):
            cmd_result = handle_command(message, queue, uow) ❷
            results.append(cmd_result)
        else:
            raise Exception(f'{message} was not an Event or Command')
    return results

```

❶ У нее все еще есть основная точка входа `handle()`, принимающая `message` — команду либо событие.

❷ Мы отправляем события и команды двум разным вспомогательным функциям, которые показаны ниже.

Вот как происходит работа с событиями:

События не могут прервать поток (`src/allocation/service_layer/messagebus.py`)

```
def handle_event(
    event: events.Event,
    queue: List[Message],
    uow: unit_of_work.AbstractUnitOfWork
):
    for handler in EVENT_HANDLERS[type(event)]: ❶
        try:
            logger.debug('handling event %s with handler %s', event,
                          handler)
            handler(event, uow=uow)
            queue.extend(uow.collect_new_events())
        except Exception:
            logger.exception('Exception handling event %s', event)
            continue ❷
```

❶ События передаются диспетчеру, который может делегировать их многочисленным обработчикам для каждого события.

❷ Он отлавливает и логирует ошибки, но не позволяет им прерывать обработку сообщений.

И вот как выполняются команды:

Команды заново иницируют исключения (`src/allocation/service_layer/messagebus.py`)

```
def handle_command(
    command: commands.Command,
    queue: List[Message],
    uow: unit_of_work.AbstractUnitOfWork
):
    logger.debug('handling command %s', command)
    try:
        handler = COMMAND_HANDLERS[type(command)] ❶
        result = handler(command, uow=uow)
        queue.extend(uow.collect_new_events())
        return result ❸
    except Exception:
        logger.exception('Exception handling command %s', command)
        raise ❷
```

❶ Диспетчер команд ожидает, что для каждой команды будет лишь один обработчик.

❷ Если иницируются какие-либо ошибки, то они быстро останавливают работу и потом всплывают.

❸ Инструкция `return result` с нами ненадолго; как уже упоминалось в подразделе «Уродливый костыль: шине сообщений приходится возвращать результаты» на с. 189, это костыль, позволяющий шине сообщений возвращать ссылку на партию товара, чтобы API мог его использовать. Мы исправим это в главе 12.

Мы также заменяем единый словарь `HANDLERS` разными словарями для команд и событий. По соглашению команды могут иметь только один обработчик.

Новые словари с обработчиками (`src/allocation/service_layer/messagebus.py`)

```
EVENT_HANDLERS = {
    events.OutOfStock: [handlers.send_out_of_stock_notification],
} # тип: Dict[Type[events.Event], List[Callable]]

COMMAND_HANDLERS = {
    commands.Allocate: handlers.allocate,
    commands.CreateBatch: handlers.add_batch,
    commands.ChangeBatchQuantity: handlers.change_batch_quantity,
} # тип: Dict[Type[commands.Command], Callable]
```

События, команды и обработка ошибок

Многие разработчики чувствуют себя некомфортно на этом этапе и спрашивают: «Что происходит, когда событие не обрабатывается? Что я должен сделать, чтобы система находилась в согласованном состоянии?» Если удастся обрабатывать половину событий в `messagebus.handle`, прежде чем процесс остановится из-за нехватки памяти, то как быть с проблемами из-за потерянных сообщений?

Начнем с худшего случая: событие обработать не получается и система остается в несогласованном состоянии. Какая ошибка могла бы к этому привести? Нередко системы оказываются в таком состоянии, когда завершена только половина операции.

Представьте: мы размещаем заказ клиента на три штуки артикула КРЕСЛО_ЖЕЛАННОЕ, `DESIRABLE_BEANBAG`, но каким-то образом у нас не

получается уменьшить количество оставшегося товара. Это привело бы к несогласованному состоянию: три штуки товара одновременно и размещены в заказе, и имеются в наличии, в зависимости от того, как вы на это посмотрите. И если позже мы разместим еще один заказ на те же самые кресла, то у службы поддержки клиентов начнутся проблемы.

Но мы уже позаботились о таких ситуациях с помощью настройки службы размещения заказов. Мы тщательно определили *агрегаты*, которые действуют как границы согласованности, и ввели паттерн *UoW*, который управляет атомарным успехом или неудачей обновления в агрегате.

Например, когда мы размещаем заказ на товар, границей согласованности является агрегат *Product*. Это означает, что мы не можем нечаянно превысить лимит размещений: конкретная товарная позиция заказа либо размещена, либо нет — тут нет места для несогласованных состояний.

По определению мы не требуем, чтобы два агрегата согласовывались сразу, поэтому если мы не сможем обработать событие и обновить только один агрегат, то система все равно может впоследствии стать согласованной. Мы не должны нарушать никаких системных ограничений.

С помощью этого примера мы можем лучше понять причину разбивки сообщений на команды и события. Когда пользователь хочет, чтобы система что-то сделала, мы представляем его запрос как *команду*. Эта команда должна изменить один *агрегат* и либо справиться с работой, либо нет. Все остальные мелкие детали вроде очистки и рассылки уведомлений можно организовать с помощью *события*. Успешность команды не зависит от успешности обработчиков событий.

Давайте рассмотрим еще один пример (из другого, воображаемого проекта), чтобы понять почему.

Представьте, что мы проектируем онлайн-магазин, который продает предметы роскоши. Отдел маркетинга хочет вознаграждать клиентов за повторные заказы. Мы будем отмечать клиентов как VIP-персон после того, как они совершат свою третью покупку, и это даст им право на приоритетное обслуживание и специальные предложения. Критерии принятия решения о клиенте по этой схеме имеют следующее содержание:

При наличии клиента с двумя заказами в своей истории, когда клиент делает третий заказ, он должен быть помечен как VIP-персона.

Когда клиент становится VIP-персоной впервые, мы должны отправить ему имейл с поздравлениями

Мы применяем технические приемы, уже рассмотренные в этой книге, и создаем новый агрегат History, который регистрирует заказы и может инициировать события предметной области, когда эти правила удовлетворяются. Мы структурируем код следующим образом:

VIP-клиент (пример кода для другого проекта)

```
class History: # Агрегат

    def __init__(self, customer_id: int):
        self.orders = set() # Set[HistoryEntry]
        self.customer_id = customer_id

    def record_order(self, order_id: str, order_amount: int): ❶
        entry = HistoryEntry(order_id, order_amount)

        if entry in self.orders:
            return

        self.orders.add(entry)

        if len(self.orders) == 3:
            self.events.append(
                CustomerBecameVIP(self.customer_id)
            )

    def create_order_from_basket(uow, cmd: CreateOrder): ❷
        with uow:
            order = Order.from_basket(cmd.customer_id, cmd.basket_items)
            uow.orders.add(order)
            uow.commit() # инициирует OrderCreated

    def update_customer_history(uow, event: OrderCreated): ❸
        with uow:
            history = uow.order_history.get(event.customer_id)
            history.record_order(event.order_id, event.order_amount)
            uow.commit() # инициирует CustomerBecameVIP

    def congratulate_vip_customer(uow, event: CustomerBecameVip): ❹
        with uow:
            customer = uow.customers.get(event.customer_id)
            email.send(
                customer.email_address,
```

```
        f'Congratulations {customer.first_name}!'
    )
```

❶ Агрегат `History` улавливает правила перевода клиента в VIP-катеорию. Отличное решение на случай, если в будущем правила станут более сложными.

❷ Первый обработчик создает заказ для клиента и инициирует событие предметной области `OrderCreated`.

❸ Второй обработчик обновляет объект `History`, регистрируя заказ.

❹ Наконец, мы отправляем имейл клиенту, когда он становится VIP-персоной.

По этому коду можно получить некоторое представление об обработке ошибок в событийно-управляемой системе.

В текущей реализации мы инициируем события об агрегате *после* того, как сохранили состояние в базе данных. Что, если мы инициировали эти события до операции сохранения и зафиксировали все изменения одновременно? Ведь, казалось бы, так можно быть уверенными в том, что вся работа была завершена. Разве это не безопаснее?

Но что произойдет, если почтовый сервер будет загружен? Если вся работа должна быть завершена в одно и то же время, то занятый почтовый сервер не даст получить деньги за заказы.

Что произойдет, если в реализации агрегата `History` есть баг? Неужели не получится забрать деньги только потому, что в клиенте не удастся распознать VIP-персону?

Благодаря разделению обязанностей получилось изолировать неудачи отдельных частей системы, что повысило ее общую надежность. Единственная часть, которая действительно *должна* завершиться — это обработчик команд, которой создает заказ. Это единственная часть, которая интересует клиента, и ставится в приоритет у стейкхолдеров.

Заметьте, как ловко мы совместили транзакционные границы с началом и концом бизнес-процессов. Используемые в коде имена соответствуют жаргону компании, а обработчики — этапам наших критериев на естественном языке. Это соответствие имен и структуры помогает лучше понимать системы по мере их разрастания.

Синхронное восстановление после ошибок

Надеемся, мы убедили вас в том, что если события отказывают независимо от команд, которые их инициировали, то это вполне нормально. Что же делать, чтобы оправиться от неизбежных ошибок?

Первое — знать, *когда* произошла ошибка, и для этого мы обычно полагаемся на логи.

Давайте еще раз посмотрим на метод `handle_event` из шины сообщений.

Текущая функция-обработчик (`src/allocation/service_layer/messagebus.py`)

```
def handle_event(
    event: events.Event,
    queue: List[Message],
    uow: unit_of_work.AbstractUnitOfWork
):
    for handler in EVENT_HANDLERS[type(event)]:
        try:
            logger.debug('Обработка события %s обработчиком %s',
                          event, handler)
            handler(event, uow=uow)
            queue.extend(uow.collect_new_events())
        except Exception:
            logger.exception('Исключение при обработке события %s',
                             event)
            continue
```

Когда мы обрабатываем сообщение в системе, то сначала записываем в лог, что собираемся сделать. Для нашего варианта использования `CustomerBecameVIP` (клиент становится VIP) логи могут выглядеть следующим образом:

```
Обработка события CustomerBecameVIP(customer_id=12345)
обработчиком <функция congratulate_vip_customer at 0x10ebc9a60>
```

Поскольку для типов сообщений мы решили использовать `dataclasses`, то получаем аккуратно выведенную сводку входящих данных, которую можно скопировать и вставить в оболочку Python, чтобы воссоздать объект.

При возникновении ошибки можно использовать записанные данные, чтобы воспроизвести проблему в юнит-тесте или сообщение в системе.

Ручное воспроизведение хорошо работает в тех случаях, когда нужно исправить баг, перед тем как обработать событие повторно. Однако системы *всегда* будут испытывать некоторый фоновый уровень самоустраняющегося отказа. Сюда входят, например, сбои в сети, взаимоблокировки таблиц и кратковременные простои, вызванные развертыванием.

В большинстве случаев можно элегантно восстановиться, попробовав еще раз. Народная мудрость гласит: «Если с первого раза не получилось, повторите операцию с экспоненциальным откатом».

Обработчик с повторной попыткой (src/allocation/service_layer/messagebus.py)

```
from tenacity import Retrying, RetryError, stop_after_attempt, wait_exponential ❶
```

```
...
```

```
def handle_event(
    event: events.Event,
    queue: List[Message],
    uow: unit_of_work.AbstractUnitOfWork
):
    for handler in EVENT_HANDLERS[type(event)]:
        try:
            for attempt in Retrying( ❷
                stop=stop_after_attempt(3),
                wait=wait_exponential()
            ):
                with attempt:
                    logger.debug('Обработка события %s обработчиком %s', event, handler)
                    handler(event, uow=uow)
                    queue.extend(uow.collect_new_events())
        except RetryError as retry_failure:
            logger.error(
                'Не получилось обработать событие %s раз, отказ!',
                retry_failure.last_attempt.attempt_number
            )
            continue
```

❶ Tenacity — это библиотека, в которой реализованы часто встречающиеся паттерны для повторных попыток.

❷ Здесь мы настраиваем шину сообщений на повторение операций до трех раз с экспоненциально увеличивающимся ожиданием между попытками.

Повторная попытка выполнения операций, которые могут завершиться неудачно, — это, вероятно, единственный лучший способ повысить отказоустойчивость ПО. Опять же из паттернов UoW и «Обработчик команд» следует, что каждая попытка начинается с согласованного состояния и не оставляет дела наполовину законченными.



В какой-то момент придется отказаться от попыток обработать сообщение. Создавать надежные системы с распределенными сообщениями сложно, и здесь мы опустим хитрости. В эпилоге есть ссылки на дополнительные источники.

Выводы

В этой книге мы решили рассказать о *событиях*, и только потом о *командах*, хотя в других книгах обычно делается наоборот. Сделать явными запросы, на которые наша система может ответить, дав им имя и собственную структуру данных, — довольно фундаментальная вещь. Вероятно, вы столкнетесь с тем, что люди называют паттерном «Обработчик команд» то, что мы делаем с паттернами «Событие», «Команда» и «Шина сообщений».

В табл. 10.2 описаны некоторые моменты, о которых следует подумать, прежде чем приступить к работе.

Таблица 10.2. Разбивка на команды и события: компромиссы

Плюсы	Минусы
- Разная трактовка команд и событий помогает понять то, что обязательно должно выполняться, а что можно привести в порядок позже. - Имя CreateBatch (создать партию товара) определенно менее запутанное, чем BatchCreated (партия товара создана). Мы выражаем намерения наших пользователей явным образом, а явное лучше, чем неявное, верно?	- Семантические различия между командами и событиями могут быть незначительными. На эту тему порой случаются жаркие споры. - Мы явно напрашиваемся на неудачу. Мы знаем, что иногда что-то ломается, и решаем справиться с этим, делая свои более мелкими и изолированными. Это может привести к тому, что система станет менее понятной и потребует более качественного мониторинга

В главе 11 поговорим об использовании событий в качестве паттерна интеграции.

Событийно-управляемая архитектура: использование событий для интеграции микросервисов

В предыдущей главе так и не обсудили, как *именно* будем получать события «размер партии товара изменился», да и вообще, как мы могли бы уведомлять внешний мир о повторном размещении.

У нас есть микросервис с веб-API, но как насчет других способов общения с другими системами? Как мы узнаем, если, скажем, поставка задерживается или количество товара изменяется? Как мы сообщим складу, что заказ размещен и должен быть отправлен клиенту?

В этой главе мы хотели бы показать, как расширить метафору событий, чтобы охватить сообщения, которые входят в систему и выходят из нее. Внутреннее ядро нашего приложения теперь представляет собой обработчик сообщений. Давайте продолжим двигаться в этом направлении и сделаем так, чтобы наше приложение снаружи тоже превратилось в обработчик сообщений. Как показано на рис. 11.1, приложение будет получать события из внешних источников через шину внешних сообщений (в качестве примера мы будем использовать очереди «издатель/подписчик» (pub/sub) хранилища Redis) и публиковать выходные данные в форме событий там же.



Код для этой главы находится в ветке `chapter_11_external_events` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_11_external_events
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_10_commands
```

¹ См. <https://oreil.ly/UiuRS>

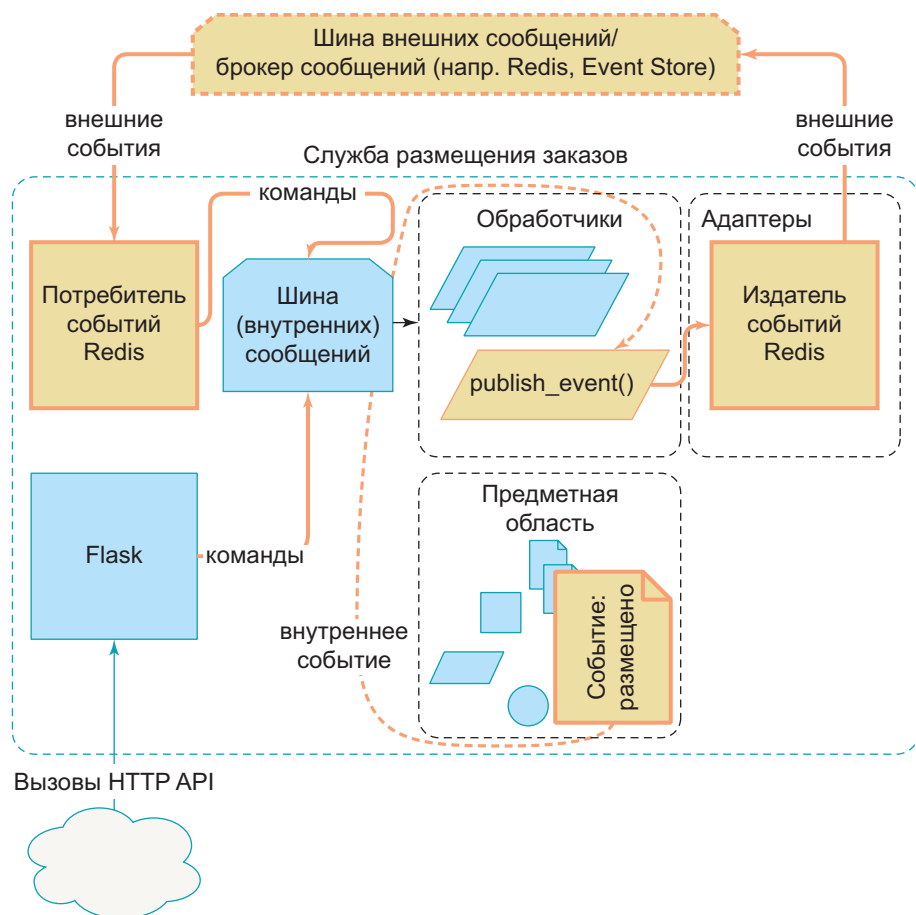


Рис. 11.1. Наше приложение — это обработчик сообщений

Распределенный комок грязи, или Мыслить существительными

Прежде чем перейти к этой теме, поговорим об альтернативах. Мы постоянно общаемся с инженерами, которые пытаются создать архитектуру на основе микросервисов. Порой они выполняют миграцию существующего приложения и при этом инстинктивно пытаются подразделить свою систему на *существительные* (nouns).

Какие существительные мы до этого использовали в нашей системе? Скажем так, у нас есть партии товара, заказы, продукты и клиенты. И поэтому

наивная попытка разделить систему могла бы выглядеть так, как показано на рис. 11.2 (обратите внимание, что мы назвали нашу систему существительным «Партии» (Batches), а не «Размещение заказов» (Allocation)).

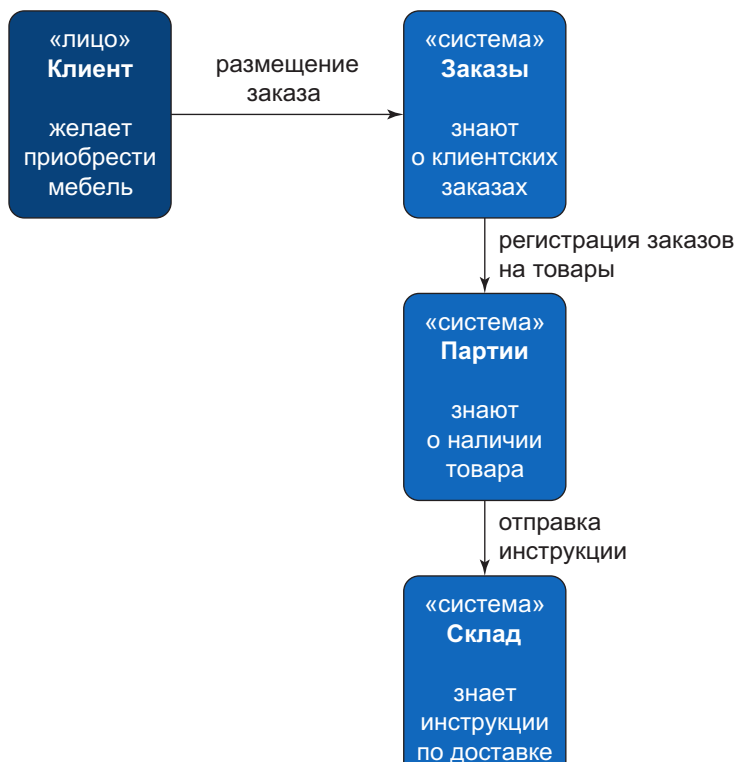


Рис. 11.2. Диаграмма контекста со службами на основе существительных

Каждая «штука» в системе имеет соответствующую службу, которая предоставляет HTTP API.

Давайте рассмотрим пример процесса по счастливому пути на рис.11.3: наши пользователи посещают веб-сайт и могут выбирать товары из имеющихся в наличии. Когда пользователь добавляет товар в корзину, мы бронируем для него этот товар. Когда заказ готов, мы подтверждаем бронирование, что заставляет нас отправить инструкции по доставке на склад. Предположим также, что если это третий заказ клиента, то мы обновляем клиентские данные и присваиваем ему VIP-статус.

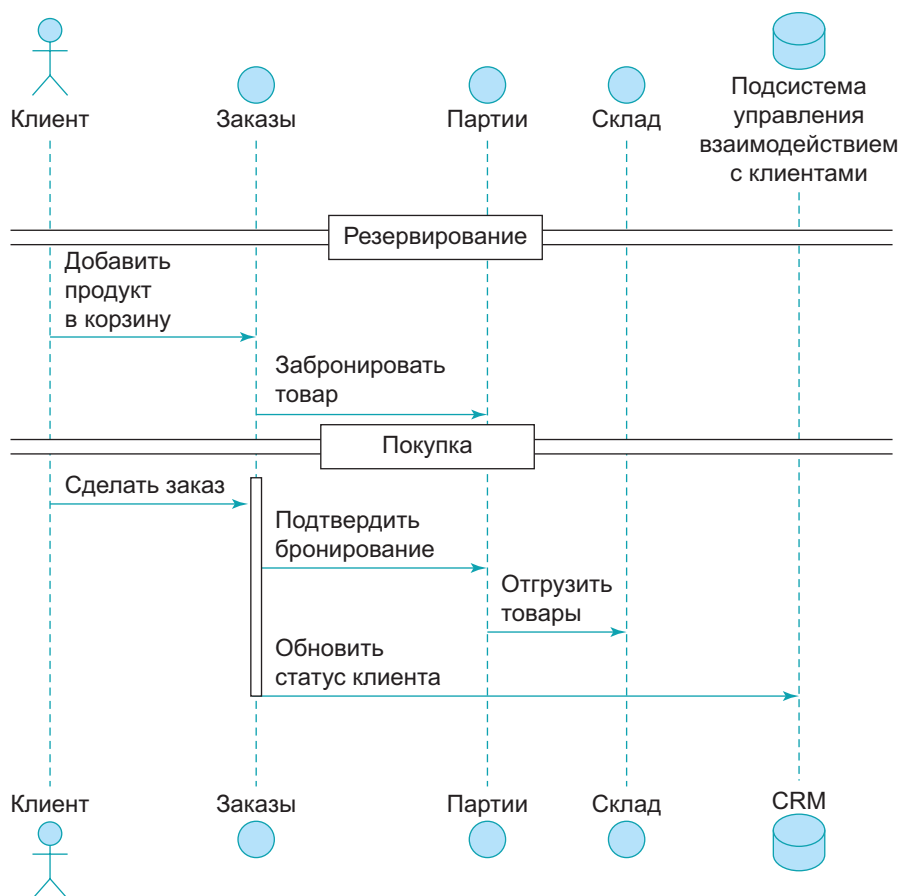


Рис. 11.3. Поток команд 1

Можно рассматривать каждый шаг как команду системы: забронировать товар (`ReserveStock`), подтвердить бронирование (`ConfirmReservation`), отгрузить товары (`DispatchGoods`), присвоить клиенту VIP-статус (`MakeCustomerVIP`) и т. д.

Такой стиль архитектуры, где для каждой таблицы базы данных создаются микросервисы, а HTTP API обрабатываются как CRUD-интерфейсы для анемичных моделей — самый популярный изначальный подход к сервис-ориентированному дизайну.

Он *прекрасно* работает с элементарными системами, но все рискует быстро скатиться в распределенный комок грязи.

Чтобы понять причину, рассмотрим еще один случай. Иногда на склад прибывает брак по вине транспортировки. Мы не можем продавать бракованные диваны, так что приходится их списывать и запрашивать больше запасов у наших партнеров. Также нужно обновлять модель запасов, что может повлечь за собой повторное размещение заказа клиента.

Куда ведет такая логика?

Ну, вообще-то, складская система знает, что товар пришел с браком, поэтому, скорее всего, этот процесс должен оставаться за ней, как показано на рис. 11.4.

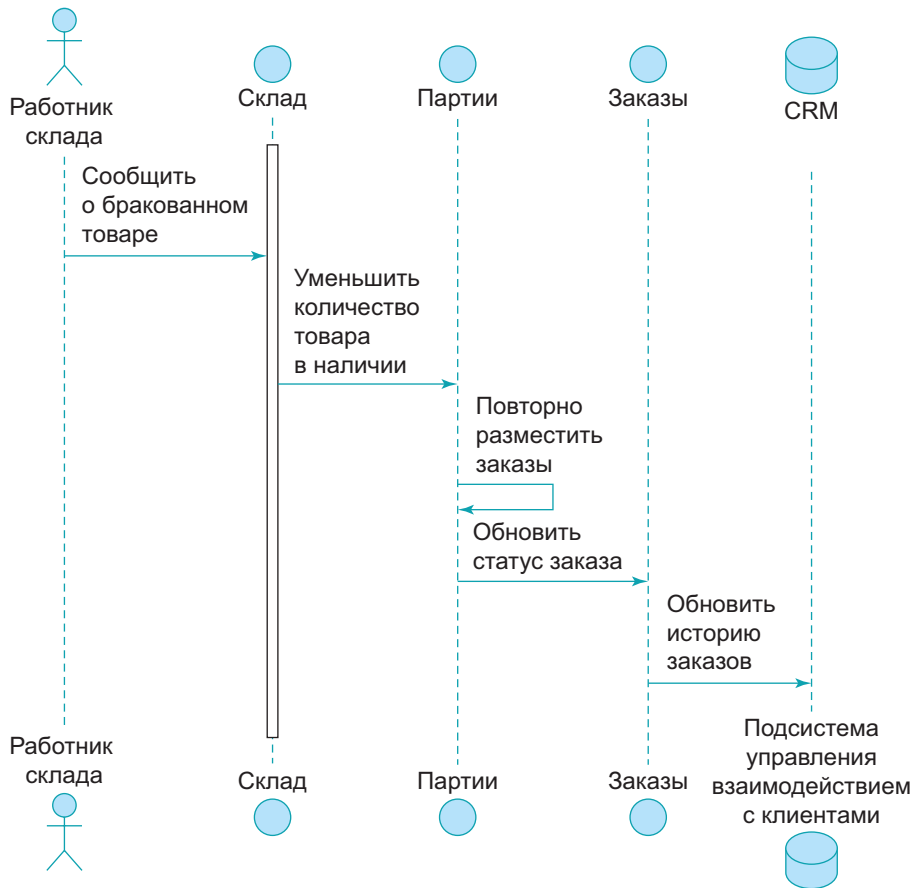


Рис. 11.4. Поток команд 2

Это тоже работает, но теперь все смешалось в графе зависимостей. Чтобы разместить заказы на товары, служба Заказов управляет системой Партией товара, которая управляет Складом; но чтобы справиться с проблемами на складе, Складская система управляет Партиями, которые управляют Заказами.

Умножьте это на все другие необходимые рабочие потоки, и вы увидите, как быстро службы становятся запутанными.

Обработка ошибок в распределенных системах

«Вещи ломаются» — это универсальный закон инженерии ПО. Что происходит в системе, когда один из запросов не выполняется? Предположим, что как только мы приняли заказ пользователя на три штуки артикула КОВРИК-ПРЕЗРЕННЫЙ, MISVEGOTTEN-RUG, происходит сбой сети, как показано на рис. 11.5.

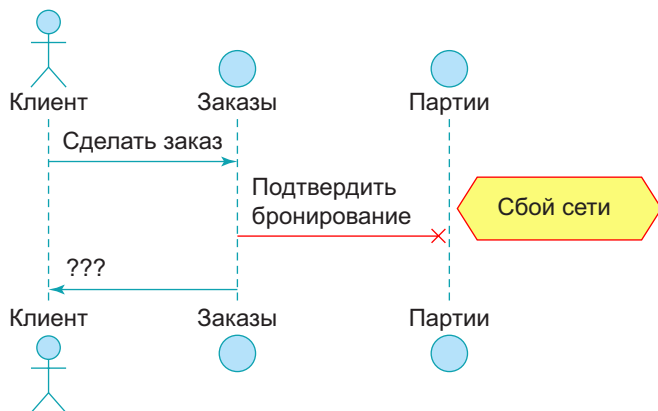


Рис. 11.5. Поток команд с ошибкой

У нас есть два варианта: мы можем принять заказ в любом случае и оставить его неразмещенным или же вовсе не принимать его, потому что не можем гарантировать его последующее размещение. Внезапный сбой в службе партий товара влияет на надежность службы заказов.

Когда две вещи приходится менять одновременно, мы говорим, что эти вещи *связаны*. Мы можем рассматривать этот каскад отказов как своего рода *временную связанность* (temporal coupling): чтобы вся система работала правильно, каждая ее часть должна правильно выполнять свои функции. Чем больше разрастается система, тем выше риск того, что какая-то из ее частей начнет сбоить.

СОГЛАСОВАННОСТЬ

Здесь мы использовали термин «связанность», но есть и другой способ описания связей между системами. *Согласованность* (connascence)¹ — это термин, используемый некоторыми авторами для описания разных типов связанности.

Согласованность неплоха сама по себе, но некоторые ее типы сильнее других. Неплохо бы иметь сильную локальную согласованность — скажем, если два класса тесно связаны. А вот между удаленными друг от друга элементами лучше оставлять слабую согласованность.

В первом примере распределенного комка грязи мы видим согласованность исполнения: для того чтобы операция была успешной, многочисленные компоненты должны знать правильный порядок работы.

Когда мы размышляем об условиях возникновения ошибки, то говорим о согласованности времени: для того чтобы операция сработала, многочисленные вещи должны произойти поочередно.

Когда мы изменяем систему, организованную в стиле удаленных вызовов процедур (RPC), в пользу событий, мы заменяем оба этих типа согласованности более слабым типом. Это и есть согласованность имени: многочисленные компоненты должны быть согласованы только по имени события и именам полей, которые оно несет.

Мы никогда не сможем полностью избежать связанности, за исключением связей между разными ПО. Мы хотим лишь одного — избежать *лишней* связанности. Согласованность предоставляет ментальную модель для понимания силы и типа связанности, присущей разным архитектурным стилям. Все по этой теме можно прочитать на веб-сайте connascence.io².

¹ Согласованность (connascence) — это метрика качества программного обеспечения, изобретенная Мейлиром Пейдж-Джонсом, позволяющая рассуждать о сложности, вызванной отношениями зависимости в объектно ориентированном проектировании, так же как это делалось для структурного проектирования. См. <https://en.wikipedia.org/wiki/Connascence>. — *Примеч. ред.*

² См. <http://www.connascence.io/>

Альтернатива: временное устранение связанности при помощи асинхронного обмена сообщениями

Как получить подходящую связанность? Часть ответа — думать глаголами, а не существительными. Модель предметной области основывается на бизнес-процессах. Это не статическая модель данных о предмете, это модель действий.

Поэтому вместо того, чтобы думать о системе для заказов и системе для партий товара, мы думаем о системе для процессов принятия и размещения заказов (с использованием отглагольных существительных) и т. д.

Когда мы разделяем все таким образом, то начинаем лучше понимать, за что отвечает каждая подсистема. Когда мы думаем о заказах, на самом деле мы имеем в виду их размещение. Все остальное может произойти *позже* — но обязательно.



Звучит знакомо, правда? Так и есть! Разделение обязанностей — это тот же самый процесс, с которым мы столкнулись при разработке агрегатов и команд.

Как и агрегаты, микросервисы должны выступать *границами согласованности*. Если у нас есть две службы, мы можем принять окончательную согласованность, так что нам не придется опираться на синхронные вызовы. Каждая служба принимает внешние команды и инициирует события, записывая результат. Другие службы могут прослушивать эти события, чтобы запускать следующие шаги в работе приложения.

Чтобы не допустить появление распределенного комка грязи, вместо временно связанных вызовов HTTP API нужно интегрировать системы, используя обмен асинхронными сообщениями. Надо сделать так, чтобы сообщения `BatchQuantityChanged` поступали как внешние сообщения от вышестоящих систем и чтобы система публиковала события `Allocated`, которые будут прослушивать нижестоящие системы.

Почему такой вариант подходит лучше? Во-первых, поскольку отдельные элементы системы могут отказывать, не затрагивая при этом ее остальные части, нам становится проще справляться с неэффективным поведением: мы по-прежнему можем принимать заказы, даже если служба размещения сегодня не в духе.

Во-вторых, мы уменьшаем степень связанности между системами. Если нужно изменить порядок операций или ввести дополнительные этапы в процесс, то можно сделать это локально.

Использование канала «издатель/подписчик» хранилища Redis для интеграции

Посмотрим, как именно все это будет работать. Понадобится какой-то способ передачи событий из одной системы в другую вроде шины сообщений, но для служб. Такой элемент инфраструктуры часто называют *брокером сообщений* (message broker). Его роль в том, чтобы принимать сообщения от издателей и доставлять их подписчикам.

В компании MADE.com используется Event Store¹. Kafka или RabbitMQ тоже сойдут как альтернативы. Легковесное решение, основанное на каналах «издатель/подписчик» хранилища Redis², также работает просто замечательно, и поскольку Redis гораздо более популярен, в этой книге мы решили использовать именно его.



Мы опускаем сложность, связанную с выбором подходящей платформы обмена сообщениями. Тут следует подумать над вопросами упорядочивания сообщений, обработки отказов и идиempотентности. Некоторые указания см. в разделе «Выстрел в ногу» на с. 287.

Новый процесс будет выглядеть так, как показано на рис. 11.6: Redis выдает событие `BatchQuantityChanged`, которое запускает весь процесс, и событие `Allocated` снова в конце публикуется в Redis.

Тестирование с помощью сквозного теста

Вот как можно начать со сквозного теста — использовать существующий API для создания партий, а затем протестировать входящие и исходящие сообщения.

¹ См. <https://eventstore.org/>

² См. <https://redis.io/topics/pubsub>

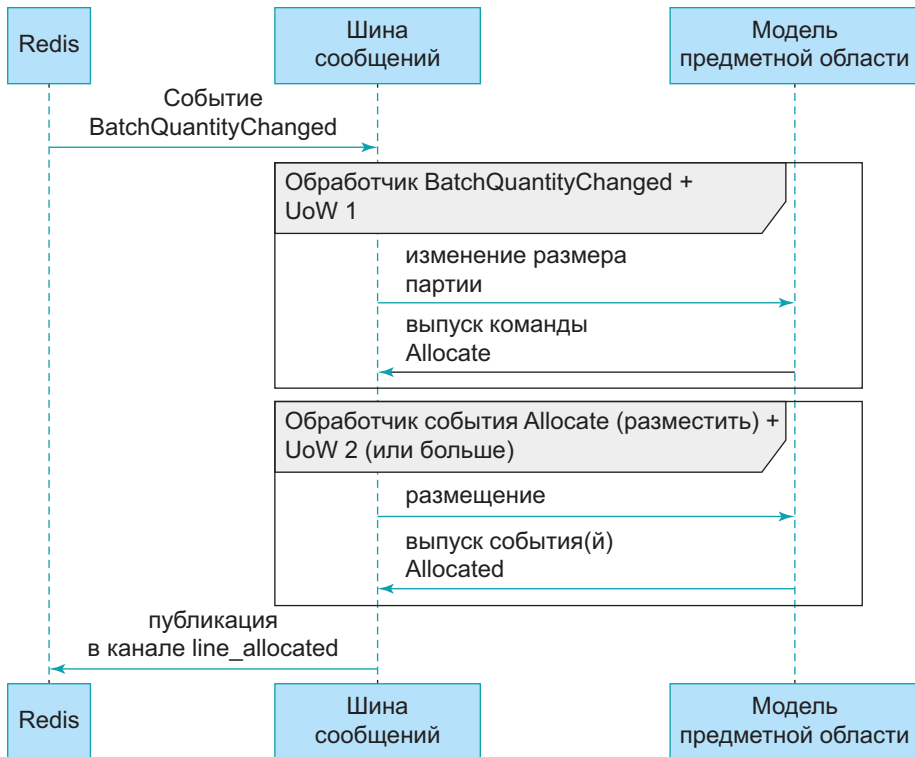


Рис. 11.6. Последовательность для потока повторного размещения

Сквозной тест для модели «издатель/подписчик» (tests/e2e/test_external_events.py)

```

def test_change_batch_quantity_leading_to_reallocation():
    # начать с двух партий и заказа, размещенного в одной из них ❶
    orderid, sku = random_orderid(), random_sku()
    earlier_batch, later_batch = random_batchref('old'), random_batchref('newer')
    api_client.post_to_add_batch(earlier_batch, sku, qty=10, eta='2011-01-02') ❷
    api_client.post_to_add_batch(later_batch, sku, qty=10, eta='2011-01-02')
    response = api_client.post_to_allocate(orderid, sku, 10) ❷
    assert response.json()['batchref'] == earlier_batch

    subscription = redis_client.subscribe_to('line_allocated') ❸

```

```
# изменить количество товара в размещенной партии,  
# чтобы оно было меньше, чем в заказе ❶  
redis_client.publish_message('change_batch_quantity', { ❸  
    'batchref': earlier_batch, 'qty': 5  
})  
  
# подождать до тех пор, пока мы не увидим сообщение ❶  
# о повторном размещении заказа  
messages = []  
for attempt in Retrying(stop=stop_after_delay(3), reraise=True): ❹  
    with attempt:  
        message = subscription.get_message(timeout=1)  
        if message:  
            messages.append(message)  
            print(messages)  
        data = json.loads(messages[-1]['data'])  
        assert data['orderid'] == orderid  
        assert data['batchref'] == later_batch
```

❶ Комментарии помогают понять, что происходит в этом тесте; мы хотим отправить в систему событие, из-за которого товарная позиция заказа размещается повторно, и видим, что повторное размещение также происходит как событие в Redis.

❷ `api_client` — это небольшой помощник, который переработан для совместной работы с двумя типами тестов; он оборачивает вызовы `requests.post`.

❸ `redis_client` — это еще один помощник в тестировании. Нам необязательно разбирать его детали; его задача — отправлять и получать сообщения из различных каналов Redis. Используем канал `change_batch_quantity` для отправки запроса на изменение размера партии и прослушаем еще один канал, `line_allocated`, для отслеживания ожидаемого повторного размещения.

❹ Из-за того что наша система асинхронна, нам приходится снова использовать библиотеку `tenacity` для добавления цикла повторных попыток. Мы делаем это прежде всего потому, что сообщение `line_allocated` может прийти не сразу, но также и потому, что в указанном канале будут и другие сообщения.

Redis — это еще один тонкий адаптер вокруг шины сообщений

Слушатель «издатель/подписчик» Redis (мы называем его *потребителем событий*) очень похож на Flask: он делает передачу из внешнего мира в наши события.

Простой слушатель сообщений Redis (src/allocation/entrypoints/redis_eventconsumer.py)

```
r = redis.Redis(**config.get_redis_host_and_port())

def main():
    orm.start_mappers()
    pubsub = r.pubsub(ignore_subscribe_messages=True)
    pubsub.subscribe('change_batch_quantity') ❶

    for m in pubsub.listen():
        handle_change_batch_quantity(m)

def handle_change_batch_quantity(m):
    logging.debug('handling %s', m)
    data = json.loads(m['data']) ❷
    cmd = commands.ChangeBatchQuantity(ref=data['batchref'],
                                       qty=data['qty']) ❷
    messagebus.handle(cmd, uow=unit_of_work.SqlAlchemyUnitOfWork())
```

❶ main() подписывает на канал change_batch_quantity при загрузке.

❷ Главная задача точки входа в систему — десериализовать JSON, конвертировать его в команду и передать ее в сервисный слой — примерно так же, как это делает Flask.

Также создаем новый нисходящий адаптер для выполнения противоположной задачи — конвертирования событий предметной области в публичные события.

Простой издатель сообщений Redis (src/allocation/adapters/redis_eventpublisher.py)

```
r = redis.Redis(**config.get_redis_host_and_port())

def publish(channel, event: events.Event): ❶
    logging.debug('publishing: channel=%s, event=%s', channel, event)
    r.publish(channel, json.dumps(asdict(event)))
```

❶ Здесь берется жестко закодированный канал, но вы также можете сохранить попарную связку между классами/именами событий и подходящим

каналом, позволяя одному или нескольким типам сообщений переходить в разные каналы.

Новое исходящее событие

Вот как будет выглядеть событие `Allocated`:

Новое событие (`src/allocation/domain/events.py`)

```
@dataclass
class Allocated(Event):
    orderid: str
    sku: str
    qty: int
    batchref: str.
```

Оно улавливает все, что нужно знать о размещении: сведения о товарной позиции заказа и о том, в какой партии товара она была размещена.

Добавляем его в модельный метод `allocate()` (естественно, предварительно добавив тест).

`Product.allocate()` выдает новое событие для регистрации того, что произошло (`src/allocation/domain/model.py`).

```
class Product:
    ...
    def allocate(self, line: OrderLine) -> str:
        ...

        batch.allocate(line)
        self.version_number += 1
        self.events.append(events.Allocated(
            orderid=line.orderid, sku=line.sku, qty=line.qty,
            batchref=batch.reference,
        ))
        return batch.reference
```

Обработчик `ChangeBatchQuantity` у нас уже есть, поэтому нужно добавить лишь обработчик, который публикует исходящее событие.

Шина сообщений растёт (`src/allocation/service_layer/messagebus.py`)

```
HANDLERS = {
    events.Allocated: [handlers.publish_allocated_event],
    events.OutOfStock: [handlers.send_out_of_stock_notification],
} # тип: Dict[Type[events.Event], List[Callable]]
```

При публикации события используется вспомогательная функция из обертки Redis.

Публикация в Redis (`src/allocation/service_layer/handlers.py`)

```
def publish_allocated_event(
    event: events.Allocated, uow: unit_of_work.AbstractUnitOfWork,
):
    redis_eventpublisher.publish('line_allocated', event)
```

Внутренние события против внешних

Лучше поддерживать четкое различие между внутренними и внешними событиями. Некоторые события могут приходить со стороны, а другие — дополняться и публиковаться снаружи системы, но так будет происходить не со всеми. Это особенно важно, если вы занимаетесь отслеживанием источников событий¹ (эта тема тянет на еще одну книгу).



К исходящим событиям особенно важно применять валидацию. Несколько принципов и примеров валидации можно найти в приложении Д в конце книги.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

В этой главе все совсем просто: сделайте так, чтобы основной вариант использования `allocate()` дополнительно мог активироваться событием в канале Redis, а также через API (или вместо него).

Скорее всего, вам придется добавить новый сквозной тест и внести некоторые изменения в `redis_eventconsumer.py`.

Выводы

События могут не только приходить снаружи системы, но также публиковаться во внешний мир — обработчик `publish` конвертирует событие в сообщение в канале Redis. Мы используем события для связи с внешним

¹ См. <https://oreil.ly/FXVil>

миром. Такое временное устранение связанности дает бóльшую гибкость в интеграциях приложений, но за это, разумеется, приходится платить.

Уведомление о событии — вещь хорошая, ведь оно означает низкий уровень связанности и довольно простую настройку. Однако это может стать проблемой, если на самом деле какой-то логический процесс перекрывает различные уведомления о событии... Такой процесс бывает трудно различить, поскольку в программных текстах он неявный... Это может затруднить отладку и модификацию.

Мартин Фаулер «Что вы подразумеваете под управлением событиями?»¹

В табл. 11.1 показано несколько компромиссов, которые стоит иметь в виду.

Таблица 11.1. Интеграция микросервисов на основе событий: компромиссы

Плюсы	Минусы
• Позволяет избежать большого распределенного комка грязи. • Службы отцеплены друг от друга: проще менять отдельные службы и добавлять новые	• Труднее увидеть совокупные потоки информации. • Конечная согласованность — это новая концепция, с которой нужно уметь работать. • Стоит обдумать нюансы надежности сообщений и выбора между доставкой «как минимум один раз» и доставкой «как максимум один раз»

В двух словах, если вы переходите от модели синхронного обмена сообщениями к асинхронному, то открываете целый ряд проблем, связанных с надежностью сообщений и конечной согласованностью. См. раздел «Выстрел в ногу» на с. 287.

¹ См. <https://oreil.ly/uaPNt>

Разделение обязанностей команд и запросов

Начнем эту главу с довольно однозначного вывода: чтение (запросы) и запись (команды) различаются, поэтому их следует трактовать по-разному (или разделять их обязанности, если угодно). Разовьем эту идею настолько далеко, насколько сможем.

Если вы немного похожи на Гарри, то поначалу все это покажется вам крайностью, но надеемся, мы сможем убедить вас, что все это *не совсем* бессмысленно. На рис. 12.1 показано, что в итоге должно получиться.



Код для этой главы находится в ветке `chapter_12_cqrs` на GitHub¹.

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_12_cqrs
# или, если пишете код по ходу чтения, возьмите за основу
# материал из предыдущей главы:
git checkout chapter_11_external_events
```

Для начала попытаемся понять, зачем вообще стоит напрягаться.

Модели предметной области для записи

Мы потратили немало времени, рассказывая о том, как создавать ПО, которое поддерживает соблюдение правил предметной области. Эти правила, или ограничения, будут различаться в зависимости от приложения и составлять ядро наших систем.

¹ См. <https://oreil.ly/YbWGT>

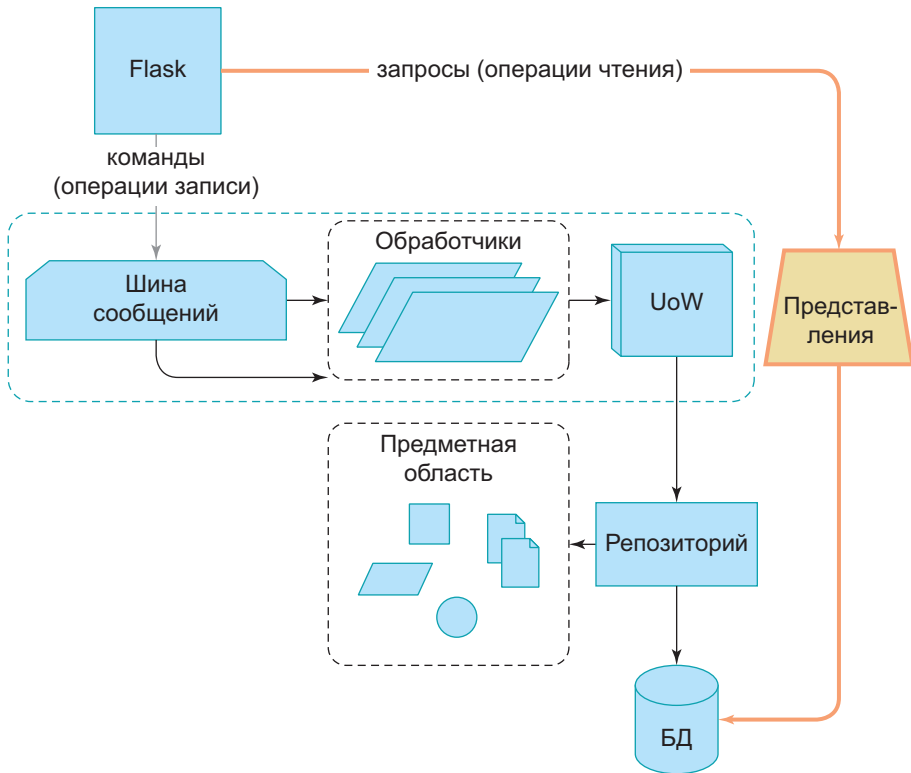


Рис. 12.1. Отделение операций чтения от операций записи

Здесь мы установили как явные ограничения, такие как «нельзя размещать заказ на большее количество товара, чем имеется в наличии», так и неявные вроде «каждая товарная позиция заказа размещается в одной партии товара».

Эти правила описаны в виде юнит-тестов в начале книги.

Базовые тесты предметной области (tests/unit/test_batches.py)

```
def test_allocating_to_a_batch_reduces_the_available_quantity():
    batch = Batch("batch-001", "SMALL-TABLE", qty=20, eta=date.today())
    line = OrderLine('order-ref', "SMALL-TABLE", 2)

    batch.allocate(line)

    assert batch.available_quantity == 18
```

...

```
def test_cannot_allocate_if_available_smaller_than_required():  
    small_batch, large_line = make_batch_and_line("ELEGANT-LAMP", 2, 20)  
    assert small_batch.can_allocate(large_line) is False
```

Для того чтобы правильно применять эти правила, нужно было обеспечить согласованность операций, и поэтому были введены такие паттерны, как UoW и «Агрегат», которые помогают выполнять фиксацию малых фрагментов работы.

Чтобы передавать изменения между этими малыми фрагментами, мы ввели паттерн «События предметной области», благодаря которому можно писать правила типа «если товар поврежден или потерян, скорректировать размер партии и при необходимости разместить заказы повторно».

Без всех этих сложностей не получится обеспечивать соблюдение правил при изменении состояния системы. Мы создали гибкий набор инструментов для записи данных. Но как насчет чтения?

Большинство пользователей не собираются покупать вашу мебель

В компании MADE.com есть система, очень похожая на службу повторного размещения. В нагруженные дни мы можем обрабатывать сотню заказов за час, для которых у нас есть большая грубая система размещения товара.

Вместе с тем в тот же самый день у нас может быть сто просмотров продукта в *секунду*. Каждый раз, когда кто-то посещает страницу продукта или его описания, нужно выяснять, есть ли продукт на складе и сколько времени потребуется, чтобы его доставить.

Предметная область та же самая — нас интересуют партии товара, дата их прибытия и количество, которое все еще есть в наличии, — но схема доступа очень отличается. Например, клиенты не заметят, если запрос устарел на несколько секунд, но если служба размещения будет несогласованной, то мы внесем беспорядок в их заказы. Можно воспользоваться преимуществом этой разницы, сделав операции чтения *согласованными в конечном счете*, чтобы заставить их работать лучше.

ДОСТИЖИМА ЛИ СОГЛАСОВАННОСТЬ ОПЕРАЦИЙ ЧТЕНИЯ?

Идея обмена согласованности на производительность поначалу заставляет многих разработчиков нервничать, поэтому кратко на этом остановимся.

Давайте представим, что запрос «получить товар в наличии» устарел на 30 секунд в момент, когда Боб посещает страницу комода с асимметричным зеркалом, **ASYMMETRICAL-DRESSER**. Между тем Гарри уже купил последний оставшийся предмет. Когда мы попытаемся разместить заказ Боба, то получим отказ и нужно будет либо отменить его заказ, либо купить больше товара и задержать его доставку.

Эта проблема *очень* беспокоит тех, кому приходилось работать только с реляционными хранилищами данных. Но давайте рассмотрим два других сценария, чтобы получить некоторую перспективу.

Во-первых, представим, что Боб и Гарри посещают страницу одновременно. Гарри уходит варить кофе, а когда возвращается, Боб уже покупает последний комод. Когда Гарри делает свой заказ, мы отправляем его в службу размещения заказов, и поскольку товаров там недостаточно, мы должны вернуть его платеж или купить больше товаров и задержать его доставку.

Едва мы открыли страницу продукта, как данные уже устарели. Вот почему несогласованность чтения — это безопасная ситуация: когда мы приступаем к размещению, нам всегда нужно проверять текущее состояние системы, потому что все распределенные системы несогласованные. Едва у вас появится веб-сервер и два клиента, как сразу же возникнет потенциал для устаревших данных.

Хорошо, давайте предположим, что мы каким-то образом решаем эту проблему: создаем волшебное, полностью согласованное веб-приложение, где никто никогда не видит устаревших данных. На этот раз Гарри первым добирается до страницы и покупает свой комод с зеркалом.

К несчастью для него, когда персонал склада пытается отправить его заказ, комод падает с погрузчика и ломается на части. И что теперь?

Единственные варианты — либо позвонить Гарри и вернуть уплаченные деньги, либо купить больше товара и отложить доставку.

Независимо от того, что мы делаем, наши программные системы никогда не будут согласовываться с реальностью, и поэтому потребуются бизнес-процессы, которые справятся с такими крайними случаями. Вполне нормально повышать производительность в обмен на согласованность чтения, так как устаревшие данные, в общем-то, неизбежны.

Можно представить, будто эти требования образуют две половины системы: сторону чтения и сторону записи, как показано в табл. 12.1.

Чудо-паттерны проектирования предметной области помогают со временем развивать систему, но не выгодна для чтения данных. Сервисный слой,

UoW и хитроумная модель предметной области — это, по сути, раздувание исходного кода.

Таблица 12.1. Чтение и запись

	Сторона чтения	Сторона записи
Поведение	Простое чтение	Сложная бизнес-логика
Кешируемость	Хорошо подходит для кеширования	Непригодная для кеширования
Согласованность	Может быть устаревшей	Должна быть транзакционно согласованной

PRG и разделение команд и запросов

Если вы занимаетесь веб-разработкой, то, скорее всего, знакомы с паттерном PRG (Post/Redirect/Get). С его помощью конечная веб-точка принимает HTTP POST и отвечает перенаправлением, чтобы увидеть результат. Например, можно принять POST в `/batches`, чтобы создать новую партию, и перенаправить пользователя в `/batches/123`, чтобы увидеть его только что созданную партию.

Такой подход устраняет проблемы, возникающие, когда пользователи обновляют страницу результатов в браузере либо пытаются добавить в закладки страницу результатов. В случае обновления это может привести к тому, что пользователи дважды отправят данные и, таким образом, купят два дивана, когда им нужен всего один. В случае закладки незадачливые клиенты в итоге получают неисправную страницу, когда попытаются получить (GET) конечную точку POST.

Обе эти проблемы возникают из-за того, что мы возвращаем данные в ответ на операцию записи. Паттерн PRG обходит проблему, разделяя операции на фазы чтения и записи.

Этот метод является простым примером принципа разделения команд и запросов (command-query separation, CQS). Здесь мы следуем одному простому правилу: функции должны либо изменять состояние, либо отвечать на вопросы. Оба варианта сразу недопустимы. Так проще думать про наше ПО: всегда должна быть возможность спросить, зажжен свет или нет, не прикасаясь к выключателю.



При создании API мы можем применять тот же метод проектирования, вернув 201 Created (Создано) или 202 Accepted (Принято) с заголовком Location (Местоположение), содержащим URI новых ресурсов. Здесь важен не используемый код статуса, а логическое разделение работы на фазу записи и фазу запроса.

Можно разделять команды и запросы ради более быстрых и масштабируемых систем, но сначала давайте исправим нарушение указанного принципа в нашем коде. Давным-давно мы ввели конечную точку `allocate`, которая принимает заказ и вызывает сервисный слой, чтобы найти нужный товарный запас. В конце вызова мы возвращаем `200 OK` и идентификатор партии. Да, мы смогли получить нужные данные, но дизайн получился кривым. Давайте исправим его так, чтобы вместо возврата простого сообщения `OK` предоставлять новую конечную точку, предназначенную только для чтения, которая будет извлекать состояние размещения заказа.

Тест API выполняет метод GET после метода POST (tests/e2e/test_api.py)

```
@pytest.mark.usefixtures('postgres_db')
@pytest.mark.usefixtures('restart_api')
def test_happy_path_returns_202_and_batch_is_allocated():
    orderid = random_orderid()
    sku, othersku = random_sku(), random_sku('other')
    earlybatch = random_batchref(1)
    laterbatch = random_batchref(2)
    otherbatch = random_batchref(3)
    api_client.post_to_add_batch(laterbatch, sku, 100, '2011-01-02')
    api_client.post_to_add_batch(earlybatch, sku, 100, '2011-01-01')
    api_client.post_to_add_batch(otherbatch, othersku, 100, None)

    r = api_client.post_to_allocate(orderid, sku, qty=3)
    assert r.status_code == 202

    r = api_client.get_allocation(orderid)
    assert r.ok
    assert r.json() == [
        {'sku': sku, 'batchref': earlybatch},
    ]

@pytest.mark.usefixtures('postgres_db')
@pytest.mark.usefixtures('restart_api')
def test_unhappy_path_returns_400_and_error_message():
    unknown_sku, orderid = random_sku(), random_orderid()
```

```

r = api_client.post_to_allocate(
    orderid, unknown_sku, qty=20, expect_success=False,
)
assert r.status_code == 400
assert r.json()['message'] == f'Недопустимый артикул {unknown_sku}'

r = api_client.get_allocation(orderid)
assert r.status_code == 404

```

Итак, как могло бы выглядеть приложение Flask?

Конечная точка для просмотра размещений (src/allocation/entrypoints/flask_app.py)

```

from allocation import views
...
@app.route("/allocations/<orderid>", methods=['GET'])
def allocations_view_endpoint(orderid):
    uow = unit_of_work.SqlAlchemyUnitOfWork()
    result = views.allocations(orderid, uow) ❶
    if not result:
        return 'not found', 404
    return jsonify(result), 200

```

❶ Отлично, вопрос с `views.py` снят; можно держать там только то, что читаем, и это будет настоящий `views.py`, а не как у Django — нечто, что знает, как создавать представления данных, предназначенные только для чтения...

Хватайте свой обед, ребята

Гм, пожалуй, мы можем просто добавить метод списка к нашему существующему объекту репозитория.

Представления выполняют... сырой SQL? (src/allocation/views.py)

```

from allocation.service_layer import unit_of_work

def allocations(orderid: str, uow: unit_of_work.SqlAlchemyUnitOfWork):
    with uow:
        results = list(uow.session.execute(
            'SELECT ol.sku, b.reference'
            ' FROM allocations AS a'
            ' JOIN batches AS b ON a.batch_id = b.id'
            ' JOIN order_lines AS ol ON a.orderline_id = ol.id'
            ' WHERE ol.orderid = :orderid',
            dict(orderid=orderid)

```



```
))  
return [{'sku': sku, 'batchref': batchref} for sku, batchref in  
results]
```

Простите? Сырой SQL?

Если вы хоть немного похожи на Гарри и впервые столкнулись с этим паттерном, то подумаете: что же, черт возьми, курил Боб?! Мы что, сворачиваем вручную собственный SQL и конвертируем строки базы данных напрямую в словари? И это после всех вложенных усилий в создание хорошей модели предметной области? А как быть с паттерном «Хранилище»? Разве он не должен быть абстракцией вокруг базы данных? Почему бы не использовать его вторично?

Что ж, рассмотрим эту, казалось бы, более простую альтернативу и посмотрим, как она выглядит на практике.

Как и прежде, сохраним представление в отдельном модуле `views.py`; обеспечение четкого различия между чтением и записью в приложении — хорошая идея. Мы применяем принцип разделения команд и запросов, и хорошо видим, какой код изменяет состояние (обработчики событий), а какой просто извлекает состояние «только для чтения» (представления).



Выделять представления только для чтения из команд и обработчиков событий, которые изменяют состояние, — вполне хорошая инициатива, даже если вы не хотите переходить к полномасштабному разделению обязанностей команд и запросов.

Тестирование представлений CQRS

Прежде чем перейти к детальному разбору, поговорим о тестировании. Какие бы подходы вы ни выбрали, вам понадобится по меньшей мере один интеграционный тест. Что-то вроде такого:

Интеграционный тест для представления (`tests/integration/test_views.py`)

```
def test_allocations_view(sqlite_session_factory):  
    uow = unit_of_work.SqlAlchemyUnitOfWork(sqlite_session_factory)  
    messagebus.handle(commands.CreateBatch('sku1batch', 'sku1', 50,  
None), uow) ❶
```

```

messagebus.handle(commands.CreateBatch('sku2batch', 'sku2', 50,
today), uow)
messagebus.handle(commands.Allocate('order1', 'sku1', 20), uow)
messagebus.handle(commands.Allocate('order1', 'sku2', 20), uow)
# добавим фальшивую партию и заказ,
# чтобы убедиться, что мы получаем правильные значения
messagebus.handle(commands.CreateBatch('sku1batch-later', 'sku1',
50, today), uow)
messagebus.handle(commands.Allocate('otherorder', 'sku1', 30), uow)
messagebus.handle(commands.Allocate('otherorder', 'sku2', 10), uow)

assert views.allocations('order1', uow) == [
    {'sku': 'sku1', 'batchref': 'sku1batch'},
    {'sku': 'sku2', 'batchref': 'sku2batch'},
]

```

❶ Мы выполняем настройку интеграционного теста с помощью публичной точки входа в наше приложение — шины сообщений. Благодаря этому тесты теряют связанность с какими-либо деталями реализации/инфраструктуры хранения.

«Очевидная» альтернатива 1: использование существующего репозитория

Как насчет добавления вспомогательного метода в репозиторий `products`?

Простое представление, в котором используется репозиторий (`src/allocation/views.py`)

```

from allocation import unit_of_work

def allocations(orderid: str, uow: unit_of_work.AbstractUnitOfWork):
    with uow:
        products = uow.products.for_order(orderid=orderid) ❶
        batches = [b for p in products for b in p.batches] ❷
        return [
            {'sku': b.sku, 'batchref': b.reference}
            for b in batches
            if orderid in b.orderids ❸
        ]

```

❶ Репозиторий возвращает объекты `Product`, и нужно найти все продукты для артикулов в заданном порядке, поэтому в репозитории создается новый вспомогательный метод с именем `.for_order()`.

❷ Теперь у нас есть продукты, но на самом деле нужны ссылки на партии, и мы получаем все возможные партии с помощью операции включения в список.

❸ Снова фильтруем, чтобы получить партии только для конкретного заказа, что, в свою очередь, опирается на способность объектов `Batch` сообщать о том, какие идентификаторы заказов они разместили.

Реализуем последнее с помощью свойства `.orderid`.

Возможно, ненужное свойство в модели (`src/allocation/domain/model.py`)

```
class Batch:
    ...

    @property
    def orderids(self):
        return {l.orderid for l in self._allocations}
```

Становится понятно, что вторично использовать существующие классы репозитория и модели предметной области не так просто, как казалось на первый взгляд. Пришлось добавить в оба класса новые вспомогательные методы, и теперь выполняется множество циклов и операций фильтрации в Python — это та работа, которую гораздо эффективнее выполняет база данных.

Так что да, с одной стороны, мы вторично используем имеющиеся абстракции, а с другой — выглядит все это неуклюже.

Модель предметной области не оптимизирована для операций чтения

То, что мы видим здесь, — это влияние модели предметной области, которая предназначена главным образом для операций записи, в то время как наши требования к операциям чтения нередко отличаются в концептуальном плане.

Таково оправдание озадаченного архитектора для разделения обязанностей между командами и запросами. Как мы уже говорили, модель предметной области не является моделью данных — мы пытаемся понять, как работает бизнес: рабочий процесс, правила изменения состояний, обмен сообще-

ниями, обязанности, связанные с реакцией системы на внешние события и вводимые пользователем данные. Большая часть всего этого не имеет никакого отношения к операциям «только для чтения».



Такое обоснование для разделения обязанностей между командами и запросами связано с обоснованием паттерна «Модель предметной области». Если вы строите простое приложение CRUD, то чтение и запись будут тесно связаны, поэтому модель предметной области или разделение ответственности между командами и запросами не нужны. Но чем сложнее ваша предметная область, тем выше вероятность того, что вам понадобится и то и другое.

Простыми словам, у классов вашей предметной области будет несколько методов для изменения состояния, но ни один из них не пригодится операциям «только для чтения».

По мере роста сложности вашей модели вам все чаще придется решать, как структурировать эту модель, что будет делать ее все более неудобной для операций чтения.

«Очевидная» альтернатива № 2: использование ORM

Вы можете подумать: «Ну хорошо, если репозиторий такой неуклюжий, как и работа с продуктами, то я могу хотя бы использовать ORM и работать с партиями товара. Ведь он для этого и существует!»

Простое представление, в котором используется ORM (src/allocation/views.py)

```
from allocation import unit_of_work, model

def allocations(orderid: str, uow: unit_of_work.AbstractUnitOfWork):
    with uow:
        batches = uow.session.query(model.Batch).join(
            model.OrderLine, model.Batch._allocations
        ).filter(
            model.OrderLine.orderid == orderid
        )
        return [
            {'sku': b.sku, 'batchref': b.batchref}
            for b in batches
        ]
```

Но действительно ли писать или понимать такое легче, чем версию с сырым SQL из примера в разделе «Хватайте свой обед, ребята» на с. 232? Возможно, там не так уж все и плохо, но должны вас предупредить, что написали мы этот код не с первой попытки и только после того, как изрядно покопались в документации SQLAlchemy. SQL — он и в Африке SQL.

Но ORM также может привести к проблемам с производительностью.

SELECT N+1 и другие соображения по поводу производительности

Так называемая проблема `SELECT N+1`¹ — распространенная проблема производительности, которая связана с ORM: при получении списка объектов ORM нередко выполняет начальный запрос, чтобы получить все идентификаторы объектов, которые ему нужны, а затем выдает индивидуальные запросы для каждого объекта, чтобы получить их атрибуты. Подобное особенно вероятно, если у ваших объектов есть какие-либо отношения по внешнему ключу.



Справедливости ради скажем, что SQLAlchemy довольно хорошо избегает проблемы `SELECT N+1`. Она не появляется в предыдущем примере, и вы можете явно запросить немедленную загрузку², чтобы избежать ее при работе с соединенными объектами.

Помимо проблемы `SELECT N+1` могут быть и другие причины, чтобы устранить связанность способа обеспечения постоянства изменений состояния со способом извлечения текущего состояния. Набор полностью нормализованных реляционных таблиц — хороший способ не допустить повреждение данных из-за операций записи. Но извлечение данных с помощью большого числа соединений бывает медленным. В таких случаях принято добавлять денормализованные представления, создавать реплики для чтения или даже добавлять уровни кэширования.

¹ См. <https://oreil.ly/OkBOS>

² См. <https://oreil.ly/XKDDm>

Время прыгать через акулу

В связи с этим¹: ну, как, убедили мы вас в том, что версия с сырым SQL не такая уж и странная, как кажется на первый взгляд? Возможно, мы преувеличивали для пущего эффекта. То ли еще будет...

Итак, разумно это или нет, но такой жестко закодированный SQL-запрос выглядит скверно, не так ли? А что, если мы сделаем его лучше...

Гораздо более приятный запрос (src/allocation/views.py)

```
def allocations(orderid: str, uow: unit_of_work.SqlAlchemyUnitOfWork):
    with uow:
        results = list(uow.session.execute(
            'SELECT sku, batchref FROM allocations_view WHERE orderid
            = :orderid',
            dict(orderid=orderid)
        ))
    ...
```

...поддерживая отдельное, денормализованное хранилище данных для модели представления?

Ха-ха-ха, никаких внешних ключей, только строковые значения, живем один раз (src/allocation/adapters/orm.py)

```
allocations_view = Table(
    'allocations_view', metadata,
    Column('orderid', String(255)),
    Column('sku', String(255)),
    Column('batchref', String(255)),
)
```

Что ж, изящно выглядящие SQL-запросы на самом деле ничего не оправдывают, но создание денормализованной копии ваших данных, оптимизированной для операций чтения, — обычное дело, если вы уже достигли

¹ «Прыжок через акулу» (jumping the shark) — метафора, используемая для обозначения момента, когда телевизионный сериал проходит пик успешности. Как только шоу «прыгает через акулу», зрители чувствуют заметное снижение качества или понимают, что шоу претерпело слишком много изменений, потеряв исходные очарование и привлекательность. См. https://ru.wikipedia.org/wiki/Прыжок_через_акулу. — *Примеч. пер.*

пределов работы с индексами. Даже с хорошо настроенными индексами реляционная база данных использует много ресурсов процессора для соединений. Самые быстрые запросы всегда будут `SELECT * from mytable WHERE key = :value`.

Однако этот подход дает нам не только скорость, но и масштаб. Когда мы записываем данные в реляционную БД. Важно иметь возможность блокировать изменяемые строки, чтобы потом не разбирать проблемы согласованности данных.

Если многочисленные клиенты изменяют данные одновременно, то мы оказываемся в странных условиях гонки. Но когда мы всего лишь читаем данные, то никаких ограничений на параллельное число клиентов, нет. Вот почему хранилища, предназначенные только для чтения, могут масштабироваться горизонтально.



Поскольку согласовывать реплики операций чтения не нужно, их может быть сколько угодно. Если вы пытаетесь масштабировать систему со сложно организованным хранилищем данных, то задумайтесь над созданием более простой модели чтения.

Непросто поддерживать модель чтения в актуальном состоянии! Представления базы данных (материализованные или нет) и триггеры — популярное решение, но оно ограничивает вас вашей базой данных. Вместо этого мы хотели бы показать вам, как использовать событийно-управляемую архитектуру вторично.

Обновление таблицы модели чтения с помощью обработчика событий

Добавляем второй обработчик в событие `Allocated`.

Событие `Allocated` получает новый обработчик (`src/allocation/service_layer/messagebus.py`)

```
EVENT_HANDLERS = {
    events.Allocated: [
        handlers.publish_allocated_event,
        handlers.add_allocation_to_read_model
    ],
}
```

Вот как выглядит код обновления модели представления:

Обновление при размещении (src/allocation/service_layer/handlers.py)

```
def add_allocation_to_read_model(
    event: events.Allocated, uow:
        unit_of_work.SqlAlchemyUnitOfWork,
):
    with uow:
        uow.session.execute(
            'INSERT INTO allocations_view (orderid, sku, batchref)'
            ' VALUES (:orderid, :sku, :batchref)',
            dict(orderid=event.orderid, sku=event.sku,
                batchref=event.batchref)
        )
        uow.commit()
```

Хотите верьте, хотите нет, но этот код в целом рабочий! *И он будет работать с теми же интеграционными тестами, что и остальные варианты.*

О'кей, теперь обработаем отмену размещения, Deallocated.

Второй слушатель для обновлений модели чтения

```
events.Deallocated: [
    handlers.remove_allocation_from_read_model,
    handlers.reallocate
],

...

def remove_allocation_from_read_model(
    event: events.Deallocated, uow:
        unit_of_work.SqlAlchemyUnitOfWork,
):
    with uow:
        uow.session.execute(
            'DELETE FROM allocations_view '
            ' WHERE orderid = :orderid AND sku = :sku',
```

На рис. 12.2 показан процесс между двумя запросами.

На рисунке можно увидеть две транзакции в операции POST/запись: одну для обновления модели записи и другую для обновления модели чтения, которая может использоваться операцией GET/чтение.

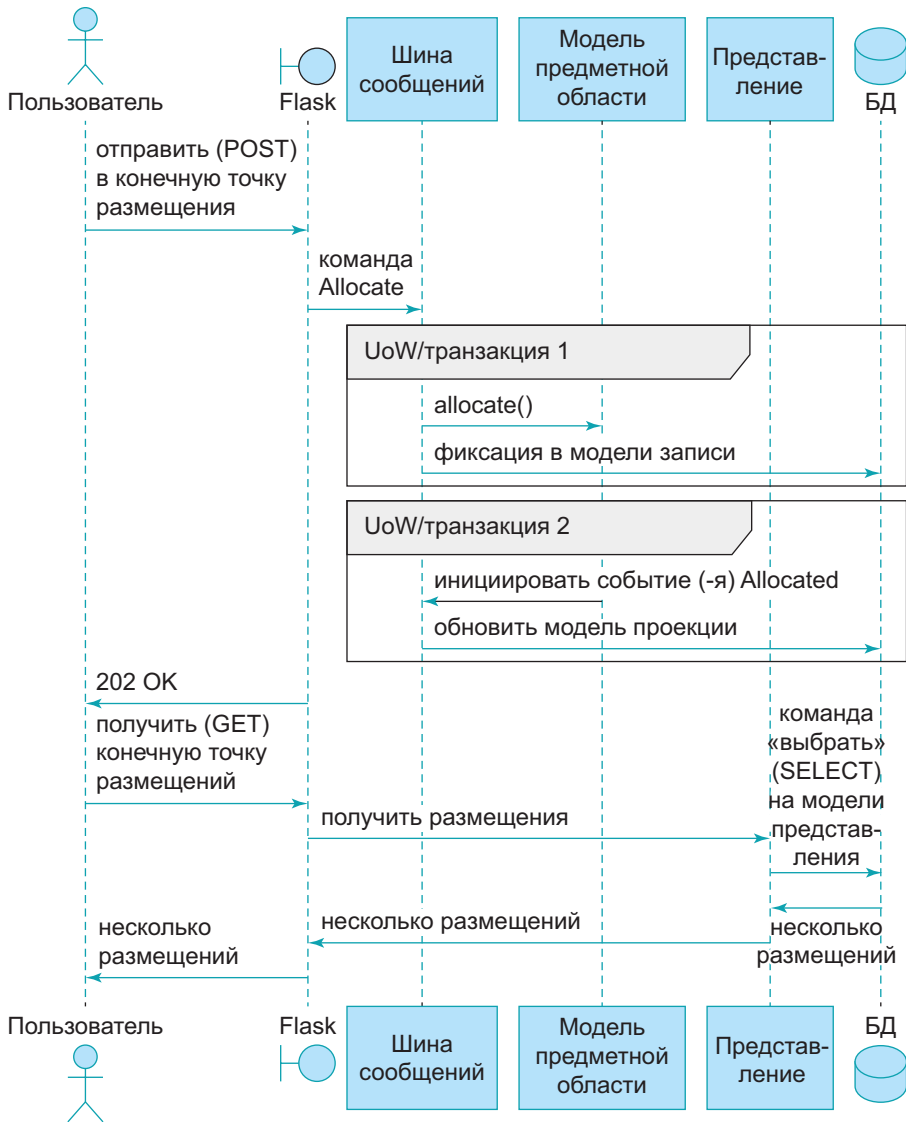


Рис. 12.2. Схема последовательности для модели чтения

ПЕРЕДЕЛКА С НУЛЯ

Как инженеры, мы в первую очередь должны спросить себя: «А что будет, если оно сломается?»

Как обращаться с моделью представления, которая не обновилась из-за бага или временного отключения? Ну, это просто еще один случай, когда события и команды могут отказывать независимо.

Если бы мы *никогда* не обновляли модель представления и артикул КОМОД-АСИММЕТРИЧНЫЙ, `ASYMMETRICAL-DRESSER`, всегда был бы в наличии, покупателей бы точно не устроило такое положение дел, но в службе `allocate` все равно происходил бы сбой, и мы приняли бы меры, чтобы исправить эту проблему.

Но перестроить модель представления очень просто. Поскольку для ее обновления мы используем сервисный слой, можно создать инструмент, который выполняет следующие действия:

- Запрашивает текущее состояние стороны записи и выясняет, что было размещено в данный момент.
 - Вызывает обработчик `add_allocate_to_read_model` (добавить размещение в модель чтения) для каждого размещенного товара. Можно использовать этот технический прием для создания совершенно новых моделей чтения из устаревших данных.
-

Изменить реализацию модели чтения очень просто

Оцените, насколько гибкой становится система благодаря событийно-управляемой модели: допустим, в один прекрасный день мы решим, что хотим реализовать модель чтения с помощью обособленного механизма хранения данных Redis.

Просто взгляните:

Обработчики обновляют модель чтения Redis (`src/allocation/service_layer/handlers.py`)

```
def add_allocation_to_read_model(event: events.Allocated, _):
    redis_eventpublisher.update_readmodel(event.orderid,
    event.sku, event.batchref)

def remove_allocation_from_read_model(event: events.Deallocated, _):
    redis_eventpublisher.update_readmodel(event.orderid, event.sku, None)
```

Помощники в модуле Redis однострочные.

Операции чтения и обновления в модели чтения Redis (src/allocation/adapters/redis_eventpublisher.py)

```
def update_readmodel(orderid, sku, batchref):  
    r.hset(orderid, sku, batchref)  
  
def get_readmodel(orderid):  
    return r.hgetall(orderid)
```

(Может быть, имя `redis_eventpublisher.py` сейчас совсем не к месту, но вы поняли идею. А само представление слегка меняется, чтобы приспособиться к своему новому бэккенду.

Представление, адаптированное к Redis (src/allocation/views.py)

```
def allocations(orderid):  
    batches = redis_eventpublisher.get_readmodel(orderid)  
    return [  
        {'batchref': b.decode(), 'sku': s.decode()}  
        for s, b in batches.items()  
    ]
```

Прежние интеграционные тесты также работают, потому что они написаны на уровне абстракции, который отвязан от реализации: при настройке сообщения помещаются в шину сообщений, а утверждения истинности выполняются относительно представления.



С помощью обработчиков событий можно здорово управлять моделью чтения, если она вам вдруг понадобится. Они также позволяют в дальнейшем легко изменять реализацию этой модели.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ

Настройте еще одно представление, на этот раз для вывода на экран размещения одной товарной позиции заказа.

Здесь компромисс между использованием жестко закодированного SQL и прохождением через репозиторий не такой уж явный. Попробуйте несколько версий (может, и переход на Redis) и посмотрите, какая вам по душе.

Выводы

В табл. 12.2 собраны плюсы и минусы по каждому из наших вариантов.

Так уж получилось, что служба размещения заказов в MADE.com действительно использует «полномасштабное» разделение обязанностей команд и запросов: модель чтения хранится в Redis, а второй уровень кэширования вообще предоставляется через Varnish. Но ее варианты использования довольно сильно отличаются от приведенных здесь. Вряд ли в таком случае вы будете использовать отдельную модель чтения и обработчики событий для ее обновления.

Зато чем сложнее со временем становится модель предметной области, тем более привлекательной выглядит упрощенная модель чтения.

Таблица 12.2. Компромиссы различных вариантов модели представления

Вариант	Плюсы	Минусы
Просто использовать репозитории	Простой согласованный подход	Возможные проблемы с производительностью при использовании сложных паттернов запросов
Использование собственных запросов с ORM	Возможность вторично использовать конфигурацию БД и определения модели	Еще один язык запросов со своими причудами и синтаксисом
Использование свернутого вручную SQL	Точный контроль над производительностью с помощью стандартного синтаксиса запросов	Изменения схемы БД должны вноситься в ваши ручные запросы и определения ORM. Сильно нормализованные схемы по-прежнему могут быть ограничены в производительности
Создание отдельных хранилищ для чтения с событиями	Копии только для чтения легко масштабируются. Представления могут строиться при изменении данных, благодаря чему запросы получаются максимально простыми	Сложный метод. Гарри будет с подозрением смотреть на такое

Нередко операции чтения будут воздействовать на те же концептуальные объекты, что и модель записи, поэтому весьма неплохо использовать ORM, добавить несколько методов чтения в свои репозитории и применить классы модели предметной области в операциях чтения.

В нашем примере операции чтения действуют на совершенно разные концептуальные сущности в модели предметной области. Служба размещения заказов использует партии товара, *Batches*, для одного-единственного артикула, но пользователи хотят заказывать из всего ассортимента, им нужно сразу несколько артикулов, поэтому ORM тут не совсем подходит. Нам бы очень хотелось обойтись представлением с сырым SQL, которое мы показали в самом начале главы.

И на этой ноте перейдем к последней главе.

Внедрение зависимостей (и начальная загрузка)

На внедрение зависимостей (dependency injection, DI) в мире Python смотрят с подозрением. И пока что мы прекрасно обходились без него.

Здесь же мы рассмотрим некоторые недостатки кода, которые заставляют задуматься об использовании внедрения зависимостей, и представим несколько вариантов его реализации. А вы сами выберете тот способ, который вам кажется наиболее питоновским.

Мы также добавим в нашу архитектуру новый компонент под названием `bootstrap.py` — он будет отвечать за внедрение зависимостей, а также за некоторые другие вещи, связанные с инициализацией. Мы объясним, почему в объектно ориентированных языках такие штуки называются *точкой сборки* (composition root) и почему сценарий начальной загрузки `bootstrap.py` прекрасно подходит для наших целей.

На рис. 13.1 показано, как выглядит наше приложение без загрузчика: точки входа выполняют большую часть инициализации и передачи нашей основной зависимости — UoW.



Если вы еще этого не сделали, прочтите главу 3, где обсуждается функциональное и объектно-ориентированное управление зависимостями.



Код для этой главы находится в ветке `chapter_13_dependency_injection` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout chapter_13_dependency_injection
```

¹ См. <https://oreil.ly/-B7e6>

```
# или, если пишете код по ходу чтения, возьмите за основу  
# материал из предыдущей главы:  
git checkout chapter_12_cqrs
```

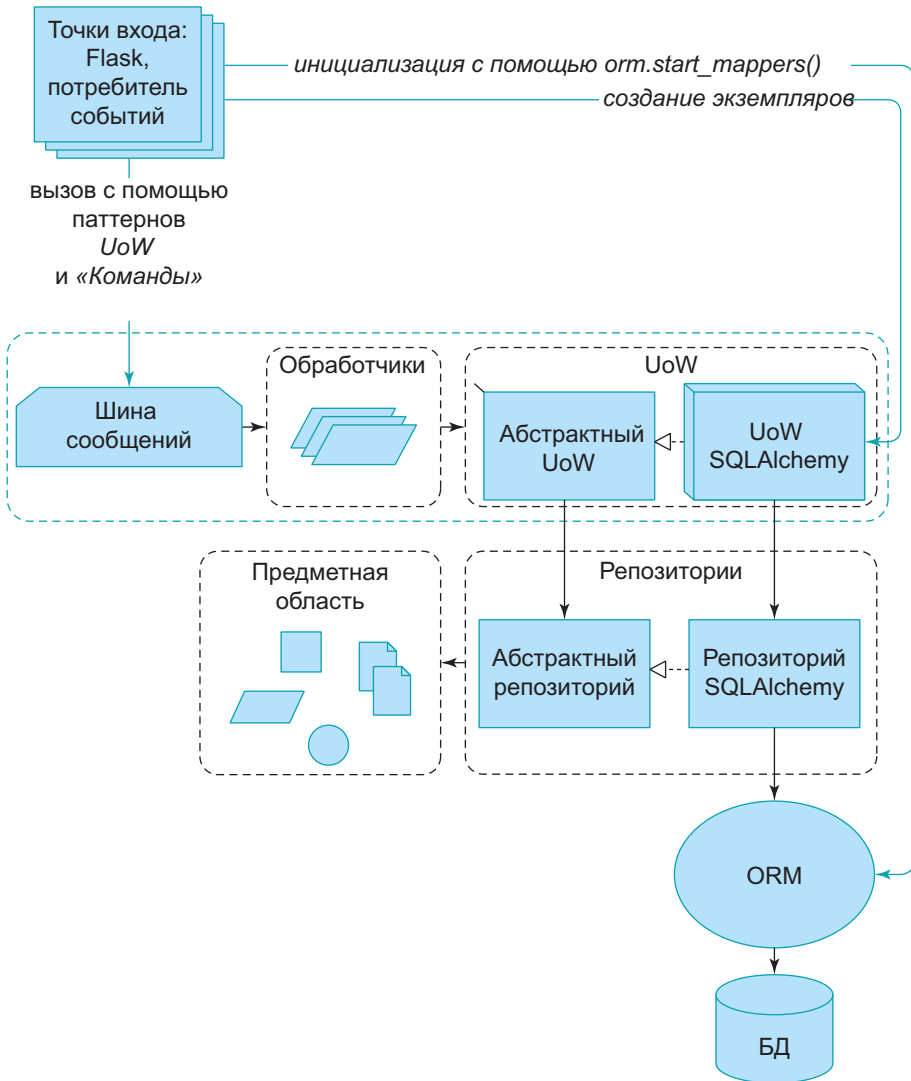


Рис. 13.1. Без загрузчика: точки входа перегружены

На рис. 13.2 показано, как загрузчик берет на себя эти обязанности.

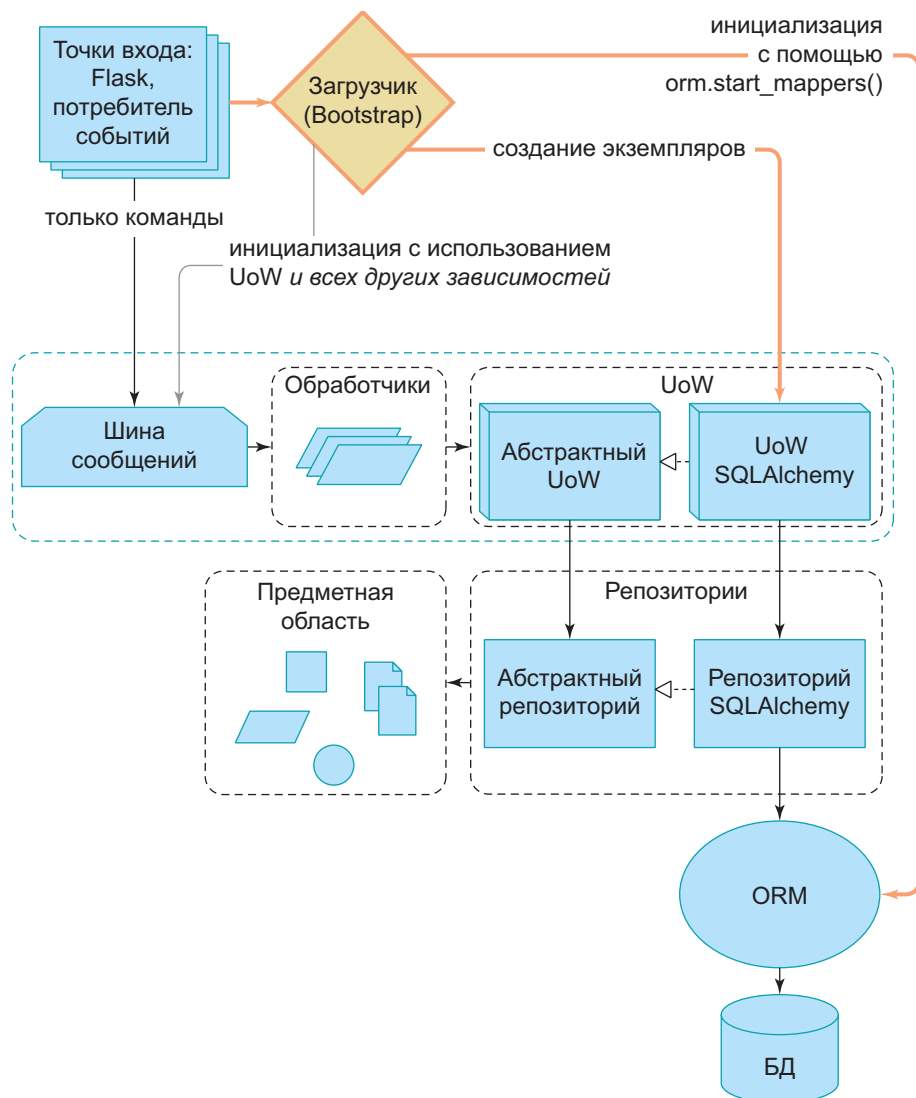


Рис. 13.2. Загрузчик (Bootstrap) позаботится обо всем в едином месте

Неявные зависимости против явных

В этот момент вы можете слегка беспокоиться. Давайте поговорим об этом. Мы показали два способа управления зависимостями и их тестирования.

Для базы данных была создана хорошо продуманная структура явных зависимостей и простых вариантов их переопределения в тестах. Основные функции-обработчики декларируют явную зависимость от UoW.

Обработчики явно зависят от UoW (src/allocation/service_layer/handlers.py)

```
def allocate(
    cmd: commands.Allocate, uow: unit_of_work.AbstractUnitOfWork
):
```

Что упрощает внесение поддельного UoW в тесты сервисного слоя.

Тесты сервисного слоя относительно поддельного UoW (tests/unit/test_services.py)

```
uow = FakeUnitOfWork()
messagebus.handle(..., uow)
```

UoW объявляет явную зависимость от фабрики сеансов.

UoW зависит от фабрики сеансов (src/allocation/service_layer/unit_of_work.py)

```
class SQLAlchemyUnitOfWork(AbstractUnitOfWork):

    def __init__(self, session_factory=DEFAULT_SESSION_FACTORY):
        self.session_factory = session_factory
        ...
```

Мы пользуемся ее преимуществами в интеграционных тестах, чтобы иногда использовать SQLite вместо Postgres.

Интеграционные тесты с другой БД (tests/integration/test_uow.py)

```
def test_rolls_back_uncommitted_work_by_default(sqlite_session_
factory):
    uow = unit_of_work.SqlAlchemyUnitOfWork(sqlite_session_factory) ❶
```

❶ Интеграционные тесты изымают `session_factory` для Postgres, предоставляя такую же фабрику для SQLite.

Разве явные зависимости не кажутся странными и Java-подобными?

Если вы привыкли к тому, как в Python обычно все происходит, то сочтете эти примеры странными. Как правило, мы объявляем зависимость неявно, просто импортируя ее, а затем, если когда-нибудь понадобится изменить ее для тестов, можем поставить обезьянью заплатку, как это принято делать в динамических языках.

Отправка имейлов как обычная зависимость на основе импорта (`src/allocation/service_layer/handlers.py`)

```
from allocation.adapters import email, redis_eventpublisher ❶
...

def send_out_of_stock_notification(
    event: events.OutOfStock, uow: unit_of_work.AbstractUnitOfWork,
):
    email.send( ❷
        'stock@made.com',
        f'Артикула {event.sku} нет в наличии',
    )
```

❶ Жестко закодированный импорт.

❷ Вызывает конкретный отправитель имейлов напрямую.

Зачем засорять код приложения ненужными аргументами только ради тестов? Благодаря вызову `mock.patch` установка обезьяньих заплаток происходит легко и непринужденно.

мок точка патч, спасибо Майклу Форду (`tests/unit/test_handlers.py`)

```
with mock.patch("allocation.adapters.email.send") as mock_send_mail:
    ...
```

Проблема в том, что наш игрушечный пример не отправляет настоящие имейлы (`email.send_mail` использует инструкцию `print`) и в реальной жизни пришлось бы вызывать `mock.patch` для *каждого отдельного теста*, который может приводить к уведомлению об отсутствии товара в наличии. Если вы работали над кодовыми базами с множеством вызовов `mock`, которые используются для предотвращения нежелательных побочных эффектов, то знаете, как раздражает этот поддельный шаблон.

И вы знаете, что имитации `mock` сильно привязывают нас к реализации. Выбирая обезьянью заплатку `email.send_mail`, мы становимся заложниками инструкции `import email`, и если когда-нибудь понадобится использовать инструкцию `from email import send_mail`, в сущности самый обыкновенный рефакторинг кода, то придется изменить все имитации `mock`.

Так что это компромисс. Да, объявление явных зависимостей, строго говоря, не является необходимым, и их использование делает код приложения сложнее. Но взамен наши тесты становятся более легкими в написании и управлении.

Кроме того, такой шаг согласуется с принципом инверсии зависимостей — вместо того чтобы иметь (неявную) зависимость от конкретной детали, мы имеем (явную) зависимость от абстракции.

Явное лучше, чем неявное.

Дзен языка Python

Явная зависимость более абстрактна (src/allocation/service_layer/handlers.py)

```
def send_out_of_stock_notification(
    event: events.OutOfStock, send_mail: Callable,
):
    send_mail(
        'stock@made.com',
        f'Артикула {event.sku} нет в наличии',
    )
```

Но если перейти к явному объявлению всех этих зависимостей, то кто и как будет их внедрять? До сих пор мы имели дело только с передачей `UoW` туда-сюда: в тестах используется поддельный `UoW`, `FakeUnitOfWork`, в то время как в точках входа с потребителями событий `Flask` и `Redis` используется реальный `UoW` и шина сообщений передает их обработчикам команд. Если добавить классы реальной и поддельной электронной почты, то кто будет создавать их и передавать дальше?

Это дополнительный (дублированный) мусор для `Flask`, `Redis` и тестов. Более того, возложение всей обязанности за передачу зависимостей нужному обработчику на шину сообщений воспринимается как нарушение принципа единственной обязанности.

Обратимся к паттерну Composition Root (сценарий начальной загрузки для вас и для нас)¹ и выполним небольшой объем «DI вручную» (внедрение зависимостей без фреймворка). См. рис. 13.3².

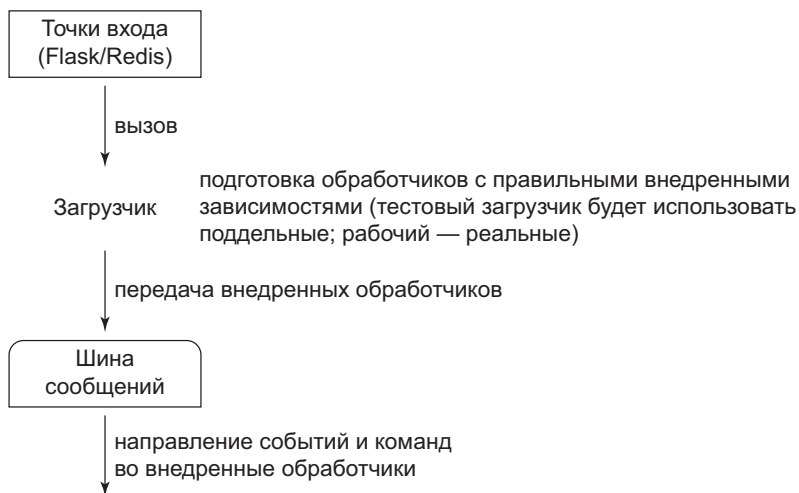


Рис. 13.3. Загрузчик между точками входа и шиной сообщений

Подготовка обработчиков: внедрение зависимостей вручную с помощью замыканий и частичных применений

Превратить функцию с зависимостями в такую, которая позже вызывается с уже внедренными зависимостями, можно, задействовав замыкания или частично примененные функции для того, чтобы составить функцию с ее зависимостями.

Примеры внедрения зависимостей с использованием замыканий или частично примененных функций

существующая функция allocate с зависимостью от абстрактной Uow
def allocate(

¹ Поскольку Python не является «чистым» объектно-ориентированным языком, то разработчикам Python непривычно компоновать набор объектов в рабочее приложение. Мы просто выбираем точку входа и запускаем код сверху вниз.

² Марк Симан называет это чистым, или классическим, внедрением зависимостей. См. <https://oreil.ly/iGpDL>

```

        cmd: commands.Allocate, uow: unit_of_work.AbstractUnitOfWork
    ):
        line = OrderLine(cmd.orderid, cmd.sku, cmd.qty)
        with uow:
            ...
# сценарий начальной загрузки подготавливает фактический UoW

def bootstrap(..):
    uow = unit_of_work.SqlAlchemyUnitOfWork()

    # подготовить версию функции allocate с зависимостью от UoW,
    # захваченного в замыкании
    allocate_composed = lambda cmd: allocate(cmd, uow)

    # либо эквивалентным образом (это обеспечивает более приятную
    # трассировку стека)
    def allocate_composed(cmd):
        return allocate(cmd, uow)

    # альтернатива с частичным применением функции
    import functools
    allocate_composed = functools.partial(allocate, uow=uow) ❶

# позже, во время выполнения, мы можем вызвать частично примененную
# функцию, и в ней уже будет привязанный UoW
allocate_composed(cmd)

```

❶ Разница между замыканиями (лямбда-выражениями или именованными функциями) и частичными применениями `functools.partial` состоит в том, что в первых используется поздняя привязка переменных¹. Это может вызывать путаницу, если какая-либо из зависимостей является мутируемой.

Вот тот же самый паттерн для обработчика уведомления об отсутствии товара в наличии, `send_out_of_stock_notification()`, который имеет другие зависимости.

Еще один пример замыкания и частично примененной функции

```

def send_out_of_stock_notification(
    event: events.OutOfStock, send_mail: Callable,
):
    send_mail(
        'stock@made.com',
        ...
# подготовить версию обработчика send_out_of_stock_notification
# с зависимостями

```

¹ См. <https://docs.python-guide.org/writing/gotchas/#late-binding-closures>

```
sosn_composed = lambda event: send_out_of_stock_notification(event,
email.send_mail)
```

```
...
```

```
# позже, во время выполнения:
```

```
sosn_composed(event) # email.send_mail уже будет внедрена
```

Альтернатива с использованием классов

Замыкания и частично примененные функции — не проблема для тех, кто знаком с функциональным программированием. Если это не про вас, то есть неплохой альтернативный вариант. Но для этого потребуются переписать все функции-обработчики в виде классов.

Внедрение зависимостей с использованием классов

```
# заменяем старое определение `def allocate(cmd, uow)` на:
class AllocateHandler:
```

```
    def __init__(self, uow: unit_of_work.AbstractUnitOfWork): ❷
        self.uow = uow
```

```
    def __call__(self, cmd: commands.Allocate): ❶
        line = OrderLine(cmd.orderid, cmd.sku, cmd.qty)
        with self.uow:
            # прежний остаток метода-обработчика
            ...
```

```
# Сценарий начальной загрузки подготавливает фактический UoW
uow = unit_of_work.SqlAlchemyUnitOfWork()
```

```
# затем подготавливает версию функции allocate с уже внедренными
# зависимостями
allocate = AllocateHandler(uow)
```

```
...
```

```
# позже, во время выполнения, можно вызвать экземпляр обработчика,
# в который уже будет внедрен UoW
allocate(cmd)
```

❶ Класс предназначен для создания вызываемой функции (callable), поэтому он имеет метод call.

❷ Но мы используем init для объявления необходимых ему зависимостей. Вам будет это знакомо, если вы когда-либо создавали на основе

классов дескрипторы либо контекстный менеджер, который принимает аргументы.

Используйте то, что удобнее вам и вашей команде.

Сценарий начальной загрузки

Нам нужно, чтобы сценарий начальной загрузки выполнял следующие действия:

- объявлял зависимости по умолчанию, но позволял их переопределять;
- делал все, что касается «инициализации», нужной для запуска приложения;
- внедрял все зависимости в обработчики;
- возвращал основной для приложения объект — шину сообщений.

Вот первая попытка:

Функция начальной загрузки (src/allocation/bootstrap.py)

```
def bootstrap(
    start_orm: bool = True, ❶
    uow: unit_of_work.AbstractUnitOfWork =
    unit_of_work.SqlAlchemyUnitOfWork(), ❷
    send_mail: Callable = email.send,
    publish: Callable = redis_eventpublisher.publish,
) -> messagebus.MessageBus:

    if start_orm:
        orm.start_mappers() ❶

    dependencies = {'uow': uow, 'send_mail': send_mail, 'publish':
        publish}
    injected_event_handlers = { ❸
        event_type: [
            inject_dependencies(handler, dependencies)
            for handler in event_handlers
        ]
        for event_type, event_handlers in handlers.EVENT_HANDLERS.items()
    }
    injected_command_handlers = { ❸
        command_type: inject_dependencies(handler, dependencies)
        for command_type, handler in handlers.COMMAND_HANDLERS.items()
    }
```

```
return messagebus.MessageBus( ❷  
    uow=uow,  
    event_handlers=injected_event_handlers,  
    command_handlers=injected_command_handlers,  
)
```

❶ `orm.start_mappers()` — это пример инициализации, которую нужно выполнять один раз при запуске приложения. Здесь также видна настройка модуля логирования `logging`.

❷ Можно использовать аргументы со значениями по умолчанию, определяя любые стандартные/производственные значения. Приятно держать их в одном месте, но иногда зависимости создают побочные эффекты. В этом случае можно установить для них значение по умолчанию `None`.

❸ Создаются внедренные версии попарных сопоставлений обработчика с помощью функции `inject_dependencies()`, которую мы покажем далее.

❹ Возвращается настроенная и готовая к использованию шина сообщений.

Вот как мы внедряем зависимости в функцию-обработчик, проверяя ее:

Внедрение зависимостей путем проверки сигнатур функций (`src/allocation/bootstrap.py`)

```
def inject_dependencies(handler, dependencies):  
    params = inspect.signature(handler).parameters ❶  
    deps = {  
        name: dependency  
        for name, dependency in dependencies.items() ❷  
        if name in params  
    }  
    return lambda message: handler(message, **deps) ❸
```

❶ Проверяем аргументы обработчика событий/команд.

❷ Сопоставляем их по имени с нашими зависимостями.

❸ Внедряем их как именованные аргументы, чтобы произвести частично примененную функцию.

ЕЩЕ БОЛЕЕ «РУЧНОЕ» И ОЧЕВИДНОЕ ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ

Если приведенный выше код кажется сложным, то вот еще более простая версия, которая может вам приглянуться. Это была первая попытка Гарри написать код для `inject_dependencies()` ради внедрения зависимостей «вручную». Когда Боб увидел этот код, то обвинил Гарри в том, что тот все лишь усложнил и вообще написал собственный фреймворк внедрения зависимостей.

Честно говоря, Гарри даже и не пришло в голову, что все это можно сделать проще, но все-таки вот пример.

Ручное создание частичных встроенных функций (src/allocation/bootstrap.py)

```
injected_event_handlers = {
    events.Allocated: [
        lambda e: handlers.publish_allocated_event(e, publish),
        lambda e: handlers.add_allocation_to_read_model(e, uow),
    ],
    events.Deallocated: [
        lambda e: handlers.remove_allocation_from_read_model(e, uow),
        lambda e: handlers.reallocate(e, uow),
    ],
    events.OutOfStock: [
        lambda e: handlers.send_out_of_stock_notification(e, send_mail)
    ]
}
injected_command_handlers = {
    commands.Allocate: lambda c: handlers.allocate(c, uow),
    commands.CreateBatch: \
        lambda c: handlers.add_batch(c, uow),
    commands.ChangeBatchQuantity: \
        lambda c: handlers.change_batch_quantity(c, uow),
}
```

Гарри говорит, что даже не мог себе представить, что напишет так много строк кода и ему придется искать столько аргументов функций вручную. Однако это решение является вполне жизнеспособным, поскольку на каждый обработчик добавляется примерно одна строка, так что такой код легко сопровождать, даже если этих обработчиков у вас десятки.

Приложение структурировано таким образом, что зависимости всегда внедряются только в одном месте — в функции-обработчике, поэтому приведенное выше «мега-ручное» решение и решение Гарри на основе `inspect()` будут работать нормально.

Если вам вдруг захочется внедрить зависимости во множество мест и в разное время, или если когда-нибудь вы попадете в *цепочки зависимостей* (где ваши зависимости имеют собственные зависимости и т. д.), то вам может пригодиться «реальный» фреймворк внедрения зависимостей.

В компании MADE мы частично использовали Inject¹, и это нормально, хотя и огорчает Pylint. Вы также можете попробовать Punq², написанный Бобом, или зависимости команды разработчиков DRY-Python³.

¹ См. <https://pypi.org/project/Inject>

² См. <https://pypi.org/project/punq>

³ См. <https://github.com/dry-python/dependencies>

Шина сообщений получает обработчики во время выполнения

Шина сообщений больше не будет статичной; ей нужно предоставлять уже внедренные обработчики. Поэтому мы преобразовываем ее из модуля в конфигурируемый класс.

Шина сообщений как класс (`src/allocation/service_layer/messagebus.py`)

```
class MessageBus: ❶

    def __init__(
        self,
        uow: unit_of_work.AbstractUnitOfWork,
        event_handlers: Dict[Type[events.Event], List[Callable]], ❷
        command_handlers: Dict[Type[commands.Command], Callable], ❷
    ):
        self.uow = uow
        self.event_handlers = event_handlers
        self.command_handlers = command_handlers

    def handle(self, message: Message): ❸
        self.queue = [message] ❹
        while self.queue:
            message = self.queue.pop(0)
            if isinstance(message, events.Event):
                self.handle_event(message)
            elif isinstance(message, commands.Command):
                self.handle_command(message)
            else:
                raise Exception(f'Сообщение {message} не было
                               Событием или Командой')
```

❶ Шина сообщений становится классом...

❷ ...который получает свои уже внедренные обработчики зависимостей.

❸ Основная функция `handle()`, по существу, та же самая, только теперь несколько атрибутов и методов в ней перемещены в `self`.

❹ Такое использование `self.queue` не безопасно в плане потоков и может привести к проблемам, если используются потоки исполнения, потому что экземпляр шины является глобальным в контексте приложения

Flask в той форме, в какой мы его написали. Здесь есть на что обратить внимание.

Что еще меняется в шине сообщений?

Логика обработчика событий и команд остается неизменной (src/allocation/service_layer/messagebus.py)

```
def handle_event(self, event: events.Event):
    for handler in self.event_handlers[type(event)]: ❶
        try:
            logger.debug('обрабатывается событие %s обработчиком %s',
                          event, handler)
            handler(event) ❷
            self.queue.extend(self.uow.collect_new_events())
        except Exception:
            logger.exception('Событие обработки исключения %s', event)
            continue

def handle_command(self, command: commands.Command):
    logger.debug('handling command %s', command)
    try:
        handler = self.command_handlers[type(command)] ❶
        handler(command) ❷
        self.queue.extend(self.uow.collect_new_events())
    except Exception:
        logger.exception('Команда обработки исключения %s', command)
        raise
```

❶ `handle_event` и `handle_command`, по сути, одинаковы, но вместо индексации в статических словарях `EVENT_HANDLERS` или `COMMAND_HANDLERS` в них используются версии с `self`.

❷ Чтобы не передавать `UoW` в обработчик, мы предполагаем, что обработчики уже имеют все свои зависимости, поэтому требуется лишь один аргумент, конкретное событие или команда.

Использование начальной загрузки в точках входа

В точках входа нашего приложения мы просто вызываем `bootstrap.bootstrap()` и получаем готовую к работе шину сообщений, а не настраиваем `UoW` и все остальное.

Flask вызывает сценарий начальной загрузки (src/allocation/entrypoints/flask_app.py)

```
-from allocation import views
+from allocation import bootstrap, views

app = Flask(__name__)
-orm.start_mappers() ❶
+bus = bootstrap.bootstrap()

@app.route("/add_batch", methods=['POST'])
@@ -19,8 +16,7 @@ def add_batch():
    cmd = commands.CreateBatch(
        request.json['ref'], request.json['sku'],
        request.json['qty'], eta,
    )
-    uow = unit_of_work.SqlAlchemyUnitOfWork() ❷
-    messagebus.handle(cmd, uow)
+    bus.handle(cmd) ❸
    return 'OK', 201
```

❶ Больше не нужно вызывать `start_orm()`; это сделают этапы инициализации в загрузочном сценарии.

❷ Нам также не нужно явно создавать конкретный тип `UoW`; об этом по умолчанию позаботится сценарий начальной загрузки.

❸ Шина сообщений теперь является конкретным экземпляром, а не глобальным модулем¹.

Внедрение зависимостей в тестах

В тестах мы можем использовать `bootstrap.bootstrap()` с переопределенными значениями по умолчанию для получения собственной шины сообщений. Вот пример интеграционного теста:

Переопределение значений по умолчанию bootstrap (tests/integration/test_views.py)

```
@pytest.fixture
def sqlite_bus(sqlite_session_factory):
```

¹ Если разобраться, она все еще глобальна в области видимости модуля `flask_app`. Это может вызвать проблемы, если вы когда-нибудь захотите протестировать свое приложение Flask в процессе с помощью Flask Test Client, а не Docker, как мы. Если хотите глубже погрузиться в тему, стоит изучить фабрики приложений Flask. См. https://oreil.ly/_a6KI

```
bus = bootstrap.bootstrap(
    start_orm=True, ❶
    uow=unit_of_work.SqlAlchemyUnitOfWork(sqlite_session_factory), ❷
    send_mail=lambda *args: None, ❸
    publish=lambda *args: None, ❸
)
yield bus
clear_mappers()

def test_allocations_view(sqlite_bus):
    sqlite_bus.handle(commands.CreateBatch('sku1batch', 'sku1', 50,
    None))
    sqlite_bus.handle(commands.CreateBatch('sku2batch', 'sku2', 50,
    date.today()))
    ...
    assert views.allocations('order1', sqlite_bus.uow) == [
        {'sku': 'sku1', 'batchref': 'sku1batch'},
        {'sku': 'sku2', 'batchref': 'sku2batch'},
    ]
```

❶ Мы по-прежнему хотим запускать ORM...

❷ ...потому что собираемся использовать реальный UoW, пусть и с базой данных прямо в памяти.

❸ Но отправлять имейлы или публиковать что-либо не нужно, поэтому мы вставляем эти пустые команды.

В юнит-тестах, напротив, можно вторично использовать поддельный UoW, FakeUnitOfWork.

Начальная загрузка в юнит-тесте (tests/unit/test_handlers.py)

```
def bootstrap_test_app():
    return bootstrap.bootstrap(
        start_orm=False, ❶
        uow=FakeUnitOfWork(), ❷
        send_mail=lambda *args: None, ❸
        publish=lambda *args: None, ❸
    )
```

❶ Запускать ORM не нужно...

❷ ...потому что поддельный UoW его не использует.

❸ Нужно также подделать имейлы и адаптеры Redis.

Так что это избавляет нас от небольшого дублирования — мы переместили несколько настроек и разумных значений по умолчанию в одно место.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ № 1

Замените все обработчики на классы, как в примере выше, и соответствующим образом измените код внедрения зависимостей в загрузчике. Так вы узнаете, какой подход вам ближе: функциональный либо на основе классов.

«Правильное» создание адаптера: рабочий пример

Чтобы по-настоящему понять, как все это работает, рассмотрим пример того, как «правильно» создать адаптер и сделать для него внедрение зависимостей. Пока что у нас есть два типа зависимостей.

Два типа зависимостей (src/allocation/service_layer/messagebus.py)

```
uow: unit_of_work.AbstractUnitOfWork, ❶  
send_mail: Callable, ❷  
publish: Callable, ❷
```

❶ UoW использует абстрактный базовый класс. Это тяжеловесный вариант для объявления и управления внешней зависимостью. Мы бы использовали это для довольно сложной зависимости.

❷ Отправитель имейлов и издатель pub/sub определяются как функции. Это прекрасно работает для простых зависимостей.

Вот что мы внедрим во время работы:

- клиент файловой системы S3;
- клиент хранилища «ключ — значение»;
- объект сеанса запросов.

У большей части этого будут более сложные API, которые не вписать в единую функцию: чтение и запись, получение и запись и т. д.

Несмотря на простоту зависимости от `send_mail`, воспользуемся ею в качестве примера, чтобы понять, каким образом можно определить более сложную зависимость.

Задаем абстрактную и конкретную реализации

Мы представим более общий API уведомлений. В один и тот же день уведомления могут рассылаться через электронную почту, SMS или Slack.

Абстрактная (с использованием абстрактных базовых классов) и конкретная реализации (src/allocation/adapters/notifications.py)

```
class AbstractNotifications(abc.ABC):

    @abc.abstractmethod
    def send(self, destination, message):
        raise NotImplementedError

...

class EmailNotifications(AbstractNotifications):

    def __init__(self, smtp_host=DEFAULT_HOST, port=DEFAULT_PORT):
        self.server = smtplib.SMTP(smtp_host, port=port)
        self.server.noop()

    def send(self, destination, message):
        msg = f'Subject: allocation service notification\n{message}'
        self.server.sendmail(
            from_addr='allocations@example.com',
            to_addrs=[destination],
            msg=msg
        )
```

Меняем зависимость в сценарии начальной загрузки.

Уведомления в шине сообщений (src/allocation/bootstrap.py)

```
def bootstrap(
    start_orm: bool = True,
    uow: unit_of_work.AbstractUnitOfWork =
    unit_of_work.SqlAlchemyUnitOfWork(),
    - send_mail: Callable = email.send,
    + notifications: AbstractNotifications = EmailNotifications(),
    publish: Callable = redis_eventpublisher.publish,
) -> messagebus.MessageBus:
```

Создаем поддельную версию для тестов

Прорабатываем и определяем поддельную версию для юнит-тестов.

Поддельные уведомления (tests/unit/test_handlers.py)

```
class FakeNotifications(notifications.AbstractNotifications):

    def __init__(self):
        self.sent = defaultdict(list) # тип: Dict[str, List[str]]

    def send(self, destination, message):
        self.sent[destination].append(message)
    ...
```

И используем ее.

Тесты изменяются незначительно (tests/unit/test_handlers.py)

```
def test_sends_email_on_out_of_stock_error(self):
    fake_notifs = FakeNotifications()
    bus = bootstrap.bootstrap(
        start_orm=False,
        uow=FakeUnitOfWork(),
        notifications=fake_notifs,
        publish=lambda *args: None,
    )
    bus.handle(commands.CreateBatch("b1", "POPULAR-CURTAINS", 9, None))
    bus.handle(commands.Allocate("o1", "POPULAR-CURTAINS", 10))
    assert fake_notifs.sent['stock@made.com'] == [
        f"POPULAR-CURTAINS нет в наличии",
    ]
```

Выясняем, как провести интеграционное тестирование реального кода

Теперь мы тестируем реальный код, обычно с помощью сквозного или интеграционного теста. Для среды разработки Docker в качестве реального почтового сервера мы использовали сервер MailHog¹.

Конфигурационный файл docker-compose с подделкой реального почтового сервера (docker-compose.yml)

```
version: "3"

services:

    redis_pubsub:
        build:
            context: .
```

¹ См. <https://github.com/mailhog/MailHog>


```

    dockerfile: Dockerfile
    image: allocation-image
    ...

api:
    image: allocation-image
    ...

postgres:
    image: postgres:9.6
    ...

redis:
    image: redis:alpine
    ...

mailhog:
    image: mailhog/mailhog
    ports:
        - "11025:1025"
        - "18025:8025"

```

В интеграционных тестах мы используем класс реальных уведомлений `EmailNotifications`, который связывается с сервером MailHog в кластере Docker.

Интеграционный тест для электронной почты (`tests/integration/test_email.py`)

```

@pytest.fixture
def bus(sqlite_session_factory):
    bus = bootstrap.bootstrap(
        start_orm=True,
        uow=unit_of_work.SqlAlchemyUnitOfWork(sqlite_session_factory),
        notifications=notifications.EmailNotifications(), ❶
        publish=lambda *args: None,
    )
    yield bus
    clear_mappers()

def get_email_from_mailhog(sku): ❷
    host, port = map(config.get_email_host_and_port().get, ['host',
                                                            'http_port'])
    all_emails = requests.get(f'http://{host}:{port}/api/v2/
                               messages').json()
    return next(m for m in all_emails['items'] if sku in str(m))

```

```
def test_out_of_stock_email(bus):
    sku = random_sku()
    bus.handle(commands.CreateBatch('batch1', sku, 9, None)) ❸
    bus.handle(commands.Allocate('order1', sku, 10))
    email = get_email_from_mailhog(sku)
    assert email['Raw']['From'] == 'allocations@example.com' ❹
    assert email['Raw']['To'] == ['stock@made.com']
    assert f'Out of stock for {sku}' in email['Raw']['Data']
```

❶ Используем загрузчик для создания шины сообщений, которая взаимодействует с классом реальных уведомлений.

❷ Выясняем, как получать имейлы с «реального» почтового сервера.

❸ Используем шину сообщений, чтобы задать условия тестирования.

❹ Несмотря ни на что, это и вправду сработало, и почти с первого раза!

Вот, собственно, и все.

УПРАЖНЕНИЕ ДЛЯ ЧИТАТЕЛЯ № 2

Относительно адаптеров можно сделать две вещи:

- Попробуйте заменить имейл-уведомления SMS-уведомлениями, например, с помощью Twilio или Slack. Сможете ли вы найти хороший аналог серверу MailHog для интеграционного тестирования?
 - Подобно тому как мы делали переход от `send_mail` к `Notifications`, попробуйте сделать рефакторинг `redis_eventpublisher`, который в настоящее время является просто вызываемой функцией, в какой-нибудь более формальный адаптер/базовый класс/протокол.
-

Выводы

Если у вас несколько адаптеров, легко устать от передачи зависимостей туда-сюда вручную, если только вы не сделаете *внедрение зависимостей*.

Настройка внедрения зависимостей — лишь одно из многих стандартных действий по настройке/инициализации, которые нужно сделать всего один раз при запуске приложения. Обычно неплохо бы объединить все это в *сценарий начальной загрузки* часто является неплохой идеей.

Сценарий начальной загрузки также хорош в качестве места указания разумной конфигурации по умолчанию для адаптеров, а также единого места для замены этих адаптеров подделками для тестов.

Фреймворк внедрения зависимостей может быть полезен, если это внедрение нужно делать на нескольких уровнях — например, если у вас есть цепочки зависимостей из компонентов, каждому из которых нужно внедрение.

В этой главе мы подробно разобрали превращение неявной/простой зависимости в «правильный» адаптер с рефакторингом абстрактных базовых классов, определением его реальной и поддельной реализаций и продумыванием интеграционного тестирования.

КРАТКО О ВНЕДРЕНИИ ЗАВИСИМОСТЕЙ И СТАРТОВОЙ ЗАГРУЗКЕ

В двух словах:

1. Определите свой API, используя абстрактные базовые классы.
 2. Реализуйте реальную сущность.
 3. Создайте подделку и используйте ее для юнит-тестов/тестов слоя служб/тестов обработчиков.
 4. Найдите менее поддельную версию, которую можно поместить в свою среду Docker.
 5. Проверьте менее поддельную «реальную» сущность.
 6. Готово!
-

Вот мы и обсудили последние паттерны и добрались до конца второй части. В эпилоге мы дадим несколько советов по применению всего этого в Реальном Мире™.

Эпилог

И что теперь?

Фух! Мы успели рассмотреть очень много вещей, и большинство читателей услышали о них впервые. Но цель этой книги вовсе не в том, чтобы сделать из вас экспертов. Мы можем лишь дать какие-то общие идеи и примеры кода, чтобы вы могли пойти дальше и написать что-то с нуля.

Наши примеры — это не закаленный в боях производственный код, а всего лишь набор кубиков, с помощью которых вы можете построить свой первый дом, космический корабль или небоскреб.

Перед нами стоят две большие задачи — поговорить о том, как начать применять эти идеи на практике в реальной системе, и предупредить вас о том, что пришлось пропустить. Мы дали вам целый арсенал, и чтобы не отстрелить себе ногу, нужно обсудить некоторые базовые правила безопасности.

Как мне добраться туда?

Скорее всего, многие из вас думают примерно следующее:

«Окей, Боб и Гарри, все это прекрасно, и если меня когда-нибудь возьмут работать над новой неосвоенной службой, я знаю, что делать. Но пока же я сижу здесь со своим большим комком грязи Django и совершенно не понимаю, как добраться до вашей милой, чистой, совершенной, незапятнанной, упрощенной модели. Только не отсюда».

Что ж... С большим комком грязи на руках трудно понять, как все исправить. Разбираться со всем нужно постепенно, шаг за шагом.

Прежде всего подумайте: какую задачу вы пытаетесь решить? Трудно ли вносить изменения в ПО? Есть ли у него проблемы с производительностью? А странные, необъяснимые баги?

Если у вас есть четкая цель, расставлять приоритеты в работе становится проще. К тому же так легче объяснить другим разработчикам, зачем все это нужно.

Компании, как правило, прагматично подходят к техническому долгу и рефакторингу, до тех пор пока инженеры могут привести разумные аргументы в пользу исправления ситуации.



Вам будет проще «продать» начальству идею о внесении серьезных изменений, если вы обоснуете ее расширением функциональности. Планируется запуск нового продукта или выход сервиса на новый рынок? Самое время, чтобы потратить инженерные ресурсы на ремонт фундамента. С проектом, который нужно завершить за полгода, легче аргументировать трехнедельную работу по чистке кода. Боб называет это *налогом на архитектуру*.

Разделение запутанных обязанностей

В начале книги мы говорили, что главной характеристикой большого ком-ка грязи является однородность: все части системы выглядят одинаково, так как мы нечетко определили обязанности каждого компонента. Для того чтобы это исправить, нужно разделить обязанности и ввести четкие границы. Первое, что можно сделать, — это начать создавать сервисный слой (рис. Э.1).

Столкнувшись с такой системой, Боб впервые узнал, как разделить ком-мок грязи, и это было сногсшибательно. Логика была *повсюду* — в веб-страницах, в менеджерах, в помощниках, в громоздких классах-службах, которые мы написали, чтобы абстрагировать менеджеры и помощники, и в ужасающе сложных объектах-командах, которые разбивали сервисы на части.

Если вы работаете с системой, которая запуталась до такого состояния, то не спешите отчаиваться: никогда не поздно начать пропалывать заросший сад. В итоге мы наняли знающего архитектора, и он помог вернуть все под контроль.

Начните с проработки *вариантов использования* вашей системы. Если имеется пользовательский интерфейс, то какие действия он выполняет?

Если есть компонент бэкенда с обработкой на сервере, то, может, каждое задание планировщика `сгон` или задание Celery будет единственным вариантом использования? Каждый из вариантов использования должен иметь свое название, причем в форме повелительного наклонения вроде «Провести платежные сборы», «Очистить заброшенные учетные записи» или «Разместить заказ на покупку».

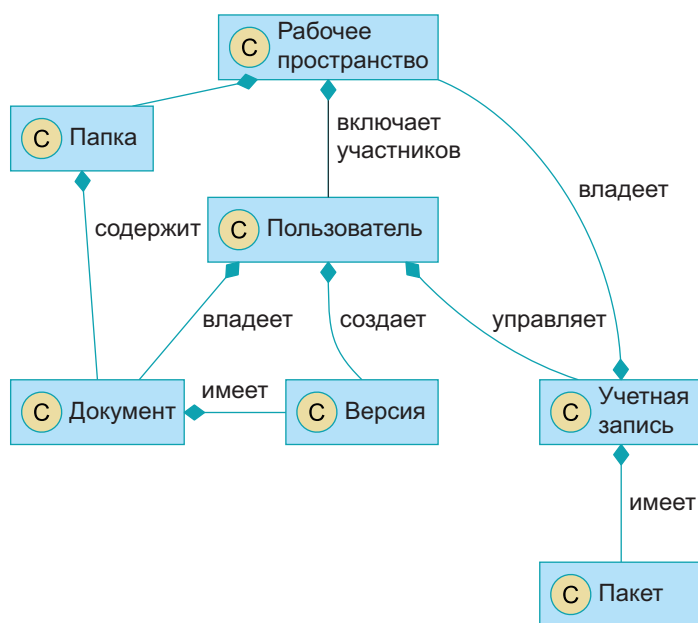


Рис. Э.1. Предметная область системы совместной работы

В нашем случае большинство вариантов использования были частью классов менеджеров и назывались, например, «Создать рабочее пространство» или «Удалить версию документа». Каждый вариант использования вызывался из пользовательского интерфейса веб-приложения.

Мы стремимся к тому, чтобы у каждой из этих поддерживаемых операций, которая занимается *оркестровкой* выполняемой работы, была всего одна функция или класс. Каждый вариант использования должен выполнять следующие действия:

- при необходимости запускать собственную транзакцию базы данных;
- доставать все необходимые данные;
- проверять все предварительные условия (см. паттерн «Обеспечение» в приложении Д);
- обновлять модели предметной области;
- сохранять любые изменения.

Каждый вариант использования должен быть успешным или неуспешным как атомарная единица. Возможно, придется вызывать один вариант использования из другого, и это нормально. Примите это к сведению и постарайтесь избегать длительных транзакций с базой данных.



Одна из самых больших проблем, с которой мы столкнулись, заключалась в том, что управляющие методы вызывали другие управляющие методы и доступ к данным мог происходить из самих объектов модели. Было трудно понять, к чему приводит каждая операция, не изучив всю кодовую базу. Сведение всей логики в один метод и использование паттерна UoW для управления транзакциями облегчили работу системы.



Ничего страшного, если функции варианта использования дублируются. Мы пытаемся не написать совершенный код, а извлечь некоторые значимые слои. Лучше дублировать код в нескольких местах, чем иметь функции варианта использования, которые вызывают друг друга в длинной цепочке.

Это хорошая возможность вытащить любой код доступа к данным или код оркестровки из модели предметной области и поместить в варианты использования. Также нужно попытаться вытащить обязанности по вводу-выводу (например, отправку электронной почты, запись файлов) из модели предметной области и поместить в функции варианта использования. Мы используем методы из главы 3, чтобы поддерживать юнит-тестирование обработчиков, даже когда они выполняют ввод-вывод.

Такие функции варианта использования главным образом будут связаны с логированием, доступом к данным и обработкой ошибок. Выполнив этот

шаг, вы поймете, что на самом деле *делает* ваша программа, и сможете убедиться, что каждая операция имеет четко определенные начало и конец. Это шаг к созданию чистой модели предметной области.

СЛУЧАЙ ИЗ ПРАКТИКИ: РАЗДЕЛЕНИЕ РАЗРОСШЕЙСЯ СИСТЕМЫ НА СЛОИ

Много лет назад Боб трудился в компании по разработке ПО, которая работала над первой версией своего приложения — онлайн-платформы для совместной работы и обмена файлами.

Когда компания начала разработку собственными силами, этот проект прошел через руки нескольких поколений разработчиков, и каждый из них все больше усложнял структуру кода.

В основе своей система представляла собой приложение ASP.NET Web Forms, созданное с помощью ORM NHibernate. Пользователи загружали документы в рабочие пространства, а затем приглашали других участников просматривать, комментировать или изменять их работу.

Большая часть сложности приложения была связана с моделью разрешений, поскольку каждый документ хранился в папке, а папки допускали чтение, запись и редактирование, как в файловой системе Linux.

Кроме того, каждое рабочее пространство было связано с учетной записью, и к ней же были прикреплены квоты с помощью пакета выставления счетов.

В результате каждая операция чтения или записи документа должна была загружать огромное число объектов из базы данных для проверки разрешений и квот. Создание нового рабочего пространства включало сотни запросов к базе данных, когда мы устанавливали структуру разрешений, приглашали пользователей и готовили образцы контента.

Часть кода для операций находилась в веб-обработчиках, которые запускались, когда пользователь нажимал кнопку или отправлял форму; часть — в объектах-менеджерах, которые содержали код для оркестровки работы, и еще немного — в модели предметной области. Объекты модели вызывали базы данных или копировали файлы на диск, да и охват тестов оставлял желать лучшего.

Для устранения этой проблемы сначала мы добавили сервисный слой, чтобы весь код для создания документа или рабочего пространства находился в одном месте и был понятен. Сюда входили извлечение кода доступа к данным из модели предметной области и помещение в обработчики команд. Точно так же мы извлекли код оркестровки из объектов-менеджеров и веб-обработчиков и поместили его в обработчики.

Получившиеся в результате обработчики команд вышли *длинными* и грязными, но процесс борьбы с хаосом был запущен.

Прочтите книгу «Эффективная работа с унаследованным кодом» Майкла Физерса (Working Effectively with Legacy Code, Michael C. Feathers, Prentice Hall), в которой даются рекомендации по тестированию унаследованного кода и началу разделения обязанностей.

Определение агрегатов и ограниченных контекстов

Часть проблемы с кодовой базой из нашего примера заключалась в том, что граф объектов был сильно связанным. Под каждой учетной записью числилось много рабочих пространств, в каждом из которых было много участников, а у тех, в свою очередь, были собственные учетные записи. В каждом рабочем пространстве было полно документов самых разных версий.

Сложно выразить весь ужас этой системы в диаграмме классов. Прежде всего, ни одна учетная запись не была связана с пользователем. Вместо этого имелось странное правило, требующее, чтобы вы перечислили все учетные записи, связанные с пользователем через рабочие области, и взяли ту, которая имеет самую раннюю дату создания.

Каждый объект в системе был частью иерархии наследования, включающей `SecureObject` и `Version`. Эта иерархия была отражена непосредственно в схеме базы данных, так что каждый запрос должен был объединить десять разных таблиц и сверяться со столбцом дискриминатора только для того, чтобы сказать, с какими объектами вы работаете. Кодовая база позволяла легко «расставлять точки» через эти объекты на вашем пути следующим образом:

```
user.account.workspaces[0].documents.versions  
[1].owner.account.settings[0];
```

Создавать так систему с помощью ORM Django или SQLAlchemy несложно, но лучше так не делать. Да, это *удобно*, но мешает правильно оценивать производительность, ведь каждое свойство может запускать поиск в базе данных.



Агрегаты — это *граница согласованности*. В общем случае каждый вариант использования должен обновлять по одному агрегату за раз. Один обработчик извлекает один агрегат из репозитория, изменяет его состояние и инициирует какие-нибудь события, которые происходят в результате. Если нужны данные из другой части системы, то можно использовать модель чтения. Но лучше не обновляйте сразу несколько агрегатов в одной транзакции. Когда мы решаем разделить код на разные агрегаты, то определенно решаем сделать их *согласованными* друг с другом *в конечном счете*.

Ряд операций требовал циклично перебирать объекты, например:

Заблокировать рабочие пространства пользователя за неуплату

```
def lock_account(user):  
    for workspace in user.account.workspaces:  
        workspace.archive()
```

Или даже рекурсивно прокручивать коллекции папок и документов:

```
def lock_documents_in_folder(folder):  
  
    for doc in folder.documents:  
        doc.archive()  
  
    for child in folder.children:  
        lock_documents_in_folder(child)
```

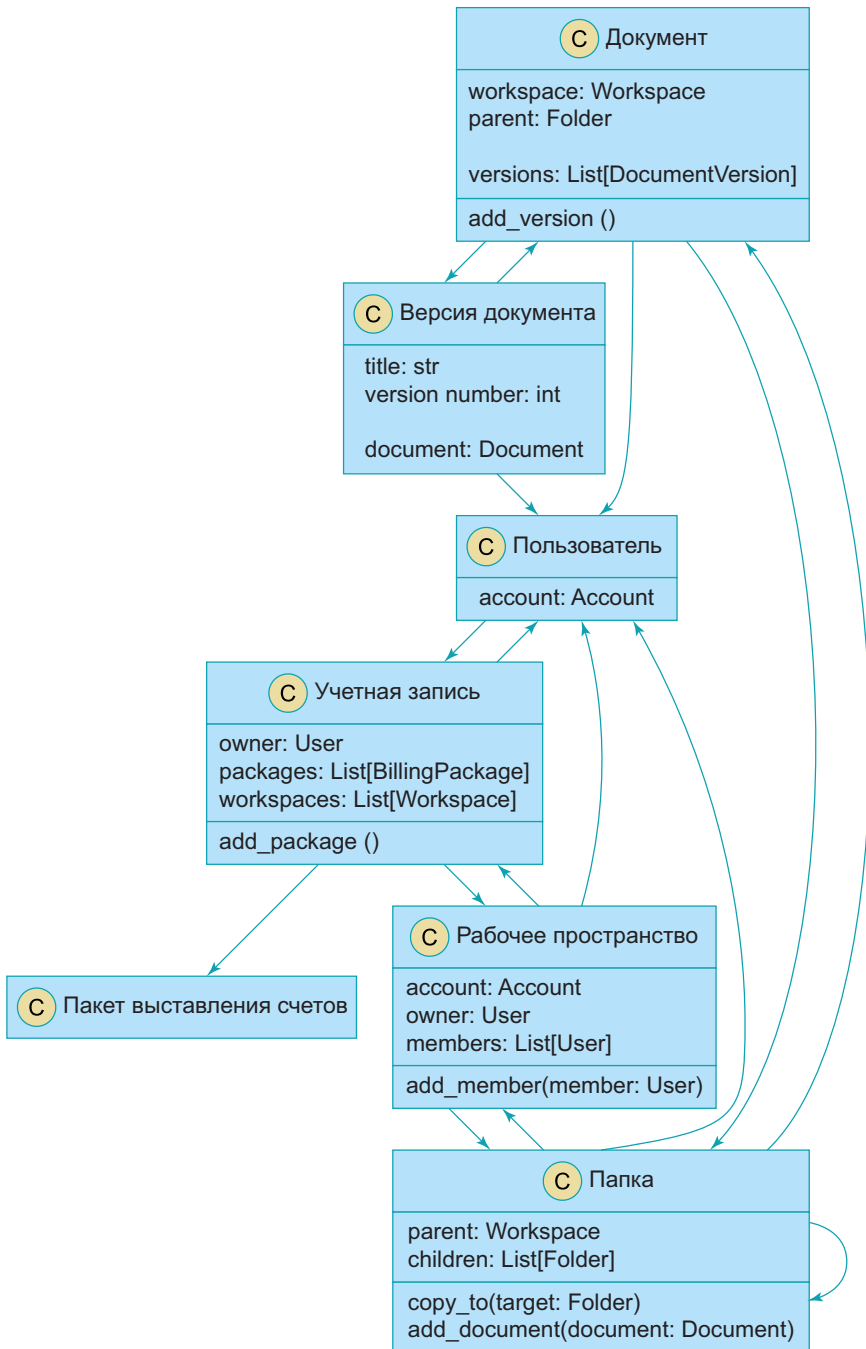
Эти операции убивали производительность, но ради их исправления пришлось бы отказаться от единого объектного графа. Вместо этого мы начали выявлять агрегаты и разрывать прямые связи между объектами.



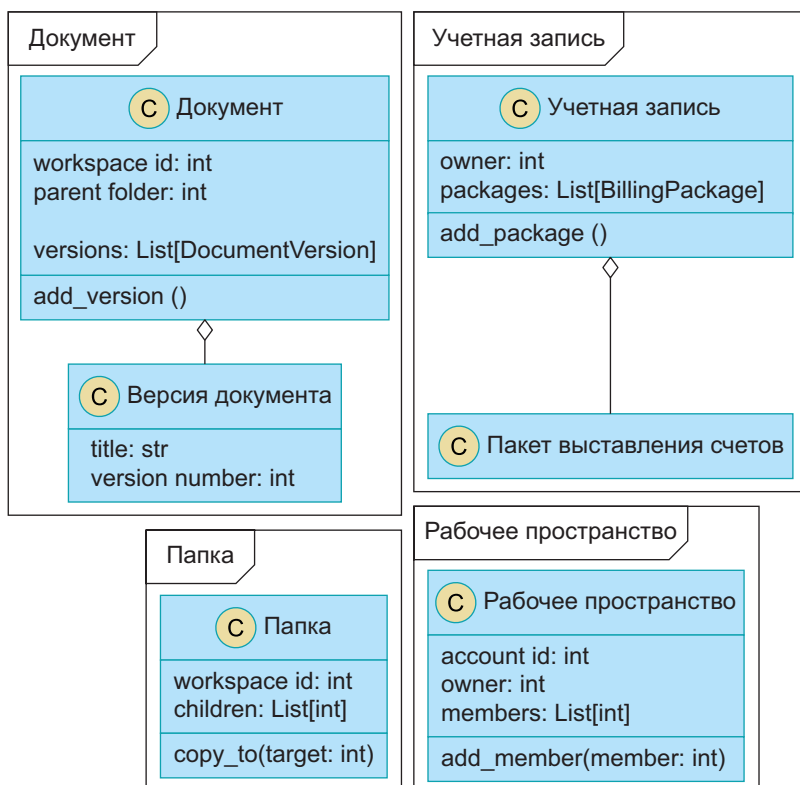
О печально известной проблеме **SELECT N+1** и о том, как можно использовать техники при чтении данных для запросов по сравнению с чтением данных для команд, рассказывалось в главе 12.

В основном мы делали это, заменяя прямые ссылки идентификаторами.

До агрегатов было так:



После моделирования с помощью агрегатов — так:



Зачастую если ссылки в вашей модели двунаправленные, значит, вы неправильно настроили агрегаты. В нашем коде документ, `Document`, знал о содержащей его папке, `Folder`, а папка, `Folder`, включала коллекцию документов, `Document`. Это позволяет легко перемещаться по объектному графу, но мешает находить правильные границы согласованности. Мы разбиваем агрегаты на части, используя вместо этого ссылки. В новой модели документ, `Document`, имел ссылку на свою родительскую папку, `parent_folder`, но не имел прямого доступа к папке, `Folder`.

Если данные нужны только для *чтения*, то мы стараемся заменить сложные циклы и преобразования на прямой SQL. Например, один из экранов представлял папки и документы в форме дерева.

Этот экран был *невероятно* тяжелым для базы данных, потому что опирался на вложенные циклы `for`, запускающие медленно загружаемый ORM.



Мы используем тот же прием в главе 11, где заменяем вложенный цикл с объектами ORM простым SQL-запросом. Это первый шаг в подходе CQRS.

После долгих размышлений мы заменили код ORM на большую «уродливо» хранимую процедуру. Код выглядел ужасно, но работал намного быстрее и помогал устранить связи между папкой, `Folder`, и документом, `Document`.

Когда нужно было *записывать* данные, мы меняли по одному агрегату за раз. Ввели шину сообщений для обработки событий. Например, в новой модели при блокировке учетной записи можно было сначала запросить все затронутые рабочие пространства с помощью команды `SELECT id FROM workspace WHERE account_id = ?`.

Затем мы могли бы создать новую команду для каждого рабочего пространства:

```
for workspace_id in workspaces:
    bus.handle(LockWorkspace(workspace_id))
```

Подход на основе событий для перехода к микросервисам через паттерн «Душител»

Паттерн «Душител» (Strangler) предполагает создание новой системы по краям старой, поддерживая ее работоспособность. Части старой функциональности постепенно перехватываются и заменяются до тех пор, пока старая система не окажется бесполезной и не будет отключена.

При создании службы доступности использовался перехват событий для перемещения функциональности из одного места в другое. Процесс включал три этапа:

1. Вызов событий для представления изменений, происходящих в системе, которую нужно заменить.

2. Создание второй системы, которая потребляет эти события и использует их для создания собственной модели предметной области.
3. Замена старой системы на новую.

Мы использовали перехват событий для перехода от схемы на рис. Э.2...

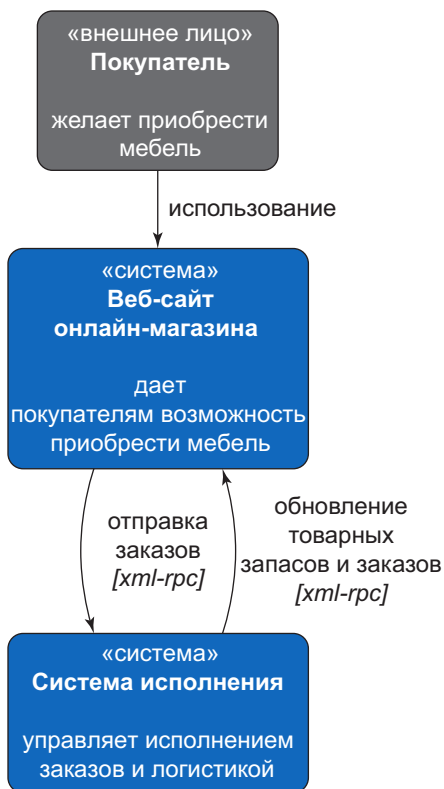


Рис. Э.2. Прежний вид: сильная двунаправленная связанность на основе вызова удаленных процедур XML-RPC

...к схеме на рис. Э.3.

Фактически это был проект, рассчитанный на несколько месяцев. Для начала надо было написать модель предметной области, которая могла бы представлять партии товара, поставки и продукты. Мы применили TDD для создания игрушечной системы, которая могла бы отвечать на один-

единственный вопрос: «Если я хочу N штук артикула КОВРИК-ОПАСНЫЙ, HAZARDOUS_RUG, то через сколько времени их доставят?»



Рис. Э.3. Итоговый вид: слабая связанность с асинхронными событиями (версию в высоком разрешении см. на сайте cosmicpython.com)



При развертывании системы, управляемой событиями, начните с «хотя бы скелета». Развертывание системы, которая просто регистрирует вводимые данные, вынуждает нас решать все вопросы инфраструктуры и начинать работу в продакшене.

СЛУЧАЙ ИЗ ПРАКТИКИ: ФОРМИРОВАНИЕ МИКРОСЕРВИСОВ ДЛЯ ЗАМЕНЫ ПРЕДМЕТНОЙ ОБЛАСТИ

MADE.com вырос на *двух* столпах: одном для внешнего приложения онлайн-магазина (фронтенд) и другом для внутренней системы выполнения заказов (бэкенд).

Эти две системы взаимодействовали через вызовы удаленных процедур XML-RPC. Периодически бэкенд просыпался и запрашивал фронтенд, чтобы узнать о новых заказах. Когда внешняя система импортировала все новые заказы, она отправляла RPC-команды для обновления количества товарных запасов.

Со временем процесс синхронизации замедлялся, пока однажды на Рождество на импорт заказов всего за один день не ушло целых 24 часа. Боба наняли, чтобы он разбил систему на набор событийно-управляемых служб.

Сперва мы установили, что медленнее всего происходили были расчет и синхронизация имеющихся товарных запасов. Нужна была система, которая могла бы прослушивать внешние события и поддерживать актуальное количество товара в наличии.

Мы стали передавать эту информацию через API, чтобы браузер пользователя мог запросить объем остатков товара и срок его доставки.

Всякий раз, когда товар заканчивался, мы инициировали новое событие, которое платформа онлайн-магазина могла бы использовать, чтобы снять товар с продажи. Поскольку мы не знали, с каким уровнем нагрузки придется иметь дело, то написали систему с паттерном CQRS. Всякий раз, когда объем товара в наличии менялся, мы обновляли базу данных Redis с помощью кешированной модели представления. Чтобы не загружать сложную модель предметной области, API веб-фреймворка Flask запрашивал эти модели представления.

В результате на вопрос: «Сколько товара осталось в наличии?» мы смогли отвечать за 2–3 миллисекунды, и теперь API нередко обрабатывает сотни запросов в секунду в течение длительных периодов.

Так что теперь вы знаете, откуда взялся наш пример приложения.

После того как появилась рабочая модель предметной области, мы переключились на создание нескольких инфраструктурных элементов. Наше первое развертывание в продакшене представляло собой крошечную систему, которая могла получать событие создания партии товара, `batch_created`,

и регистрировать его JSON-представление в логе. Эта система была пробной версией событийно-управляемой архитектуры в стиле Hello World. Это заставило нас развернуть шину сообщений, подключить производителя и потребителя, создать конвейер развертывания и написать простой обработчик сообщений.

Получив конвейер развертывания, необходимую инфраструктуру и базовую модель предметной области, мы приступили к работе. Через пару месяцев мы уже обслуживали реальных клиентов.

Как убедить стейкхолдеров попробовать что-то новое

Если вы подумываете о том, чтобы сформировать новую систему из большого комка грязи, то, вероятно, страдаете от проблем с надежностью, производительностью, технической сопровождаемостью или от всего этого сразу. Серьезные проблемы требуют решительных мер!

В качестве первого шага рекомендуем заняться *моделированием предметной области*. Во многих разросшихся системах инженеры, владельцы продуктов и клиенты говорят на разных языках. Стейкхолдеры описывают систему с помощью абстрактных понятий, ориентированных на процесс, в то время как разработчики вынуждены говорить о системе в той форме, в какой она существует физически в своем хаотичном состоянии.

Найти подходящий способ моделирования своей предметной области — довольно сложная задача, и сама по себе она является темой многих хороших книг. Нам нравится использовать интерактивные приемы, такие как событийный штурм и моделирование по принципу CRC, потому что люди хорошо сотрудничают через игру. *Моделирование событий* — это еще один прием, который объединяет инженеров и владельцев продуктов для того, чтобы понять систему с точки зрения команд, запросов и событий.



Несколько замечательных руководств по визуальному моделированию систем с событиями можно найти по адресам www.eventmodeling.org и www.eventstorming.org.

СЛУЧАЙ ИЗ ПРАКТИКИ: МОДЕЛЬ ПОЛЬЗОВАТЕЛЯ

Ранее мы упоминали, что учетная запись и модель пользователя в нашей первой системе были привязаны друг к другу «странным правилом». Это прекрасный пример того, как инженеры и стейкхолдеры дрейфуют в противоположные стороны.

В этой системе *учетные записи* являются родительскими *рабочими пространствами*, а пользователи — *участниками* рабочих пространств. Рабочие пространства были фундаментальной единицей для применения разрешений и квот. Если пользователь *вступал* в рабочее пространство и у него еще не было *учетной записи*, то мы ассоциировали его с учетной записью, которой принадлежало это рабочее пространство.

Несмотря на бардак, все работало нормально до того дня, пока владелец продукта не попросил добавить новую функцию:

«Когда пользователь присоединяется к компании, мы хотим добавить его в несколько рабочих пространств, используемых по умолчанию, таких как рабочее пространство HR или рабочее пространство объявлений компании».

Пришлось объяснить, что такого понятия, как «компания», не существует, а присоединять пользователя к учетной записи бессмысленно. Кроме того, «компания» может поддерживать много учетных записей, которые принадлежат разным пользователям, и нового пользователя могли приглашать в любую из них.

Годы добавления костылей и обходных путей в поломанную модель наконец достигли нас, и пришлось переписать всю функцию управления пользователями как совершенно новую систему.

Цель состоит в том, чтобы иметь возможность обсуждать систему на одном языке — так все участники могли бы понять, какая ее часть самая сложная.

Мы обнаружили, что рассматривать проблемы предметной области как ката по TDD весьма полезно. Например, первым кодом, который мы написали для службы доступности, была модель партии и строки заказа. Можно относиться к этому как к воркшопу или как к всплеску в начале проекта. Как только вы сможете показать ценность моделирования, вам станет проще привести аргумент в пользу оптимизации структуры проекта.

СЛУЧАЙ ИЗ ПРАКТИКИ: ДЭВИД СЕДДОН О ТОМ, КАК ДЕЛАТЬ МАЛЕНЬКИЕ ШАГИ

Привет! Я Дэвид, один из научных редакторов этой книги. Я работал над несколькими сложными Django-монолитами и поэтому знал, какую боль Боб и Гарри обещали унять.

Первое знакомство с описанными здесь паттернами привело меня в восторг. Я уже успешно использовал некоторые из этих техник в малых проектах, но здесь был план гораздо более крупных систем с базами данных, вроде той, над которой я тогда работал. Я стал думать, как бы реализовать этот план в своей компании.

Я решил заняться проблемами кодовой базы, которые всегда меня беспокоили. Начал с ее реализации в качестве варианта использования, но столкнулся с неожиданными вопросами. Имелись места, которые я не учел при чтении, и теперь не понимал, что делать. Заключалась ли проблема в том, что мой вариант использования взаимодействовал с двумя разными агрегатами? Мог ли один вариант использования вызывать другой? И имел ли он право на жизнь в системе, которая подчинялась разным архитектурным принципам, не приводя к ужасному беспорядку?

И что же произошло с этим столь многообещающим планом? Был ли он вообще пригоден для моего случая? Достаточно ли хорошо я понял эти идеи, чтобы применить их на практике? И даже если бы это было так, то согласился бы кто-нибудь из моих коллег на такое серьезное изменение? Или же все это было лишь милыми фантазиями, не применимыми к реальной жизни?

Лишь спустя некоторое время я понял, что могу начать с малого. Не нужно было быть приверженцем чистоты или делать все правильно с первого раза: я мог экспериментировать, отыскивая то, что работало в моем случае.

И вот что я сделал. Мне удалось частично применить некоторые идеи. Я создал новую функциональность, чья бизнес-логика могла тестироваться без базы данных или имитаций. Наша команда ввела сервисный слой, чтобы помочь определить выполняемые системой задания.

Если вы начнете применять эти паттерны в своей работе, то испытаете сходные чувства. Когда хорошая теория из книги встречается с реальностью кодовой базы, эффект может быть деморализующий.

Мой совет: сосредоточьтесь на конкретной задаче и спросите себя, как вы могли бы использовать описанные здесь идеи, пускай частично и неидеально. Возможно, как и я, вы обнаружите, что первая задача будет слишком трудной; если это так, то переходите к другой. Не пытайтесь вскипятить океан и не бойтесь ошибиться. Это будет опытом познания, и вы будете уверены в том, что движетесь в том же направлении, которое другие сочли правильным.

Так что если вы тоже не знаете, с чего начать, попробуйте этот подход. Не думайте, что нужно особое разрешение, чтобы перестроить архитектуру. Поищите для начала что-нибудь крохотное. И прежде всего делайте это, чтобы решить конкретную задачу. Если вы преуспеее в ее решении, то будете знать, что у вас есть что-то правильное — и у других тоже.

Вопросы наших научных редакторов, которые мы не включили в основной текст

Вот некоторые вопросы, которые нам задали на этапе подготовки книги, но которым не нашлось места в ее финальной версии.

Должен ли я сразу все это применять? Или же можно делать это по чуть-чуть?

Вы можете принять эти техники на вооружение по кусочкам, постепенно. Если у вас уже есть система, то рекомендуем создать сервисный слой, чтобы попытаться держать оркестровку в одном месте. Как только вы это сделаете, будет гораздо проще внедрить логику в модель и протолкнуть пограничные обязанности, такие как проверка или обработка ошибок, в точки входа.

Сервисный слой пригодится, даже если у вас все еще есть большой грязный ORM Django — благодаря нему вы начнете понимать границы операций.

Извлечение вариантов использования ломает большую часть моего нынешнего кода; он слишком запутан.

Просто скопируйте и вставьте. Вполне нормально, что в краткосрочной перспективе возникнет больше работы по дублированию. Думайте об этом как о многоступенчатом процессе. Ваш код сейчас находится в плохом состоянии, поэтому копируйте его и вставляйте в новое место, а затем сделайте этот новый код чистым и аккуратным.

Таким образом, вы сможете заменить вызовы старого кода вызовами нового и наконец устранить беспорядок. Исправление больших кодовых баз — грязная работенка. Не ожидайте, что все сразу станет лучше, и не переживайте, что некоторые части вашего приложения останутся грязными.

Нужно ли мне разделять обязанности между командами и запросами? Как-то странно. Разве я не могу просто использовать репозитории?

Конечно, можете! Представленные в этой книге техники призваны облегчить вашу жизнь. Это же не какая-то аскетическая дисциплина, которой нужно себя наказывать.

В нашей первой системе, созданной как вариант использования, у нас было много объектов строителя представлений (View Builder), которые

использовали репозитории для извлечения данных, а затем выполняли некоторые преобразования, чтобы вернуть немые модели чтения. Преимущество было в том, что при возникновении проблем с производительностью можно было легко переписать View Builder для использования собственных запросов или сырого SQL.

Как должны взаимодействовать варианты использования в рамках более крупной системы? Есть ли какие-либо проблемы с вызовами последней со стороны первого?

Это может быть промежуточным шагом. Опять же в первом варианте использования у нас были обработчики, которые должны были вызывать другие обработчики. Однако это и вправду становится запутанным, и гораздо лучше перейти к использованию шины сообщений, чтобы разделить эти задачи.

Как правило, ваша система будет иметь одну-единственную реализацию шины сообщений и несколько подпредметных областей, которые сосредоточены на определенном агрегате или наборе агрегатов. Завершенный вариант использования может инициировать событие, а обработчик событий может запускаться в другой точке системы.

Можно ли считать признаком кода «с душком» случай, когда репозитории/агрегатов много, и если да, то почему?

Агрегат — это граница согласованности, поэтому если ваш вариант использования нуждается в атомарном обновлении двух агрегатов (в рамках одной транзакции), то с вашей границей согласованности явно что-то не так. В идеале следует подумать о переходе на новый агрегат, который будет охватывать все то, что вы хотите изменить в одно и то же время.

Если вы обновляете только один агрегат и используете другой (-е) для доступа только для чтения, то это нормально, хотя вместо этого можно подумать о создании модели чтения/просмотра, чтобы получать эти данные, — все становится чище, если каждый вариант использования имеет только один агрегат.

Если вам действительно нужно модифицировать два агрегата, а эти две операции не обязательно должны быть в одной транзакции/UoW, то подумайте о разделении работы между двумя разными обработчиками и использовании события предметной области для передачи

информации между ними. В статьях Вона Вернона по проектированию агрегатов можно узнать подробности¹.

А что, если у меня есть система только для чтения, но с тяжелой бизнес-логикой?

Модели представления могут иметь сложную логику. Рекомендуем разделить модели чтения и записи, поскольку они отличаются согласованностью и пропускной способностью. В принципе, можно использовать более простую логику для чтения, но такой путь не всегда правильный. В частности, модели разрешения и авторизации нередко сильно усложняют сторону чтения.

Мы писали системы, в которых для моделей представления нужно было писать масштабные юнит-тесты. В этих системах мы отделяем конструктор представления от поставщика представления, как показано на рис. Э.4.

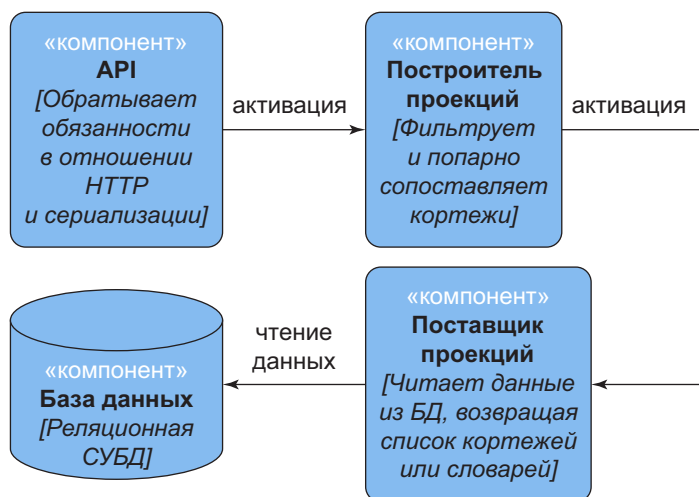


Рис. Э.4. Конструктор представлений и сборщик представлений (версию в высоком разрешении см. на сайте cosmicpython.com)

+ Это позволяет легко тестировать View Builder, передавая ему поддельные данные (например, список словарей). «Причудливое CQRS» с обработчиками событий на самом деле является способом запуска

¹ См. <https://oreil.ly/sufKE>

сложной логики представления всякий раз, когда мы выполняем операцию записи, во избежание ее запуска при чтении.

Нужно ли для всего этого создавать микросервисы?

Конечно же, нет! Эти методы появились примерно на десять лет раньше микросервисов. Агрегаты, события предметной области и инверсия зависимостей — все это способы управления сложностью в крупных системах. Так уж получается, что когда вы создали набор вариантов использования и модель для бизнес-процесса, переместить их в собственную службу будет довольно легко, хотя это и не обязательно.

Я использую Django. Могу ли я все равно это делать?

Читайте приложение Г, написанное целиком для вас.

Выстрел в ногу

Итак, мы дали вам поиграть в целую кучу новых игрушек. Вот важная информация мелким шрифтом. Гарри и Боб не рекомендуют вам копирастить их код в продакшене и перестраивать свою автоматическую торговую платформу на шаблоне «издатель/подписчик» Redis. В целях экономии бумаги и простоты мы махнули рукой на массу сложных тем. Вот что, по нашему мнению, вы должны знать, прежде чем опробовать все это на реальном коде:

Надежный обмен сообщениями очень сложно организовать.

Канал pub/sub хранилища Redis ненадежный и не должен использоваться в качестве общецелевого инструмента для обмена сообщениями. Мы выбрали его потому, что хорошо его знаем и им легко управлять. В компании MADE мы запускаем Event Store в качестве инструмента для обмена сообщениями, но у нас есть опыт работы с RabbitMQ и Amazon EventBridge.

У Тайлера Трита есть несколько отличных постов на сайте bravenewgeek.com. Почитайте хотя бы статьи «Вы не можете иметь строго одно-разовую доставку»¹ и «То, чего вы хотите, — это то, чего вы не хотите: понимание компромиссов в распределенных сообщениях»².

¹ You Cannot Have Exactly-Once Delivery. См. <https://oreil.ly/pcstD>

² What You Want Is What You Don't: Understanding Trade-Offs in Distributed Messaging. См. <https://oreil.ly/j8bmF>

Мы намеренно выбираем малые, сфокусированные транзакции, которые могут отказывать независимо.

В главе 8 мы обновляем процесс так, что отмена размещения товарной позиции заказа и повторное ее размещение теперь происходят в двух отдельных UoW. Понадобятся мониторинг, чтобы знать, когда эти транзакции отказывают, и инструменты для воспроизведения событий. Некоторые из них упрощаются с помощью журнала транзакций в качестве брокера сообщений (например, Kafka или EventStore). Вы также можете обратиться к паттерну «Исходящие транзакционные сообщения» (Outbox)¹.

Мы не затрагиваем идемпотентность.

Мы еще не задумывались над тем, что происходит, когда обработчики вызываются повторно. На практике лучше сделать обработчики идемпотентными, чтобы многократный вызов их с одним и тем же сообщением не приводил к повторным изменениям состояния. Этот ключевой прием выстраивания надежности позволяет безопасно повторять события, когда они отказывают.

По идемпотентной обработке сообщений есть много хороших материалов. Попробуйте начать со статей «Как обеспечить идемпотентность в приложении, согласованном в конечном счете с дизайном на основе DDD/SQRS»² и «(Не)надежность в обмене сообщениями»³.

События со временем должны будут изменить свою схему.

Нужно будет найти какой-то способ записи событий и обмена схемой с потребителями. Нам нравится использовать схему в формате JSON и упрощенную разметку Markdown, потому что это легкий путь, но есть и другой источник информации. Грег Янг написал целую книгу о событийно-управляемых системах: «Управление версиями в системе, управляемой событиями» (Versioning in an Event Sourced System, Leanpub).

¹ Outbox Pattern. См. <https://oreil.ly/sLfnp>

² How to Ensure Idempotency in an Eventual Consistent DDD/CQRS Application. См. <https://oreil.ly/yERzR>

³ (Un)Reliability in Messaging. См. <https://oreil.ly/Ekuhi>

Книги для обязательного прочтения

Вот еще несколько книг, которые мы хотели бы порекомендовать:

- «Чистые архитектуры на Python» Леонардо Джордани (Clean Architectures in Python, Leonardo Giordani, Leanpub), изданная в 2019 году, — одна из немногих предварительных книг по архитектуре приложений на Python.
- «Шаблоны интеграции корпоративных приложений» Грегора Хопы и Бобби Вульфа (Enterprise Integration Patterns, Gregor Hohpe, Bobby Woolf, AddisonWesley Professional). Хорошее вступление в тему паттернов обмена сообщениями.
- «От монолита к микросервисам» Сэма Ньюмена (Monolith to Microservices, Sam Newman, O'Reilly) и первая книга Ньюмена «Создание микросервисов»¹ (Building Microservices, O'Reilly). Паттерн «Душитель» упоминается как предпочтительный наряду со многими другими. Прочтите их, если планируете переходить на микросервисы. А еще эти книги неплохо описывают паттерны интеграции и соображения по интеграции на основе асинхронных сообщений.

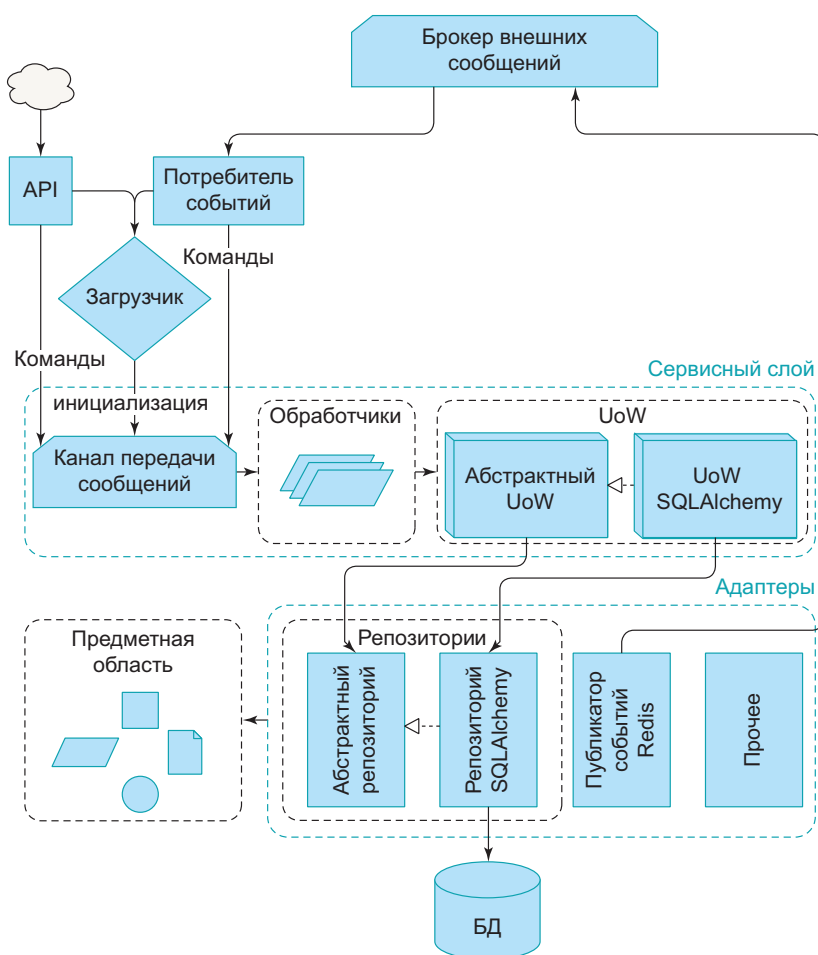
Выводы

Ух! Советов и предложений по дальнейшему изучению получилось немало, и мы надеемся, что вас это не спугнет. Цель этой книги — дать достаточно знаний и представлений, чтобы вы могли применять описанные здесь техники. Будем рады услышать о том, как у вас идут дела и с какими проблемами вы сталкиваетесь в собственных системах. Мы на связи по адресу www.cosmicpython.com.

¹ Ньюмен С. Создание микросервисов. СПб.: Питер, 2016. 304 с.

Сводная диаграмма и таблица

Вот как выглядит наша архитектура к концу книги:



В табл. А.1 кратко описывается каждый паттерн и его поведение.

Таблица А.1. Компоненты архитектуры и их поведение

Слой	Компонент	Описание
Предметная область <i>Определяет бизнес-логику</i>	Сущность	Объект предметной области, у которого со временем могут меняться атрибуты, но при этом он все же остается узнаваемым
	Объект-значение	Немутируемый объект предметной области, атрибуты которого полностью его определяют. Он взаимозаменяем с другими идентичными объектами
	Агрегат	Кластер связанных объектов, который мы рассматриваем как единый юнит для изменения данных. Определяет и применяет границу согласованности
	Событие	Представляет нечто, что произошло
	Команда	Представляет задание, которое система должна выполнить
Сервисный слой <i>Определяет задания, которые система должна выполнять, и оркестрирует работу разных компонентов</i>	Обработчик	Принимает команду или событие и выполняет то, что должно произойти
	UoW	Абстракция вокруг целостности данных. Каждый UoW представляет атомарное обновление. Делает доступными репозитории. Отслеживает новые события в извлеченных агрегатах
	Шина (внутренних сообщений)	Обрабатывает команды и события, направляя их в соответствующий обработчик
Адаптеры (вторичные) <i>Конкретные реализации интерфейса, который направлен из нашей системы во внешний мир (I/O)</i>	Репозиторий	Абстракция издателя событий вокруг системы постоянного хранения данных. Каждый агрегат имеет собственный репозиторий
	Издатель событий	Выталкивает события в шину внешних сообщений
Точки входа (первичные адаптеры) <i>Транслируют внешние входные данные в вызовы в сервисном слое</i>	Веб	Получает веб-запросы и преобразует их в команды, передавая их в шину внутренних сообщений
	Потребитель событий	Считывает события из шины внешних сообщений и транслирует их в команды, передавая их в шину внутренних сообщений
<i>Не используется (N/A)</i>	Шина внешних сообщений (брокер сообщений)	Часть инфраструктуры, которую разные службы используют для взаимодействия с помощью событий

Шаблонная структура проекта

В главе 4 мы перешли от простого хранения всех элементов в одной папке к более структурированному дереву и подумали, что было бы интересно рассмотреть его в деталях.



Код для этого приложения находится в ветке `appendix_project_structure` на GitHub¹:

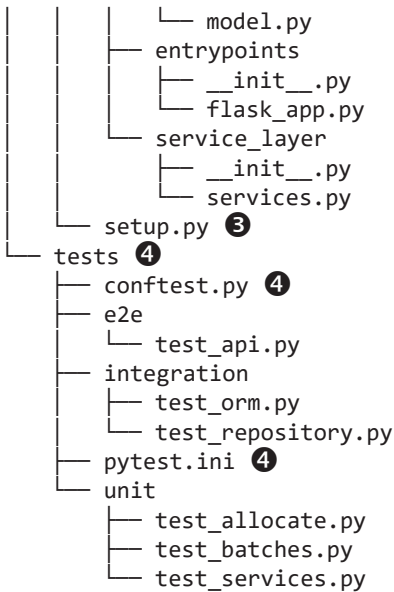
```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout appendix_project_structure
```

Базовая структура папок выглядит следующим образом:

Дерево проекта

```
.
├── Dockerfile ❶
├── Makefile ❷
├── README.md
├── docker-compose.yml ❶
├── license.txt
├── mypy.ini
├── requirements.txt
├── src ❸
│   ├── allocation
│   │   ├── __init__.py
│   │   ├── adapters
│   │   │   ├── __init__.py
│   │   │   ├── orm.py
│   │   │   └── repository.py
│   ├── config.py
│   ├── domain
│   └── __init__.py
```

¹ См. <https://oreil.ly/1rDRC>



❶ Файлы `docker-compose.yml` и `Dockerfile` — основные элементы конфигурации контейнеров, на базе которых работает наше приложение. А еще они запускают тесты (для непрерывной интеграции). В более сложном проекте может быть сразу несколько файлов `Dockerfile`, хотя мы пришли к тому, что лучше снизить число образов¹.

❷ `Makefile` предоставляет точку входа для всех типичных команд, которые понадобится выполнить разработчику (или серверу непрерывной интеграции) в рамках стандартного рабочего процесса: `make build`, `make test` и т. д.². Но использовать его необязательно. Вы можете просто применить `docker-compose` и `pytest` напрямую, но если ничего другого нет, то неплохо бы иметь все «популярные команды» где-то в списке, и в отличие от документации, `Makefile` — это код, поэтому он не так быстро устаревает.

¹ Иногда действительно лучше разделить образы для производства и тестирования, но мы все же считаем, что попытки разграничения образов для разных типов кода приложения (например, веб-API и клиент канала «издатель/подписчик») лишь добавляют проблем; слишком высока цена с точки зрения сложности и времени повторной сборки/непрерывной интеграции. Но вы можете с нами не согласиться.

² Чисто питоновской альтернативой инструкциям `Makefile` является `Invoke` (<http://www.pyinvoke.org/>). Советуем попробовать поработать с ним, если кто-нибудь в вашей команде знает Python (или по крайней мере знает его лучше, чем Bash).

③ Весь код нашего приложения, включая модель предметной области, приложение Flask и инфраструктурный код, располагается в питоновском пакете внутри папки `src`¹, который мы устанавливаем с помощью команды `pip install -e` и файла `setup.py`. Это облегчает импорт. В нашем примере структура внутри этого модуля полностью плоская, но в более сложном проекте можно ожидать роста иерархии папок, которая включает `domain_model/`, `infrastructure/`, `services/` и `api/`.

④ Тесты располагаются в отдельной папке. Вложенные папки различают разные типы тестов и позволяют выполнять их отдельно. Мы можем держать общие оснастки (`conftest.py`) в главной папке тестов и вкладывать более конкретные, если захотим. Здесь также хранится `pytest.ini`.



Документация `pytest`² действительно хороша в том, что касается компоновки тестов и обеспечения импортопригодности.

Рассмотрим некоторые из этих файлов и концепций подробнее.

Переменные окружения, 12 факторов. Конфигурирование внутри и снаружи контейнеров

Базовая задача, которую мы пытаемся решить, связана с настройками конфигурации, которые должны быть разными для следующих целей:

- выполнения кода или тестов непосредственно на вашей рабочей машине, возможно, связываясь с соотнесенными портами из контейнеров Docker;
- выполнения на самих контейнерах, с «реальными» портами и сетевыми именами;
- разные контейнерные среды (для разработки, подготовки, производства и т. д.).

¹ В статье Хайнека Шлавака «Тестирование и упаковка» (Testing and Packaging) можно больше узнать о папках `src`. См. <https://hynek.me/articles/testing-packaging>

² См. <https://oreil.ly/QVb9Q>

Эта задача решается конфигурированием с помощью переменных окружения, как это предложено в 12-факторном манифесте¹. Но если говорить конкретно, то как реализовать это в коде и контейнерах?

Config.py

Всякий раз, когда коду приложения требуется доступ к какой-либо конфигурации, он получает его из файла `config.py`. Вот несколько примеров из нашего приложения.

Пример конфигурационных функций (`src/allocation/config.py`)

```
import os

def get_postgres_uri(): ❶
    host = os.environ.get('DB_HOST', 'localhost') ❷
    port = 54321 if host == 'localhost' else 5432
    password = os.environ.get('DB_PASSWORD', 'abc123')
    user, db_name = 'allocation', 'allocation'
    return f"postgresql://{user}:{password}@{host}:{port}/{db_name}"

def get_api_url():
    host = os.environ.get('API_HOST', 'localhost')
    port = 5005 if host == 'localhost' else 80
    return f"http://{host}:{port}"
```

❶ Используем функции для получения текущей конфигурации, а не константы, имеющиеся во время импорта, так как это позволяет клиентскому коду при необходимости изменять `os.environ`.

❷ `config.py` также определяет некоторые настройки по умолчанию, предназначенные для работы при выполнении кода с локальной машины разработчика².

¹ См. <https://12factor.net/config>

² Это настройка локальной разработки, которая «просто работает» (насколько возможно). Вместо нее вы можете предпочесть жесткий сбой при отсутствующих переменных среды, в особенности если какие-либо из значений по умолчанию будут небезопасны для использования в продакшене.

Стоит попробовать элегантный питоновский пакет под названием *environ-config*¹, если вы устали вручную прописывать собственные функции конфигурации на основе среды.



Не позволяйте этому конфигурационному модулю превратиться в свалку, куда сначала скидывается, а затем везде импортируется все, что хотя бы отдаленно напоминает конфигурацию. Нужно, чтобы все было немутируемым, а изменения следует вносить только с помощью переменных окружения. Если вы решите использовать сценарий начальной загрузки, то можете сделать его единственным местом (кроме тестов), куда импортируется конфигурация.

Docker-Compose и конфигурация контейнеров

Мы используем легковесный инструмент оркестровки контейнеров Docker под названием *docker-compose*. Его основная конфигурация осуществляется посредством файла YAML (увы)².

Конфигурационный файл *docker-compose* (*docker-compose.yml*)

```
version: "3"
services:
  app: ❶
    build:
      context: .
      dockerfile: Dockerfile
    depends_on:
      - postgres
    environment: ❸
      - DB_HOST=postgres ❹
      - DB_PASSWORD=abc123
      - API_HOST=app
      - PYTHONDONTWRITEBYTECODE=1 ❺
    volumes: ❻
      - ./src:/src
      - ./tests:/tests
    ports:
```

¹ См. <https://github.com/hynek/envron-config>

² Гарри уже устал от YAML. Эта разметка повсюду, и, тем не менее, он все никак не может запомнить ее синтаксис и правила расстановки отступов.


```
- "5005:80" ⑦

postgres:
  image: postgres:9.6 ②
  environment:
    - POSTGRES_USER=allocation
    - POSTGRES_PASSWORD=abc123
  ports:
    - "54321:5432"
```

❶ В файле `docker-compose` мы определяем разные *службы* (контейнеры), которые нужны для нашего приложения. Обычно один главный образ содержит весь код, и мы можем использовать его для выполнения нашего API, тестов или любой другой службы, которой нужен доступ к модели предметной области.

❷ Вероятно, у вас будут и другие инфраструктурные службы, включая базу данных. В производстве вместо контейнера вы можете использовать облачный провайдер, но `docker-compose` позволяет создавать похожую службу для разработки или непрерывной интеграции.

❸ Раздел `environment` позволяет задавать переменные среды для ваших контейнеров, сетевых имен и портов, как видно внутри кластера Docker. Если контейнеров так много, что информация о них начинает дублироваться в этих разделах, то можно использовать `environment_file`. Мы обычно называем наш раздел `container.env`.

❹ Внутри кластера инструмент `docker-compose` настраивает сетевое взаимодействие так, чтобы контейнеры были доступны друг другу через сетевые имена, созданные по имени службы.

❺ Совет профессионалов: если вы монтируете тома для совместного использования исходных папок между локальной машиной разработчика и контейнером, то переменная среды `PYTHONDONTWRITEBYTECODE` дает команду Python не писать файлы `.pyc`, что избавит вас от миллионов корневых файлов, разбросанных по всей локальной файловой системе, которые все время приходится удалять. К тому же они становятся причиной странных ошибок компилятора Python.

❻ Благодаря монтированию исходного и тестового кода в виде томов, `volume`, не нужно выполнять повторную сборку контейнеров всякий раз, когда в код вносятся изменения.

❷ Раздел `ports` позволяет выставлять порты из контейнеров во внешний мир¹ — они соответствуют портам по умолчанию, которые мы задаем в `config.py`.

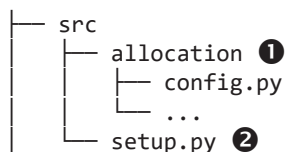


Внутри Docker другие контейнеры доступны через сетевые имена, созданные на основе имени службы. Вне Docker они доступны на `localhost`, в порте, определенном в разделе `ports`.

Установка исходного кода в виде пакета

Весь код приложения (на самом деле весь код, кроме тестов) располагается в папке `src`.

Папка `src`



❶ Вложенные папки определяют имена модулей верхнего уровня. При необходимости их может быть несколько.

❷ И еще `setup.py` — это файл, который делает модуль пригодным для установки через менеджер пакетов `pip`, как показано далее.

Модули, пригодные для установки через `pip` в три строки (`src/setup.py`)

```
from setuptools import setup
```

```
setup(
    name='allocation',
    version='0.1',
    packages=['allocation'],
)
```

И все. `packages=` задает имена вложенных папок, которые вы хотите устанавливать в качестве модулей верхнего уровня. Элемент `name` носит кос-

¹ На сервере непрерывной интеграции, скорее всего, не получится надежно выставлять произвольные порты, и это делается только для удобства локальной разработки. Можно попытаться сделать эти соотнесения портов необязательными (например, с помощью `docker-compose.override.yml`).

метический характер, но обязателен. Для пакета, который на самом деле никогда не попадет в PyPI, это будет прекрасно¹.

Файл Dockerfile

Файлы Dockerfile специфичны для каждого конкретного проекта, но вот несколько ключевых этапов, которые, скорее всего, будут общими:

Наш файл Dockerfile (Dockerfile)

```
FROM python:3.8-alpine
```

```
❶  
RUN apk add --no-cache --virtual .build-deps gcc postgresqldev musl-  
dev python3-dev  
RUN apk add libpq
```

```
❷  
COPY requirements.txt /tmp/  
RUN pip install -r /tmp/requirements.txt  
  
RUN apk del --no-cache .build-deps
```

```
❸  
RUN mkdir -p /src  
COPY src/ /src/  
RUN pip install -e /src  
COPY tests/ /tests/
```

```
❹  
WORKDIR /src  
ENV FLASK_APP=allocation/entrypoints/flask_app.py  
FLASK_DEBUG=1 PYTHONUNBUFFERED=1  
CMD flask run --host=0.0.0.0 --port=80
```

❶ Установка зависимостей системного уровня.

❷ Установка питоновских зависимостей (возможно, вы захотите отделить свои зависимости среды разработки от зависимостей производственной среды; ради упрощения кода мы здесь этого не сделали).

¹ Дополнительные советы по `setup.py` см. в статье Хайнека о сборке пакетов по ссылке: <https://oreil.ly/KMWDz>.

- ❸ Копирование и установка исходного кода.
- ❹ Дополнительно можно настроить команду запуска по умолчанию (ее придется переопределять из командной строки).



Следует отметить, что мы устанавливаем компоненты в порядке частотности, точнее, того, как часто они могут меняться. Это позволяет максимизировать вторичное использование кэша сборки Docker. Не можем выразить словами всю досаду и разочарование, которые лежат в основе этого урока. В отношении этого и многих других советов по улучшению питоновского файла Dockerfile ознакомьтесь со статьей «Сборка пакетов, готовых к эксплуатации в производственной среде Docker»¹.

Тесты

Тесты хранятся вместе со всем остальным, как показано здесь:

Дерево папок с тестами

```
├─ tests
│   ├── conftest.py
│   ├── e2e
│   │   └── test_api.py
│   ├── integration
│   │   ├── test_orm.py
│   │   └── test_repository.py
│   ├── pytest.ini
│   └── unit
│       ├── test_allocate.py
│       ├── test_batches.py
│       └── test_services.py
```

Ничего выдающегося здесь нет, просто разделены некоторые типы тестов, которые вы, может быть, захотите выполнять отдельно, и несколько файлов для совместных фикстур, конфигураций и пр.

В тестовых папках нет папки `src` или файла `setup.py`, потому что обычно делать тесты пригодными для установки через `pip` не приходилось. Но при трудностях с путями импорта такой способ может помочь.

¹ Production-Ready Docker Packaging. См. <https://pythonspeed.com/docker>

Выводы

Вот наши базовые строительные блоки:

- код в папке `src`, пригодный для установки через `pip` с использованием файла `setup.py`;
- конфигурация `Docker` для раскручивания локального кластера, который максимально отражает производственную среду;
- конфигурация посредством переменных окружения, централизованных в питоновском файле `config.py`, с настройками по умолчанию, позволяющими компонентам работать вне контейнеров;
- файл `Makefile` для полезных команд, `гм`, командной строки.

Сомневаемся, что кто-то в итоге придет точно к таким же решениям, но надеемся, что наша система вдохновит вас.

Замена инфраструктуры: делаем все с помощью CSV

Это приложение показывает преимущества от использования паттернов «Репозиторий», UoW и «Сервисный слой». По замыслу, оно вытекает из главы 6.

Пока мы заканчиваем с нашим API Flask и готовим его к релизу, приходят стейкхолдеры и говорят, что не готовы использовать API. Они спрашивают, можно ли создать что-то, что читает только партии товаров и заказы из пары CSV-файлов и выводит третий CSV-файл с размещениями.

Обычно после такого разработчики начинают чертыхаться и плевать. Но только не мы! Мы позаботились о том, чтобы инфраструктурные обязанности были четко отделены от модели предметной области и сервисного слоя. Переход на CSV-файлы всего лишь сведется к написанию нескольких новых классов `Repository` и `UnitOfWork`. Тогда мы сможем вторично использовать всю логику из слоя предметной области и сервисного слоя.

Вот сквозной тест, который покажет входной и выходной потоки CSV-файлов:

Первый тест CSV (`tests/e2e/test_csv.py`)

```
def test_cli_app_reads_csvs_with_batches_and_orders_and_outputs_
allocations(
    make_csv
):
    sku1, sku2 = random_ref('s1'), random_ref('s2')
    batch1, batch2, batch3 = random_ref('b1'), random_ref('b2'),
    random_ref('b3')
    order_ref = random_ref('o')
    make_csv('batches.csv', [
        ['ref', 'sku', 'qty', 'eta'],
        [batch1, sku1, 100, ''],
        [batch2, sku2, 100, '2011-01-01'],
```

```

    [batch3, sku2, 100, '2011-01-02'],
])
orders_csv = make_csv('orders.csv', [
    ['orderid', 'sku', 'qty'],
    [order_ref, sku1, 3],
    [order_ref, sku2, 12],
])

run_cli_script(orders_csv.parent)

expected_output_csv = orders_csv.parent / 'allocations.csv'
with open(expected_output_csv) as f:
    rows = list(csv.reader(f))
assert rows == [
    ['orderid', 'sku', 'qty', 'batchref'],
    [order_ref, sku1, '3', batch1],
    [order_ref, sku2, '12', batch2],
]
```

Погрузившись в реализацию и отбросив мысли о репозиториях и всем в этом роде, возможно, вы начнете с чего-то такого:

Первая проба компонента, который читает/пишет CSV-файлы (src/bin/allocate-from-csv)

```

#!/usr/bin/env python
import csv
import csv import sys
from datetime import datetime
from pathlib import Path

from allocation import model

def load_batches(batches_path):
    batches = []
    with batches_path.open() as inf:
        reader = csv.DictReader(inf)
        for row in reader:
            if row['eta']:
                eta = datetime.strptime(row['eta'], '%Y-%m-%d').date()
            else:
                eta = None
            batches.append(model.Batch(
                ref=row['ref'],
                sku=row['sku'],
                qty=int(row['qty']),
                eta=eta
            ))
```

```

        ))
    return batches

def main(folder):
    batches_path = Path(folder) / 'batches.csv'
    orders_path = Path(folder) / 'orders.csv'
    allocations_path = Path(folder) / 'allocations.csv'

    batches = load_batches(batches_path)

    with orders_path.open() as inf, allocations_path.open('w') as outf:
        reader = csv.DictReader(inf)
        writer = csv.writer(outf)
        writer.writerow(['orderid', 'sku', 'batchref'])
        for row in reader:
            orderid, sku = row['orderid'], row['sku']
            qty = int(row['qty'])
            line = model.OrderLine(orderid, sku, qty)
            batchref = model.allocate(line, batches)
            writer.writerow([line.orderid, line.sku, batchref])

if __name__ == '__main__':
    main(sys.argv[1])

```

Выглядит не так уж и плохо! И мы вторично используем объекты модели предметной области и службу предметной области.

Но это не будет работать. Существующие размещения также должны быть частью постоянного CSV-хранилища. Можно написать второй тест, который улучшит ситуацию.

И еще один, с существующими размещениями (tests/e2e/test_csv.py)

```

def test_cli_app_also_reads_existing_allocations_and_can_append_to_them(
    make_csv
):
    sku = random_ref('s')
    batch1, batch2 = random_ref('b1'), random_ref('b2')
    old_order, new_order = random_ref('o1'), random_ref('o2')
    make_csv('batches.csv', [
        ['ref', 'sku', 'qty', 'eta'],
        [batch1, sku, 10, '2011-01-01'],
        [batch2, sku, 10, '2011-01-02'],
    ])
    make_csv('allocations.csv', [
        ['orderid', 'sku', 'qty', 'batchref'],

```



```

        [old_order, sku, 10, batch1],
    ])
    orders_csv = make_csv('orders.csv', [
        ['orderid', 'sku', 'qty'],
        [new_order, sku, 7],
    ])

    run_cli_script(orders_csv.parent)

    expected_output_csv = orders_csv.parent / 'allocations.csv'
    with open(expected_output_csv) as f:
        rows = list(csv.reader(f))
    assert rows == [
        ['orderid', 'sku', 'qty', 'batchref'],
        [old_order, sku, '10', batch1],
        [new_order, sku, '7', batch2],
    ]

```

Можно продолжать и дальше добавлять в функцию `load_batches` лишние строки, а также придумать какой-то способ отслеживания и сохранения новых размещений — но у нас уже есть модель, которая все это делает! И это паттерны «Репозиторий» и UoW.

Требуется сделать всего одну вещь — реализовать те же самые абстракции, но в основе которых лежат CSV-файлы, а не база данных. И как вы увидите, сделать это и правда несложно.

Реализация паттернов «Репозиторий» и UoW для CSV-файлов

Вот как может выглядеть репозиторий на основе CSV-файлов. Он абстрагирует всю логику чтения CSV-файлов с диска, учитывая то, что он должен читать два разных CSV-файла (один для партий товара и другой для размещений). Получаем уже знакомый API `.list()`, который позволяет создавать коллекцию объектов предметной области прямо в памяти.

Репозиторий, использующий CSV-файлы в качестве механизма хранения (`src/allocation/service_layer/csv_uow.py`)

```

class CsvRepository(repository.AbstractRepository):

    def __init__(self, folder):
        self._batches_path = Path(folder) / 'batches.csv'

```

```
self._allocations_path = Path(folder) / 'allocations.csv'
self._batches = {} # тип: Dict[str, model.Batch]
self._load()

def get(self, reference):
    return self._batches.get(reference)

def add(self, batch):
    self._batches[batch.reference] = batch

def _load(self):
    with self._batches_path.open() as f:
        reader = csv.DictReader(f)
        for row in reader:
            ref, sku = row['ref'], row['sku']
            qty = int(row['qty'])
            if row['eta']:
                eta = datetime.strptime(row['eta'],
                                         '%Y-%m-%d').date()
            else:
                eta = None
            self._batches[ref] = model.Batch(
                ref=ref, sku=sku, qty=qty, eta=eta
            )
    if self._allocations_path.exists() is False:
        return
    with self._allocations_path.open() as f:
        reader = csv.DictReader(f)
        for row in reader:
            batchref, orderid, sku = row['batchref'],
            row['orderid'], row['sku']
            qty = int(row['qty'])
            line = model.OrderLine(orderid, sku, qty)
            batch = self._batches[batchref]
            batch._allocations.add(line)

def list(self):
    return list(self._batches.values())
```

А вот как будет выглядеть паттерн UoW для CSV-файлов.

UoW для CSV-файлов: фиксация = `csv.writer(src/allocation/service_layer/csv_uow.py)`

```
class CsvUnitOfWork(unit_of_work.AbstractUnitOfWork):

    def __init__(self, folder):
        self.batches = CsvRepository(folder)

    def commit(self):
        with self.batches._allocations_path.open('w') as f:
            writer = csv.writer(f)
            writer.writerow(['orderid', 'sku', 'qty', 'batchref'])
            for batch in self.batches.list():
                for line in batch._allocations:
                    writer.writerow(
                        [line.orderid, line.sku, line.qty,
                         batch.reference]
                    )

    def rollback(self):
        pass
```

И как только мы это сделаем, CLI-приложение для чтения и записи партий и размещений в CSV-файлы станет таким, каким оно должно быть, — немного кода для чтения товарных позиций заказа и еще немного для вызова существующего сервисного слоя.

Размещение с помощью CSV-файлов в девяти строках (`src/bin/allocate-from-csv`)

```
def main(folder):
    orders_path = Path(folder) / 'orders.csv'
    uow = csv_uow.CsvUnitOfWork(folder)
    with orders_path.open() as f:
        reader = csv.DictReader(f)
        for row in reader:
            orderid, sku = row['orderid'], row['sku']
            qty = int(row['qty'])
            services.allocate(orderid, sku, qty, uow)
```

Та-да! Ну? Вы впечатлены или как?

С любовью,
Боб и Гарри

Паттерны «Репозиторий» и UoW с Django

Предположим, что вы хотите использовать Django вместо SQLAlchemy и Flask. Как все будет выглядеть? Первое, что нужно сделать, — это выбрать место установки. Мы помещаем его в отдельный пакет рядом с основным кодом размещения заказов:

```
├── src
│   ├── allocation
│   │   ├── __init__.py
│   │   └── adapters
│   │       └── __init__.py
│   └── ...
├── ...
├── djangoproject
│   ├── alloc
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── migrations
│   │   │   └── 0001_initial.py
│   │   └── __init__.py
│   ├── models.py
│   ├── views.py
│   ├── django_project
│   │   ├── __init__.py
│   │   ├── settings.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   └── manage.py
├── setup.py
├── tests
│   ├── conftest.py
│   ├── e2e
│   │   └── test_api.py
│   ├── integration
│   └── test_repository.py
└── ...
```



Код для этого приложения находится в ветке `appendix_django` на GitHub¹:

```
git clone https://github.com/cosmicpython/code.git
cd code
git checkout appendix_django
```

Паттерн «Репозиторий» с Django

Мы использовали плагин под названием `pytest-django`² для управления тестовой базой данных.

Переписывание первого теста репозитория свелось к минимуму — просто замена сырого SQL вызовом ORM/QuerySet Django:

Первый тест репозитория после адаптации (`tests/integration/test_repository.py`)

```
from djangoproject.alloc import models as django_models

@pytest.mark.django_db
def test_repository_can_save_a_batch():
    batch = model.Batch("batch1", "RUSTY-SOAPDISH", 100,
                       eta=date(2011, 12, 25))

    repo = repository.DjangoRepository()
    repo.add(batch)

    [saved_batch] = django_models.Batch.objects.all()
    assert saved_batch.reference == batch.reference
    assert saved_batch.sku == batch.sku
    assert saved_batch.qty == batch._purchased_quantity
    assert saved_batch.eta == batch.eta
```

Второй тест более сложный, так как имеет дело с размещениями, но при этом все еще состоит из знакомого кода Django:

Сложный второй тест репозитория (`tests/integration/test_repository.py`)

```
@pytest.mark.django_db
def test_repository_can_retrieve_a_batch_with_allocations():
    sku = "PONY-STATUE"
```

¹ См. <https://oreil.ly/A-I76>

² См. <https://github.com/pytest-dev/pytest-django>

```

d_line = django_models.OrderLine.objects.create(orderid="order1",
        sku=sku, qty=12)
d_b1 = django_models.Batch.objects.create(
    reference="batch1", sku=sku, qty=100, eta=None
)
d_b2 = django_models.Batch.objects.create(
    reference="batch2", sku=sku, qty=100, eta=None
)
django_models.Allocation.objects.create(line=d_line, batch=d_batch1)

repo = repository.DjangoRepository()
retrieved = repo.get("batch1")

expected = model.Batch("batch1", sku, 100, eta=None)
assert retrieved == expected # Batch.__eq__ only compares
                             reference
assert retrieved.sku == expected.sku
assert retrieved._purchased_quantity == expected._purchased_
                             quantity
assert retrieved._allocations == {
    model.OrderLine("order1", sku, 12),
}

```

Вот как в итоге выглядит фактический репозиторий:

Репозиторий Django (src/allocation/adapters/repository.py)

```

class DjangoRepository(AbstractRepository):

    def add(self, batch):
        super().add(batch)
        self.update(batch)

    def update(self, batch):
        django_models.Batch.update_from_domain(batch)

    def _get(self, reference):
        return django_models.Batch.objects.filter(
            reference=reference
        ).first().to_domain()

    def list(self):
        return [b.to_domain() for b in django_models.Batch.objects.all()]

```

Как видите, реализация основана на моделях Django, имеющих некоторые настраиваемые методы для трансляции в модель предметной области и из нее¹.

Специализированные методы в классах ORM Django для трансляции в модель предметной области и из нее

Эти специализированные методы выглядят примерно так:

ORM Django со специализированными методами конверсии модели предметной области (src/djangoproject/alloc/models.py)

```
from django.db import models
from allocation.domain import model as domain_model

class Batch(models.Model):

    reference = models.CharField(max_length=255)
    sku = models.CharField(max_length=255)
    qty = models.IntegerField()
    eta = models.DateField(blank=True, null=True)

    @staticmethod
    def update_from_domain(batch: domain_model.Batch):
        try:
            b = Batch.objects.get(reference=batch.reference) ❶
        except Batch.DoesNotExist:
            b = Batch(reference=batch.reference) ❶
        b.sku = batch.sku
        b.qty = batch._purchased_quantity
        b.eta = batch.eta ❷
        b.save()
        b.allocation_set.set(
            Allocation.from_domain(1, b) ❸
            for 1 in batch._allocations
        )

    def to_domain(self) -> domain_model.Batch:
        b = domain_model.Batch(
            ref=self.reference, sku=self.sku, qty=self.qty, eta=self.eta
```

¹ Разработчики из проекта DRY-Python создали инструмент mappers, который, как нам кажется, способен минимизировать стереотипный код для подобных решений. См. <https://mappers.readthedocs.io/en/latest>

```

    )
    b._allocations = set(
        a.line.to_domain()
        for a in self.allocation_set.all()
    )
    return b

```

```

class OrderLine(models.Model):
    #...

```

❶ Для объектов-значений `objects.get_or_create` может работать, но для сущностей понадобится явный блок `try-get/except` для обработки вставок новых строк в данные или их обновление, если они существуют (`upsert`)¹.

❷ Мы показали здесь самый сложный пример. Если вы все же решите его повторить, то имейте в виду, что будет много стереотипного кода! К счастью, он несложный.

❸ Отношения также нуждаются в осторожном индивидуальном подходе.



Как и в главе 2, мы используем инверсию зависимостей. Объектно-реляционное отображение (Django) зависит от модели, а не наоборот.

Паттерн UoW с Django

Тесты не слишком меняются.

Адаптированные тесты UoW (`tests/integration/test_uow.py`)

```

def insert_batch(ref, sku, qty, eta): ❶
    django_models.Batch.objects.create(reference=ref, sku=sku,
        qty=qty, eta=eta)

def get_allocated_batch_ref(orderid, sku): ❶
    return django_models.Allocation.objects.get(
        line__orderid=orderid, line__sku=sku
    ).batch.reference

@pytest.mark.django_db(transaction=True)
def test_uow_can_retrieve_a_batch_and_allocate_to_it():

```

¹ @mr-bo-jangles предположил, что вы можете использовать `update_or_create` (<https://oreil.ly/HTq1r>), но это выходит за рамки нашего кунг-фу Django.


```

insert_batch('batch1', 'HIPSTER-WORKBENCH', 100, None)

uow = unit_of_work.DjangoUnitOfWork()
with uow:
    batch = uow.batches.get(reference='batch1')
    line = model.OrderLine('o1', 'HIPSTER-WORKBENCH', 10)
    batch.allocate(line)
    uow.commit()

batchref = get_allocated_batch_ref('o1', 'HIPSTER-WORKBENCH')
assert batchref == 'batch1'

@pytest.mark.django_db(transaction=True) ❷
def test_rolls_back_uncommitted_work_by_default():
    ...

@pytest.mark.django_db(transaction=True) ❷
def test_rolls_back_on_error():
    ...

```

❶ Поскольку в этих тестах было мало вспомогательных функций, их основная часть практически такая же, как и в SQLAlchemy.

❷ `mark.django_db(transaction=True)` из `pytest-django` требуется для тестирования наших форм поведения, связанных с транзакциями и откатом.

Реализация оказалась довольно простой, хотя нам потребовалось несколько попыток, чтобы выяснить, какой именно вызов Django будет работать.

UoW после адаптации для Django (`src/allocation/service_layer/unit_of_work.py`)

```

class DjangoUnitOfWork(AbstractUnitOfWork):

    def __enter__(self):
        self.batches = repository.DjangoRepository()
        transaction.set_autocommit(False) ❶
        return super().__enter__()

    def __exit__(self, *args):
        super().__exit__(*args)
        transaction.set_autocommit(True)

    def commit(self):
        for batch in self.batches.seen: ❸
            self.batches.update(batch) ❸

```

```
transaction.commit() ❷
```

```
def rollback(self):
    transaction.rollback() ❷
```

❶ Метод `set_autocommit(False)` был лучшим способом дать команду Django немедленно прекратить автоматическую фиксацию каждой операции ORM и начать транзакцию.

❷ Затем мы используем явный откат и фиксации.

❸ Единственная трудность: поскольку, в отличие от SQLAlchemy, мы не оборудуем сами экземпляры модели предметной области, команда `commit()` должна явно пройти через все объекты, которые были затронуты каждым репозиторием, и вручную обновить их обратно в ORM.

API: представления Django — это адаптеры

Django-файл `views.py` в конечном итоге почти идентичен старому `flask_app.py`, так как из нашей архитектуры следует, что мы имеем дело с очень тонкой оболочкой вокруг сервисного слоя (который, кстати, совсем не изменился):

Приложение Flask → представления Django (`src/djangoproject/alloc/views.py`)

```
os.environ['DJANGO_SETTINGS_MODULE'] = 'djangoproject.django_
                                     project.settings'

django.setup()
```

```
@csrf_exempt
def add_batch(request):
    data = json.loads(request.body)
    eta = data['eta']
    if eta is not None:
        eta = datetime.fromisoformat(eta).date()
    services.add_batch(
        data['ref'], data['sku'], data['qty'], eta,
        unit_of_work.DjangoUnitOfWork(),
    )
    return HttpResponse('OK', status=201)
```

```
@csrf_exempt
def allocate(request):
    data = json.loads(request.body)
```

```
try:
    batchref = services.allocate(
        data['orderid'],
        data['sku'],
        data['qty'],
        unit_of_work.DjangoUnitOfWork(),
    )
except (model.OutOfStock, services.InvalidSku) as e:
    return JsonResponse({'message': str(e)}, status=400)

return JsonResponse({'batchref': batchref}, status=201)
```

Почему все так сложно?

О'кей, это работает, но, похоже, и вправду требует бóльших усилий по сравнению с Flask/SQLAlchemy. Почему так?

Основная причина низкоуровневая — ORM Django работает по-другому. У нас нет эквивалента классического ORM SQLAlchemy, поэтому наши `ActiveRecord` и модель предметной области не могут быть одним и тем же объектом. Вместо этого требуется создавать слой ручной трансляции позади репозитория. И это дополнительная работа (зато после этого техническое сопровождение будет относительно простым).

Поскольку Django тесно связан с базой данных, следует использовать `pytest-django` и тщательно продумывать тестовые базы данных с самой первой строки кода, чего не нужно было делать, когда мы начинали с чистой модели предметной области.

Высокоуровневая причина всей прелести Django — он разработан с учетом упрощенного создания приложений CRUD с минимальным набором шаблонов. Но все советы в этой книге крутятся вокруг того, что делать, если ваше приложение перестало быть простым приложением CRUD.

В такой ситуации Django больше мешает, чем помогает. Тот же самый администратор Django, изначально такой полезный, становится опасным, если ваше приложение строится на основе сложного набора правил и создания модели вокруг рабочего потока по изменению состояния. Администратор Django все это обходит.

Что делать, если у вас уже есть Django

Итак, что вы должны делать, если хотите применить какие-то паттерны к приложению Django? Мы бы сказали следующее:

- Адаптация паттернов «Репозиторий» и UoW займет много времени. Что они дадут в краткосрочной перспективе, так это более быстрые юнит-тесты. Поэтому оцените, разумно ли это в вашем случае. В долгосрочной перспективе они отвязывают ваше приложение от Django и базы данных, поэтому если в будущем вы захотите совершить миграцию любого из них, использовать «Репозиторий» и UoW — идея вполне неплохая.
- Паттерн «Сервисный слой» подходит, если в работе у вас много дублирования. `views.py` будет хорошим способом обдумать варианты использования отдельно от конечных веб-точек.
- Теоретически вы по-прежнему можете применять предметно-ориентированное проектирование и моделировать предметную область с помощью моделей Django, тесно связанных с базой данных. Миграции, может, и затормозят работу, но не сломают все окончательно. До тех пор пока ваше приложение не слишком сложное, а тесты — не слишком медленные, вы можете получить преимущество из подхода на основе *толстых моделей* (fat models): протолкните как можно больше логики в свои модели и примените такие паттерны, как «Сущность», «Объект-значение» и «Агрегат». Однако обратите внимание на нижеследующее предостережение.

С учетом сказанного в сообществе Django поговаривают¹, что подход на основе толстых моделей сам по себе сталкивается с проблемами масштабируемости, особенно в том, что касается управления взаимозависимостями между приложениями. В этих случаях много говорится о выделении бизнес-логики или слоя предметной области, который будет располагаться между представлениями данных, формами и `models.py`, которые затем можно свести к минимуму.

¹ См. <https://oreil.ly/Nbpjj>

Шаги на пути

Предположим, что вы работаете над проектом Django и не уверены, что он будет достаточно сложным, чтобы оправдать рекомендуемые нами паттерны, но все равно хотите что-то предпринять, чтобы в среднесрочной перспективе, а также при добавлении каких-нибудь паттернов в будущем ваша жизнь была проще. Подумайте вот о чем:

- Один из советов — вложить `logic.py` в каждое приложение Django с самого первого дня. Это даст возможность поместить бизнес-логику и в то же время не связывать ее с формами, представлениями и моделью. Позже это может стать ступенькой для перехода к полностью *не связанной* модели предметной области и/или сервисному слою.
- Слой бизнес-логики может начать работать с объектами модели Django и только позже полностью отвязаться от фреймворка и работать с простыми структурами данных Python.
- Что касается чтения, то можно получить некоторые преимущества разделения обязанностей между командами и запросами, вложив чтение в одно место во избежание разбросанных повсюду вызовов ORM.
- При разбивке модулей для чтения и для логики предметной области, возможно, стоит устранить связанность с иерархией приложений Django. Обязанности бизнеса будут их пересекать.



Хотели бы поблагодарить Дэвида Седдона и Ашию Завадук за то, что они приняли участие в обсуждении некоторых идей, изложенных в этом приложении. Они сделали все возможное, чтобы мы не наговорили глупостей на тему, в которой у нас мало опыта, но, возможно, им это не удалось.

Прочие размышления и реальный опыт работы с существующими приложениями приведены в эпилоге.

Валидация

Всякий раз, когда мы рассказываем обо всех этих приемах, из раза в раз возникает один и тот же вопрос: «Где я должен выполнять валидацию? Должна ли она относиться к моей бизнес-логике в модели предметной области или же это все-таки инфраструктурная обязанность?»

Как и в любом вопросе, касающемся архитектуры, ответ такой: все зависит от обстоятельств!

Самый важный пункт — стремление поддерживать код в явно разделенном состоянии, чтобы каждая часть системы была простой. Мы не хотим загромождать код ненужными деталями.

И вообще, что такое валидация?

Когда люди используют слово «валидация» (validation), то обычно имеют в виду процесс, когда тестируют данные на входе в операцию, чтобы убедиться, что они соответствуют определенным критериям. Входные данные, соответствующие этим критериям, считаются *валидными*, а не соответствующие — *невалидными*.

Если входные данные невалидные, то операция не может продолжаться и должна завершиться с какой-то ошибкой. Другими словами, валидация — это создание *предварительных условий*. Мы считаем, что лучше разделять предварительные условия на три подтипа: синтаксис, семантику и прагматику.

Валидация синтаксиса

В лингвистике *синтаксис* языка — это множество правил, которые управляют структурой грамматических предложений. Например, в русском языке предложение «Разместить 3 шт. артикула ЛАМПА-БЕЗВКУСНАЯ в за-

каз 27» правильное с точки зрения грамматики, в то время как выражение «чуки пуки ток» — нет. Мы можем описать грамматически правильные предложения как хорошо сформулированные.

Как это соотносится с нашим приложением? Вот несколько примеров синтаксических правил:

- команда размещения заказа `Allocate` должна иметь идентификатор заказа, артикул и количество;
- количество — это положительное целочисленное значение;
- артикул — это строковое значение.

Все это является правилами формы и структуры входных данных. Команда `Allocate` без артикула или идентификатора заказа не является валидным сообщением. Это эквивалент фразы «разместить три в».

Обычно мы проверяем эти правила по краю системы. Эмпирическое правило: обработчик сообщений всегда должен получать только хорошо сформированное сообщение, содержащее всю необходимую информацию.

Один из вариантов — помещать логику валидации в сам тип сообщений.

Валидация на классе сообщений (`src/allocation/commands.py`)

```
from schema import And, Schema, Use

@dataclass
class Allocate(Command):

    _schema = Schema({ ❶
        'orderid': int,
        sku: str,
        qty: And(Use(int), lambda n: n > 0)
    }, ignore_extra_keys=True)

    orderid: str
    sku: str
    qty: int

    @classmethod
    def from_json(cls, data): ❷
        data = json.loads(data)
        return cls(**_schema.validate(data))
```

❶ Библиотека `schema`¹ позволяет описывать структуру и валидацию сообщений в приятной декларативной форме.

❷ Метод `from_json` читает строку как JSON и превращает ее в тип сообщений.

Но здесь мы рискуем столкнуться с повторением, так как нужно указывать поля дважды, поэтому можно ввести вспомогательную библиотеку, которая унифицирует валидацию и объявление типов сообщений.

Фабрика команд со схемой (`src/allocation/commands.py`)

```
def command(name, **fields): ❶
    schema = Schema(And(Use(json.loads), fields), ignore_extra_
                      keys=True) ❷
    cls = make_dataclass(name, fields.keys())
    cls.from_json = lambda s: cls(**schema.validate(s)) ❸
    return cls

def greater_than_zero(x):
    return x > 0

quantity = And(Use(int), greater_than_zero) ❹

Allocate = command( ❺
    orderid=int,
    sku=str,
    qty=quantity
)

AddStock = command(
    sku=str,
    qty=quantity
```

❶ Функция `command` принимает имя сообщения плюс именованные аргументы `kwargs` для полей с полезной для сообщения информацией, где имя `kwarg` — это имя поля, а значение — синтаксический анализатор.

❷ Используем функцию `make_dataclass` из модуля `dataclass` для динамического создания типа сообщений.

❸ Патчим метод `from_json`, направляя в динамический класс данных.

¹ См. <https://pypi.org/project/schema>

④ Можем создавать многоразовые парсеры для количества, артикула и т. д., чтобы держать сущности сухими (DRY — неповторяемыми).

⑤ Объявление типа сообщений становится однострочным.

Все это происходит за счет потери типов в вашем классе данных, так что не забываете об этом компромиссе.

Закон Постеля и паттерн «Толерантный читатель»

Закон Постеля, или принцип надежности, гласит: «Будьте либеральны в том, что вы получаете от пользователя, и консервативны в том, что отдаете ему». Считаем, что этот закон особенно хорошо применим в контексте интеграции с другими нашими системами. Здесь идея в том, что мы должны быть строгими, когда посылаем сообщения в другие системы, но как можно более либеральными, когда получаем сообщения от них.

Например, наша система *может* выполнять валидацию формата артикула. Мы используем составные артикулы, такие как ПОДУШКА-НЕУМОЛИМАЯ, UNFORGIVING-CUSHION, и ПУФИК-ПРЕЗРЕННЫЙ, MISBEGOTTEN-POUFFE. Они следуют простой схеме: два слова, разделенные дефисом, где второе слово — это тип продукта, а первое слово — прилагательное¹.

Разработчики любят проверять такие штуки в своих сообщениях и отметить все, что выглядит как невалидный артикул. Это приводит в дальнейшем к ужасным проблемам, когда какой-нибудь бунтарь выпускает продукт под названием КРЕСЛО-ЛЕЖАНКА-КОМФОРТАБЕЛЬНОЕ, COMFY-CHAISE-LONGUE или когда сбой у поставщика приводит к отгрузке артикула КОВЕР-ДЕШЕВЫЙ-2, CHEAP-CARPET-2.

Системы размещения заказов вовсе *не касается* то, каким может быть формат артикула. Нужен лишь идентификатор, поэтому можно просто описать его как строковое значение. Это означает, что система закупок может менять формат, когда ей заблагорассудится, и системе заказов будет все равно.

¹ В русском переводе, следуя принятому деловому обиходу, артикулы именуются наоборот — сначала родовое существительное, потом видовое описательное прилагательное. — *Примеч. пер.*

ПОСТЕЛЬ ВСЕГДА ПРАВ?

Кого-то может раздражать одно лишь упоминание Постеля. Вам скажут, что именно Постель виноват в том, что в интернете все неисправно и ничего хорошего там быть не может. Но спросите как-нибудь Хайнека о SSLv3.

Нам нравится подход на основе паттерна «Толерантный читатель» в конкретном контексте интеграции на основе событий между контролируруемыми нами службами, потому что он допускает независимое развитие этих служб.

Если вы отвечаете за API с публичным доступом в большом и страшном интернете, то у вас есть веские причины для того, чтобы быть консервативнее в отношении того, какие входные данные вы считаете допустимыми.

Этот же принцип применим к номерам заказов, телефонным номерам покупателей и многому другому. По большей части мы можем игнорировать внутреннюю структуру строковых значений.

Точно так же разработчики любят выполнять валидацию входящих сообщений с помощью Schema JSON или создавать библиотеки, которые валидируют входящие сообщения и делятся ими между системами. Все это также ненадежно.

Давайте представим, например, что система закупок добавляет новые поля в сообщение об изменении размера партии товара, `ChangeBatchQuantity`, которые регистрируют причину изменения и почтовый адрес пользователя, ответственного за это изменение.

Поскольку эти поля не имеют никакого значения для службы размещения заказов, их следует просто проигнорировать. Можно сделать это в библиотеке `schema`, передав именованный аргумент `ignore_extra_keys=True`.



Валидируйте как можно меньше. Читайте только те поля, которые вам нужны, и не уточняйте их содержимое. Это поможет системе оставаться надежной, когда другие системы меняются со временем. Не поддавайтесь искушению поделиться определениями сообщений между системами: вместо этого упростите определение данных, от которых вы зависите. См. статью Мартина Фаулера о паттерне «Толерантный читатель»¹.

¹ См. https://oreil.ly/YL_La

Указанный паттерн, при котором извлекаются только интересующие нас поля и выполняется их минимальная валидация, называется паттерном «Толерантный читатель» (Tolerant Reader).

Валидация по краям системы

Мы упомянули, что не хотим загромождать код ненужными деталями. В частности, мы хотим не писать защитный код внутри модели предметной области, а убедиться, что запросы действительно валидны, прежде чем их увидят модель предметной области или обработчики вариантов использования. Это помогает исходному коду долго оставаться чистым и пригодным для технического сопровождения. Иногда мы называем это *валидацией по краям системы* (validating at the edge of the system).

Вдобавок к поддержанию вашего кода чистым и свободным от бесконечных проверок и подтверждений истинности имейте в виду, что невалидные данные, блуждающие по вашей системе, — это бомба замедленного действия; чем глубже она проникает, тем больше вреда может нанести и тем меньше инструментов у вас есть, чтобы решить проблему. Еще в главе 8 мы говорили, что шина сообщений — это отличное место для размещения сквозных обязанностей и валидация является прекрасным тому примером. Вот как мы можем изменить шину сообщений, чтобы она выполняла валидацию за нас:

Валидация

```
class MessageBus:
```

```
    def handle_message(self, name: str, body: str):
        try:
            message_type = next(mt for mt in EVENT_HANDLERS if mt.__
                                name__ == name)
            message = message_type.from_json(body)
            self.handle([message])
        except StopIteration:
            raise KeyError(f"Неизвестное имя {name} сообщения")
        except ValidationError as e:
            logging.error(
                f'невалидное сообщение типа {name}\n'
                f'{body}\n'
                f'{e}'
            )
            raise e
```

Вот как можно использовать этот метод из конечной точки Flask API:

API выталкивает ошибки валидации вверх (src/allocation/flask_app.py)

```
@app.route("/change_quantity", methods=['POST'])
def change_batch_quantity():
    try:
        bus.handle_message('ChangeBatchQuantity', request.body)
    except ValidationError as e:
        return bad_request(e)
    except exceptions.InvalidSku as e:
        return jsonify({'message': str(e)}), 400

def bad_request(e: ValidationError):
    return e.code, 400
```

И теперь его можно подключить к асинхронному процессору сообщений.

Ошибки валидации при обработке сообщений Redis (src/allocation/redis_pubsub.py)

```
def handle_change_batch_quantity(m, bus: messagebus.MessageBus):
    try:
        bus.handle_message('ChangeBatchQuantity', m)
    except ValidationError:
        print('Пропуск невалидного сообщения')
    except exceptions.InvalidSku as e:
        print(f'Не получается изменить товар – отсутствующий артикул {e}')
```

Обратите внимание, что точки входа занимаются исключительно тем, что получают сообщения из внешнего мира и сообщают об успехе или неудаче. Шина сообщений занимается валидацией наших запросов и маршрутизацией их в правильный обработчик, а обработчики сосредоточены исключительно на логике варианта использования.



Когда вы получаете невалидное сообщение, обычно мало что можно сделать, кроме как зарегистрировать ошибку и продолжить. В компании MADE мы используем метрики для подсчета числа получаемых системой сообщений и доли успешно обработанных, пропущенных или невалидных из них. Инструменты мониторинга предупредят нас, если мы увидим всплеск «плохих» сообщений.

Валидация семантики

В то время как синтаксис связан со структурой сообщений, семантика относится к изучению их *смысла*. Предложение «Отменить никаких собак из многоточия четыре» синтаксически правильное и имеет ту же структуру, что и предложение «Разместить один чайник в заказ пять», но оно бессмысленно.

Мы можем прочитать этот документ JSON как команду `Allocate`, но не сможем успешно ее выполнить, потому что это *бессмыслица*.

Бессмысленное сообщение

```
{
  "orderid": "superman",
  "sku": "zygote",
  "qty": -1
}
```

Мы привыкли проверять семантические обязанности в слое обработчика сообщений с помощью своего рода контрактного программирования.

Предварительные условия (src/allocation/ensure.py)

```
"""
```

Этот модуль содержит предварительные условия, которые мы применяем к обработчикам.

```
"""
```

```
class MessageUnprocessable(Exception): ❶
```

```
    def __init__(self, message):
        self.message = message
```

```
class ProductNotFound(MessageUnprocessable): ❷
```

```
    """
```

Это исключение инициируется, когда мы пытаемся выполнить действие с продуктом, которого нет в базе данных.

```
    """
```

```
    def __init__(self, message):
        super().__init__(message)
        self.sku = message.sku
```

```
    def product_exists(event, uow): ❸
```

```
product = uow.products.get(event.sku)
if product is None:
    raise ProductNotFound(event)
```

❶ Используем общий базовый класс для ошибок, свидетельствующих о невалидности сообщения.

❷ Использование конкретного типа ошибок для этой задачи упрощает отчет об ошибке и ее обработку. Например, в Flask легко сопоставить `ProductNotFound` с 404.

❸ `product_exists` — предварительное условие. Если условие является ложным, то инициируем ошибку.

Это поддерживает главный поток логики в сервисном слое чистым и декларативным.

Обеспечить вызовы в службах (`src/allocation/services.py`)

```
# services.py
from allocation import ensure

def allocate(event, uow):
    line = mode.OrderLine(event.orderid, event.sku, event.qty)
    with uow:
        ensure.product_exists(uow, event)

        product = uow.products.get(line.sku)
        product.allocate(line)
        uow.commit()
```

Можем расширить эту технику для проверки идемпотентного использования сообщений. Например, нужно убедиться, что мы не вставляем партию товара более одного раза.

Если нас попросят создать уже существующую партию, то мы внесем в лог предупреждение и перейдем к следующему сообщению.

Инициация исключения `SkipMessage` для игнорируемых событий (`src/allocation/services.py`)

```
class SkipMessage (Exception):
    """
    Это исключение возникает, когда сообщение не может быть
    обработано, но нет никакого неправильного поведения. Например,
    мы можем получить одно и то же сообщение несколько раз или
    сообщение, которое уже устарело.
    """
```

```
def __init__(self, reason):
    self.reason = reason

def batch_is_new(self, event, uow):
    batch = uow.batches.get(event.batchid)
    if batch is not None:
        raise SkipMessage(f"Партия с ИД {event.batchid} уже
        существует")
```

Введение исключения «Пропустить сообщение», `SkipMessage`, позволяет обрабатывать эти случаи в общем виде в шине сообщений.

Шина сообщений теперь знает, как пропускать сообщения (src/allocation/messagebus.py)

```
class MessageBus:
```

```
    def handle_message(self, message):
        try:
            ...
        except SkipMessage as e:
            logging.warn(f"Пропуск сообщения {message.id}, поскольку
            {e.reason}")
```

Здесь есть пара подводных камней, о которых следует знать. Во-первых, следует убедиться, что мы применяем одинаковые UoW для главной логики нашего варианта использования. В противном случае могут возникнуть ошибки параллелизма.

Во-вторых, нужно постараться не вкладывать всю бизнес-логику в эти проверки предварительных условий. В качестве эмпирического подхода: если правило *может* быть протестировано внутри модели предметной области, то там оно и *должно* быть протестировано.

Валидация прагматики

Прагматика — это то, как мы понимаем язык в контексте. После того как мы разобрали сообщение и поняли его смысл, его остается обработать в контексте. Например, если вы получаете комментарий к запросу на включение измененного кода со словами «думаю, что это очень смело», то это может означать, что рецензент восхищается вашей смелостью — если только он не англичанин и в этом случае пытается до вас донести, что ваши намерения слишком рискованны и только дурак станет это делать. Контекст — это наше все.

КРАТКО О ВАЛИДАЦИИ

Люди по-разному понимают валидацию

Говоря о валидации, убедитесь, что вы четко представляете себе то, что вы валидируете. Считаем, что есть смысл задуматься о синтаксисе, семантике и прагматике: структуре сообщений, содержательности сообщений и бизнес-логике, управляющей ответами на сообщения.

Валидируйте по краю, когда это возможно

Валидировать обязательные поля и допустимые диапазоны чисел скучно, и мы хотим, чтобы это не входило в нашу красивую чистую кодовую базу. Обработчики всегда должны получать только валидные сообщения.

Валидируйте только то, что требуется

Используйте паттерн «Толерантный читатель»: читайте только те поля, которые нужны вашему приложению, и не конкретизируйте чрезмерно их внутреннюю структуру. Рассматривая поля как непрозрачные строковые значения, вы приобретаете большую гибкость.

Потратьте время на написание помощников для валидации

Наличие хорошего декларативного способа валидации входящих сообщений и применения предварительных условий к вашим обработчикам сделает кодовую базу намного чище. Стоит потратить время на то, чтобы сделать скучный код простым в техническом сопровождении.

Локализуйте каждый из трех типов валидации в нужном месте

Валидация синтаксиса может выполняться в классах сообщений, валидация семантики — в сервисном слое или в канале передачи сообщений, а валидация прагматики — в модели предметной области.



После того как вы выполнили валидацию синтаксиса и семантики ваших команд по краям вашей системы, предметная область становится местом для валидации всего остального. Валидация прагматики часто является главной частью ваших бизнес-правил.

С точки зрения ПО прагматика операции обычно управляется моделью предметной области. Когда мы получаем сообщение типа «разместить заказ на три миллиона штук артикула ЧАСЫ-ДЕФИЦИТНЫЕ для заказа 76 543», оно синтаксически и семантически является верным, но мы не сможем его выполнить, потому что у нас нет столько товара.

Об авторах

Гарри Персиваль несколько лет работал консультантом по менеджменту и невыносимо страдал. Вскоре он вновь открыл свою истинную природу гика — ему повезло попасть в группу фанатиков экстремального программирования (ХР), первопроходцев в разработке Resolver One — программы для работы с электронными таблицами, которая, к сожалению, прекратила свой жизненный цикл. Работал в PythonAnywhere LLP, продвигая разработку через тестирование в интервью, на семинарах и конференциях по всему миру. Сейчас работает в компании MADE.com.

Боб Грегори — архитектор ПО в компании MADE.com. Вот уже более десяти лет создает событийно-управляемые системы через предметно-ориентированное проектирование.

Об обложке

Змея на обложке — темный тигровый питон (*Python bivittatus*) родом из Юго-Восточной Азии. В наши дни обитает в джунглях и болотах в Южной Азии, Мьянме, Китае и Индонезии; также завезен в Национальный парк Эверглейдс, штат Флорида.

Тигровые питоны — одни из самых крупных змей в мире. Могут вырастать до семи метров и весить 90 кг. Самки крупнее самцов. В одной кладке может быть до ста яиц. В дикой природе тигровые питоны живут в среднем от 20 до 25 лет.

Отметины на теле тигрового питона начинаются со светло-коричневого пятна в форме стрелы на голове и продолжаются вдоль тела прямоугольниками, которые выделяются на фоне коричневой чешуи. Прежде чем тигровые питоны вырастут окончательно, что занимает два-три года, они живут на деревьях, охотясь на мелких млекопитающих и птиц. Они также умеют плавать, обходясь без кислорода до 30 минут.

Из-за разрушения среды обитания тигровый питон получил охранный статус как уязвимый вид. Многие животные, изображенные на обложках издательства O'Reilly, находятся под угрозой исчезновения, и все они важны для нашего мира.

Гарри Персиваль, Боб Грегори

**Паттерны разработки на Python:
TDD, DDD и событийно-ориентированная архитектура**

Перевел с английского А. Логунов

Заведующая редакцией	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>А. Юринова</i>
Технический редактор	<i>М. Петруненко</i>
Литературные редакторы	<i>А. Жужулин, М. Петруненко</i>
Обложка	<i>В. Мостипан</i>
Корректоры	<i>Е. Павлович, М. Одинокова</i>
Верстка	<i>Е. Неволайнен</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,

Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 11.2021. Наименование: книжная продукция.

Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортёр в Беларусь: ООО «ПИТЕР М», 220020, РБ,

г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 15.10.21. Формат 70х100/16. Бумага офсетная. Усл. п. л. 27,090. Тираж 700. Заказ